数理工学実験

2026年度版

実験演習ワーキンググループ編

数理工学実験 (2026 年度版)

京都大学 工学部情報学科 数理工学コース

授業担当：
川越 大輔 助教
吉渡 叶 助教
筒 広樹 助教
原田 健自 助教

京都大学工学部情報学科数理工学コース
実験演習ワーキンググループ編

目 次

第 1 章　レポートの書法 **8**

1.1 レポートの内容 8
1.1.1 構成 8
1.1.2 形式 9
1.1.3 文章 9
1.1.4 物理量について 10
1.1.5 Activity 1 11
1.2 LaTeX の使い方 12
1.2.1 最初の一歩 12
1.2.2 節の作り方 12
1.2.3 文章の書き方 12
1.2.4 数式の書き方 13
1.2.5 図の入れ方 13
1.2.6 表の入れ方 14
1.2.7 参考文献の書き方 15
1.2.8 newcommand 15
1.2.9 Activity 2 15
1.3 gnuplot で図を作成する 16
1.3.1 enhanced を使った図の作り方 16
1.3.2 LaTeX 数式を使った図の作り方 16
1.3.3 データファイルのプロット法 18
1.3.4 Activity 3 18

第 2 章　数値線形代数 **19**

2.1 目的 19
2.2 準備 19
2.2.1 ベクトルと行列のノルム 19
2.2.2 内積 20
2.3 連立 1 次方程式の数値解法 20
2.3.1 消去法 20
2.3.2 行列の LU 分解 22
2.3.3 LU 分解による解法 23
2.4 行列固有値問題の数値解法 23
2.4.1 べき乗法 24
2.4.2 逆反復法 26

2.4.3 行列の QR 分解 26
2.4.4 QR 法 27
2.4.5 QR 法の高速化 28
2.5 課題 29

第 3 章 連続最適化 **31**
3.1 目的 31
3.2 関数の零点探索と Python 導入 31
3.2.1 関数の零点探索 31
3.2.2 Python 導入 32
3.3 数理最適化の基礎と降下法 33
3.3.1 関数の最大化・最小化 33
3.3.2 降下法 34
3.3.3 最急降下法とニュートン法 35
3.4 直線探索と最適化手法の収束性 36
3.4.1 直線探索法 36
3.4.2 反復法の収束性について 37
3.5 最適化手法の実用的な修正方法と総合演習 39
3.5.1 ニュートン方向の修正 39
3.5.2 総合演習 39
3.6 （追加）制約付き凸二次計画問題 40

第 4 章 高速フーリエ変換 **42**
4.1 はじめに 42
4.2 フーリエ級数 42
4.3 フーリエ変換 44
4.4 離散フーリエ変換 (DFT) 45
4.5 高速フーリエ変換 (FFT) 48
4.6 パワースペクトル密度と窓関数 50
4.7 問題 52

第 5 章 常微分方程式の数値解法 **56**
5.1 目的 56
5.2 離散化 57
5.3 代表的な数値解法 57
5.3.1 Euler 法 57
5.3.2 Crank-Nicolson 法 58
5.3.3 多段法 58
5.3.4 高次精度一段法 59
5.4 安定性 60
5.5 爆発解 61
5.6 カオス 62
5.7 演習問題 63

第 6 章　組合せ最適化 **65**
6.1 目的 . . . 65
6.2 準備 . . . 65
6.3 最短経路問題 . . . 65
6.4 分枝限定法 . . . 66
6.5 最短路問題に対する分枝限定法 . . . 67
6.6 実験課題 . . . 67

第 7 章　モンテカルロシミュレーション **70**
7.1 はじめに . . . 70
7.2 疫病の確率的な伝染モデル . . . 70
7.2.1 サンプル状態列 . . . 71
7.3 期待値のモンテカルロ計算 . . . 73
7.3.1 円周率の計算 . . . 74
7.3.2 サンプルの作成方法 . . . 74
7.4 マルコフ連鎖モンテカルロ法 . . . 75
7.4.1 マルコフ過程とその状態の出現頻度の分布 . . . 75
7.4.2 マルコフ連鎖モンテカルロ法のアルゴリズム . . . 76
7.4.3 マルコフ連鎖モンテカルロ法で注意すべき点 . . . 77
7.5 イジングモデル . . . 77
7.5.1 エネルギー . . . 77
7.5.2 トーラス上のイジングモデルの相転移現象（臨界現象） . . . 78
7.5.3 対称性の自発的破れ . . . 78
7.5.4 イジングモデルのメトロポリス法 . . . 79
7.6 おわりに . . . 80

第 8 章　熱方程式の差分法 **81**
8.1 目的 . . . 81
8.2 拡散方程式 . . . 81
8.3 数値解法 . . . 81
8.3.1 Euler 陽解法 . . . 82
8.3.2 Crank-Nicolson 法 . . . 83
8.3.3 その他の陰解法 . . . 85
8.4 安定性解析 . . . 86
8.5 演習問題 . . . 86

付 録 A 最小二乗法 **90**
1.1 目的 . . . 90
1.2 最小二乗法とその評価方法 . . . 90
1.2.1 回帰問題 . . . 92
1.2.2 重み付き最小二乗法 . . . 96
1.3 推定値の合成と逐次最小二乗法 . . . 98
1.3.1 異なるデータセットからの推定値の合成 . . . 98

1.3.2 逐次最小二乗法 99
1.4 Kalman フィルタと Kalman スムーザ 102
1.4.1 Kalman フィルタ 102
1.4.2 Kalman スムーザ 103
1.5 交互最小二乗法 104
1.5.1 交互最小二乗法 104
1.5.2 K-平均法 105
付録 データの取り扱い 107
付録 1 データの読み込み 107
付録 2 データのプロット 108

付 録 B 関数の補間と数値積分 110
2.1 目的 110
2.2 Lagrange 補間と Newton-Cotes 公式 110
2.2.1 Lagrange 補間 110
2.2.2 Newton-Cotes 公式 111
2.2.3 複合公式 112
2.2.4 Newton-Cotes 公式の誤差 113
2.2.5 2.2 節の課題 113
2.3 Gauss 型積分公式 114
2.3.1 直交多項式 114
2.3.2 Gauss 型積分公式 115
2.3.3 複合公式 116
2.3.4 Gauss 型積分公式の誤差 116
2.3.5 2.3 節の課題 117
2.4 有限要素法 117
2.4.1 概要 117
2.4.2 Strum-Liouville 型微分方程式と弱形式 118
2.4.3 弱形式の離散化 118
2.4.4 2.4 節の課題 119

付 録 C ニューラルネットワークによる機械学習 121
3.1 目的 121
3.2 数学的準備 121
3.2.1 ニューラルネットワーク 121
3.2.2 関数回帰と最適化 122
3.2.3 勾配降下法による学習 122
3.3 計算機実装 124
3.3.1 Google Colaboratory 124
3.3.2 レポートの作成 124
3.3.3 NN による画像認識 125
3.4 次元削減による特徴量抽出 125
3.4.1 オートエンコーダ 125

3.4.2　手書き文字の特徴量抽出 125
3.5　生成モデル学習 126
3.5.1　変分オートエンコーダ 126

第1章　レポートの書法

本稿では, まず第 1.1 節でレポートの構成・形式・文章作成上の注意点などを述べる. 次に, レポート作成に使用する LaTeX の簡単な使い方を第 1.2 節で, gnuplot を使って図を作成する方法を第 1.3 節で紹介する.

1.1　レポートの内容

実験のレポートとは, その実験の報告書であり, 実験内容を知らない第三者が読んでも理解できるような内容でなければならない. 他人に読んで貰うためのものである以上, 読者の意欲を損なわないよう読みやすく仕上げることを心がける.

読みやすくするために注意すべきこととして,

- 構成 (第 1.1.1 節)
- 形式 (第 1.1.2 節)
- 文章 (第 1.1.3 節)

を順に説明する. また科学実験に付随する問題として

- 物理量の記法 (第 1.1.4 節)

を述べる.

1.1.1　構成

表紙

本実験のレポートには表紙を付ける. 表紙には以下の項目を記入すること.

- 科目名
- 実験テーマ
- 実験の実施年月日
- 実験の実施場所
- レポートの提出年月日
- 提出者の学年, 学生番号および氏名

レポートの構成の一例

1. はじめに
   何をしたいのかという目的や, 目的を達成するための方法の概略を書く. 方法の詳細は次の「原理」に譲る.
2. 原理・方法の詳細
   配布冊子の内容をそのまま写してはいけない. 原理を一旦自分なりに理解した後, 説明に必要な項目を取捨選択し, 必要であれば補足して記す.
3. 実験方法
   レポートで報告されている実験を, 読者が再現できるように書く. 読者が再現できなければ, それはレポートに書かれている情報 (方法の詳細やパラメータの値など) が足りていないということである.
4. 結果
   実験結果をわかりやすくまとめる. 必要に応じて, 図や表を用いる. 図を gnuplot を用いて用意する方法は第 1.3 節を参照せよ.
5. 考察・議論
   実験から得られた結果の意味や正当性の吟味, 問題点の議論などを行う. 理論値があればそれとの対比を行ない, 理論値と実験値が異なればその理由を考察する.

   「有意義だった」「大変だった」などの個人の主観による感想は要らない. 科学的文書に必要なのは事実とそれに基づく論理的推論である.
6. 結論
   実験結果と考察から得られた内容のまとめを行う. 読者は「はじめに」と「結論」に注目することが多いので, これまでに述べたことでも大事なことは再記載・強調してもよい.

7. 参考文献
   本文中で引用した参考文献を記すこと. 本文に記載した事の根拠とそれが誰の業績かを明らかにするため, 正しく引用することは重要である.

1.1.2 形式

- 用紙は A4 判を使用する.
- LaTeX を使う.
  数式が混じった文章を綺麗に書くソフトとして LaTeX がある. LaTeX の簡単な使用法については第 1.2 節を参照すること.
- 使用する記号にはすべて定義を与える.
- 式, 図, 表の参照は番号で行う.
  文中で式, 図, 表を参照する場合は番号を使用する. 例えば,

  式 (2) を式 (10) に代入すると…
  測定結果を表 2 にまとめる.

  などと使用する. 式番号は ( ) で囲み, 図や表の番号は囲まない. これらの番号は文中に現れる順番による. 文章構成の変更にともない式などの順番が変わっても自動的に正しく参照する方法については第 1.2 節を見ること.
- 図表の適切な使用.
  図や表は一つの独立した要素であるので, それを見ただけで何を表したものであるかが明確にされていなければならない. また適切なグラフ形式 (線形・片対数・両対数など) を選択し, 縦軸, 横軸は何を表しているのか, そしてそれらの単位は何であるかを明らかにしなければならない.

1.1.3 文章

ネイティブな日本語話者であっても、正しくかつ読みやすい日本語文章を書くには訓練が必要である. ここではいくつかの注意点を述べる. どう書くのが良いか迷った場合には, 文の自然さや格調を多少犠牲にしても, 意味の明確さを優先することを勧める. 文献 [1] も参考にすること.

主張は何か?

レポートを含め科学的文書の目的は,「自分の主張を読者に伝える」ことである. まずは「自分の主張が何であるか」をよく考え, それを強く意識しながら書く.

構成

主張すべきことが決まったら, その主張が明確になるよう構成を考える. だいたいは第 1.1.1 節で示したような例に沿えば良いが, 必要であれば順番を入れ替えたり, 項目を追加・削除することも考える.

段落

段落は気分で分けるものではない. 意味の塊ごとに段落を改める. ただしあまりに長い段落は読みにくさのもとなので, バランスを取って適切に段落分けをする.

例えば A1, A2, B1, B2, B3 というトピック毎のブロックがあるとする. (A1+A2), (B1+B2+B3) がそれぞれ適切な長さであれば, 2 つの段落に分ける. A1 などそれぞれのブロックが十分に長い場合は, 5 つの段落に分けることを考える.

文の順番

× A 君は朝起きた. B 君も朝起きて大学に行った. A 君は大学に行く途中で買い物をした. A 君と B 君は大学で会った.

内容は理解できるが, A 君のトピックが第 1,3 文に別れており読みにくい. 次のように交通整理する.

○ A 君は朝起きた. A 君は大学に行く途中で買い物をした. B 君も朝起きて大学に行った. A 君と B 君は大学で会った.

文の長さ

一文がとても長くなると読みにくさのもとになる. 時に一文が 10 行ほどにも渡る場合を見かけるが, 避ける方が賢明である. ただし一文一文があまりに短いと文章が幼い印象になる. その意味では先の例文も改善の余地がある.

○ A 君は朝起き, 大学に行く途中で買い物をした. B 君も朝起きて大学に行った. 二人は大学で会った.

多重の意味に取れることを避ける

× ぼくは A 君と B 君と C 君の家に行った.

これは多重に意味が取れる.
1. A 君, B 君, C 君の家にそれぞれ行った.
2. A 君と一緒に, B 君, C 君の家にそれぞれ行った.
3. A 君,B 君と一緒に, C 君の家に行った.

どの意味であるかわからないような文は避けるべきである.

主語と述語

命令形など特殊な場合を除き, 主語と述語は文の最小セットである[1]. 主語に対する述語が正しく書けているか注意する. 途中で形容する部分が長く入る場合には特に注意する.

× 関数 $f(x)$ は原点付近で $f(x) = f(0)+xf'(x)+\cdots$ とテイラー展開する.

関数が主体になってテイラー展開することはないので、次のように改善する.

○ (我々は) 関数 $f(x)$を原点付近で $f(x) = f(0)+xf'(x)+\cdots$ とテイラー展開する.

○ 関数 $f(x)$ は原点付近で $f(x) = f(0)+xf'(x)+\cdots$ とテイラー展開される.

[1]以下の第 2 例文のように, 日本語では主語を書かないほうが自然な文になる場合も多いが, ここではこの事情に深入りしない.

代名詞

代名詞はその名の通り「名詞の代わり」だから代名詞をそれが指す名詞で置き換えても文章が成立していなければならない. 例えば

○ A 君は僕の友達だ. 彼は日本人だ.

において, 2 文目では「彼」=「A 君」だから

○ A 君は僕の友達だ. A 君は日本人だ.

と書いてもよい.

代名詞が指す名詞が明確か否かを常に意識すること. 明確でなければ, 代名詞が指す名詞をそのまま書く.

× 方法 A と B がある. ここではこれを用いる.

○ 方法 A と B がある. ここでは方法 B を用いる.

助詞

「て, に, を, は」 が間違っていないか, 意味が取りやすいか, に注意する. また「の」が連続する文章は読みにくいので避ける.

× 上の方程式の右辺の第 1 項の変数 x の次数は 2 である.

○ 上の方程式の右辺において、第 1 項に現れる変数 x の次数は 2 である.

接続詞

「および」「あるいは」「よって」「しかし」「一方」等, 文などを繋ぐ言葉である。論理的な文章を書く上では重要である.

1.1.4 物理量について

物理量についての注意を述べる. 数値計算による実験においては該当しないこともあるので取捨選択すること.

有効数字

実験で測定された結果は有限桁の精度しかない. 有効桁だけを残したものを有効数字という. 例えば有効桁 3 桁で 1.23×10^1 と書けば, この数値には 0.01×10^1 の誤差を含んでおり真値は $[12.25, 12.35)$ の間にあることを意味する. 1.230×10^1 の真値は $[12.295, 12.305)$ の間にあり, 両者の意味が異なることがわかる.

2 つの有効数字を四則演算した場合, 計算結果の有効桁は次の通り:

- 乗算・除算: 有効桁のうち小さい方
  $(1.2 \times 10^1) \times (3.45 \times 10^2) = 4.140 \times 10^3 \to 4.1 \times 10^3$
  (有効桁 2 桁と 3 桁の積の有効桁は 2 桁)
- 加算・減算: 有効数字末端の桁位の大きい方
  $1.250 + 56.7 = 57.950 \to 58.0$
  (10^{-3} の桁までと 10^{-1} の桁までの和は 10^{-1} の桁まで有効だから 10^{-2} の桁を四捨五入する)

字体の使い分け

1. 物理量の添字が物理量あるいは変数を表す場合にはイタリック体を用いる. それ以外はローマン体を用いる.
   周波数 f に対する物理量 A の値を添え字で書くと A_f だが, 例えば前方 (forward) の略字を意図する場合は A_{f} と書く.
2. 単位を表す記号はローマン体を用いる.
   質量の単位は kg ではなく kg である.
3. 数学定数を表す文字はローマン体である.
   自然対数は e (e ではない), 虚数単位は i (i ではない) と表す.
4. 特殊な数学関数 ($\sin, \cos, \log$ など) や常微分演算子 $\frac{\mathrm{d}}{\mathrm{d}x}$ はローマン体で表す.

詳しい説明は [2] を参照のこと.

関係式の表記

物理量を文字を使って表す場合の注意についても [2] に説明されている. 重要な点は, 物理量相互間の関係式は単位に無関係に普遍的に成立するということである.

良く見かける不適切な例を [2] から引用しておこう.

× 電磁波の波長 λ[m] と周波数 f[Hz] の間には以下の関係がある.

$$\lambda = 3.0 \times 10^8 / f$$

3.0×10^8 という無次元量を使った関係式は λ を m, f を Hz という「ものさし」を用いた場合に有効であって, そうでない場合は成立しない. λ と f の単位をどのようにとっても成立する関係式は, 以下のように書けばよい.

○ 電磁波の波長 λ と周波数 f の間には以下の関係がある.

$$\lambda = (3.0 \times 10^8 \mathrm{m/sec}) / f$$

図表の数値

図や表に現れる数値は無次元量で書かなければならない. 例えば長さ l を cm 単位で測った値を数直線上に表すとしよう. 数直線上の値が 1.2 という点は $l = 1.2$cm を表す. よって数直線のラベルを l/cm としておくと 1.2 は無次元量となる. 測定の単位を m に変えて $l = 1.2 \times 10^{-3}$m となるときはラベルを $10^3 l/\mathrm{m}$ とすればよい. この方法を使うと, グラフから読み取った値を 10^{-3} 倍するのか, 読み取った値はすでに 10^{-3} 倍されているのかという曖昧さを無くすことができる.

1.1.5 Activity 1

I. 「二次方程式 $x^2 - 2x - 3 = 0$ の解を求めよ」という問題に対して,
「$(x-1)^2 = 4.\ x = 3, -1.$」
という答案が提出された. この答案を改善するた

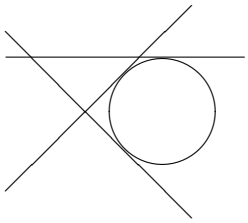
めに, 答案提出者へのアドバイスを考えよ. 箇条書きでも良い.

II. 先の問題に対する次の答案を改善せよ.

「$x^2-2x-3=0$ の問題を考える. 平方完成すればよい. この問題は平方完成できる. 解を求めると $x=3,-1$.」

III. $\sqrt{2}$ が無理数であることの証明を書け. 誰が読んでも誤解なくまた読みやすく書くこと. [2]

IV. 次の図形を文章のみで描写せよ. 図形を見たことがない人が読んだだけで描けるようにせよ. 大きさや多少の誤差は気にしなくて良い.



1.2 LaTeX の使い方

LaTeX とは, ソースファイルに書かれたコマンドをコンパイル (機械がわかるように翻訳) することにより, 数式が混じった文章を綺麗に仕上げるためのソフトである. ここでは基本的な使い方, および節・式・図・表番号の自動参照の仕方を説明する.

1.2.1 最初の一歩

最小限の LaTeX コマンドセットは

```
\documentclass{jarticle}
\begin{document}
本文を書く所
\end{document}
```

である. \begin と \end で挟まれた箇所を LaTeX では「環境」と呼ぶ. 上の例では document 環境が作成されたことになる. 節の題名や本文・式・図・表はすべて document 環境内に書く. documentclass が jarticle になっているのは日本語用だからであり, 英語用には article を用いる. なお環境によっては \ (バックスラッシュ) の代わりに ¥ (円マーク) が使われる場合もある.

1.2.2 節の作り方

```
\section{節の題名}

\subsection{小節 1 の題名}

\subsection{小節 2 の題名}
```

と書くと節や小節が作られ, 自動的に節・小節の番号が付けられる.

節番号を参照するときに手で書いていては, 構成を変更したときに面倒になる. 自動的に参照できるようにするには

```
\section{節の題名}
\label{sec:example}
....
第\ref{sec:example}節では...
```

と \section の後に \label でラベルを付けておいて, 参照したい箇所で \ref にラベルを書けばよい. 以下で見るように, 番号の自動参照は式番号, 図番号, 表番号でも可能である. ここでは節のラベルであることがわかるようにラベルの名前を sec:example としたが, ラベルの名前は自由にしてよい. ただしラベル毎に名前は変えなければならない.

なお正しく参照させるためには \label を付けて (あるいは変更して) から LaTeX ソースファイルを2 回コンパイルしなければならない.

1.2.3 文章の書き方

文章は LaTeX が適当なところで改行してくれる. ソースでの改行は LaTeX 文章には反映されな

[2] 証明の方法は, $\sqrt{2}$ が有理数になっている, つまり $\sqrt{2}=p/q$ (p,q は互いに素な整数, $q\neq 0$) なる p,q が存在すると仮定し, 両辺を二乗して矛盾を導けばよい.

い．コンパイル後の文章中で強制的に改行したい場所には \\ を書く[3]．

段落を改めたい場合は，前の段落の終わりから一行間を開けて書き始めればよい．段落の先頭では自動的に一字さがるがこれはこのままで良い。

1.2.4 数式の書き方

文章中では $ で挟むことにより数式を書く。例えば $\alpha=\beta$ と書くと $\alpha = \beta$ と出力される．

式を別行立てにするには

```
\begin{align}
  \alpha = \sin^{2}\beta
\end{align}
```

とする．出力は

$$\alpha = \sin^2 \beta \tag{1.1}$$

となる．数式を改行したい場合は \\ を入力する．この場合，&を各行に入れることにより位置を揃えることができる．式番号は式が現れる順に付される．本稿では実は documentclass が jarticle ではなく jbook のために (1.1) という番号になっているが，documentclass を jarticle にすると (1) などの番号になる．

式番号を自動的に参照するには，

```
\begin{align}
  \alpha = \sin^{2}\beta
  \label{eq:example}
\end{align}
```

と align 環境の中で \label を付けておく．参照する箇所では \eqref を使って

```
式 \eqref{eq:example} によると...
```

と書くと，

式 (1.1) によると...

と出力される．

番号なしの式は

```
\[ a=b \]
```

あるいは

```
\begin{align*}
  a=b
\end{align*}
```

と書くと

$$a = b$$

と出力される．

1.2.5 図の入れ方

図のファイル形式は様々あるが，第 1.3 節で見るように gnuplot の epslatex を使うと LaTeX の書式で美しい数学記号を EPS 形式の図中に入れられる．そこでここでは EPS ファイル example1.eps と example2.eps があるとしてこれを LaTeX 中に取り入れる方法を述べる．

手順 1: \documentclass と \begin{document} の間で graphicx パッケージを dvipdfmx オプションで読み込む．

```
\documentclass{jarticle}
\usepackage[dvipdfmx]{graphicx} %←ココ
\begin{document}
...
\end{document}
```

手順 2: example1.eps と example2.eps を LaTeX ソースと同じディレクトリ (フォルダ) に入れた後，図を入れたい場所に figure 環境で取り入れる．

```
\begin{figure}
 \centering
 \includegraphics{example1.eps}
 \includegraphics{example2.eps}
 \caption{図の取り入れ方法の例．第\ref{sec:gnuplot}節に示した二通りの作成方法の出力を比較している．}
 \label{fig:example}
\end{figure}
```

[3] 見た目を整える以外には使わない．

図の大きさを変えたい場合は,
\includegraphics[width=6cm]{example1.eps}
などとサイズを指定する. このようにサイズ指定して取り込んだ例を図 1.1 に示す.

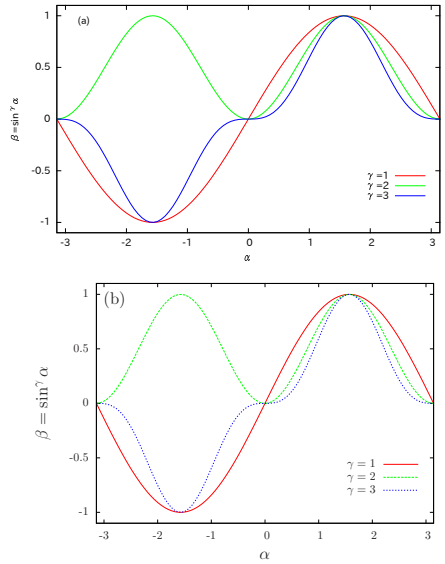


図 1.1: 図の取り入れ例. 第 1.3 節に示した二通りの作成方法の出力を比較している. (a) enhanced を使用. (b) epslatex を使用. パネル (b) だけが示された図に対するキャプション例は次の通り: $\beta = \sin^\gamma \alpha$ のグラフ. $\gamma = 1$ (赤, 実線), 2 (緑, 破線), 3 (青, 一点鎖線).

図についての注意

- 一つの figure 環境に複数枚の図がある場合には, (a) (b) などで区別する. 参照するときも図 1.1(a) などと書く.
- \caption の中には図の表題や説明を書く. 「実験装置」など一語だけのこともあれば, パラメータの値などを全部書いて 10 行以上に及ぶこともある. これは分野の文化による. いずれにせよ, 本文中あるいは \caption 中に必要な説明を与えるべきである.
- 式番号と同様に \label で図番号も参照できる. もちろんラベルの名前は自由でよい.
- LaTeX において図の位置調整は難しい.

```
\begin{figure}[htbp]
```

と h (here), t (top), b (bottom), p (page: 図用ページ) のオプションがある. これらオプションを取捨選択したり, figure 環境のブロックをソースファイル中で上下に動かしたりすると, ある程度は図の位置を動かせるが, 完全に思い通りにならないこともある.

1.2.6 表の入れ方

表を入れるには table 環境を使う. 以下のソースに対する出力は表 1.1 となる.

```
\begin{table}[h]
  \centering
  \caption{表の出力例}
  \begin{tabular}{l|ll}
    \hline
    a & 1 & 2 \\
    b & 3 & 4 \\
    \hline
  \end{tabular}
  \label{tab:example}
\end{table}
```

表 1.1: 表の出力例

a	1	2
b	3	4

表についての注意

- caption の内容については, 図と同様である.
- 図の caption は下に置くが, 表の caption は上に置く. \caption を \tabular よりも上に置けばよい.
- 式番号, 図番号と同様に \label で表番号も参照できる. もちろんラベルの名前は自由でよい.
- 図と同様に h, t, b, p のオプションがあるが, やはり位置の調整は簡単ではない.

1.2.7 参考文献の書き方

参考文献は, それを出力する位置に以下のように書く.

```
\begin{thebibliography}{99}
\bibitem{Eexp} 京都大学工学部電気系教室編，電気電子工学基礎実習.
\end{thebibliography}
```

Eexp は文献のラベルであり, 参照したい位置で \cite{Eexp} (\ref{Eexp} ではない) と書くと [2] と出る. 文献が複数ある場合は \bibitem を増やしていくと, 文献に付く番号が順に大きくなる. 文中に現れる順ではなく, thebibliography 環境に書いた順に番号が付されることに注意せよ.

文献情報の書式は分野の文化による. ここではその一例を紹介する.

書籍の場合

```
\bibitem{Feynman1} R. P. ファインマン，ファインマン物理学 1 力学（新装版)，坪井忠二訳（岩波書店，1986).
\bibitem{LL1} L. ランダウ，E. リフシッツ，力学 (増訂第 3 版）ランダウ=リフシッツ理論物理学教程（東京図書，1986).
```

それぞれ 1986 は出版年で, その前は出版社名である.

雑誌論文の場合

```
\bibitem{EPR} A. Einstein, B. Podolsky, and N. Rosen, Can quantum-mechanical description of physical reality be considered complete?,
Phys. Rev. {\bf 47}, 777 (1935).
```

必要な情報は,

- 著者名 (全員. 英語では姓以外はイニシャル)
- 雑誌名 (各雑誌に通例の略し方がある)
- 巻数 (太字で書く)
- 始まりのページ番号 (論文番号の場合あり)
- 出版年

である. 例のように, 論文の題名を入れる場合もある.

1.2.8 newcommand

何度も使う記号や命令はなるべく簡単にタイプしたい. こういう場合は \newcommand を使って \documentclass と \begin{document} の間に新しいコマンドを定義する. 例えば,

```
\documentclass{jarticle}
\newcommand{\al}{\alpha}
\begin{document}
...
```

としておくと、本文中では $\al$ と書くと α が出力される. 引数を持つコマンドも可能で

```
\newcommand{\be}[1]{\beta_{#1}}
```

として $\be{3}$ と書くと β_3 が出力される.

```
\newcommand{\ga}[2]{\gamma_{#1}^{#2}}
```

として $\ga{3}{4}$ と書くと γ_3^4 が出力される. 当然ながら、コマンドの名前は重複がないようにすべきである.

1.2.9 Activity 2

I. 次の出力になるように LaTeX のソースを作成せよ. 式番号は \ref を使って参照せよ. 式番号の参照さえできれば, 式番号は問題の番号でなくともよい.

関数

$$f(x,y) = x^2 + y \tag{1.2}$$

および

$$g(x,y) = y^2 + x \tag{1.3}$$

を考える. 関数 (1.2) および (1.3) の値が同時に 0 となる (x, y) を求める.

II. table 環境を使って, 今後 1 週間の週間天気予報を作れ (予報は仮定でもよい). caption も適切に書け.

III. 式番号の参照 (\ref{...}) で ( ) を書く手間を減らすために newcommand で \eqref を定義せよ. \eqref{eq:example} と打てばラベル eq:example を持つ式番号が ( ) で挟まれるようにせよ.

IV. 常微分 $\frac{\mathrm{d}x}{\mathrm{d}t}$ をタイプする手間を減らすために newcommand を使って \fracd を定義し \fracd{x}{t} と打てば $\frac{\mathrm{d}x}{\mathrm{d}t}$ が出力されるようにせよ. なお d の部分は \mathrm{d} で出る.

1.3 gnuplot で図を作成する

gnuplot で図を作成するには二通りの方法がある.

- gnuplot を立ち上げて一行ずつコマンドを打ち込む.
- スクリプトファイル (実行すべきコマンドを順に書いたファイル) を gnuplot に読み込ませる.

作った図の修正が容易なため, 後者を勧める. 本稿でも後者の方法を説明する. gnuplot のバージョンは 4 以上であることを確認されたい.

軸のラベルなどにギリシャ文字の出力を試したいので, 以下では例として, $\beta = \sin\alpha$, $\beta = \sin^2\alpha$, $\beta = \sin^3\alpha$ のグラフを描く. ただし gnuplot で関数をプロットするには sin(x) などとしなければならないことに注意せよ.

1.3.1 enhanced を使った図の作り方

手順 1

次のようなスクリプトファイル (example1.gp とする) を用意する. スクリプトファイル中で, 行頭が # の行はコメント行となる.

example1.gp

```
set terminal eps color enhanced
set output "example1.eps"
set xlabel "{/Symbol a}"
set ylabel "{/Symbol b}"
set label 1 '{(a)}' at -2.8,0.9
set key at 3, -0.5
plot [-pi:pi][-1.1:1.1]
 sin(x) title "{{/Symbol g}=1}",
 sin(x)**2 title "{{/Symbol g}=2}",
 sin(x)**3 title "{{/Symbol g}=3}"
#上の 4 行は実際には 1 行で書く
quit
```

enhanced を使うことによって, {/Symbol a} でギリシャ文字 α を出力できる. (他も試してみよ) label の (a) で at の後ろは出力する座標なので適切に指定する.

手順 2

ターミナル上で gnuplot に読み込ませる.

```
gnuplot example1.gp
```

これで図 1.1(a) に示すような example1.eps というファイルができる. example1.eps という名前は 2 行目の set output で指定されており, example1.gp と example1.eps という名前の一致は必要ではなくバラバラでも構わない.

1.3.2 LaTeX 数式を使った図の作り方

図中の数式をより美しく仕上げる方法として, gnuplot の epslatex を使った方法を紹介する. (") と (') の違いに注意せよ. 事前に以下の include.tex を用意しておく.

include.tex

```
\documentclass{article}
\usepackage{graphicx,color}
\pagestyle{empty}
\begin{document}
\begin{figure}[h]
\input{./temp.tex}
\end{figure}
\end{document}
```

include.tex に書かれている temp.tex は, 次の手順 1 におけるスクリプトファイル example2.gp 内に現れる temp.eps と関連している. temp という名前は変更してもよいが, include.tex と example2.gp のセットでの変更が必要である. 最終的な EPS ファイルの名前は手順 4 で選べる.

手順 1

gnuplot のスクリプトファイルを準備する.

example2.gp

```
set terminal epslatex color
set output "temp.eps"
set xlabel '{\Large $\alpha$}'
set ylabel '{\Large $\beta$}'
set label 1 '{\Large (b)}' at -2.8,0.9
set key at 3, -0.5
plot [-pi:pi][-1.1:1.1]
 sin(x) title "{{/Symbol g}=1}",
 sin(x)**2 title "{{/Symbol g}=2}",
 sin(x)**3 title "{{/Symbol g}=3}"
 lt 5 lc rgb "blue"
#上の 5 行は実際には 1 行で書く
quit
```

$\gamma = 3$ のグラフは, 線種 (lt 5) と色 (lc rgb "blue") を指定している. 必要に応じて他のグラフも同様に指定できる. 点種や線種の番号は gnuplot のバージョンに応じて異なる可能性があるので, 実際に作成して確認すること. gnuplot のプロンプトで "test" とタイプすると情報が出る.

手順 2

ターミナル上で gnuplot に example2.gp を読み込ませる.

```
gnuplot example2.gp
```

これで temp.eps と temp.tex ができる.

手順 3

ターミナル上で include.tex をコンパイルする.

```
latex include.tex
```

手順 4

ターミナル上で次の一連のコマンドを実行する.

```
dvips -E include.dvi
epstopdf --outfile=include.pdf include.ps
pdftops -eps include.pdf example2.eps
```

最後の行の example2.eps は, 最終的に得られる eps ファイルの名前指定であり, 自由に変えられる. 図 1.1(b) は以上の手順で作成した図である.

手順 3 でコンパイルに失敗するときには

データファイルを使ってプロットしている場合, データファイルの名前に data_1.dat の "_" など LaTeX で使うコマンドが含まれているとコンパイルができない場合がある. この問題の解決方法として次の 3 つが考えられる.

1. 該当するファイルの名前を変える.
   data_1.dat → data1.dat
2. ファイル名が表示されないように gnuplot のスクリプトの中で unset key を使う.
3. gnuplot スクリプトの plot コマンドで title 'This is the title' のように title を設定することでデータファイル名が表示されるのを避ける. 当然, 意味のあるタイトルにすべきである.

さらに便利に

上の手順 2,3,4 をシェルスクリプトに書いておくと一括処理ができて便利である. 意欲がある人は挑戦してみられたい.

1.3.3 データファイルのプロット法

上の例では関数 sin(x) などを直接プロットしていたが, データファイル data.dat に格納された点をプロットする場合は,

```
plot sin(x)
```

の代わりに

```
plot "data.dat"
```

とする. なお data.dat は第 1 列に x 軸の値, 第 2 列に y 軸の値が来るように作成する:

```
x1 y1
x2 y2
x3 y3
 :  :
```

データファイルに第 3 列以降があってもよい. 第 2 列を横軸の値, 第 3 列を縦軸の値に使いたい場合は

```
plot "data.dat" using 2:3
```

とする. さらに, グラフを左に 1 だけずらしたい場合には

```
plot "data.dat" using ($2-1):3
```

なども使える.

1.3.4 Activity 3

I. $\alpha \in [0.01, 1]$ の範囲において,

- $\beta = 2\alpha$
- $\beta = \exp(\alpha)$
- $\beta = \alpha^2$

のグラフを同時に描いた図を作成せよ. それぞれが, 線形関数, 指数関数, 冪関数であることが一見してわかるようにプロットの方法を工夫して 3 枚作成せよ. (a)(b)(c) で区別を付け, それぞれ軸の名前も書くこと.

ヒント 1: グラフのうち一見してそれとわかるのは直線である. よって各関数のグラフが直線になるようにプロットを工夫すればよい. 例えば $\beta = \exp(\alpha)$ では横軸 α, 縦軸 β では直線にならないが, 縦軸を $\log\beta$ にするとどうだろう? gnuplot で縦軸を対数スケールにするには

```
set log y
```

と打てば良い. ただし, 軸の目盛りと整合するように軸の名前を付けること. (軸を対数スケールにしたら軸名は $\log\beta$ なのか, β のままなのか?)

ヒント 2: 軸の目盛りが 1000000 などでは見にくいので, 10^n のうち n だけを出力したい場合がある. y 軸でこれを行うには, gnuplot のスクリプトファイル中に

```
set format y "%T"
```

と指定すればよい. もちろん y 軸の名前はこれに合わせて適切に選ばなければならない.

II. 図ができたら LaTeX のソースに figure 環境で取り込んでみよ. caption には図の説明 (軸をどうしたか, 傾きがどうか, 等) を必要十分に入れよ. 本文 (figure 環境ではない所) には, プロットをどのように工夫したかの説明を書け.

[山口義幸]

参考文献

[1] 木下是雄, 理科系の作文技術 (中公新書, 1981).

[2] 京都大学工学部電気系教室編, 電気電子工学基礎実習.

第2章　数値線形代数

2.1　目的

数値線形代数の主要テーマである連立1次方程式や行列固有値問題は，科学技術計算から日常生活まで，ありとあらゆるところに現れる基本問題である．例えば（他テーマで扱うであろう）物理現象の数値シミュレーションでは，コアの部分は連立1次方程式の計算であったりする．また大規模データ解析の場面では，しばしば巨大なデータ行列の固有値･固有ベクトルの計算が必要になる．

本章では，そうした数値線形代数の基礎的な知識と技術を身につけることを目的として，連立1次方程式

$$A\boldsymbol{x} = \boldsymbol{b} \tag{2.1}$$

および行列固有値問題

$$A\boldsymbol{x} = \lambda\boldsymbol{x} \tag{2.2}$$

の数値解法を実装し，数値実験によりその性能を調べる．

連立1次方程式 (2.1) の解は（係数行列 A が正則ならば）$\boldsymbol{x} = A^{-1}\boldsymbol{b}$ と表せる．しかし逆行列を計算して解を求める方法は誤差に弱く，計算量にも利点はないため, 用いられることは滅多にない．Cramer の公式を用いる方法も同様である．

実用的な連立1次方程式の数値解法は，直接法と反復法の2つに大分類される．直接法では有限回の四則演算で（演算が無誤差で行えるならば真の）解を求める．反復法（Jacobi 法，Gauss-Seidel 法，SOR 法，共役勾配法など）では解に収束する近似解の列を逐次生成する．本章では直接法の代表格である消去法と，行列の LU 分解による解法を扱う．

行列 A の固有値は特性多項式 $\varphi(z) := \det(zI - A)$ の根として特徴付けられる．しかし固有値を数値的に求めるために，元の問題の線形構造を無視して，代数方程式 $\varphi(z) = 0$ を直接解くようなことはしない．

実用的な行列固有値問題の解法の多くは，真の固有値･固有ベクトルに収束する近似列をつくる反復法である．そこでは対象とする行列の構造など様々な要請に応じて，多種多様な解法（Jacobi 法，二分法，分割統治法，LR 法，dqds 法など）が開発されている．本章では古典的な解法であるべき乗法と，最も広く利用されている（であろう）QR 法を主に扱う．

数値線形代数を詳しく扱った書籍としては [1, 2] などがある．また線形計算のためのソフトウェアライブラリも数多く公開されている．LAPACK [3] などオープンソースのものもあるので興味のある人は参考にしてほしい．

注意 2.1.1 本資料の作成時に用いた実験環境は次の通りである．

OS	macOS Catalina 10.15.4
CPU	Intel Core i7-8569U 2.8GHz
Memory	16 GB
Compiler	Clang version 11.0.3

2.2　準備

2.2.1　ベクトルと行列のノルム

ベクトルの大きさはノルムで測る.（複素）ベクトル $\boldsymbol{x} = (x_i) \in \mathbb{C}^n$ および実数 $p \geq 1$ に対して

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}}$$

を $\boldsymbol{x}$ の p **ノルム**という．特に2ノルム $\|\boldsymbol{x}\|_2$ はユークリッドノルムとも呼ばれる．また

$$\|\boldsymbol{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

を ∞ **ノルム**という．数値計算では高速に計算できる $\|\boldsymbol{x}\|_\infty$ が採用されることが多い．

どのノルムでも $\|\boldsymbol{x}\| \geq 0$ である．ただし $\|\boldsymbol{x}\| = 0$ となるのは $\boldsymbol{x} = \boldsymbol{0}$（零ベクトル）の場合に限る．またスカラー倍に関して $\|c\boldsymbol{x}\| = |c|\|\boldsymbol{x}\|$ である．

注意 2.2.1 このようにノルムによらない議論をする場合や，どのノルムを用いてもよい場面では，任意のノルムを表すために記号 $\|\boldsymbol{x}\|$ を用いることにする．特定のノルムを用いる場合は $\|\boldsymbol{x}\|_2$ などノルムの種類を明示する．

行列の大きさを測るためのノルムもある．（複素）行列 $A = (a_{i,j}) \in \mathbb{C}^{n\times n}$ に対して，理論上重要なのは（ベクトルのノルムを用いて定義される）作用素ノルム

$$\|A\| = \max_{\boldsymbol{x}\neq\boldsymbol{0}} \frac{\|A\boldsymbol{x}\|}{\|\boldsymbol{x}\|} = \max_{\|\boldsymbol{x}\|=1} \|A\boldsymbol{x}\|$$

である．実際の数値計算では値を簡単に求められる**最大値ノルム**

$$\|A\|_{\max} = \max_{1\leq i,j\leq n} |a_{i,j}|$$

などが重用される．

ベクトルの場合と同様にどの行列ノルムでも $\|A\| \geq 0$ である．ただし $\|A\| = 0$ となるのは $A = O$（零行列）の場合に限る．またスカラー倍に関して $\|cA\| = |c|\|A\|$ である．

2.2.2 内積

ベクトル $\boldsymbol{x} = (x_i),\ \boldsymbol{y} = (y_i) \in \mathbb{C}^n$ に対して

$$(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^*\boldsymbol{y} = \sum_{i=1}^{n} \overline{x_i} y_i$$

を**（エルミート）内積**という．ただし（ベクトルを含む）行列 A に対して $A^* = (\overline{A})^{\mathsf{T}}$（共役転置）である．特に 2 ノルムに関して

$$(\boldsymbol{x}, \boldsymbol{x}) = \|\boldsymbol{x}\|_2^2$$

が成り立つ．

実ベクトルの場合

$$(\boldsymbol{x}, \boldsymbol{y}) = 0 \tag{2.3}$$

ならばベクトル $\boldsymbol{x}, \boldsymbol{y}$ は直交する．（つまり $\boldsymbol{x}, \boldsymbol{y}$ のなす角が $\pi/2$ になる．）複素ベクトルの場合にも (2.3) が成り立つとき（形式的に） $\boldsymbol{x}, \boldsymbol{y}$ は**直交する**という．

2.3 連立 1 次方程式の数値解法

行列 $A = (a_{i,j}) \in \mathbb{C}^{n\times n}$ およびベクトル $\boldsymbol{b} = (b_i) \in \mathbb{C}^n$ に対して，連立 1 次方程式 (2.1) の解 $\boldsymbol{x} \in \mathbb{C}^n$ を数値的に求めることを考える．ただし A は正則であるとする．

方程式 (2.1) の数値解 $\tilde{\boldsymbol{x}} \in \mathbb{C}^n$ の精度は，真の解 $\boldsymbol{x}$ との相対誤差

$$\frac{\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|}{\|\boldsymbol{x}\|} \tag{2.4}$$

で測る．

注意 2.3.1 方程式 (2.1) の解法を実装したプログラムに誤りがないか確認するには，数値解 $\tilde{\boldsymbol{x}}$ の残差ノルム

$$\|\boldsymbol{b} - A\tilde{\boldsymbol{x}}\| \tag{2.5}$$

を調べるのが便利である．この値は $\tilde{\boldsymbol{x}}$ が真の解であれば 0 になる．このため極端に大きな値になる場合はプログラムに誤りがある可能性が高い．

2.3.1 消去法

（*Gauss* の）**消去法**では連立 1 次方程式 (2.1) の解を次のように求める．

消去法

次の手続きでベクトル $\boldsymbol{x}$ を求める．

(1) 拡大係数行列 $(A, \boldsymbol{b})$ を行基本変形により行階段形にする：

$$(A', \boldsymbol{b}') = \begin{pmatrix} a'_{1,1} & a'_{1,2} & \cdots & a'_{1,n} & b'_1 \\ 0 & a'_{2,1} & \cdots & a'_{2,n} & b'_2 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \cdots & 0 & a'_{n,n} & b'_n \end{pmatrix}.$$

(2) 方程式 $A'\boldsymbol{x} = \boldsymbol{b}'$ の解 $\boldsymbol{x}$ を漸化式

$$x_i = \frac{1}{a'_{i,i}} \left(b'_i - \sum_{j=i+1}^{n} a'_{i,j} x_j \right)$$

$(i = n, n-1, \ldots, 1)$ により求める．

消去法において方程式 $A'\boldsymbol{x} = \boldsymbol{b}'$ は元の方程式 (2.1) と同値であることに注意する．(1) で $(A, \boldsymbol{b})$ を行階段形にするには，例えば次のようにすればよい．

前進消去

次の手続きを $k = 1, 2, \ldots, n-1$ の順に繰り返して拡大係数行列 $(A, \boldsymbol{b})$ を逐次更新する．

(1) 第 k 列の下三角成分 $a_{k,k}, \ldots, a_{n,k}$ の中から $a_{i,k} \neq 0$ をひとつ選ぶ．（この $a_{i,k}$ を**ピボット**という．）

(2) $i \neq k$ ならば第 k 行と第 i 行を入れ替える．

(3) （$a_{k+1,k}, \ldots, a_{n,k}$ がすべて 0 になるように）各 $k+1 \leq i \leq n$ に対して第 i 行から第 k 行の $\frac{a_{i,k}}{a_{k,k}}$ 倍を引く．

注意 2.3.2 (1) でピボット $a_{i,k}$ として 0 に近いものを選ぶと，後の演算で桁落ちが起こったり，情報落ちのため精度が悪化したりする可能性がある．このため通常は絶対値最大の $a_{i,k}$ をピボットとして選ぶ．

単精度（`float`）で実装した消去法により，連立 1 次方程式 (2.1) の解を求める数値実験を行った．100 次の行列 A とベクトル $\boldsymbol{b}$ のペアをランダムに 100 個生成し，各ペアに対して連立 1 次方程式 (2.1) の数値解を求めた．計算結果をまとめたものを図 2.1–2.3 に示す．

図 2.1 のグラフは，消去法で求めた数値解 $\tilde{\boldsymbol{x}}$ の残差ノルム (2.5) を，各ペア $A, \boldsymbol{b}$ に対して（対数）プロットしたものである．残差ノルムは概ね小さいので，実装したプログラムは正しく動作していると思われる．

図 2.2 のグラフは，真の解との相対誤差 (2.4) を同様にプロットしたものである．（点線はその平均値である．）ただし不明な真の解の代わりに，倍精度（`double`）の消去法で求めた数値解 $\boldsymbol{x}$ を用いている．相対誤差はおおよそ 10^{-6}〜10^{-5} 程度なので，求めた数値解には 10 進 5 桁程度の精度があると考えられる．

図 2.3 のグラフは，数値解を求めるのに要した

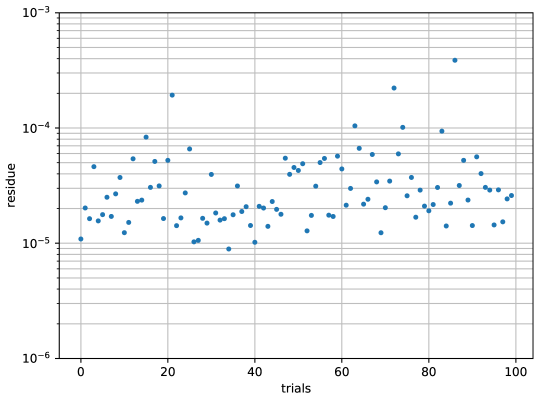


図 2.1: 消去法による残差ノルム．

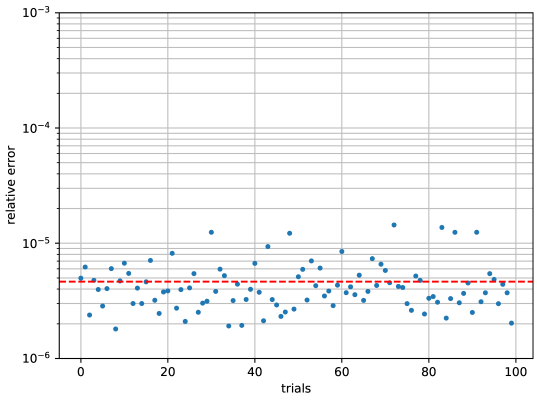


図 2.2: 消去法による相対誤差．（点線は平均値．）

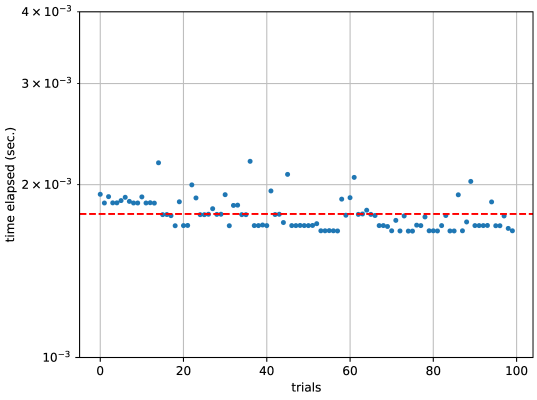


図 2.3: 消去法による計算時間．（点線は平均値．）

計算時間を同様にプロットしたものである．（点線はその平均値である．）少しばらつきはあるが 2×10^{-3} 秒弱の辺りに固まっている．消去法は直接法であり，解を求めるまでの四則演算の回数は次数 n のみで決まる．このため計算時間が試行によらずほぼ一定になっていると考えられる．

2.3.2 行列の LU 分解

行列 A を単位下三角行列 L と上三角行列 U を用いて

$$A = LU$$

の形に表すことを LU **分解**という．ただし単位下三角行列とは対角成分がすべて 1 の下三角行列のことである．

行列 A が正則ならば, $1,\dots,n$ の置換 π が存在して,

$$P_\pi A = LU$$

の形に LU 分解できる．ただし $P_\pi = (\delta_{i,\pi(i)})$（置換行列）である．この事実は消去法と同じ考え方により示される．

消去法では前進消去の手続き (1)–(3) を繰り返して A を上三角行列 $A'(=U)$ にする．この操作を行列積で表すと

$$L_{n-1}P_{\tau_{n-1}}\cdots L_1P_{\tau_1}A = U \qquad (2.6)$$

である．ただし前進消去において k 回目の (1) で選んだピボットを $a_{i_k,k}$ とおくとき

$$\tau_k = (k\ i_k)$$

（k と i_k を入れ替える互換）である．ただし $i_k = k$ ならば τ_k は恒等置換である．また k 回目の (2) を終えた直後の $A=(a_{i,j})$ に対して

$$L_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\frac{a_{k+1,k}}{a_{k,k}} & 1 & \\ & & \vdots & & \ddots \\ & & -\frac{a_{n,k}}{a_{k,k}} & & 1 \end{pmatrix}$$

である．

いま，$i_k \ge k$ より，置換

$$\pi_k = \tau_{n-1}\cdots\tau_k$$

は $k,\dots,n$ のみに作用する．これより行列

$$L'_k = P_{\pi_{k+1}}L_kP_{\pi_{k+1}}^{\mathsf{T}}$$

は（L_k と同じ形の）下三角行列である．このとき (2.6) より $(L'_{n-1}\cdots L'_1)P_{\pi_1}A = U$ である．したがって

$$L = (L'_1)^{-1}\cdots(L'_{n-1})^{-1}$$

とおくと

$$P_{\pi_1}A = LU \qquad (2.7)$$

である．L は単位下三角行列なので (2.7) は行列 $P_{\pi_1}A$ の LU 分解である．

以上の議論から LU 分解の求め方が分かる．

LU 分解

最初 π を恒等置換とする．次の手続きを $k=1,2,\dots,n-1$ の順に繰り返して，行列 A,L および置換 π を逐次更新する．

(1) A の第 k 列の下三角成分 $a_{k,k},\dots,a_{n,k}$ の中からピボット $a_{i,k}\neq0$ をひとつ選ぶ．

(2) $i\neq k$ ならば A,L の第 k 行と第 i 行を入れ替える．また π に左から互換 $(k\ i)$ を掛ける．

(3) 各 $k+1\le i\le n$ に対して A の第 i 行から第 k 行の $a_{i,k}/a_{k,k}$ 倍を引く．

(4) L の第 k 列を

$$\begin{pmatrix} \mathbf{0}_{k-1} \\ 1 \\ \frac{a_{k+1,k}}{a_{k,k}} \\ \vdots \\ \frac{a_{n,k}}{a_{k,k}} \end{pmatrix}$$

とする．ただし $\mathbf{0}_d$ は次数 d の零ベクトルである．

最後の A を U とする．（このとき最初の A に対して $P_\pi A = LU$ である．）

2.3.3 LU 分解による解法

LU 分解は連立 1 次方程式 (2.1) の解を求めるのに応用できる．$P_\pi A = LU$ のとき (2.1) は連立方程式

$$L\boldsymbol{y} = P_\pi \boldsymbol{b}, \qquad U\boldsymbol{x} = \boldsymbol{y}$$

と同値である．

LU 分解による解法

次の手続きでベクトル $\boldsymbol{x}$ を求める．

(1) LU 分解により $P_\pi A = LU$ を満たす単位下三角行列 $L = (\ell_{i,j})$ と上三角行列 $U = (u_{i,j})$ および置換 π を求める．

(2) 方程式 $L\boldsymbol{y} = P_\pi \boldsymbol{b}$ の解 $\boldsymbol{y} = (y_i)$ を漸化式

$$y_i = b_{\pi(i)} - \sum_{j=1}^{i-1} \ell_{i,j} y_j$$

（$i = 1, 2, \ldots, n$）により求める．

(3) 方程式 $U\boldsymbol{x} = \boldsymbol{y}$ の解 $\boldsymbol{x} = (x_i)$ を漸化式

$$x_i = \frac{1}{u_{i,i}} \left(y_i - \sum_{j=i+1}^{n} u_{i,j} x_j \right)$$

（$i = n, n-1, \ldots, 1$）により求める．

2.4 行列固有値問題の数値解法

行列 $A = (a_{i,j}) \in \mathbb{C}^{n \times n}$ に対して方程式 (2.2) を満たすスカラー $\lambda \in \mathbb{C}$ と非零ベクトル $\boldsymbol{x} = (x_i) \in \mathbb{C}^n$ があるとき，λ を A の**固有値**といい，$\boldsymbol{x}$ を（固有値 λ に対する）**固有ベクトル**という．A の固有値と固有ベクトルを数値的に求めることを考える．

A の固有値を $\lambda_1, \ldots, \lambda_n$ とおく．A が正則でない場合や，絶対値が同じ固有値が複数ある場合は議論が煩雑になるため，ここでは

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > 0 \tag{2.8}$$

を仮定する．特に λ_i を**第 i 固有値**といい，λ_i に対する固有ベクトルを**第 i 固有ベクトル**という．

各 $1 \le i \le n$ に対して $\boldsymbol{v}_i$ を第 i 固有ベクトルとする．固有値を並べた対角行列

$$\Lambda = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$$

および固有ベクトルを並べた行列

$$V = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n)$$

に対して

$$AV = V\Lambda \tag{2.9}$$

である．（これを A の対角化という．）

数値的に求めた第 i 固有値 $\tilde{\lambda}_i$ の精度は，真値 λ_i との相対誤差

$$\frac{|\lambda_i - \tilde{\lambda}_i|}{|\lambda_i|} \tag{2.10}$$

で測る．

数値的に求めた第 i 固有ベクトル $\tilde{\boldsymbol{v}}_i$ の精度は，真の第 i 固有ベクトル $\boldsymbol{v}_i$ との相対誤差

$$\frac{\|\boldsymbol{v}_i - \tilde{\boldsymbol{v}}_i\|}{\|\boldsymbol{v}_i\|} \tag{2.11}$$

で測る．ただし固有ベクトルにはスカラー倍の自由度があるので，次の 2 条件を満たすように $\boldsymbol{v}_i, \tilde{\boldsymbol{v}}_i$ を正規化してから (2.11) を求める．

(1) $\|\boldsymbol{v}_i\| = \|\tilde{\boldsymbol{v}}_i\|$.

(2) ある $1 \le k \le n$ に対して $\boldsymbol{v}_i, \tilde{\boldsymbol{v}}_i$ の第 k 成分はともに正実数である．

注意 2.4.1 行列固有値問題の解法を実装したプログラムに誤りがないか確認するには，数値的に求めた第 i 固有値 $\tilde{\lambda}_i$ および第 i 固有ベクトル $\tilde{\boldsymbol{v}}_i$ に対して，固有方程式 (2.2) の残差ノルム

$$\|\tilde{\lambda}_i \tilde{\boldsymbol{v}}_i - A\tilde{\boldsymbol{v}}_i\| \tag{2.12}$$

を調べるのが便利である．この値は真の固有値・固有ベクトルであれば 0 になる．このため極端に大きな値になる場合はプログラムに誤りがある可能性が高い．

2.4.1 べき乗法

べき乗法は第 1 固有値 λ_1 を求めるための算法であり，第 1 固有ベクトルも同時に求めることができる．べき乗法の考え方は次の観察に基づいている．

> ベクトル $\boldsymbol{x}$ に繰り返し A を掛ける：
>
> $$\boldsymbol{x},\ A\boldsymbol{x},\ A^2\boldsymbol{x},\ A^3\boldsymbol{x},\ \ldots.$$
>
> このとき $A^k\boldsymbol{x}$ は第 1 固有ベクトルに近付いていく．

この現象が起こる仕組みを見てみよう．任意のベクトル $\boldsymbol{x} \in \mathbb{C}^n$ に対して $\boldsymbol{x} = V\boldsymbol{c}$ によりベクトル $\boldsymbol{c} = (c_i) \in \mathbb{C}^n$ を定める．このとき (2.9) より

$$A^k\boldsymbol{x} = V\Lambda^k\boldsymbol{c} = \lambda_1^k\left\{c_1\boldsymbol{v}_1 + \sum_{i=2}^{n} c_i\left(\frac{\lambda_i}{\lambda_1}\right)^k \boldsymbol{v}_i\right\}$$

である．(2.8) より $\lim_{k\to\infty}(\lambda_i/\lambda_1)^k = 0$ $(i \neq 1)$ である．したがって $c_1 \neq 0$ ならば $A^k\boldsymbol{x}$ は $k \to \infty$ のとき $\boldsymbol{v}_1$ 成分が支配的になる．

$A^k\boldsymbol{x}$ を適当に正規化すれば，$\boldsymbol{v}_1$ 成分を残しつつ，$\boldsymbol{v}_1$ 以外の成分を $\boldsymbol{0}$ に収束させることができる．これがべき乗法の原理である．

> **べき乗法**
>
> 適当な初期ベクトル $\boldsymbol{x}_0$ から次の手続きを $k = 0, 1, 2, \ldots$ の順に反復してベクトル $\boldsymbol{x}_k = (x_{k;i})_i$ およびスカラー μ_k を求める．
>
> (1) $\boldsymbol{y}_k = A\boldsymbol{x}_k$ とする．
>
> (2) $\boldsymbol{x}_k$ の成分の中から $x_{k;i} \neq 0$ をひとつ（例えば絶対値最大のものを）選び $\mu_k = \dfrac{y_{k;i}}{x_{k;i}}$ とする．ただし $\boldsymbol{y}_k = (y_{k;i})_i$ である．
>
> (3) 数列 $\{\mu_k\}$ が十分収束していれば反復を終了する．
>
> (4) $\boldsymbol{x}_{k+1} = \dfrac{\boldsymbol{y}_k}{\|\boldsymbol{y}_k\|}$ とする．
>
> 反復終了時の μ_k を第 1 固有値 λ_1 として採用する．必要ならば $\boldsymbol{x}_k$ を第 1 固有ベクトルとして採用する．

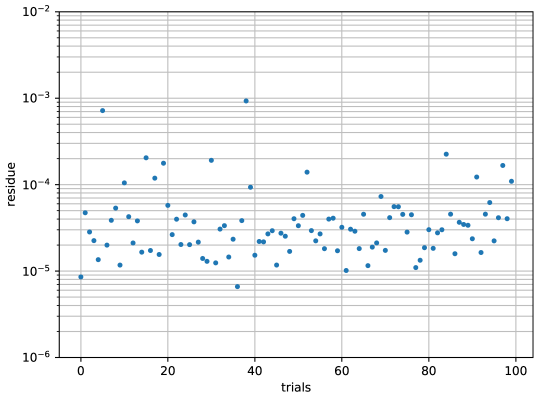


図 2.4: べき乗法による残差ノルム．

べき乗法では理論上

$$\lim_{k\to\infty} \mu_k = \lambda_1$$

である．しかし実際の計算では真の収束は期待できないため，(3) で近似的な収束判定を行っている．具体的には例えば，予め定めた閾値 $\varepsilon > 0$ に対して，μ_k の相対的な変化量

$$\frac{|\mu_k - \mu_{k-1}|}{|\mu_k|}$$

が ε より小さくなった時点で十分収束したと判定する．

べき乗法の収束の速さは $\lim_{k\to\infty}(\lambda_2/\lambda_1)^k = 0$ の収束の速さと同じである．つまり収束率 $|\lambda_2|/|\lambda_1|$ の 1 次収束である．これより第 1 固有値と第 2 固有値の絶対値の比 $|\lambda_2|/|\lambda_1|$ が小さいほど収束は速く，1 に近いほど遅い．

単精度（`float`）で実装したべき乗法により，行列 A の第 1 固有値と第 1 固有ベクトルを求める数値実験を行った．100 次行列 A をランダムに 100 個生成し，各 A に対して第 1 固有値と第 1 固有ベクトルを数値的に求めた．ただし収束判定の閾値は $\varepsilon = 1.0 \times 10^{-6}$ とした．計算結果をまとめたものを図 2.4–2.6 に示す．

図 2.4 のグラフは，べき乗法で数値的に求めた第 1 固有値 $\tilde{\lambda}_1$ と第 1 固有ベクトル $\tilde{\boldsymbol{v}}_1$ に対する固有方程式の残差ノルム (2.12) を，各 A に対して（対数）プロットしたものである．残差ノルムは概ね小さいので，実装したプログラムは正しく動作していると思われる．

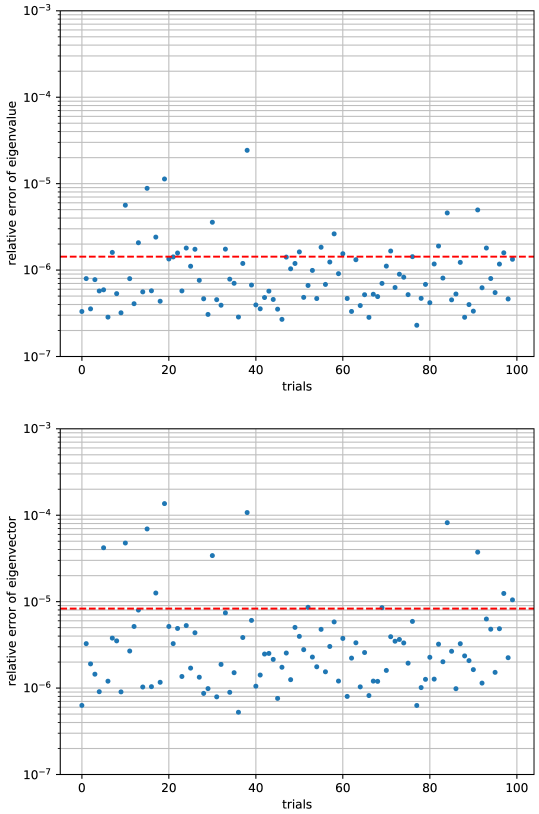


図 2.5: べき乗法による第 1 固有値の相対誤差（上）と第 1 固有ベクトルの相対誤差（下）.（点線は平均値.）

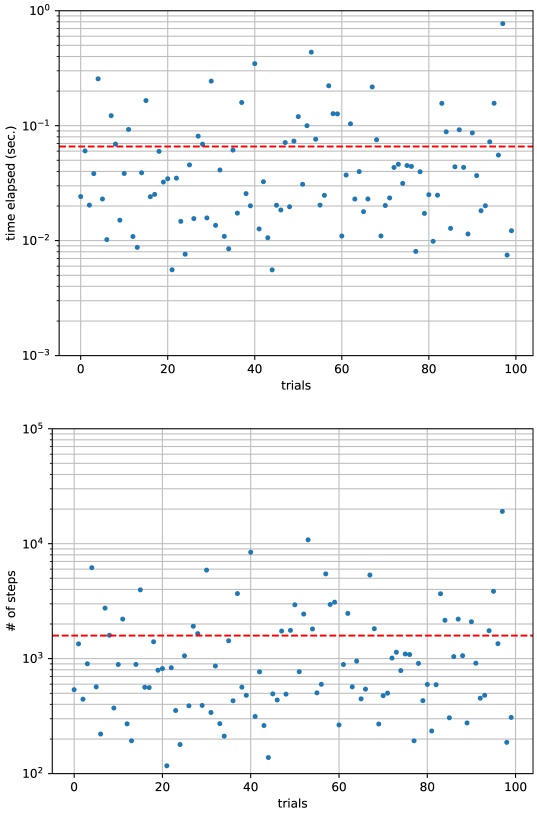


図 2.6: べき乗法による計算時間（上）と反復回数（下）.（点線は平均値.）

図 2.5 の 2 つのグラフは，$\tilde{\lambda}_1$ の相対誤差 (2.10) と $\tilde{\boldsymbol{v}}_1$ の相対誤差 (2.11) を同様にプロットしたものである．(点線はそれぞれの平均値である．) ただし不明な真の固有値・固有ベクトルの代わりに，倍精度のべき乗法で数値的に求めた第 1 固有値 λ_1 と第 1 固有ベクトル $\boldsymbol{v}_1$ を用いている．

$\tilde{\lambda}_1$ の相対誤差は，極端に大きいものを除けば 10^{-6} 程度である．これより数値的に求めた第 1 固有値には，概ね 10 進 6 桁程度の精度があると考えられる．(収束判定の閾値を 10^{-6} と設定したのだから，これは自然な結果である．) $\tilde{\boldsymbol{v}}_1$ の相対誤差は，極端に大きいものを除けば 10^{-6}〜10^{-5} 程度である．これより数値的に求めた第 1 固有ベクトルには，概ね 10 進 5 桁程度の精度があると考えられる．

図 2.6 の 2 つのグラフは，収束までに要した計算時間と反復回数を同様にプロットしたものである．(点線はそれぞれの平均値である．) 2 つのグラフはほとんど同じ形をしている．これは計算時間が反復回数に比例するためである．(べき乗法の 1 ステップに含まれる四則演算の回数は次数 n のみで決まる．)

計算時間は行列 A により大きくばらついている．数 100 回の反復ですぐに収束する例もあれば，収束までに 1 万回以上の反復を要した例もある．これは A の固有値の配置の違いにより，べき乗法の収束率 $|\lambda_2|/|\lambda_1|$ が変化したためと考えられる．

2.4.2 逆反復法

べき乗法の考え方は，実は第 1 固有値・固有ベクトル以外にも適用することができる．スカラー $\sigma \in \mathbb{C}$ は A の固有値でないとする．このとき行列 $(A-\sigma I)^{-1}$ の固有値は

$$\frac{1}{\lambda_1-\sigma}, \quad \cdots, \quad \frac{1}{\lambda_n-\sigma}$$

である．この中で絶対値最大 (つまり $(A-\sigma I)^{-1}$ の第 1 固有値) を実現するのは σ に最も近い λ_i である．これより $(A-\sigma I)^{-1}$ に対してべき乗法を適用すると，A の固有値の中で σ に最も近い λ_i が $\frac{1}{\lambda_i-\sigma}$ の形で求まる．この原理を用いたのが**逆反復法**である．

逆反復法

適当な初期ベクトル $\boldsymbol{x}_0$ から次の手続きを $k = 0, 1, 2, \ldots$ の順に反復してベクトル $\boldsymbol{x}_k = (x_{k;i})_i$ およびスカラー μ_k を求める．

(1) 連立 1 次方程式

$$(A-\sigma I)\boldsymbol{y}_k = \boldsymbol{x}_k \tag{2.13}$$

の解 $\boldsymbol{y}_k$ を求める．

(2) $\boldsymbol{x}_k$ の成分の中から $x_{k;i} \neq 0$ をひとつ (例えば絶対値最大のものを) 選び $\mu_k = \dfrac{y_{k;i}}{x_{k;i}}$ とする．ただし $\boldsymbol{y}_k = (y_{k;i})_i$ である．

(3) 数列 $\{\mu_k\}$ が十分収束していれば反復を終了する．

(4) $\boldsymbol{x}_{k+1} = \dfrac{\boldsymbol{y}_k}{\|\boldsymbol{y}_k\|}$ とする．

反復終了時の $\frac{1}{\mu_k} + \sigma$ を σ に最も近い固有値 λ_i として採用する．必要ならば $\boldsymbol{x}_k$ を λ_i に対する固有ベクトルとして採用する．

逆反復法では，べき乗法での (1) に相当する

$$\boldsymbol{y}_k = (A-\sigma I)^{-1}\boldsymbol{x}_{k-1}$$

を計算する代わりに同値な連立 1 次方程式 (2.13) を解く．その計算には LU 分解を用いるのが効率的である．(係数行列 $A-\sigma I$ は不変なので，LU 分解の計算は最初の 1 回だけでよい！) なお逆反復法の収束判定はべき乗法と同様である．

逆反復法の収束の速さは収束率 $|\lambda_i-\sigma|/|\lambda_j-\sigma|$ の 1 次収束である．ただし λ_i, λ_j はそれぞれ σ に最も近い固有値と 2 番目に近い固有値である．特に σ が λ_i に近いほど，収束率が 0 に近付くため収束は速くなると期待される．逆反復法の計算途中で σ をより λ_i に近いものに取り替えたらどうなるか考えてみよ．

2.4.3 行列の QR 分解

行列 A をユニタリ行列 Q と上三角行列 R を用いて

$$A = QR \tag{2.14}$$

の形に表すことを QR **分解**という．ただし Q がユニタリ行列であるとは $Q^*Q = I$ が成り立つことをいう．

QR 分解は Gram-Schmidt （の直交化）法で求めることができる．$A = (\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n)$ の列ベクトルから漸化式

$$\boldsymbol{b}_j = \boldsymbol{a}_j - \sum_{i=1}^{j-1} (\boldsymbol{q}_i, \boldsymbol{a}_j)\boldsymbol{q}_i, \tag{2.15a}$$

$$\boldsymbol{q}_j = \frac{\boldsymbol{b}_j}{\|\boldsymbol{b}_j\|_2} \tag{2.15b}$$

（$j = 1, \ldots, n$）によりベクトル $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n$ をつくる．このとき $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n$ は $(\boldsymbol{q}_i, \boldsymbol{q}_j) = \delta_{i,j}$ を満たすので，行列

$$Q = (\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n)$$

はユニタリ行列である．また (2.15) より $r_{i,j} = (\boldsymbol{q}_i, \boldsymbol{a}_j)$（$i < j$）および $r_{j,j} = \|\boldsymbol{b}_j\|_2$ とおくと，上三角行列

$$R = \begin{pmatrix} r_{1,1} & r_{1,2} & \cdots & r_{1,n} \\ 0 & r_{2,2} & \cdots & r_{2,n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & r_{n,n} \end{pmatrix}$$

は (2.14) を満たす．

Gram-Schmidt 法の漸化式 (2.15) は誤差に弱く，数値計算では行列 Q の直交性が悪くなる（つまり Q^*Q と単位行列 I の差が小さくならない）傾向にある．この弱点に対処したのが**修正** *Gram-Schmidt* **法**である．

修正 *Gram-Schmidt* 法

次の手続きを $j = 1, \ldots, n$ の順に繰り返してベクトル $\boldsymbol{q}_j$（$1 \le j \le n$）とスカラー $r_{i,j}$（$1 \le i \le j \le n$）を求める．

(1) ベクトル $\boldsymbol{b}_{1,j} = \boldsymbol{a}_j$ から漸化式

$$r_{i,j} = (\boldsymbol{q}_i, \boldsymbol{b}_{i,j}),$$
$$\boldsymbol{b}_{i+1,j} = \boldsymbol{b}_{i,j} - r_{i,j}\boldsymbol{q}_i$$

$i = 1, \ldots, j-1$）により $r_{i,j}$（$1 \le i < j$）とベクトル $\boldsymbol{b}_{i,j}$（$1 \le i \le j$）を求める．

(2) $r_{j,j} = \|\boldsymbol{b}_{j,j}\|_2$ および $\boldsymbol{q}_j = \dfrac{\boldsymbol{b}_{j,j}}{r_{j,j}}$ とする．

（このとき $Q = (\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n)$ はユニタリ行列であり，上三角行列

$$R = \begin{pmatrix} r_{1,1} & r_{1,2} & \cdots & r_{1,n} \\ 0 & r_{2,2} & \cdots & r_{2,n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & r_{n,n} \end{pmatrix}$$

とともに $A = QR$ を満たす．）

修正 Gram-Schmidt 法は，数学的には Gram-Schmidt 法と同値である．しかしペア $\boldsymbol{q}_i, \boldsymbol{q}_j$（$i < j$）の直交性の悪さが $\boldsymbol{q}_i$ 以外のペアに影響しないように漸化式が改良されている．

注意 2.4.2 QR 分解を実装したプログラムに誤りがないか確認するには，行列ノルム

$$\|A - QR\|, \qquad \|I - Q^*Q\|$$

を調べるのが便利である．

2.4.4 QR 法

QR 法はすべての固有値を同時に求めることができる算法である．QR 法の考え方は次の観察に基づいている．

同時反復法

適当な初期行列 X_0 から次の手続きを $k = 1, 2, 3, \ldots$ の順に反復して行列 X_k を求める．

(1) $Y_k = AX_k$ とする．

(2) Y_k を QR 分解する：$Y_k = Q_k R_k$.

(3) $X_{k+1} = Q_k$ とする．

このとき行列 $V^{-1}X_k$ は $k \to \infty$ のとき上三角行列に近付く．（つまり X_k の第 j 列ベクトルは $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j$ のみの 1 次結合で表せるようになっていく．）

この現象が起こる仕組みは，実はべき乗法の原理とよく似ている.（詳細は [1] を見よ.）特に X_k の第 1 列のふるまいはべき乗法のベクトル $\boldsymbol{x}_k$ と全く同じである.

QR 法

初期行列 $A_0 = A$ から次の手続きを $k = 0, 1, 2, \ldots$ の順に反復して行列 A_k を求める.

(1) A_k の狭義下三角部分が十分 0 に収束していれば反復を終了する.

(2) A_k を QR 分解する：$A_k = Q_k R_k$.

(3) $A_{k+1} = R_k Q_k$ とする.

反復終了時の A_k の対角成分を，上から順に固有値 $\lambda_1, \ldots, \lambda_n$ として採用する.

QR 法の反復は

$$A_{k+1} = Q_k^* A_k Q_k \tag{2.16}$$

と表せるので，数学的には（ユニタリ行列による）相似変形の繰り返しである．特に各 A_k は初期行列 $A_0 = A$ と同じ固有値を持つ．したがって A_k が $k \to \infty$ のとき上三角行列に近付けば，その対角成分は A の固有値に収束する.（三角行列の固有値はその対角成分である.）

A_k が上三角行列に近付く仕組みを見る．QR 法において

$$X_k = Q_0 Q_1 \cdots Q_{k-1}, \qquad Y_k = A X_k$$

とおくと，X_k はユニタリ行列であり

$$Y_k = X_{k+1} R_k$$

が成り立つ．これより QR 法は同時反復法を初期行列 $X_0 = I$ から実行するのと同値である．一方 (2.16) より

$$A_k = X_k^* A X_k$$

である．さらに (2.9) より

$$A_k = (V^{-1} X_k)^{-1} \Lambda V^{-1} X_k$$

である．X_k は同時反復法に従うので $V^{-1} X_k$（および逆行列 $(V^{-1} X_k)^{-1}$）は $k \to \infty$ のとき上三角行列に近付く．したがって A_k も上三角行列に近付く．以上が QR 法の原理である.

QR 法では $A_k = (a_{k;i,j})_{i,j}$ の全ての狭義下三角成分 $a_{k;i,j}$（$i > j$）が十分 0 に収束した時点で反復を終了する．具体的には例えば，予め定めた閾値 $\varepsilon > 0$ に対して，同じ行の対角成分と比較した

$$\frac{|a_{k;i,j}|}{|a_{k;i,i}|}$$

が ε より小さくなった時点で，第 (i, j) 成分は十分 0 に収束したと見なす.

QR 法では A_k の狭義下三角成分 $a_{k;i,j}$ は，収束率

$$\rho_{i,j} = \frac{|\lambda_i|}{|\lambda_j|}$$

で（0 に）1 次収束する．これより QR 法の収束の速さは，収束率

$$\max_{1 \le i < n} \frac{|\lambda_{i+1}|}{|\lambda_i|}$$

の 1 次収束である．このため絶対値の近い固有値のペア λ_i, λ_{i+1} が 1 つでもあると収束は遅い.

注意 2.4.3 固有ベクトルまで求めるには，QR 法で数値的に求めた固有値 λ_i を σ として逆反復法を適用すればよい.

2.4.5 QR 法の高速化

シフト

行列 A の対角成分から定数 σ を引く：

$$A' = A - \sigma I.$$

この操作を**（原点）シフト**という．シフト前の A の固有値 $\lambda_1, \ldots, \lambda_n$ に対して，シフト後の A' の固有値は $\lambda_1 - \sigma, \ldots, \lambda_n - \sigma$ である.

シフトの考え方は QR 法の収束を加速させるのに役立つ．QR 法の計算途中で，例えば行列 A_k を求めた時点で，第 n 固有値 λ_n の近似値 σ が見つかったとする.（例えば何回か反復した後の A_k の第 (n, n) 成分 $a_{k;n,n}$ など.）このとき A_k を σ だけシフトして（つまり A_k を $A_k - \sigma I$ に取り替えて）QR 法の計算を続ける.

σ が λ_n に十分近ければ，A_k の狭義下三角成分のシフト後の収束率

$$\rho'_{i,j} = \frac{|\lambda_i - \sigma|}{|\lambda_j - \sigma|}$$

はシフト前の $\rho_{i,j}$ より小さい．特に A_k の第 n 行成分の収束率 $\rho'_{n,j}$ はとても 0 に近い．このためシフト後の A_k の第 n 行成分は，QR 法の反復計算によりとても速く収束すると期待される．

減次

QR 法の反復計算が進んで，行列 $A_k = (a_{k;i,j})_{i,j}$ の第 n 行成分が（シフトなどの効果もあって）ほとんど収束したとする．つまり収束判定の閾値 $\varepsilon > 0$ に対して

$$\frac{|a_{k;n,j}|}{|a_{k;n,n}|} < \varepsilon \qquad (1 \le j < n) \tag{2.17}$$

が満たされたとする．このとき A_k は行列

$$\begin{pmatrix} & & * \\ & B_k & \vdots \\ & & * \\ 0 & \cdots\ 0 & a_{k;n,n} \end{pmatrix}$$

に近い．ただし B_k は A_k から第 n 行と第 n 列を除いた小行列である．特に A_k の固有値は B_k の固有値と $a_{k;n,n}$ により近似される．

そこで次のような戦略が考えられる．

減次戦略

(2.17) が満たされた時点で $a_{k;n,n}$ を固有値の 1 つとして採用し，残りの固有値は小行列 B_k から求める．

このように途中で行列の次数を下げて計算量を抑える工夫を**減次**という．

2.5 課題

課題 1 消去法により，連立 1 次方程式 (2.1) の解を求める数値実験を行う．$n = 100, 200, 400, 800$ のそれぞれに対して，n 次の行列 A とベクトル $\boldsymbol{b}$ のペアをランダムに 100 個生成し，各ペアに対して連立 1 次方程式 (2.1) の数値解を求めよ．

(1) 各 n に対して，数値解 $\tilde{\boldsymbol{x}}$ の残差ノルム (2.5)，相対誤差 (2.4)，計算時間をそれぞれプロットせよ．ただし真の解は不明なので，MATLAB や Python における数値計算パッケージで得られるものを真の解として用いよ．

(2) 数値解の精度と次数 n の関係について考察せよ．

(3) 計算時間と次数 n の関係について考察せよ．

課題 2 課題 1 と同じ内容を, LU 分解による解法に替えて行え．

課題 3 連立 1 次方程式に対する消去法と LU 分解による解法を，数値解の精度と計算速度の観点から比較せよ．その際，理論的な計算量（加減算，乗算，除算それぞれの回数）についても考察せよ．

課題 4 べき乗法により，行列 A の第 1 固有値と第 1 固有ベクトルを求める数値実験を行う．$n = 50, 100, 200, 400$ のそれぞれに対して，n 次行列 A をランダムに 100 個生成し，各 A の第 1 固有値と第 1 固有ベクトルを数値的に求めよ．

(1) 各 n に対して，数値的に求めた第 1 固有値 $\tilde{\lambda}_1$ と第 1 固有ベクトル $\tilde{\boldsymbol{v}}_1$ に対する固有方程式の残差ノルム (2.12)，$\tilde{\lambda}_1$ の相対誤差 (2.10)，$\tilde{\boldsymbol{v}}_1$ の相対誤差 (2.11)，計算時間，収束までに要した反復回数をそれぞれプロットせよ．

(2) $\tilde{\lambda}_1, \tilde{\boldsymbol{v}}_1$ の精度と次数 n の関係について考察せよ．

(3) 計算時間と次数 n の関係について考察せよ．

注意 2.5.1 べき乗法や QR 法の収束性は, 与えられた行列の固有値の絶対値がすべて相異なる場合に保証される．しかし, ランダムな実行列は共役な複素固有値を含む可能性があり, その場合にはアルゴリズムの収束が保証されない．そこで, 生成した実行列 A にその転置 A^{T} を加えるなどして実対称行列を生成し, それを用いて実験するなどの工夫をすると良い．

課題 5 べき乗法と逆反復法を組み合わせて，第 1 固有値と第 1 固有ベクトルを数値的に求めるための算法として，べき乗法より高速なものをつくれ．つくった算法とべき乗法を精度と速度の面から比較せよ．

課題 6 QR 法により，行列 A の全ての固有値を求める数値実験を行う．$n = 10, 20, 40, 80$ のそれぞれに対して，n 次行列 A をランダムに 100 個生成し，各 A の固有値 $\lambda_1, \ldots, \lambda_n$ を数値的に求めよ．

(1) 各 n に対して，数値的に求めた固有値 $\tilde{\lambda}_1, \ldots, \tilde{\lambda}_n$ に対する固有方程式の残差ノルムの最大値

$$\max_{1 \le i \le n} \|(A - \tilde{\lambda}_i I)\boldsymbol{v}_i\|,$$

相対誤差の最大値

$$\max_{1 \le i \le n} \frac{|\lambda_i - \tilde{\lambda}_i|}{|\lambda_i|},$$

計算時間，収束までに要した反復回数をそれぞれプロットせよ．ただし真の固有値 λ_i と固有ベクトル $\boldsymbol{v}_i$ は不明なので，MATLAB や Python における数値計算パッケージで得られるものを真の固有値および固有ベクトルとして用いよ．

(2) $\tilde{\lambda}_1, \ldots, \tilde{\lambda}_n$ の精度と次数 n の関係について考察せよ．

(3) 計算時間と次数 n の関係について考察せよ．

課題 7 シフトと減次を用いて QR 法を高速化せよ．つくった算法と元々の QR 法を精度と速度の面から比較せよ．

参考文献

[1] 杉原正顯・室田一雄，線形計算の数理，岩波書店，2009.

[2] 山本有作・曽我部知広，計算科学のための基本数理アルゴリズム，張紹良編，共立出版，2019.

[3] LAPACK, http://www.netlib.org/lapack/.

[川越大輔]

第3章　連続最適化

3.1　目的

与えられた関数 $f\colon \mathbb{R}^n \to \mathbb{R}$ の零点，および最小値または最大値を与える点 $x \in \mathbb{R}^n$ を求める手法を学び，演習を通じてその手法を理解する．本章は Python による説明を随所にしており，また課題において「Python」を用いるよう指示している箇所があるが，**担当教員の指示によっては必ずしも Python を使う必要はない**．内容は，3.2〜3.5 節の大きく 4 つに分かれており，3.2 節は関数の零点を求める二分法とニュートン法について学び，Python を導入する．3.3 節は降下法の概要を述べ，最適化手法として根幹を成す最急降下法およびニュートン法について学ぶ．3.4 節は，直線探索を用いたニュートン法および最適化手法の収束性について学ぶ．3.5 節は，ニュートン法に修正を加えた手法について学び，総合演習を行う．

3.2　関数の零点探索と Python 導入

ここでは，関数の零点探索のための二分法とニュートン法を学び，Python の導入を行う．

3.2.1　関数の零点探索

与えられた関数 $f\colon \mathbb{R} \to \mathbb{R}$ に対して，$f(x) = 0$ を満たす点 x のことを関数 f の零点と呼ぶ．関数の零点探索は，様々な場面で必要とされる．例えば，行列 $A \in \mathbb{R}^{n\times n}$ の固有値を求める際には，固有多項式 $p_A(x) = \det(xI - A)$ の零点を求めることに帰着され，一方，連続関数 $F\colon \mathbb{R} \to \mathbb{R}$ の最適化では，導関数 F' の零点を求めることに帰着される．

このように場面に応じて関数 f は異なるが，零点探索はそれぞれの場面で重要な役割を果たしている．ここでは，計算機を用いた関数の零点探索を考える．はじめに，関数の零点探索のための反復法として知られる二分法を以下に与える．

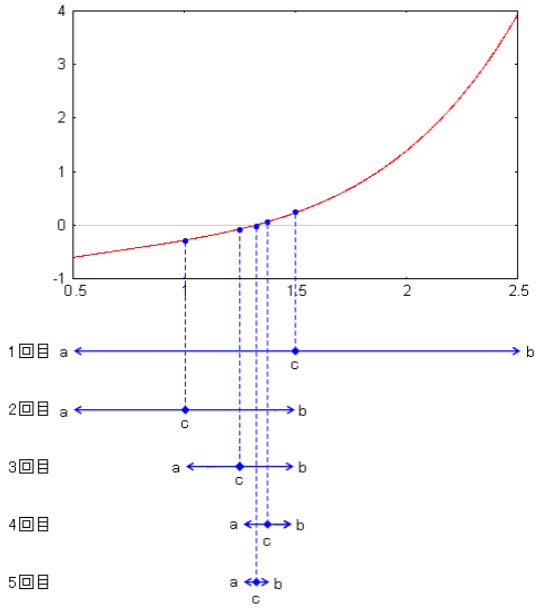


図 3.1: 二分法の生成点列

二分法

Step 0: $f(a) < 0$, $f(b) \geq 0$ を満たす初期点 a, b を選び，終了条件 $\varepsilon > 0$ を決める．

Step 1: a と b の中間点 $c := (a+b)/2$ を求める．もし，c が終了条件 $|f(c)| \leq \varepsilon$ を満たしていれば，c を解として終了する．

Step 2: もし，$f(c) < 0$ であれば $a \leftarrow c$ とし，$f(c) \geq 0$ であれば $b \leftarrow c$ として，Step 1 へ戻る．

二分法は，関数 f が $\mathbb{R}$ 上で連続であれば必ず収束する．実際，Step 1 の初期条件を満たす a, b を選べば，関数 f の零点は a と b の間に存在するので，a と b の間隔を繰り返し 1/2 に縮小していけば，中間点 c は以下の図で示すように関数 f の零点へ近づいていくことがわかる．

二分法は，アルゴリズムが簡易的であり，二つの異なる初期点 a, b の間に零点が存在すれば，必ず収束するという優れた性質を持つ．しかし，一

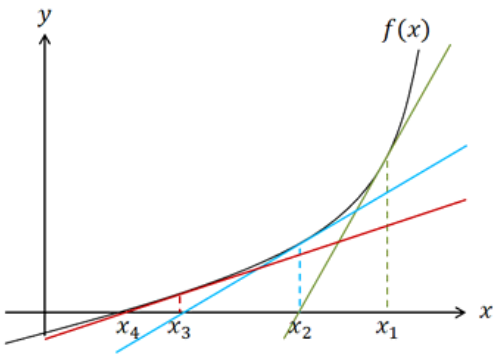


図 3.2: ニュートン法の生成点列

般的には収束が遅いことで知られる．より収束が速い手法としては，次に与えるニュートン法が良く知られている．

ニュートン法

Step 0: 初期点 x_0 を選び，終了条件 $\varepsilon > 0$ を決める．$k := 0$ とする．

Step 1: ニュートン方程式

$$f'(x_k)\Delta x_k = -f(x_k)$$

を解き，Δx_k を求める．

Step 2: 点列を $x_{k+1} := x_k + \Delta x_k$ により更新する．もし，$|f(x_{k+1})| \leq \varepsilon$ を満たしていれば，x_{k+1} を解として終了する．

Step 3: $k \leftarrow k+1$ として，Step 1 へ戻る．

ニュートン法は，点 x_k の周りで関数 f を一次近似し，接線の方程式 $y = f'(x_k)(x-x_k)+f(x_k)$ を考える．この方程式と x 軸の交点を求め，更新点とする（つまり，$f'(x_k)(x-x_k)+f(x_k)=0$ を解き，$x = x_k - f(x_k)/f'(x_k)$ を更新点とする）ことで点列を生成する反復法である．一般的に収束は速いが，解の近くに初期点を選ばなければ収束しないということが知られている．

3.2.2 Python 導入

ここでは，Python の導入を行う．インターネット上のチュートリアルを参考にし，Anaconda3 を用いて Python をインストールせよ．例えば，

- `https://sukkiri.jp/technologies/ides/anaconda-win_install.html`

が参考となる．Python 3.0，数値計算ライブラリの Numpy，グラフ描写ライブラリの Matplotlib が PC にインストールされている場合は飛ばしても良い．

Python ヒント

Python はスクリプト言語なので，実行方法が 2 通りある：(a) 直接プログラムを 1 行ずつ打ち込む方法と，(b) プログラムファイルを作成し，インタプリタに一気に実行させる方法．(a) の場合は，簡単な計算の確認などのテストをするために便利であり，ターミナル上に `python` コマンドを入力することによってインタプリタが実行される．(b) の場合は，「`.py`」という拡張子のファイル（例えば，`test.py`）を用いて，ターミナル上に `python test.py` を入力し，プログラムを実行する．また，anaconda3 をインストールした際に，Spyder と呼ばれる Python 専用のエディターが同時にインストールされるので，こちらを用いるのも良い．

Python ヒント

本演習では，ライブラリ Numpy・Matplotlib が必要となる．インポートするために

```
import numpy as np
import matplotlib.pyplot as plt
```

を実行する，またはファイルに入力する．ここでは，Numpy の組込み関数を有効に使う．コンパイラ言語 C と違って，例えば，

```
w = np.random.rand(4)
```

を入力すると，ランダムな 4 次元ベクトルが `while` 文や `for` 文なしで生成される．同様に，

```
y = np.random.rand(4)
sum = w + y
```

を追加すると，2 つのランダムなベクトルの和が簡単に得られる．

課題 1 次の関数 $f\colon \mathbb{R} \to \mathbb{R}$ を考える．

$$f(x) := x^3 + 2x^2 - 5x - 6$$

Python を用いて，以下の課題に取り組め．

(a) 関数 f を $-10 \leq x \leq 10, -10 \leq y \leq 10$ の範囲で描写せよ．

(b) 二分法により関数 f の全ての零点を求めよ．ただし，二つの異なる初期点は，(a) で描写した関数 f の概形を参考にして適切に設定せよ．

(c) ニュートン法により関数 f の全ての零点を求めよ．ただし，(a) で描写した関数 f の概形を参考にして，零点の近くに初期点を設定せよ．

Python ヒント

グラフ描写は以下を参考にすると良い．例えば，$y = \sin(x)$ を $-5 \leq x \leq 5, -5 \leq y \leq 5$ の平面内に描写する際は，Numpy・Matplotlib をインポートしてから以下を入力する．

```
x = np.arange(-5,5,0.1)
y = np.sin(x)
plt.plot(x, y, 'red')
plt.xlim(-5,5)
plt.ylim(-5,5)
plt.axhline(0, c='black')
plt.axvline(0, c='black')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graph of y=sin(x)')
plt.show()
```

3.3 数理最適化の基礎と降下法

ここでは，まず数理最適化における基本を学ぶ．さらに，最も基本的な最適化手法として知られる降下法の概要を説明し，その具体例として最急降下法およびニュートン法を学ぶ．

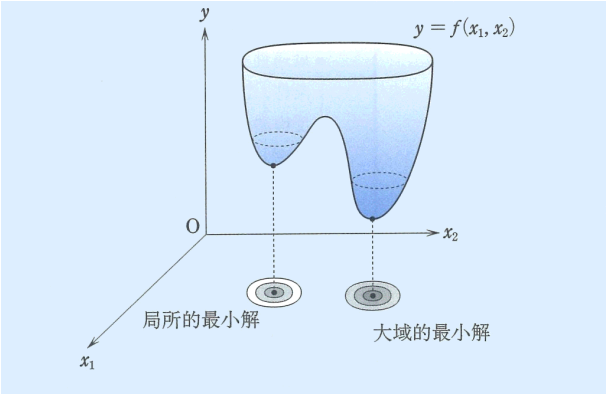


図 3.3: 大域的最小解，局所的最小解

3.3.1 関数の最大化・最小化

工学的分野に限らず，様々な分野において，与えられた関数の最小値，または最大値を求めることは重要である．例えば，企業においては，その利益を最大化することを目的としており，また，統計の分野における最小二乗法は，与えられたデータとの誤差を最小にする近似曲線を求めることを目的としている．

各分野において，その最小化または最大化しようとする関数は異なるが，ここでは，一般に $f\colon \mathbb{R}^n \to \mathbb{R}$ で与えられているとする．このとき，その関数の最小化問題は，

$$\begin{aligned} &\text{minimize} && f(x) \\ &\text{subject to} && x \in X \end{aligned} \tag{3.1}$$

として定式化される．これは,「集合 $X \subseteq \mathbb{R}^n$ の中から，関数値 $f(x)$ を最小とする x を求めよ」という問題である．ここで，f をこの最小化問題の**目的関数**と呼び，集合 X を**制約集合**と呼ぶ．さらに，制約条件を満たす点を**実行可能解**という．制約集合 X が $X = \mathbb{R}^n$ のとき，問題 (3.1) を**無制約最小化問題**という．本演習では，制約無し最小化問題のみを扱う．また，最大化問題については，最大化する関数が $\tilde{f}$ で与えられているとき，この問題は $f(x) := -\tilde{f}(x)$ の最小化問題へと同値に変形することができるため，ここでは最小化問題のみを考える．

最小化問題において，全ての実行可能解 x の関数値よりも大きくならない関数値をもつ点 $\bar{x} \in \mathbb{R}^n$

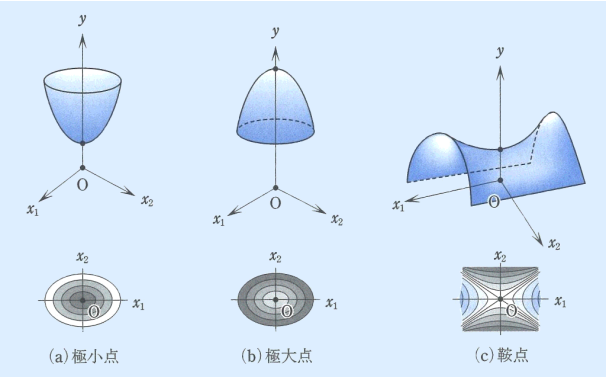


図 3.4: 局所的最大解と鞍点

を，その問題の**大域的最小解**と呼ぶ (図 3.3)．つまり，次が成り立つ．

$$f(\bar{x}) \leq f(x) \quad \text{for all } x \in \mathbb{R}^n$$

しかし，大域的最小解であるかどうかの判定には，$\mathbb{R}^n$ 上の全ての点の情報が必要となるため，一般的に大域的最小解を求めることは難しい．このため，次の局所的最小解という概念が重要となる．ここで，$\hat{x}$ が**局所的最小解**とは，$\hat{x}$ を含むある近傍 $N \subseteq \mathbb{R}^n$ を取ることができて，その近傍内では $f(\hat{x})$ よりも関数値が小さくなる実行可能解が存在しないことである (図 3.3)．つまり，$\hat{x}$ を含むある近傍 N が存在し，次を満たすことである．

$$f(\hat{x}) \leq f(x) \quad \text{for all } x \in N$$

もし，$N = \mathbb{R}^n$ ならば，局所的最小解は大域的最小解となる．関数 f が微分可能であるとき，

$$\nabla f(x) := \left(\frac{\partial f(x)}{\partial [x]_0}, \ldots, \frac{\partial f(x)}{\partial [x]_{n-1}} \right)^\top = 0$$

を満たす点 x のことを f の停留点と呼ぶ．ただし，$[x]_k$ $(k = 0, 1, \ldots, n-1)$ はベクトル x の第 k 成分であり，$\top$ は転置を表す．また，$\nabla f(x)$ は点 x における f の勾配ベクトルと呼ばれる．もし，$\hat{x}$ が局所的最小解であれば，$\hat{x}$ は f の停留点となる．しかし，f の停留点が f の局所的最小解であるとは限らない．局所的最大解や鞍点の場合もあるからである (図 3.4)．

本演習で扱う最急降下法およびニュートン法は，基本的に停留点を求める手法であり，必ずしも局所的最小値を求められるわけではない．しかしながら，大抵の問題に対しては，それらの手法によって局所的最小解を求めることができる．また，局所的最小値を求めることができる複雑な手法も，その根本的なところで，最急降下法やニュートン法を用いている．このため，それらの手法を理解することは重要である．

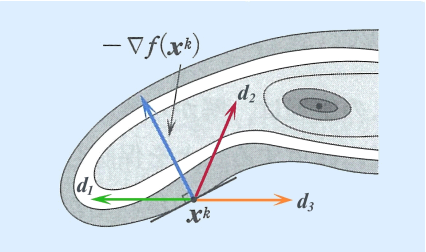


図 3.5: 降下方向

3.3.2 降下法

最急降下法やニュートン法は降下法と呼ばれる手法である．降下法は，関数の値を減少させる点列 $\{x^0, x^1, x^2, \ldots\}$ を生成していく反復法である[1]．つまり，

$$f(x^0) > f(x^1) > \cdots$$

となる点列 $\{x^k\}$ を生成する．この点列は

$$x^{k+1} := x^k + t_k d^k$$

で与えられる．ここで，d^k を**探索方向**，t_k を**ステップサイズ**と呼ぶ．降下法では，前提として，d^k は次の条件を満たすと仮定する．

$$\langle d^k, \nabla f(x^k) \rangle < 0$$

ここで，$\langle y, z \rangle$ はベクトル $y, z \in \mathbb{R}^n$ の内積を表している．つまり，$\langle y, z \rangle := \sum_{i=0}^{n-1} [y]_i [z]_i$ である．このような条件を満たしてるベクトル d^k のことを**降下方向**と呼ぶ．

[1] 点列を表す際は，上付き添え字を用いる．つまり，x^k は $\mathbb{R}^n$ の点列 $\{x^0, x^1, x^2, \ldots\}$ における k 番目の項を表す．なお，$\mathbb{R}$ や $\mathbb{R}^{n,m}$ の点列を表す際は，冪乗との混同を避けるため下付き添え字を用いる．

図 3.5 は関数 f の等高線を表しており，図中の d_1 および d_3 は降下方向であるが，d_3 は降下方

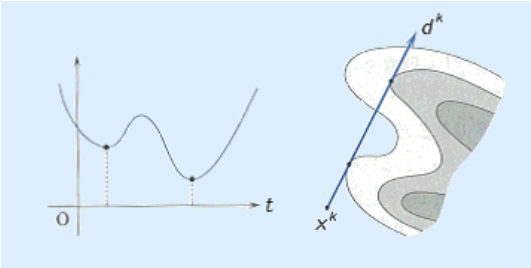


図 3.6: 直線探索

向ではない．降下方向へ適切に進めば，関数値は減少するが，進みすぎると増加する場合がある．そのため，$f(x^k) > f(x^{k+1})$ となるように，t_k をうまく設定する必要がある．この t_k を決める方法は**直線探索法**と呼ばれ，いくつか提案されているが，本テキストではバックトラック法を学ぶ．バックトラック法の詳細は 3.4 節で述べる．図 3.6（右）は探索方向を表す．図 3.6（左）は探索方向 d^k へ進んだ際の目的関数値の変化を表すグラフである．

バックトラック法を組み込んだ降下法を以下に与える．ただし，$\|z\| := (\sum_{i=0}^{n-1}[z]_i^2)^{1/2}$ である．

降下法

Step 0: 初期点 x^0，終了条件 $\varepsilon > 0$ を決める．$k := 0$ とする．

Step 1: もし，x^k が終了条件 $\|\nabla f(x^k)\| \leq \varepsilon$ を満たしていれば，x^k を解として終了する．

Step 2: 次式を満たす降下方向 d^k を定める．

$$\langle d^k, \nabla f(x^k)\rangle < 0$$

Step 3: バックトラック法を用いて，ステップサイズ t_k を決定する．

Step 4: 点 x^k を $x^{k+1} := x^k + t_k d^k$ と更新する．$k \leftarrow k+1$ として，Step 1 へ戻る．

注意 3.3.1 Step 1 における終了条件のパラメータ ε は，$\varepsilon = n \times 10^{-6}$ のように設定されることが多い．これは，一般に問題の次元 n が大きくなるにつれて，計算誤差が大きくなるため，この誤差を許容するという意味がある．

注意 3.3.2 反復回数 (k) が十分大きいとき，「解が求まらない」と判断する．よって，許容される最大反復回数の設定が必要である．

この降下法によって生成される点列 $\{x^k\}$ に対して，以下で与える収束に関する定理が示されている．証明に興味がある方は文献 [1] を読んでもらいたい．

定理 3.3.1 点列 $\{x^k\}$ および $\{d^k\}$ は，それぞれ降下法により生成される有界列とし，$\{d^k\}$ は降下方向の列であると仮定する．また，ある正の定数 γ が存在し，全ての $k \in \mathbb{N} \cup \{0\}$ に対して，以下の不等式が成り立つと仮定する．

$$\langle d^k, \nabla f(x^k)\rangle \leq -\gamma \|\nabla f(x^k)\|^2$$

このとき，点列 $\{x^k\}$ の任意の集積点 x^* は，関数 f の停留点となる．つまり，$\nabla f(x^*) = 0$ が成り立つ．

この定理により，上手に降下方向を選んでやれば，問題の停留点 (多くの場合，局所最小解) を求めることができる．次節以降で紹介する最急降下法とニュートン法は，この定理の降下方向に関する仮定を満たす手法である．

3.3.3 最急降下法とニュートン法

ここでは，最急降下法およびニュートン法について説明をする．

最急降下法は，点 x^k が与えられたときに，降下方向として以下を用いる手法である．

$$\text{[最急降下方向]} \qquad d^k := -\nabla f(x^k)$$

いま，$\nabla f(x^k) \neq 0$ と仮定する．(もし，$\nabla f(x^k) = 0$ であれば，点 x^k は既に f の停留点となっている．) このとき，

$$\langle d^k, \nabla f(x^k)\rangle = -\|\nabla f(x^k)\|^2 < 0$$

が成り立つため，d^k は降下方向であり，定理 3.3.1 の仮定を満たすことがわかる．（実際，上記の等号により，定理 3.3.1 における正の定数 γ として，$\gamma = 1$ の存在が確かめられる．）

一方，ニュートン法は，x^k が与えられたときに，降下方向として以下を用いる手法である．

[ニュートン方向] $\quad d^k := -\nabla^2 f(x^k)^{-1} \nabla f(x^k)$

ただし，$\nabla^2 f(x)$ は x における f のヘッセ行列を表しており，

$$\begin{bmatrix} \frac{\partial^2 f(x)}{\partial [x]_0^2} & \frac{\partial^2 f(x)}{\partial [x]_0 [x]_1} & \cdots & \frac{\partial^2 f(x)}{\partial [x]_0 [x]_{n-1}} \\ \frac{\partial^2 f(x)}{\partial [x]_1 [x]_0} & \frac{\partial^2 f(x)}{\partial [x]_1^2} & \cdots & \frac{\partial^2 f(x)}{\partial [x]_1 [x]_{n-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial [x]_{n-1} [x]_0} & \frac{\partial^2 f(x)}{\partial [x]_{n-1} [x]_1} & \cdots & \frac{\partial^2 f(x)}{\partial [x]_{n-1}^2} \end{bmatrix}$$

で与えられる．ここで，$\{\nabla^2 f(x^k)\}$ は有界かつ一様正定値（詳細は 3.5 節で解説する）であると仮定する．これは，ある $\mu > 0$ が存在して，全ての $k \in \mathbb{N} \cup \{0\}$ と $v \in \mathbb{R}^n$ に対して，

$$\frac{1}{\mu}\|v\|^2 \leq \langle \nabla^2 f(x^k)^{-1} v, v \rangle \leq \mu \|v\|^2$$

が成り立つことと同値である．このとき，すべての k に対して，$\nabla f(x^k) \neq 0$ であれば，

$$\begin{aligned} \langle d^k, \nabla f(x^k) \rangle &= -\langle \nabla^2 f(x^k)^{-1} \nabla f(x^k), \nabla f(x^k) \rangle \\ &\leq -\frac{1}{\mu} \|\nabla f(x^k)\|^2 \\ &< 0 \end{aligned}$$

となるため，d^k は降下方向であり，定理 3.3.1 の仮定を満たすことがわかる．

課題 2 関数 $f\colon \mathbb{R} \to \mathbb{R}$ を以下で定義する．

$$f(x) := \frac{1}{3}x^3 - x^2 - 3x + \frac{5}{3}$$

Python を用いて，

(a) 最急降下法を実装し，関数 f の停留点を求めよ．ただし，初期点は $x_0 := 1/2$ と設定し，各反復 $k \in \mathbb{N} \cup \{0\}$ において，ステップサイズは $t_k := 1/(k+1)$ を採用せよ．

(b) ニュートン法を実装し，関数 f の停留点を求めよ．ただし，初期点は $x_0 := 5$ と設定し，各反復 $k \in \mathbb{N} \cup \{0\}$ において，ステップサイズは $t_k := 1$ を採用せよ．

3.4 直線探索と最適化手法の収束性

ここでは，前半で 3.3 節の降下法の概説で触れた直線探索法について学ぶ．特に，直線探索法の一種であるバックトラック法を詳説する．後半では，反復法の収束性について学ぶ．

3.4.1 直線探索法

直線探索法は，ステップサイズ $t_k > 0$ を決定するための手法であり，点 x_k を x_{k+1} へ更新する際に，$f(x_{k+1}) < f(x_k)$ を満たすようにするために行う．これまでにいくつかの直線探索法が提案されているが，以下に与えるバックトラック法は理論的にも実用的にも優れていることが知られている．

バックトラック法

Step 0: $\xi \in (0,1)$，$\rho \in (0,1)$，初期ステップサイズ $\bar{t} > 0$ を選ぶ．$t = \bar{t}$ とする．

Step 1: 次式を満たすまで $t \leftarrow \rho t$ とする．

$$f(x^k + td^k) \leq f(x^k) + \xi t \langle d^k, \nabla f(x^k) \rangle$$

Step 2: $t_k = t$ として終了．

Step 1 の不等式は**アルミホ条件**と呼ばれ，その意味は図 3.7 で表す．横軸はステップサイズ t，縦軸は $\varphi(t) := f(x^k + td^k)$ を表す．原点における φ の接線は $y = f(x^k) + t\langle d^k, \nabla f(x^k) \rangle$ である．アルミホ条件を満たすステップサイズを選ぶことは，φ の接線の傾きを緩和して得られる直線 $y = f(x^k) + \xi t \langle d^k, \nabla f(x^k) \rangle$ よりも関数値が小さくなる t の区間からステップサイズを選ぶことに対応していると言える．ξ を小さく設定すれば，

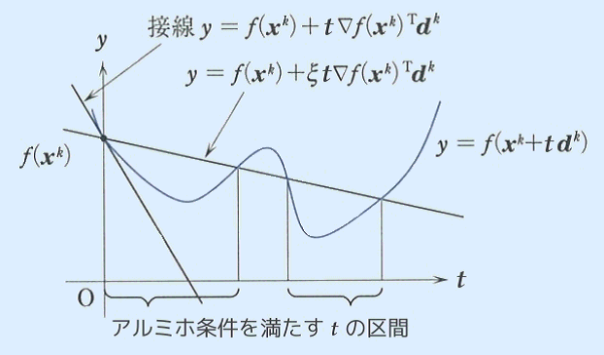


図 3.7: アルミホ条件を満たすステップサイズ

アルミホ条件を満たすステップサイズを速く見つけることができる．

3.4.2 反復法の収束性について

ここまで，最急降下法およびニュートン法について学び，定理 3.3.1 によって，どちらの手法を用いても問題の解（多くの場合，局所最小解）を求められることがわかった．しかし，ここまでの議論では，どれだけ速く解を得ることができるかはわからない．これについて議論するためには，反復法の解への収束の速さに関する概念が必要となる．

ここで，x^* を求める問題の解とし，$\{x_k\}$ は x^* へ収束するとしよう．このとき，ある $\kappa \in (0,1)$ と $m_0 \in \mathbb{N}$ が存在して，

$$\|x^{k+1} - x^*\| \leq \kappa \|x^k - x^*\| \quad \text{for all } k \geq m_0$$

が成り立つとき，点列 $\{x^k\}$ は解 x^* に **1 次収束**するという．一方，

$$\lim_{k\to\infty} \frac{\|x^{k+1} - x^*\|}{\|x^k - x^*\|^2} < \infty$$

が成り立つとき，点列 $\{x^k\}$ は解 x^* に **2 次収束**するという．これは，ある $\eta > 0$ と $n_0 \in \mathbb{N}$ が存在して，

$$\|x^{k+1} - x^*\| \leq \eta \|x^k - x^*\|^2 \quad \text{for all } k \geq n_0$$

となることを意味しており，一旦 $\|x^k - x^*\| < 1$ となると，非常に速く収束する．

表 3.1: 1 次収束と 2 次収束

k	y_k	z_k
0	1	1
1	0.5	0.5
2	0.25	0.125
3	0.125	0.0078125
4	0.0625	0.0000305
⋮	⋮	⋮

ここで，次の点列 $\{y^k\}$ と $\{z^k\}$ を考えてみよう．

$$y_k = 0.5^k, \quad z_k = 0.5 z_{k-1}^2$$

ただし，$z_0 = 1$ とする．点列 $\{y_k\}$ は，0 に 1 次収束することが確かめられる．一方, 点列 $\{z_k\}$ は，0 に 2 次収束することが確かめられる．これらの点列を計算してみると，表 3.1 となる．表 3.1 より，2 次収束する点列は，1 次収束する点列に比べて，極めて速い収束性を持つことがわかる．

ここまでに学んだ最急降下法は，適切な仮定の下で一次収束することが示されている．しかし，最小化する関数の一階微分（つまり一次の情報）しか用いていないため，二次収束しないことが多い．一方，ニュートン法は関数のヘッセ行列（つまり二次の情報）を用いた方法であり，適切な仮定の下で二次収束することが知られている．よって，最急降下法とニュートン法の解への収束性を比較すると，一般にニュートン法に軍配が上がる．

このように収束性の観点から両手法を見ると，ニュートン法の方が最急降下法に比べて有能な反復法であるかのように思える．しかし，ニュートン法は毎回の反復において，降下方向の生成のためにヘッセ行列 $\nabla^2 f(x^k)$ および勾配 $\nabla f(x^k)$ を計算し，ニュートン方程式 $\nabla^2 f(x^k) d^k = -\nabla f(x^k)$ を解かなければならないため，勾配 $\nabla f(x^k)$ を計算するだけの最急降下法に比べて一反復当たりの計算負荷が大きい．従って，問題の規模が大きくなれば，最急降下法では計算が可能であっても，ニュートン法では計算が困難となることも有り得る．このように，解への収束が速いニュートン法の方が，最急降下法に比べて有能な反復法である

とは限らないことに注意して欲しい．

課題 3 次の 2 次元の最適化問題を考える．

$$\begin{aligned} \text{minimize} \quad & f(x) := x_0^2 + e^{x_0} + x_1^4 + x_1^2 \\ & \qquad -2x_0x_1 + 3 \\ \text{subject to} \quad & x := (x_0, x_1)^\top \in \mathbb{R}^2 \end{aligned}$$

Python を用いて，

(a) 点 x が与えられたとき，目的関数の値 $f(x)$ を出力する関数を作成せよ．

(b) 点 x が与えられたとき，勾配ベクトル $\nabla f(x)$ を出力する関数を作成せよ．

(c) 点 x が与えられたとき，ヘッセ行列 $\nabla^2 f(x)$ を出力する関数を作成せよ．

Python ヒント

- 課題 3 を実施する前に，$f(x) := x_0^2 + x_1$ の場合を考える．このとき，

$$\nabla f(x) = \begin{bmatrix} 2x_0 \\ 1 \end{bmatrix}, \nabla^2 f(x) = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$$

である．

- 以下のコードをファイルに入力し，実行してみる．コードも理解しておく[a]．

```
#!/usr/bin/env python
# -*- coding:  utf-8 -*-

import numpy as np

def evalf(x):
  # 目的関数の値 f(x) を計算する
  f = x[0]**2 + x[1]
  return f

def evalg(x):
  # f の勾配を計算する
  g = np.array([2*x[0], 1])
  return g

def evalh(x):
  # f のヘッセ行列を計算する
  H = np.array([[2, 0], [0, 0]])
  return H

def main():
  x = np.array([0.3, 5])
  f = evalf(x)
  g = evalg(x)
  H = evalh(x)
  print('f =', f, '\ng =', g, \
    '\nH = \n', H)
if __name__ == '__main__':
  main()
```

[a] Python 3 を使った例である．

課題 4 Python を用いて，

(a) バックトラック法を用いた最急降下法を実装

することで課題 3 の最適化問題を解き，最適解，最適値，および反復回数を与えよ．ただし，初期点は $x^0 = (1,1)^\top$ と設定し，またバックトラック法における ξ，ρ，$\bar{t}$ は，それぞれ $\xi = 10^{-4}$，$\rho = 0.5$，$\bar{t} = 1$ と設定せよ．

(b) バックトラック法を用いたニュートン法を実装することで課題 3 の最適化問題を解き，最適解，最適値，および反復回数を与えよ．ただし，初期点は $x^0 = (1,1)^\top$ と設定し，またバックトラック法における ξ，ρ，$\bar{t}$ は，それぞれ $\xi = 10^{-4}$，$\rho = 0.5$，$\bar{t} = 1$ と設定せよ．

3.5 最適化手法の実用的な修正方法と総合演習

3.3 節で学習したニュートン法は，$\{\nabla^2 f(x^k)\}$ が一様正定値であることを仮定していたが，この仮定は非常に強い仮定であり，成り立たないこともしばしばある．ここでは，この仮定を満たさない場合にも適用可能となるように，ニュートン法を修正する方法について学ぶ．また，Python を用いて最急降下法とニュートン法を実装し，いくつかの最適化問題を解く総合演習を行う．

3.5.1 ニュートン方向の修正

まず，対称行列の正定値性と固有値の関係について学ぶ．対称行列 $M \in \mathbb{R}^{n\times n}$ が正定値であるとは，

$$\langle Mv, v\rangle > 0 \quad \text{for all } v \in \mathbb{R}^n\backslash\{0\}$$

を満たすことである．以降，対称行列 M が正定値であることを $M \succ O$ と表す．次の定理は，正定値性と固有値の関係を表すものであり，非常に重要な性質である．

定理 3.5.1 以下の二つの命題は同値である．

- 対称行列 $M \in \mathbb{R}^{n\times n}$ が正定値である．
- 対称行列 $M \in \mathbb{R}^{n\times n}$ の全ての固有値 $\lambda_j(M)$ $(j = 1,\ldots,n)$ は正である

また，$\mathbb{R}^{n\times n}$ の点列 $\{M_k\}$ が有界かつ一様正定値であるとは，ある $\kappa > 0$ が存在して，全ての $k \in \mathbb{N}\cup\{0\}$ に対して，

$$\frac{1}{\kappa}\|v\|^2 \le \langle M_k v, v\rangle \le \kappa\|v\|^2 \quad \text{for all } v \in \mathbb{R}^n$$

が成り立つことであり，これは，

$$\frac{1}{\eta}\|v\|^2 \le \langle M_k^{-1} v, v\rangle \le \eta\|v\|^2 \quad \text{for all } v \in \mathbb{R}^n$$

と同値である．ただし，$\eta := 1/\kappa$ である．

3.3 節で学んだニュートン法は，「$\{\nabla^2 f(x^k)\}$ が有界かつ一様正定値である」と仮定したが，一様正定値性については成り立たないことも多々あるため，このような場合に対応するために，次に述べるような修正を行う．各反復 k に対して，ヘッセ行列 $\nabla^2 f(x^k)$ に単位行列 I の τ 倍を加え，次の方程式を考える．

$$\big(\nabla^2 f(x^k) + \tau I\big)d^k = -\nabla f(x^k) \tag{3.2}$$

ただし，$\tau > 0$ は $\nabla^2 f(x^k) + \tau I \succ O$ となるようにとる．もし，$\nabla^2 f(x^k) \succ O$ ならば，$\tau = 0$ とすれば良いので，$\nabla^2 f(x^k) \not\succ O$ の場合を考えよう．いま，$\nabla^2 f(x^k) + \tau I$ の固有値は $\lambda_j(\nabla^2 f(x^k)) + \tau$ $(j = 1,\ldots,n)$ となることを考慮すると，例えば τ として，$\nabla^2 f(x^k)$ の最小固有値の絶対値にある定数（10^{-1} や 10^{-2} など）を加えた値を設定すれば，$\lambda_j(\nabla^2 f(x^k)) + \tau > 0$ $(j = 1,\ldots,n)$ となり，定理 3.5.1 より $\nabla^2 f(x^k) + \tau I \succ O$ となる．よって，この τ を用いて，式 (3.2) から探索方向 d^k を生成する．このようなニュートン方向の修正は実用上，良く行われるものであり有用である．

3.5.2 総合演習

ここでは，Python を用いて，いくつかの最適化問題を解く演習を行う．

課題 5 Python を用いて，最急降下法およびニュートン法を実装することで次の最適化問題を解き，最適解および各手法の反復回数を与えよ．ただし，初期点は $x^0 = [2,0]^\top$ と設定し，またバックト

ラック法における ξ，ρ，$\bar{t}$ は，それぞれ $\xi = 10^{-4}$，$\rho = 0.5$，$\bar{t} = 1$ と設定せよ．

$$\begin{aligned} \text{minimize} \quad & f(x) := \sum_{i=0}^{2} f_i(x)^2 \\ \text{subject to} \quad & x \in \mathbb{R}^2 \end{aligned}$$

ただし，$f_i(x) := y_i - [x]_0(1 - [x]_1^{i+1})$ $(i = 0, 1, 2)$ と定義し，$y_0 = 1.5$, $y_1 = 2.25$, $y_2 = 2.625$ である．(参考：最適値は 0，最適解は $[3, 0.5]^\top$ である．)

ヒント

課題 5 の f の勾配とヘッセ行列は，それぞれ次のようになる．

$$\begin{aligned} \nabla f(x) &= 2\textstyle\sum_{i=0}^{2} f_i(x)\nabla f_i(x) \\ \nabla^2 f(x) &= 2\textstyle\sum_{i=0}^{2} \left(f_i(x)\nabla^2 f_i(x) \right. \\ &\quad \left. + \nabla f_i(x)\nabla f_i(x)^\top\right) \end{aligned}$$

参考文献

[1] D.P. Bertsekas, “Nonlinear Programming”, 2nd edition, Athena Scientific (1999).

[2] 福島雅夫,「新版・数理計画入門」，朝倉書店 (2011).

[3] J. Nocedal and S.J. Wright, “Numerical Optimization”, 2nd edition, Springer Verlag (2006).

[山川雄也]

3.6 （追加）制約付き凸二次計画問題

連続最適化の本文では，勾配計算に基づく各種反復解法について学んだ．とくに無制約条件のない場合の最適化問題について，局所最適解を求めるためのアルゴリズムとその収束性を学んだ．この発展版として，制約のある最適化問題とそのアルゴリズムを考えよう．議論を難しくしないために，ここでは凸最適化と呼ばれる問題のみを扱う．とくに基本的な凸最適化問題である，凸二次計画問題と呼ばれる問題を，課題で扱う．二次計画問題とは，目的関数が二次で，制約条件が線形である問題をいい，次の形で表される．

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & \frac{1}{2}x^\top Q x + c^\top x \\ \text{subject to} \quad & Ax \le b, \end{aligned} \tag{3.3}$$

ここで $Q \in \mathbb{R}^{n\times n}$ は対称行列，$c \in \mathbb{R}^n$，$A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ であり，不等号はベクトルの要素毎に成り立つことを意味する．

一般には線形の「等式」制約条件も課すが，不等式制約で等式制約を表すことができるし（$Ax \le b$ と $Ax \ge b$ を制約に入れればよい），その逆もスラック変数と呼ばれる冗長な決定変数を用意すれば可能である．このとき，次の課題に答えよ．

課題 6

1. 二次計画問題として定式化することが自然な問題を 1 つ挙げよ．

2. 二次計画問題 (3.3) は，Q が非負の固有値のみを持つ場合（Q が半正定値行列の場合），**凸二次計画問題**と呼ばれる．Q が正負両方の固有値を持つ場合に，二次計画問題がどのように難しくなるか調べよ．また文献も載せること（生成 AI の回答を参照する場合は，その根拠となる文献まで調べて載せること）．

3. 凸二次計画法の解法を調べ，そのアルゴリズムを述べよ．またその文献も載せること（生成 AI の回答を参照する場合は，その根拠となる文献まで調べて載せること）．

4. 二次計画問題 (3.3) として，次のパラメータを持つ問題を考える．

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix},\ c = -\begin{bmatrix} 1 \\ 1 \end{bmatrix},$$
$$A = \begin{bmatrix} 1 & 1 \end{bmatrix},\ b = 0.$$

このとき，最適解 x^* を数値的に求めよ．ソルバを使っても構わない．またこの問題は解析的に解けるので，解析解とも比較せよ．

5. （最適制御問題）$T = 3$ とし，次の問題を考える．

$$\begin{aligned} \min_{u} \quad & u^\top u, \\ \text{subject to } & x_{k+1} = Ax_k + Bu_k, \\ & x_0 = \begin{bmatrix} 10 \\ 1 \end{bmatrix},\ x_T = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \\ & u = \begin{bmatrix} u_0 \\ \vdots \\ u_{T-1} \end{bmatrix} \in \mathbb{R}^T. \end{aligned}$$

ただし

$$A = \begin{bmatrix} 1.2 & 1 \\ 0 & 0.9 \end{bmatrix},\ B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

この問題を凸二次計画問題に帰着させ，最適解 $u^* \in \mathbb{R}^T$ を求めよ．

最後の問題は，任意の初期ベクトル $x_0 \in \mathbb{R}^2$ から最小のエネルギーで指定された場所（この場合は原点）に移動させるという，可到達性に関係する最適制御問題の 1 つである．

注意 3.6.1 二次計画問題は，一般の凸最適化問題を解くときの近似解法（逐次二次計画法）として非常によく用いられる手法である．

[大木健太郎]

第4章　高速フーリエ変換

4.1　はじめに

高速フーリエ変換（Fast Fourier Transform; FFT）は、離散フーリエ変換を効率よく計算するためのアルゴリズムであり、音声データをはじめとする時系列データや画像データの解析･加工、数値シミュレーションなど、様々な分野で広く利用されている．本演習では、1965 年に J. W. Cooley（ジェームズ･クーリー）と J. W. Tukey（ジョン・テューキー）によって発表された形式の高速フーリエ変換アルゴリズムを学び、その基本的な仕組みを理解するとともに、様々な問題に応用できるようになることを目標とする．

4.2　フーリエ級数

周期 T をもつ関数 $x(t)$ （$x(t+mT)=x(t)$、m: 整数、以下 $x(t)$ は実数値関数とする）を、基本角周波数 $2\pi/T$ の整数倍の角周波数をもつ三角関数の系列、

$$\cos\left(\frac{2\pi k}{T}t\right) \quad \text{および} \quad \sin\left(\frac{2\pi k}{T}t\right),$$

$(k=1,2,3,\ldots)$ の重ね合わせ（足し合わせ）で表すことをフーリエ級数展開とよぶ．そこで、次のような関数 $x_M(t)$ を考えてみよう．

$$x_M(t) \equiv \frac{a_0}{2} + \sum_{k=1}^{M}\left[a_k\cos\left(\frac{2\pi k}{T}t\right) + b_k\sin\left(\frac{2\pi k}{T}t\right)\right] \tag{4.1}$$

係数 a_k と b_k（k 次のフーリエ係数）は、$x_M(t)=x(t)$ の場合を考えることにより決める．すなわち、両辺に $\cos(2\pi kt/T)$ または $\sin(2\pi kt/T)$ を掛けて積分し、関係式

$$\frac{1}{T}\int_0^T \cos\left(\frac{2\pi k}{T}t\right)\cos\left(\frac{2\pi m}{T}t\right)\,dt = \frac{1}{2}\delta_{k,m},$$
$$\frac{1}{T}\int_0^T \sin\left(\frac{2\pi k}{T}t\right)\sin\left(\frac{2\pi m}{T}t\right)\,dt = \frac{1}{2}\delta_{k,m},$$
$$\frac{1}{T}\int_0^T \cos\left(\frac{2\pi k}{T}t\right)\sin\left(\frac{2\pi m}{T}t\right)\,dt = 0$$

（ただし、$\delta_{k,m}$ はクロネッカーのデルタ）を用いると、次のように決まる．

$$a_0 = \frac{2}{T}\int_0^T x(t)\,dt, \tag{4.2}$$
$$a_k = \frac{2}{T}\int_0^T x(t)\cos\left(\frac{2\pi k}{T}t\right)\,dt, \tag{4.3}$$
$$b_k = \frac{2}{T}\int_0^T x(t)\sin\left(\frac{2\pi k}{T}t\right)\,dt \tag{4.4}$$

上のように、$x(t)$ が周期 $\frac{T}{k}$（$k=1,2,\cdots,M$）の三角関数を足し合わせたものであれば、$x_M(t)$ で $x(t)$ を表すことができる．しかし、任意の周期関数 $x(t)$ について、M を大きくすれば $x_M(t)$ が $x(t)$ に近づくかどうかは、$x(t)$ の性質に依存する [1]．結論だけをいうと、次のことが知られている．

> I. $x(t)$ と $x'(t)\equiv\frac{d}{dt}x(t)$ が（ある有限個の点を除いて）区分的に連続であり、かつ、不連続な点（代表として t_0 とする）で左極限と右極限が存在すれば、$x_M(t)$ は $M\to\infty$ において、$x(t)$ が連続な各点 t で $x(t)$ に収束する．すなわち、t が $x(t)$ の連続点であるとき、
>
> $$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty}\left[a_k\cos\left(\frac{2\pi k}{T}t\right) + b_k\sin\left(\frac{2\pi k}{T}t\right)\right] \tag{4.5}$$
>
> が成り立つ．ただし、a_0、a_k および b_k は、式 (4.2)–(4.4) で定義される．

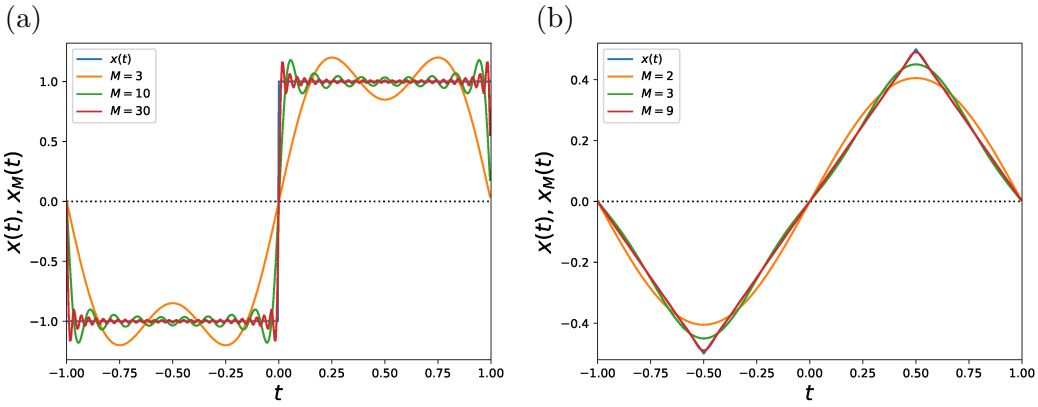


図 4.1: 矩形波 (a) と三角波 (b) に対する $x_M(t)$ の収束の様子

一方、不連続点 t_0 においては、$x(t)$ の左右極限の平均を

$$\overline{x}(t_0) \equiv \frac{x(t_0-0)+x(t_0+0)}{2} \tag{4.6}$$

と定義すると、$x_M(t_0)$ は $M \to \infty$ で $\overline{x}(t_0)$ に収束する．すなわち、

$$\lim_{M\to\infty} x_M(t_0) = \overline{x}(t_0) \tag{4.7}$$

また、次のことも知られている．

II. $x(t)$ がリプシッツ（Lipschitz）連続、すなわち、ある実数 $L > 0$ が存在し、任意の実数 s と t について

$$|x(t) - x(s)| \le L\,|t-s| \tag{4.8}$$

が成り立つ場合は、$x_M(t)$ は $M \to \infty$ で $x(t)$ に一様収束する．

I と II の具体例として、
(a) 矩形波

$$x(t) = \begin{cases} 1, & 0 < t < 1, \\ -1, & -1 < t < 0 \end{cases}, \tag{4.9}$$

(b) 三角波

$$x(t) = \begin{cases} t, & -\frac{1}{2} < t < \frac{1}{2}, \\ 1-t, & \frac{1}{2} \le t < \frac{3}{2}. \end{cases} \tag{4.10}$$

を考える．ただし、いずれも示された領域以外では、$x(t+2m) = x(t)$ (m は整数) を満たす周期 2 の周期関数とする．これらのフーリエ係数は、それぞれ、

$$\text{(a)}\quad a_k = b_{2m+2} = 0,\ b_{2m+1} = \frac{4}{\pi(2m+1)},$$

$$\text{(b)}\quad a_k = b_{2m+2} = 0,\ b_{2m+1} = \frac{4(-1)^m}{\pi^2(2m+1)^2},$$

となる．ただし、$k = 0, 1, 2, \ldots$ および $m = 0, 1, 2, \ldots$ である．

図 4.1 に $x(t)$ と

$$x_M(t) = \sum_{m=0}^{(M-1)/2} b_{2m+1} \sin[\pi(2m+1)t]$$

を示す．ここでは、M を奇数としている．

(a) の矩形波の場合をみると、不連続点 $t = \cdots, -1, 0, 1, \cdots$ 以外の領域では、M を大きくすると、ゆっくりとではあるが $x_M(t)$ は $x(t)$ に近づいている．しかし、不連続点の近傍には、振幅が跳ね上がるような振動部分が残っている．この上下へのはみ出しは M を増やしても無くならないことが知られており、ギブス（Gibbs）現象と呼ばれている [2]．

一般に、$x(t)$ の不連続点 t_0 における飛びを

$$x(t_0+0) - x(t_0-0) = a$$

とすると、M を十分大きくしたとき、不連続点の近傍における $x_M(t)$ は $\overline{x}(t_0)$ を基準として振動

し、次のように振る舞う [1, 2].

$$\sup_{t\uparrow t_0}|x_M(t)-x(t_0-0)|\simeq 0.08949\,|a|,$$
$$\sup_{t\downarrow t_0}|x_M(t)-x(t_0+0)|\simeq 0.08949\,|a|. \quad (4.11)$$

ただし、ここでの sup(上限) は、t_0 の十分近傍における t について取るものとし、$t\uparrow t_0$ および $t\downarrow t_0$ は、それぞれ $t<t_0$ および $t>t_0$ を保ちながら t が t_0 に近づくことを表す.

一方、図 4.1(b) の三角波の場合をみると、矩形波の場合に比べて、格段に速く収束している様子が見える.

物理現象（音波や電磁波など）で現れる物理量の時間変動や空間変動を表す関数は、多くの場合、上で述べた I または II の条件を満たす. したがって、不連続点におけるギブス現象などに注意すれば、多くの物理量の変動をフーリエ級数で表すことができると考えてよい.

4.3 フーリエ変換

フーリエ変換とは、フーリエ級数展開 (4.1) におけるフーリエ係数を決める公式 (4.2)–(4.4) に対応するものである. フーリエ級数展開 (4.1) では、周期が有限である関数を対象としているが、フーリエ変換では、一般の関数も対象に含まれる.

ここでは、フーリエ変換を簡潔に定義するため、まず周期 T をもつ関数 $x(t)$ を考え、$x(t)$ は II の性質（リプシッツ連続性）を満たすとする. すなわち、式 (4.1) において $M\to\infty$ としたとき、$x_M(t)$ は $x(t)$ に一様収束するとする. したがって、$x(t)$ は複素フーリエ級数を用いて次のように表すことができる.

$$x(t)=\frac{1}{T}\sum_{k=-\infty}^{\infty}X_k\exp\left(i\frac{2\pi k}{T}t\right) \quad (4.12)$$

ここで、$i\equiv\sqrt{-1}$、X_k は複素数（複素フーリエ係数）である. 式 (4.12) は、オイラーの公式

$$e^{it}=\cos t+i\sin t$$

を用いると、式 (4.1) のような表現に帰着される. 因子 $1/T$ を付けた理由は、後の便宜のためである. ここで、

$$\int_{-T/2}^{T/2}\exp\left[i\frac{2\pi(k-l)}{T}t\right]dt=T\delta_{k,l}$$

を用いると、X_k は次のように表される.

$$X_k=\int_{-T/2}^{T/2}x(t)\exp\left(-i\frac{2\pi k}{T}t\right)dt \quad (4.13)$$

変数 t を時刻と見なすと、式 (4.12)、(4.13) に現れた $\frac{2\pi k}{T}$ は角振動数（角周波数、角速度）、$\frac{k}{T}$ は周波数（振動数）である. T は基本周期とも呼ばれる. 次に、式 (4.12)、(4.13) において T を無限大にすることを考える. 周波数を

$$f\equiv\frac{k}{T}$$

とおくと、隣り合う周波数の間隔は $\Delta f=\frac{1}{T}$ である. したがって、$T\to\infty$ において $\Delta f\to df$ とみなし、X_k を周波数 f の関数 $X(f)$ とみなすと、式 (4.13)、(4.12) はそれぞれ次のような積分に書き換えることができる.

$$X(f)=\int_{-\infty}^{\infty}x(t)e^{-2\pi ift}\,dt, \quad (4.14)$$
$$x(t)=\int_{-\infty}^{\infty}X(f)e^{2\pi ift}\,df \quad (4.15)$$

式 (4.14) で定義される $X(f)$ をフーリエ変換、式 (4.15) を逆フーリエ変換と呼ぶ.

周波数 f の代わりに角振動数 $\omega\equiv 2\pi f$ を用いる場合、上の 2 式は次のように表現される.[1]

$$\widetilde{X}(\omega)=\int_{-\infty}^{\infty}x(t)e^{-i\omega t}\,dt, \quad (4.16)$$
$$x(t)=\frac{1}{2\pi}\int_{-\infty}^{\infty}\widetilde{X}(\omega)e^{i\omega t}\,d\omega \quad (4.17)$$

[1] $x(t)$ が周期関数の場合、式 (4.14) あるいは式 (4.16) の積分は通常の意味では収束しない. この場合、ある非常に小さな実数 $\epsilon>0$ を用いて、

$$\widetilde{X}(\omega)=\int_{-\infty}^{\infty}x(t)e^{-\epsilon|t|-i\omega t}\,dt$$

と一旦定義し、最後に $\epsilon\to 0$ とするような方法も用いられる. 数学的に厳密には、デルタ関数を含む超関数（分布、例えば参考文献 [1] の 6 章参照）を用いて取り扱うことができる. このような取り扱いについては、後で cos 関数のフーリエ変換を考える際に改めて触れる.

フーリエ変換を理論的に用いる場合は、角振動数を用いた式 (4.16)、(4.17) がよく使われる．しかし、次節以降で取り扱う離散フーリエ変換のように実用的に取り扱う場合は、周波数を用いた表現 (4.14)、(4.15) が便利な場合が多い．

4.4 離散フーリエ変換 (DFT)

連続時間 t に関して周期的あるいは非周期的に、ある有限な範囲で変動する原信号 $x(t)$ があるとする．$x(t)$ を離散的な時点で標本抽出（サンプリング）し、得られた離散数列を時系列データ、あるいは単にデータと呼ぶ．

有限個の離散データ列に対するフーリエ変換は、離散フーリエ変換（Discrete Fourier Transform、略して DFT）と呼ばれる．以下では、間隔 τ の離散時刻 $n\tau$（$n = 0, 1, 2, \cdots, N-1$）における $x(n\tau)$ の値からなるデータ $\{x_n\}_{n=0}^{N-1}$ について、その離散フーリエ変換を考える．ここで、

$$x_n \equiv x(n\tau)$$

と定義する．τ は標本化（サンプリング）周期、$1/\tau$ は標本化（サンプリング）周波数と呼ばれる．また、N をデータ長と呼ぶことにする．

N 個の値から成るデータ $\{x_n\}_{n=0}^{N-1}$ の離散フーリエ変換（DFT）およびその逆変換は、次のように定義される．

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi kn}{N}}, \tag{4.18}$$

$$x_n = \frac{1}{N}\sum_{k=0}^{N-1} X_k e^{i\frac{2\pi kn}{N}}. \tag{4.19}$$

ここで、$n, k = 0, 1, \cdots, N-1$ であり、X_k は k 番目のフーリエ係数 (フーリエ成分) である．

まず、フーリエ変換 (4.14) および逆変換 (4.15) と対応する DFT(4.18) およびその逆変換 (4.19) とは、どのような関係にあるのか、また、どのような条件で、式 (4.18)、(4.19) が式 (4.14)、(4.15) の良い近似になるのか、といったことについて考察しよう．

式 (4.19) を n の任意の整数に対して考えると、

$$x_{n+mN} = x_n \quad (m : \text{整数})$$

が成り立つ．すなわち、x_n は基本周期 N をもつ周期的な数列とみなすことができる．同様に、式 (4.18) より、

$$X_{k+mN} = X_k \quad (m : \text{整数})$$

が成り立ち、X_k も基本周期 N をもつことがわかる．

一般的には、これらの周期性は元々の $x(t)$ および $X(f)$ にはなかった性質である．前者は、データ長を有限にしたこと、後者は時間を離散化したことに、それぞれ関係している．これらの詳細については、以下で示す．

初めに式 (4.14) と式 (4.18) との対応関係を見てみよう．式 (4.14) について、その積分を離散時刻に関する和で近似し、周波数 f を離散的な周波数

$$f_k = \frac{k}{N\tau}$$

に限定すると、DFT で扱うデータの時間区間が有限であることを考慮して、

$$X(f_k) \simeq \tau \sum_{n=-\infty}^{\infty} w(n\tau)x(n\tau)e^{-i\frac{2\pi kn}{N}} \tag{4.20}$$

と書くことができる．ここで、$w(t)$ は、$t < 0$ および $t \geq N\tau$（$N\tau \equiv T$）で $w(t) = 0$ となる関数である．つまり、$w(t)x(t)$ は $0 \leq t < T$ の範囲の $x(t)$ を切り取ったものであり、$w(t)$ を窓関数と呼ぶ．特に、$0 \leq t < T$ で $w(t) = 1$、それ以外で 0 となるものは矩形窓と呼ばれる．ここで、

$$X_k \equiv \frac{X(f_k)}{\tau}$$

と定義すると、式 (4.20) は

$$X_k \simeq \sum_{n=0}^{N-1} w(n\tau)x(n\tau)e^{-i\frac{2\pi kn}{N}} \tag{4.21}$$

となる．さらに、

$$x_n \equiv w(n\tau)x(n\tau)$$

とおけば、式 (4.18) と対応する形

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi kn}{N}} \tag{4.22}$$

が得られる．式 (4.19) での x_n は、このようにして得られたフーリエ係数 X_k を用いて、周期 N の数列として拡張されている．すなわち、

$$x_{n+mN} = w(n\tau)x(n\tau), \tag{4.23}$$

$(0 \le n < N$、m：整数) である．

$x(t)$ が周期 T の関数である場合、$w(t)$ は $0 \le t < T$ の間だけで 1 となる矩形窓でよい．一方、$x(t)$ が周期関数でない場合、あるいは基本周期が未知である場合には、一般に $x(0)$ と $x(T)$ は一致しない．このとき、式 (4.23) による周期的な拡張では、x_{N-1} と $x_N = x_0$ の間に不連続性が生じる．フーリエ級数における不連続性の影響は前に述べたとおりであり、この不連続性はフーリエ変換の結果にも影響を及ぼす．

そこで、一般の $x(t)$ を扱う場合には、x_{N-1} と $x_N = x_0$ ができるだけ滑らかにつながるように、矩形窓以外の窓関数を用いることがある．例えば、$0 \le w(t) \le 1$、$w(T/2) = 1$、$t = T/2$ に関して対称であり、$t = 0$ および T で連続的に 0 となるような連続関数が考えられる．窓関数の具体例については §4.6 で示す．

次に時間を離散化したことに起因する問題を取り扱う．式 (4.15) を時刻と周波数に関して離散化すると、

$$x(n\tau) = \frac{1}{\tau N}\sum_{p=-\infty}^{\infty} X\left(\frac{p}{\tau N}\right) e^{i\frac{2\pi pn}{N}}$$
$$= \frac{1}{N}\sum_{k=0}^{N-1}\left[\frac{1}{\tau}\sum_{m=-\infty}^{\infty} X\left(\frac{k+mN}{\tau N}\right)\right] e^{i\frac{2\pi kn}{N}}$$

$(\because p = k + mN)$ より、$x(n\tau) = x_n$、

$$X_k = \sum_{m=-\infty}^{\infty} \frac{1}{\tau} X\left(\frac{k+mN}{\tau N}\right) \tag{4.24}$$

とおくと、式 (4.15) と式 (4.19) は対応付けられる．

式 (4.24) では、上に述べた周期性 $X_{k+mN} = X_k$ (m: 整数) が満たされている．X_k の 1 周期に対する k の定義域を

$$-\frac{N}{2} \le k < \frac{N}{2}$$

ととる．この範囲において、

$$X_k \approx \frac{1}{\tau} X\left(\frac{k}{\tau N}\right)$$

が成り立つならば、DFT によるフーリエ係数 X_k は、元のフーリエ変換 $X(f)$ を離散周波数 $f = k/(\tau N)$ 上でよく近似していると言える．しかし、一般には式 (4.24) の $m \neq 0$ の項も寄与するため、この近似が成り立つとは限らない．

そこで、次の量

$$\Delta X_k \equiv \frac{1}{\tau} X\left(\frac{k}{\tau N}\right) - X_k \tag{4.25}$$

$\left(-\frac{N}{2} \le k < \frac{N}{2}\right)$ を考える．式 (4.24) より、

$$\Delta X_k = -\frac{1}{\tau}\sum_{m\neq 0} X\left(\frac{k+mN}{\tau N}\right) \tag{4.26}$$

と書くことができる．すなわち、ΔX_k は、時間を離散化したことによって、本来の周波数 $f = k/(\tau N)(-\frac{N}{2} \le k < \frac{N}{2})$ 以外の周波数成分が X_k に重なったことによる誤差を表しており、エイリアシング（Aliasing）誤差（折り返し雑音）と呼ばれる [3].

エイリアシング誤差が無くなるような $X(f)$ および τ の条件について考察しよう．式 (4.26) において、$\Delta X_k = 0$ の場合、$X(f)$ は

$$\sum_{m\neq 0} X\left(\frac{k+mN}{\tau N}\right) = 0 \tag{4.27}$$

$\left(-\frac{N}{2} \le k < \frac{N}{2}\right)$ を満たさなければならない．そこで、$X(f)$ は $|f| > f_m$ で $X(f) = 0$ であると仮定しよう．このとき、式 (4.27) において $m \neq 0$ の項がすべて 0 となるためには、

$$f_m < \frac{1}{2\tau} \equiv f_c$$

が満たされればよいことがわかる [3]．f_c はナイキスト周波数（Nyquist frequency）と呼ばれる．

この結果は、標本化（サンプリング）定理と呼ばれるものと同じ内容である．標本化定理とは、

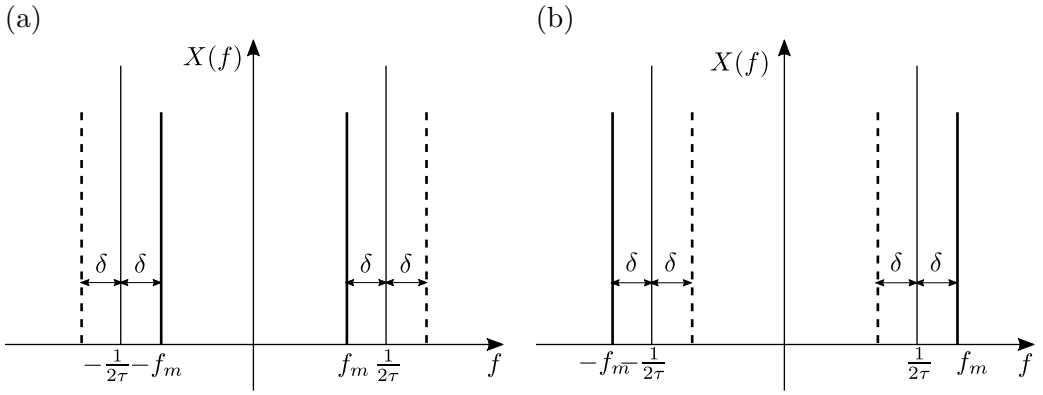


図 4.2: $x(t) = \cos(2\pi f_m t)$ のフーリエ変換と DFT によるフーリエ成分（概念図）．(a) $f_m = f_c - \delta$ および (b) $f_m = f_c + \delta$（$0 < \delta < f_c$）の場合を示す．ただし、$f_c = \frac{1}{2\tau}$ はナイキスト周波数である．太線は $x(t)$ のフーリエ変換（周波数 f_m および $-f_m$ における線スペクトル）を表す．波線は DFT の周期性によって現れる、$f = f_m - \frac{1}{\tau}$ および $f = -f_m + \frac{1}{\tau}$ に対応するフーリエ成分の実部を表す．フーリエ変換と DFT によるフーリエ係数は式 (4.24) によって関係づけられ、ここでは、各 f に近い離散周波数 $\frac{k}{\tau N} \approx f$ を仮定している．

ある原信号 $x(t)$ があり、そのフーリエ変換 $X(f)$ が $|f| > f_m$ で 0 となるとき、$x(t)$ を $\frac{1}{2f_m}$ より小さい標本化周期で標本化すれば、標本化された時系列データから原信号 $x(t)$ を完全に復元できるというものである．

具体例として、$x(t) = \cos(2\pi f_m t)$ の DFT を考えよう．図 4.2 には、(a) $f_m = f_c - \delta$ と (b) $f_m = f_c + \delta$ ($f_c = \frac{1}{2\tau}$、τ は標本化周期、$0 < \delta < f_c$) の場合について、DFT によるフーリエ係数と $x(t)$ のフーリエ変換 $X(f)$ を模式的に示している．$X(f)$ は、周波数 f_m および $-f_m$ に集中した線スペクトル (太い実線) [2] となる．

標本化された離散数列 $\{x(\tau n)\}_{n=0}^{N-1}$ について、$f_m = f_c + \delta$ の場合を考えると、

$$x(\tau n) = \cos\left[2\pi\left(\frac{1}{2\tau} + \delta\right) n\tau\right]$$

となる．DFT のフーリエ係数が周波数に関して周期 $\frac{1}{\tau}$ をもつことから、$x(\tau n)$ は、

$$x(\tau n) = \cos\left[2\pi\left(\frac{1}{2\tau} + \delta - \frac{1}{\tau}\right) n\tau\right] \qquad (4.28)$$

とも書くことができる．実際、

$$e^{2\pi i(f-1/\tau)n\tau} = e^{2\pi i f n\tau} e^{-2\pi i n} = e^{2\pi i f n\tau}$$

である．したがって、$\cos t = (e^{it} + e^{-it})/2$ より、cos 関数を指数関数に直して考えると、周波数 $f_m = \frac{1}{2\tau} + \delta$ と $-f_m = -\frac{1}{2\tau} - \delta$ の線スペクトルは、それぞれ、

$$f_m - \frac{1}{\tau} = -\left(\frac{1}{2\tau} - \delta\right), \quad -f_m + \frac{1}{\tau} = \frac{1}{2\tau} - \delta$$

に現れる．すなわち、周波数 f_m と $-f_m$ の線スペクトルは、ナイキスト周波数 $f_c = \frac{1}{2\tau}$ を境として折り返され、それぞれ $-(f_c - \delta)$ と $f_c - \delta$ に現れる．

(a) の場合、図の破線は DFT の周期性（周波数に関して周期 $\frac{1}{\tau}$ である）による $f_m - \frac{1}{\tau}$ および $-f_m + \frac{1}{\tau}$ の線スペクトルである．$|f| < f_c$ の範囲では、フーリエ変換と DFT によるフーリエ成分の位置は一致する．

(b) の場合、フーリエ変換の線スペクトルは $(-f_c, f_c)$ の外側にある．これらが DFT の周期性

[2] $2\pi f_m \equiv \Omega$ として、$x(t) = \cos(\Omega t)$ のフーリエ変換を

$$\widetilde{X}(\omega) = \lim_{\epsilon \to +0} \int_{-\infty}^{\infty} \cos(\Omega t) e^{-\epsilon|t| - i\omega t}\, dt$$

($\omega \equiv 2\pi f$) より計算すると、

$$\widetilde{X}(\omega) = \lim_{\epsilon \to +0} \left[\frac{\epsilon}{(\omega - \Omega)^2 + \epsilon^2} + \frac{\epsilon}{(\omega + \Omega)^2 + \epsilon^2}\right]$$

となる．すなわち、$\omega = \pm\Omega$ （$f = \pm f_m$）に集中したデルタ関数であり、ここではこれを線スペクトルと呼んでいる．因子 $e^{-\epsilon|t|}$ は窓関数と見なすことができる．

によって $(-f_c, f_c)$ 内に現れたものが、$f_m - \frac{1}{\tau}$ および $-f_m + \frac{1}{\tau}$ の破線である．これは、上で述べたように、$-f_m$ および f_m をそれぞれ $-\frac{1}{2\tau}$ および $\frac{1}{2\tau}$ の軸に関して折り返したものと見ることができる．

ここでは、cos 関数を用いて DFT によるフーリエ係数の実部を見たが、§4.6 で述べるパワースペクトル密度を用いれば、sin 関数や位相をずらした $\cos(2\pi f_m t + \phi_0)$ についても同じことが言える．

以上のことから、原信号 $x(t)$ の標本化に際して、$x(t)$ のフーリエ変換 $X(f)$ が $|f| > f_m$ の範囲で無視できるほど小さく、標本化周期を $\frac{1}{2f_m}$ より十分小さくとることができ、かつ、$x(t)$ の低周波数領域における（興味の対象となる）特徴を損なわないような十分長い時間にわたって標本を採取すれば、標本化された信号は原信号を十分よく表していると考えてよいだろう．

4.5 高速フーリエ変換 (FFT)

高速フーリエ変換 (Fast Fourier Transform) は、DFT の計算を高速に行うために考案されたアルゴリズムであり、以下ではこれを FFT と呼ぶ．まず、その原理を解説しよう．

式 (4.18) を簡潔に表現するため、

$$W_N \equiv e^{-i\frac{2\pi}{N}} \tag{4.29}$$

という記号を導入し、式 (4.18) を

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{kn} \tag{4.30}$$

と書き表す．x_n を N 次元ベクトルと見ると、W_N^{kn} は DFT 行列の (k, n) 成分と考えることができる．

まず、W_N^{kn} の周期的性質を利用せず、W_N^{kn} を互いに独立な行列要素として扱って DFT を計算した場合の計算量について考えてみよう．式 (4.30) における複素数乗算の回数は N^2 と見積もることができる．これは、データの長さ N とともに計算量が N^2 に比例して増加することを意味する．実際に計算機で DFT の計算を行うと、入力するデータが大きくなるにつれて、計算時間が急速に長くなることが分かる．

これに対して FFT では、W_N^{kn} のもつ周期的な性質や規則性を利用することで、乗算回数を $N \log_2 N$ 程度まで減らすことができる．その結果、DFT を直接計算する場合に比べて、計算に要する時間を著しく短縮できることが後で分かる．

FFT は 1965 年に J. W. Cooley と J. W. Tukey が発表した形式のものが一般によく知られているが、その約 20 年前にも、同様の方法を独立に発見した研究者がいた [4, 5]．FFT の初期の発見の一つである Danielson–Lanczos の補題もその一つであり、FFT の本質を示している．Danielson–Lanczos の補題では、

$$W_N^{2kn} = W_{N/2}^{kn} = e^{-i\frac{2\pi kn}{N/2}}$$

という性質を利用することで、長さ N の DFT を長さ $\frac{N}{2}$ の二つの DFT に分解できることを示している．具体的に説明しよう．

N が偶数であるとして、式 (4.30) の和の中の n を偶数 $\{2m \mid m = 0, \cdots, \frac{N}{2} - 1\}$ と奇数 $\{2m+1 \mid m = 0, \cdots, \frac{N}{2} - 1\}$ の場合に分けると、

$$\begin{aligned} X_k &= \sum_{m=0}^{\frac{N}{2}-1} x_{2m} W_N^{2mk} + \sum_{m=0}^{\frac{N}{2}-1} x_{2m+1} W_N^{(2m+1)k} \\ &= \sum_{m=0}^{\frac{N}{2}-1} x_{2m} W_{N/2}^{mk} + W_N^k \sum_{m=0}^{\frac{N}{2}-1} x_{2m+1} W_{N/2}^{mk} \\ &= X_k^{(0)} + W_N^k X_k^{(1)} \end{aligned} \tag{4.31}$$

ここで、[3]

$$X_k^{(\alpha)} \equiv \sum_{n=0}^{\frac{N}{2}-1} x_{2n+\alpha} W_{N/2}^{kn}, \qquad \alpha \in \{0, 1\} \tag{4.32}$$

は、$\frac{N}{2}$ 個の偶数番目（$\alpha = 0$）または奇数番目（$\alpha = 1$）のデータに対する DFT である．また、$X_k^{(\alpha)}$ は k に関して $\frac{N}{2}$ を周期とする．すなわち、

$$X_{k+\frac{N}{2}}^{(\alpha)} = X_k^{(\alpha)}$$

が成り立つ．

$N = 2^K$ だとすると、式 (4.31) の手続きを再帰的に実行できる．式 (4.32) の $X_k^{(\alpha)}$ に対して式

[3] 上付き添字の “$(\cdot)$” は、冪乗との混同を避けるための記法である．

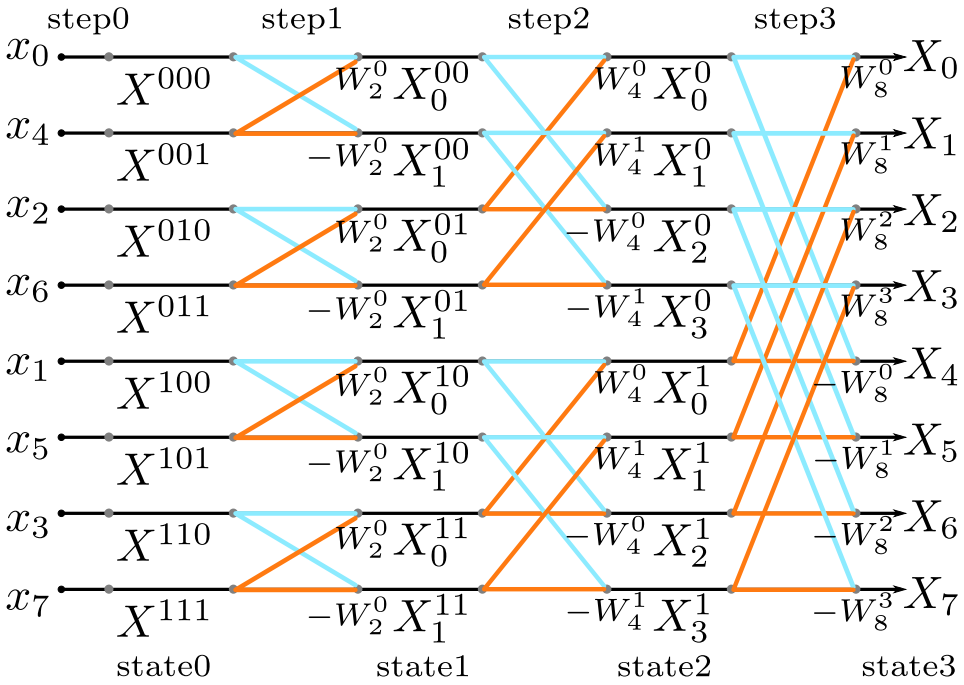


図 4.3: FFT($N = 8$) のバタフライ演算．本文中の $X_l^{(\alpha\beta\cdots\chi)}$ は、図中では $X_l^{\alpha\beta\cdots\chi}$ と表示されている．$W_2^0 = 1$．$W_N^j = e^{-i\frac{2\pi j}{N}}$．

(4.31) の手続きを用いると、$X_k^{(\alpha)}$ を長さ $\frac{N}{4}$ の二つの DFT の和で表すことができる．具体的には、

$$\begin{aligned} X_k^{(\alpha)} &= \sum_{m=0}^{\frac{N}{4}-1} x_{2(2m)+\alpha} W_{N/2}^{2mk} \\ &\quad + \sum_{m=0}^{\frac{N}{4}-1} x_{2(2m+1)+\alpha} W_{N/2}^{(2m+1)k} \\ &= \sum_{m=0}^{\frac{N}{4}-1} x_{2^2m+\alpha} W_{N/4}^{mk} \\ &\quad + W_{N/2}^k \sum_{m=0}^{\frac{N}{4}-1} x_{2^2m+2+\alpha} W_{N/4}^{mk} \\ &= X_k^{(\alpha 0)} + W_{N/2}^k X_k^{(\alpha 1)} \end{aligned} \tag{4.33}$$

となる．ここで、

$$X_k^{(\alpha\beta)} \equiv \sum_{n=0}^{\frac{N}{4}-1} x_{2^2n+2\beta+\alpha} W_{N/4}^{kn}, \qquad \alpha, \beta \in \{0, 1\} \tag{4.34}$$

は、長さ $\frac{N}{4}$ のデータ列に対する DFT である．ただし、このデータ列は元のデータ列から 4 点おきに取り出したものである．$X_k^{(\alpha\beta)}$ は、周波数 k に関して $\frac{N}{4}$ を周期とする．すなわち、

$$X_{k+\frac{N}{4}}^{(\alpha\beta)} = X_k^{(\alpha\beta)}$$

である．

この手続きは、長さ 1 の DFT になるまで繰り返すことができる．すなわち、

$$X_k^{(\alpha\beta\cdots\eta\zeta)} = x_{2^{K-1}\zeta+2^{K-2}\eta+\cdots+2\beta+\alpha} \tag{4.35}$$

となるまで分解できる．このとき、$X_k^{(\alpha\beta\cdots\eta\zeta)}$ は周波数 k に関して 1 周期であるため、便宜的に

$$X_k^{(\alpha\beta\cdots\eta\zeta)} = X_0^{(\alpha\beta\cdots\eta\zeta)}$$

とおく．ここで、添字の () の中に現れる $\alpha\beta\cdots\eta\zeta$ と、元のデータ列の添字 n との関係を考えよう．n を K 桁の 2 進数で

$$n = (\zeta\eta\cdots\beta\alpha)_2$$

と表すと、$\alpha\beta\cdots\eta\zeta$ は、この 2 進数のビット列を反転したものになっている．したがって、

$$x_{(\zeta\eta\cdots\beta\alpha)_2} \quad と \quad X_0^{(\alpha\beta\cdots\eta\zeta)}$$

は一対一に対応している．

FFT のアルゴリズムの一つである時間引き FFT アルゴリズムは、最後に示した式 (4.35) から開始して、式 (4.33)、式 (4.31) の手続きを逆に遡るものである．これを図 4.3 の $K = 3$ ($N = 8$) の例を見ながら説明する．

まず、N（$= 2^K$）個の等間隔に並んだ時点におけるデータ $\{x_n\}_{n=0}^{N-1}$ が、一次元配列（0 から $N-1$ までのアドレスを割り振られた記憶領域）に読み込まれたとしよう．

ステップ 0　$\{x_n\}_{n=0}^{N-1}$ の各時刻 n について、K 桁の 2 進数表現

$$n = (\zeta\eta\cdots\beta\alpha)_2$$

を求め、そのビット列を反転した数

$$(\alpha\beta\cdots\eta\zeta)_2 \tag{4.36}$$

をアドレスとする $X^{(\alpha\beta\cdots\eta\zeta)}$ $(= X_0^{(\alpha\beta\cdots\eta\zeta)})$ の配列に x_n を代入する[4] (図 4.3 の左端と state 0 の列)．

ステップ k　$1 \le k < K$ として、長さ 2^{k-1} の DFT の組から、長さ 2^k の DFT を構成する．すなわち、state $k-1$ の 2^{K-k} 組の DFT から、state k の 2^{K-k} 組の DFT を作る：

$$X_l^{(\alpha\beta\cdots\chi)} = X_l^{(\alpha\beta\cdots\chi 0)} + W_{2^k}^l X_l^{(\alpha\beta\cdots\chi 1)}, \tag{4.37}$$

$(0 \le l < 2^k)$．ここで、χ は、式 (4.36) に示したビット反転後の K 桁のビット列 $\alpha\beta\cdots\chi\cdots\eta\zeta$ において、下から k 桁目に対応するビットである．式 (4.33) および式 (4.31) は、それぞれ $k = K-1$ および $k = K$ の場合に対応する．$X_l^{(\alpha\beta\cdots\chi 0)}$ と $X_l^{(\alpha\beta\cdots\chi 1)}$ は、l に関して 2^{k-1} を周期とする．また、state k の $X_l^{(\alpha\beta\cdots\chi)}$ を格納する配列のアドレスは、$(\alpha\beta\cdots\chi)_2\, 2^k + l$ となる．したがって、$X_l^{(\alpha\beta\cdots\chi 0)}$ と $X_l^{(\alpha\beta\cdots\chi 1)}$ の配列上のアドレスの差は 2^{k-1} である．そこで、式 (4.37) を $0 \le l < 2^{k-1}$ と $2^{k-1} \le l < 2^k$ とに分けて書くと、

$$\begin{aligned} X_g^{(\alpha\beta\cdots\chi)} &= X_g^{(\alpha\beta\cdots\chi 0)} + W_{2^k}^g X_g^{(\alpha\beta\cdots\chi 1)}, \\ X_{g+2^{k-1}}^{(\alpha\beta\cdots\chi)} &= X_g^{(\alpha\beta\cdots\chi 0)} - W_{2^k}^g X_g^{(\alpha\beta\cdots\chi 1)}, \end{aligned} \tag{4.38}$$

$(0 \le g < 2^{k-1})$．図では、色の異なる実線で $X_g^{(\alpha\beta\cdots\chi 0)}$ と $X_g^{(\alpha\beta\cdots\chi 1)}$ から $X_g^{(\alpha\beta\cdots\chi)}$ と $X_{g+2^{k-1}}^{(\alpha\beta\cdots\chi)}$ を作る演算を示している．この演算はバタフライ演算と呼ばれる．バタフライ演算を基本単位として各ステップの構造を見ると、step 1 では孤立したバタフライ演算が 2^{K-1} 個あり、step 2 では 2 つのバタフライ演算からなる群が 2^{K-2} グループ、step 3 では 4 つのバタフライ演算からなる群が 2^{K-3} グループ、というようになる．一般に、step k では、2^{k-1} 個のバタフライ演算からなる群が 2^{K-k} グループある．アルゴリズムを実装する際の定石の一つは、このような群の左上端位置を順次移動させながら、各群内でバタフライ演算を行うことである．

ステップ K　$k = K$ のとき、長さ $2^K = N$ の DFT が得られ、$\{X_l\}_{l=0}^{N-1}$ が求まる．

ここで、再び計算量について考えよう．一連の計算が終了するまでのステップ数は $K = \log_2 N$ 回である．各ステップでは、$\frac{N}{2}$ 個のバタフライ演算が行われる．式 (4.38) を見ると、各バタフライ演算では、$W_{2^k}^g$ による複素数の乗算が 1 回必要である．ただし、$W_2^0 = 1$ など、W が 1 となる場合にはこの乗算を省略できる．したがって、W による複素数の乗算回数は、おおよそ $\frac{N}{2}\log_2 N$ 回である．より正確には、W が 1 となる乗算を省略するなどの実装上の工夫により、実際の乗算回数はこれより少なくなる．いずれにしても、計算量は $N\log_2 N$ の程度であり、最初に述べた DFT の計算量 N^2 に比べて非常に少ない．

4.6　パワースペクトル密度と窓関数

原信号 $x(t)$ に関して、

$$\lim_{T\to\infty} \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} |x(t)|^2\, dt \tag{4.39}$$

は信号の平均パワーと呼ばれる．これは、式 (4.12) を用いると、適切な条件のもとで、次のように書

[4] $X^{(\alpha\beta\cdots\eta\zeta)}$ は複素数であることに注意．x_n が実数である場合には、$X^{(\alpha\beta\cdots\eta\zeta)}$ の虚部を 0 に初期化しておく．

き換えることができる．

$$\begin{aligned}&\lim_{T\to\infty}\frac{1}{T}\int_{-\frac{T}{2}}^{\frac{T}{2}}|x(t)|^2\,dt\\&=\lim_{T\to\infty}\frac{1}{T^2}\sum_{k=-\infty}^{\infty}|X_k|^2\\&=\int_{-\infty}^{\infty}\lim_{T\to\infty}\frac{1}{T}|X(f)|^2\,df\end{aligned}\tag{4.40}$$

これはパーセバルの定理（Parseval の定理）と呼ばれるものである．また、

$$S(f)\equiv\lim_{T\to\infty}\frac{1}{T}|X(f)|^2\tag{4.41}$$

はパワースペクトル密度と呼ばれる．$S(f)\,df$ は、信号に含まれる周波数 f の近傍の微小な周波数幅 df に含まれるパワーを表す量であり、

$$\int_{f_1}^{f_2}S(f)\,df$$

は、周波数 f_1 から f_2 までの範囲に含まれるパワーを表す．したがって、式 (4.40) は、

$$\lim_{T\to\infty}\frac{1}{T}\int_{-\frac{T}{2}}^{\frac{T}{2}}|x(t)|^2\,dt=\int_{-\infty}^{\infty}S(f)\,df$$

と書くことができる．$x(t)$ が実数のとき、

$$X(-f)=X(f)^*$$

（$*$ は複素共役を表す）であるので、パワースペクトル密度 $S(f)$ は f に関して偶関数である．

離散時刻 $n\tau$ $(n=0,1,\cdots,N-1)$ で標本化された時系列データ $\{x_n\}_{n=0}^{N-1}$ に対する、式 (4.40) に対応する平均パワーの関係式は、式 (4.19) を用いると、次のように得られる．

$$\begin{aligned}\frac{1}{N}\sum_{n=0}^{N-1}|x_n|^2&=\frac{1}{N^2}\sum_{k=0}^{N-1}|X_k|^2\\&\equiv\sum_{k=0}^{N/2}S_k\end{aligned}\tag{4.42}$$

ただし、$\{X_k\}_{k=0}^{N-1}$ は、離散周波数

$$f_k=\frac{k}{\tau N}\qquad(k=0,1,\cdots,N-1)$$

におけるフーリエ係数である．x_n が実数の場合には、$X_{N-k}=X_k^*$ であることから、正の周波数側と負の周波数側の成分をまとめて、S_k を

$$S_k=\begin{cases}\dfrac{|X_0|^2}{N^2}, & k=0,\\[2ex]\dfrac{|X_k|^2+|X_{N-k}|^2}{N^2}, & 1\le k\le\frac{N}{2}-1,\\[2ex]\dfrac{|X_{N/2}|^2}{N^2}, & k=\frac{N}{2}\end{cases}\tag{4.43}$$

と定義できる．ここで、S_k は周波数

$$f_k=\frac{k}{\tau N}\qquad\left(0\le k\le\frac{N}{2}\right)$$

に対応するパワーを表している．

この S_k を用いて、有限長の時系列データからパワースペクトル密度を推定する方法をピリオドグラム (periodogram) という．したがって、S_k は、有限長の離散時系列データから得られるパワースペクトル密度の推定値とみなすことができる．以下では、特に断らない限り、S_k をパワースペクトル密度とも呼ぶことにする．

上で定義した S_k は、窓関数として矩形窓を用いた場合に対応している．一般の窓関数 $w(t)$ を用いて、x_n を

$$x_n=w(n\tau)x(n\tau),\quad n=0,1,\cdots,N-1$$

のように定義した場合には、S_k は次のように定義される [5].

$$S_k=\begin{cases}\dfrac{|X_0|^2}{W_{\rm ss}}, & k=0,\\[2ex]\dfrac{|X_k|^2+|X_{N-k}|^2}{W_{\rm ss}}, & 1\le k\le\frac{N}{2}-1,\\[2ex]\dfrac{|X_{N/2}|^2}{W_{\rm ss}}, & k=\frac{N}{2}.\end{cases}\tag{4.44}$$

ただし、$W_{\rm ss}$ は窓関数の二乗和（window squared and summed）に関係する量であり、

$$W_{\rm ss}\equiv N\sum_{n=0}^{N-1}w_n^2,\quad w_n\equiv w(n\tau)\tag{4.45}$$

とする．一般の $0 \le w(t) \le 1$ の窓関数を掛けると、窓関数を掛けた時系列データの平均パワー

$$\frac{1}{N}\sum_{n=0}^{N-1}|w(n\tau)x(n\tau)|^2$$

は、矩形窓の場合に比べて小さくなる．そこで、窓関数によって生じるこのパワーの減少を補正するため、上の式では窓関数の二乗和に基づく $W_{\rm ss}$ で規格化している．

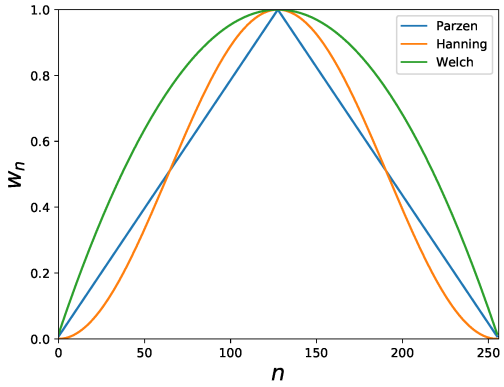


図 4.4: 代表的な窓関数．$N = 256$.

窓関数としてよく用いられるものをいくつか挙げておく（図 4.4）．ここでは、

$$w_n \equiv w(n\tau), \quad n = 0, 1, \cdots, N-1$$

とする．

1. Parzen 窓

$$w_n = 1 - \left|\frac{n - \frac{1}{2}(N-1)}{\frac{1}{2}(N+1)}\right| \tag{4.46}$$

2. Hann 窓

$$w_n = \frac{1}{2}\left[1 - \cos\left(\frac{2\pi n}{N-1}\right)\right] \tag{4.47}$$

3. Welch 窓

$$w_n = 1 - \left(\frac{n - \frac{1}{2}(N-1)}{\frac{1}{2}(N+1)}\right)^2 \tag{4.48}$$

4.7 問題

以下の問題への答案をレポートとして提出せよ．レポートには、それぞれの問題について、問題文（あるいはそれを簡潔にまとめたもの）、答案（計算結果や図など）、および考察を記述すること．

提出するレポートは PDF 形式で保存し、ファイル名を「report 学籍番号.pdf」とせよ．また、問題 2, 3, 5, 6 で作成したプログラムのソースコードは、フォルダー「src 学籍番号」の下に整理して格納し、tar+gz（または zip）で圧縮すること．圧縮したファイルのファイル名は、「Src 学籍番号.tgz」（または「Src 学籍番号.zip」）とすること．

以上の 2 つのファイル（PDF および tgz または zip）を、KLUMS から提出すること．

次の 2 つの周期関数について、以下の問いに答えよ．

$$\text{A:}\quad x(t) = t - [t],$$
$$\text{B:}\quad x(t) = \left|\cos\left(\frac{\pi}{2}t\right)\right|.$$

ただし、$[t]$ はガウス記号であり、n を整数として t が $n \le t < n+1$ を満たすとき、$[t] = n$ とする．

(a) 上の A、B の周期関数をフーリエ級数展開するときの k 次のフーリエ係数 a_k および b_k

$$a_k = \frac{2}{T}\int_{-T/2}^{T/2} x(t)\cos\left(\frac{2\pi k}{T}t\right)\,dt,$$
$$b_k = \frac{2}{T}\int_{-T/2}^{T/2} x(t)\sin\left(\frac{2\pi k}{T}t\right)\,dt$$

を求めよ．ただし、T は $x(t)$ の基本周期である．

(b) それぞれについて、M 次までのフーリエ級数展開

$$x_M(t) = \frac{a_0}{2} + \sum_{k=1}^{M}\left[a_k\cos\left(\frac{2\pi k}{T}t\right) + b_k\sin\left(\frac{2\pi k}{T}t\right)\right]$$

を求め、$x(t)$ とともに図示せよ．M については、$x_M(t)$ が $x(t)$ に近づく様子がわかるように、3 つの場合を適切に選べ．

(c) 上の A、B の周期関数について、M を大きくしていったときの $x_M(t)$ の収束の仕方の違いを比較し、その相違点を述べよ.

問題２ (動作テスト)

次のような時系列データ $\{x_n\}_{n=0}^{7}$:

$$x_0 = 1, \quad x_1 = 1, \quad x_2 = 1, \quad x_3 = 1,$$
$$x_4 = 0, \quad x_5 = 0, \quad x_6 = 0, \quad x_7 = 0$$

が与えられたとき、以下の問に答えよ.

(a) DFT [本文の式 (4.18)] の定義式に従ってフーリエ係数 X_k を計算し、可能な限り厳密な形で求めよ.

(b) FFT を用いてフーリエ係数 X_k を数値的に求めよ. また、その結果 ($X_k^{\rm FFT}$) と (a) の結果 ($X_k^{\rm exact}$) との差から最大誤差

$$\max_k \left| X_k^{\rm FFT} - X_k^{\rm exact} \right|$$

を評価し、誤差の大きさを 10^{-K} のオーダー[5]で表せ. さらに、誤差の原因について述べよ.

問題３ (標本化定理とエイリアシング)

次の信号

$$x(t) = \frac{1}{2}\cos(\pi f_s t) + \frac{1}{4}\sin(2\pi f_s t) + \frac{1}{6}\cos(2\sqrt{3}\pi f_s t) \qquad \text{(P3)}$$

を考えよう. この信号を標本化周期 $\tau = 1/f$ の離散時刻 $t = n\tau$ $(n = 0, 1, \cdots, N-1)$ で標本化する.

$f = 10$、$N = 256$ として、(i) $f_s = f/10$、(ii) $f_s = f/\sqrt{14}$、(iii) $f_s = f/\sqrt{10}$ のそれぞれの場合について、標本化された時系列データ $\{x(n\tau)\}_{n=0}^{N-1}$ からピリオドグラムによりパワースペクトル密度を推定し、図示せよ. また、それぞれの場合について、信号 $x(t)$ のパワースペクトル密度と比較し、エイリアシングによって生じる違いについて説明せよ.

[5]最大の誤差が $x \times 10^{-K}$、$1 \le |x| < 10$、K は整数、であることを意味する.

問題４ (近い周波数の線スペクトルの識別)

二つの近い周波数の正弦波からなる信号:

$$x(t) = \sin\left[2\pi(f-\delta)t\right] + \sin\left[2\pi(f+\delta)t\right] \quad \text{(P4)}$$

$(0 < \delta \ll f)$ があり、これを標本化周期 $\tau = \frac{1}{5f}$ の離散時刻 $t = n\tau$ $(n = 0, 1, \cdots, N-1)$ で標本化したものを考える. 以下の問に答えよ.

	$\delta = 0.2$	$\delta = 0.1$	$\delta = 0.05$
矩形窓			
Parzen 窓			
Hann 窓			

表 4.1: δ および窓関数の組み合わせに対する二つの線スペクトルを識別できる最小データ長 N_c.

(a) 周波数が近い 2 つの正弦波がある場合、うなりと呼ばれる包絡線がゆっくり変動する振動が生じる. 式 (P4) の場合に、式を変形して、うなりの周波数（振幅が最大になる間隔に対応する周波数）を求めよ.

(b) $f = 10$ として、(I) $\delta = 0.2$、(II) $\delta = 0.1$、(III) $\delta = 0.05$、および、窓関数が (i) 矩形窓、(ii) Parzen 窓、(iii) Hann 窓の各々の組み合わせについて、データ長を $N = 256, 512, 1024, 2048, 4096, \cdots$ と変えて標本化した時系列データ $\{x(n\tau)\}_{n=0}^{N-1}$ のパワースペクトル密度を計算せよ. 二つの周波数 $f-\delta$ と $f+\delta$ に対応するピークを区別できると判断した最小のデータ長を N_c とし、表 4.1 の各欄に N_c を書き入れよ. また、N_c を判断した根拠となるパワースペクトル密度の図を示せ.

(c) 前問の結果より N_c と δ についてどの様な関係 (たとえば、比例や反比例など) が予想されるか、解答せよ.

(d) 問 (b) で取り扱った窓関数の中で、近接した周波数の線スペクトルを識別・推定するという目的に、最もかなっていたものはどれか (甲乙つけがたい場合は複数挙げよ)、(パワース

ペクトル密度などの) 図や理由を付けて答えよ．(目的にかなう窓関数は標本化周期のとり方、窓の開始･終了時刻の取り方、「推定の正しさ」の基準などに依存して変わるので、この小問に関しては、正解を設定しない．目的は問題５と問題６で用いる窓関数を決定することである．)

問題５ (太陽表面の黒点に関するデータの解析)

ウォルフ黒点相対数（Wolf number）は、太陽表面の黒点および黒点群の総量を数値化したものである．KLUMS 上の指定された場所（演習期間中に指定する.）からデータ（元のデータは [6] のものを使用）をダウンロードして、下記の設問に答えよ．

(a) データ `SN_y_tot_V2.0.txt` を用いて、横軸を時間（年）として、黒点相対数の時系列を図示せよ．

(b) 月単位での観測データ `SN_m_tot_V2.0.txt` を用いて、データの先頭から 2^K 個（K は整数）のデータを取り出し、そのデータから黒点相対数の変動のパワースペクトル密度を求め、図示せよ．周波数の単位は 1/year とする．窓関数は、問題４ (d) で選んだものを用いよ．

(c) $f > 0$ の範囲でパワースペクトル密度が最大となる周波数から周期を求めよ．周期は年単位で表し、1 年未満は四捨五入せよ．

問題６ (氷期ー間氷期サイクルの解析)

Milankovitch 理論では、地球の気候変動における氷河期と間氷期の到来を、地球の公転軌道要素の周期的変化によって生じる日射量の変化と関係づけている．具体的には、公転軌道における楕円軌道の離心率、地軸傾斜、地軸の歳差運動の周期的変動に関連する数十万年から 100 万年周期の気候変動があるという (発表 : 1920 年代)．理論の発表後しばらくして、1970 年代より、海洋底のボーリング調査により採取されたサンプルに遺された微生物（有孔虫）化石の酸素同位体比[6] から気候変動の周期が測定されるようになり、Milankovitch 理論を支持する結果が報告されるようになってきた．(詳細は、文献やインターネットなどで調べよ．)

KLUMS 上の指定された場所 (演習期間中に指定する．) からデータ (元のデータは [7] のものを使用) をダウンロードして、下記の設問に答えよ．

(a) 横軸を時間 [ka BP] [7] として、酸素同位体比 (δ^{18}O)[‰] の時系列を図示せよ．

(b) データ長が 2^K (K は整数) になるようにデータを切り詰めて、そのデータからパワースペクトル密度を求め結果を図示せよ．[窓関数は、問題４ (d) で答えたものを用いよ．]

(c) パワースペクトル密度に現れる顕著な極大の周波数から、楕円軌道の離心率、地軸傾斜、および地軸の歳差運動に関連するとされる 3 つの代表的な周期を千年単位で推定せよ．ただし、時系列には長期的なトレンド（観測期間全体にわたる緩やかな増加・減少傾向）が含まれているため、周波数が 0 付近に現れるスペクトルの極大は無視すること．

また、データ長を 2^K とする際の窓の開始点を変えて、推定される周期が変化するかどうかを確認せよ．周期が変化する場合には、3〜5 個程度の異なる開始点について周期を求め、その平均値を最終的な推定値として答えよ．

参考文献

[1] 中村周, フーリエ解析. 応用数学基礎講座 4 / 岡部靖憲, 和達三樹, 米谷民明編集. 朝倉書店,

[6] 海水中の水（H_2O）は、酸素原子が ^{16}O の軽い水と、^{18}O の重い水（存在比は小さい）とから構成されている．海水中を漂う石灰質有孔虫は海水を使って石灰質の殻を作り、その殻には水温が低いほど、重い海水が多く取り込まれるので、酸素同位体比 ^{18}O/^{16}O は氷期に大きく、間氷期に小さくなる．

[7] t ka BP は、現在から $t \times 1000$ 年前を意味する．

2003.

[2] 福田礼次郎, フーリエ解析, 理工系の基礎数学 新装版 6, 岩波書店, 2022.

[3] 佐川雅彦, 貴家仁志, 辻井重男. 高速フーリエ変換とその応用. ディジタル信号処理シリーズ / 辻井重男企画・編集, 第 2 巻. 昭晃堂, 1992.

[4] E. Oran Brigham. *The Fast Fourier Transform and Its Applications*. Prentice-Hall, Inc., USA, 1988.

[5] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. ニューメリカルレシピ・イン・シー : C 言語による数値計算のレシピ : 日本語版. 技術評論社, 東京, Japan, 1993. 翻訳 : 丹慶 勝市 奥村 晴彦 佐藤 俊郎 小林 誠.

[6] SILSO World Data Center, Royal Observatory of Belgium, Brussels. The international sunspot number. *International Sunspot Number Monthly Bulletin and online catalogue*, `http://www.sidc.be/silso`.

[7] John D Imbrie, James D Hays, Douglas G Martinson, Andrew McIntyre, Alan C Mix, Joseph J Morley, Nicklas G Pisias, Warren L Prell, and Nicholas J Shackleton (1984). Stacked and smoothed oxygen-isotope record as a function of age in the SPECMAP time scale (SPECMAP-Stack). PANGAEA, `https://doi.org/10.1594/PANGAEA.56063`.

[筒広樹]

第5章　常微分方程式の数値解法

5.1　目的

自然現象や社会現象の数理モデルには，多くの場合で微分方程式が現れる．それゆえ，微分方程式を解くことは数学的な興味だけでなく現象を理解する上でも重要であるが，限られた場合を除いて解析的に解を得ることはできない．そのため，数値解析の必要性が生じる．本章では，微分方程式の中でも，独立変数をひとつしか持たない常微分方程式の数値解析手法について学習する．

独立変数を t，未知変数を $\boldsymbol{u}(t)$ とする．ただし，$\boldsymbol{u}$ は n 次元のベクトルで，$\boldsymbol{u} = (u_1, u_2, ..., u_n)$ とする．物理的には t は時間変数を表すことが多い．次の 1 階の常微分方程式の初期値問題を考える：

$$\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}t} = \boldsymbol{f}(t, \boldsymbol{u}(t)), \quad \boldsymbol{u}(0) = \boldsymbol{u}_0. \tag{5.1}$$

ここで，$\boldsymbol{f} = (f_1, f_2, ..., f_n)$ は既知関数である．特に，$\boldsymbol{f}$ が $\boldsymbol{u}$ に対して線形（あるいは非線形）であるとき，常微分方程式は線形（あるいは非線形）であるという．また，$\boldsymbol{f}$ が t に依存しないとき，常微分方程式は自励系であるという．常微分方程式の初期値問題の解の存在や一意性については本章の範囲を超えるため，興味がある人は参考文献（例えば [1–3]）を参照せよ．表記の簡略化のため，必要に応じて $\mathrm{d}\boldsymbol{u}/\mathrm{d}t$ を $\boldsymbol{u}'$ と表す．

常微分方程式の例を知っておくことは理解の一助となるので，いくつか代表的なものを紹介しておく．なお，下記の二つの例はともに線形方程式である．

例 5.1.1 質点の運動

x 軸上を運動する質量 m の質点を考える．時間変数を t とし，質点の位置と速度をそれぞれ $x(t)$ および $v(t)$ とする．質点には速度に比例する摩擦力 $-cv(t)$ $(c > 0)$ が働く．また，初期位置と初期速度はそれぞれ $x(0) = x_0$ および $v(0) = v_0$ である．既に学習しているように，この質点の運動を表す常微分方程式と初期条件は

$$\begin{cases} x' = v, \quad mv' = -cv, \\ x(0) = x_0, \quad v(0) = v_0 \end{cases}$$

で与えられる．

式 (5.1) の表記に合わせると，$u_1 = x$, $u_2 = v$, $f_1 = v$, $f_2 = -(c/m)v$ となる．一方で，物体の運動方程式は x に対する 2 階の常微分方程式と見ることもできることに注意する．この例から，1 つの n 階の常微分方程式は，n 個の 1 階常微分方程式のシステムに置換できることが分かる．

例 5.1.2 熱方程式

t を時間，x を位置変数とし，空間一次元の偏微分方程式を考える．具体的には，一様な材質からなる長さ L の棒の伝熱問題，すなわち棒の温度 $T(t, x)$ の振舞いを調べる．棒の初期温度は $\bar{T}(x)$ とし，棒の両端 $x = 0$ および L における温度はそれぞれ $T(t, 0) = T_0$ および $T(t, L) = T_L$ で一定とする．また，初期条件は $\bar{T}(0) = T_0$ および $\bar{T}(L) = T_L$ を満たしている．このとき，棒の温度 $T(t, x)$ は以下の熱方程式に従う：

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \tag{5.2a}$$

境界条件: $T(t, 0) = T_0$, $T(t, L) = T_L$, (5.2b)

初期条件: $T(0, x) = \bar{T}(x)$. (5.2c)

ただし, $\alpha(> 0)$ は温度伝導率である．

式 (5.2) は偏微分方程式なので, 本章の学習の範囲外と思うかもしれないが，以下のように考えると式 (5.2) も本章の対象となっていることが分かる．まず，区間 $x \in [0, L]$ を $M - 1$ 個に分割しよう．すなわち，整数 $i = 1, 2, \ldots, M$ に対して $x_i = (i - 1)\Delta x$ $(\Delta x = L/(M - 1))$ とする．合わせて，$T_i(t) = T(t, x_i)$ と表しておこう．このとき, 式 (5.2a) の右辺は，2 階の中心差分を用いて以下のように近似できる：

$$\alpha \frac{\partial^2 T}{\partial x^2} \approx \alpha \frac{T_{i-1}(t) - 2T_i(t) + T_{i+1}(t)}{\Delta x^2}.$$

この近似の下で，$\boldsymbol{u} = (T_1, T_2, ..., T_M)$ と考えれば，式 (5.2a) は M 個の連立常微分方程式になっていることが分かるだろう．もちろん，精度良く解を近似するためには M を大きくする必要があるため，偏微分方程式を解く際は巨大な連立方程式となることは想像に難くない．なお，境界条件の議論や $\boldsymbol{f}$ の具体形までは本章では紹介しない．

上記二つの例から想像ができるように，我々の身の回りの現象に対する数理モデルの多くは常微分方程式と結びついている．本章では，その数値解法の初歩的な部分を自分で構築し，実際の問題に適用できるようになることを目指す．

5.2 離散化

まず常微分方程式の変数の離散化について述べる．これは，例 5.1.2 でも既に登場しているように，微分方程式を数値的に解く場合にしばしば行われる方法である．

簡単のため，$t \in [0, T]$ で定義される 1 変数の常微分方程式の初期値問題を考える：

$$u' = f(t, u(t)), \quad u(0) = u_0. \tag{5.3}$$

ここで，$n = 0, ..., N$ とし，以下の離散変数を導入する：

$$t_n = n\Delta t, \quad U_n = u(t_n) \quad (\Delta t = T/N). \tag{5.4}$$

これは t を等間隔に分割した離散化の一例であり，t_n の形は必ずしも (5.4) のとおりでなくてよい．Δt をステップ幅と呼ぶ．これらの離散変数を用いると，例えば Taylor 展開を用いて u_n を

$$\begin{aligned} U_n &= u(t_{n-1} + \Delta t) \\ &= u(t_{n-1}) + u'(t_{n-1})\Delta t + O(\Delta t^2) \end{aligned}$$

と表すことで，u' は

$$u'(t_{n-1}) = \frac{U_n - U_{n-1}}{\Delta t} + O(\Delta t)$$

と近似される．つまり，離散化した変数を用いれば，式 (5.3) は

$$U_n \approx U_{n-1} + f(t_{n-1}, u(t_{n-1}))\Delta t \tag{5.5}$$

と変形できる．ここで，式 (5.5) の右辺の第二項目は $f(t_{n-1}, u(t_{n-1}))$ の代わりに $f(t_n, u(t_n))$ としても構わない．どちらを採用するか（あるいは他の形を採用するか）は数値解析手法の性質を決める重要な問題であり，次節以降で議論する．ここでは，初期条件 u_0 が与えられれば, 式 (5.5) を踏まえた漸化式

$$u_n = u_{n-1} + f(t_{n-1}, u_{n-1})\Delta t \tag{5.6}$$

によって U_n の相当値である u_n $(n = 1, 2, ..., N)$ が逐次的に求められることに気づけば十分である．

5.3 代表的な数値解法

本節では, 常微分方程式の初期値問題 (5.3) に対するいくつかの代表的な数値解法やその分類について説明する．ただし，$u(t_m)$ の相当値である u_m $(m = 0, 1, ..., n-1)$ は既知として，$u(t_n)$ の相当値である u_n を求める方法を記述する．また，ステップ幅は Δt で一定とする．必要に応じて，$f_n = f(t_n, u_n)$ と簡略化した表記を用いる．

5.3.1 Euler 法

さて，再び問題 (5.3) に立ち戻ろう．この式を $t \in [t_{n-1}, t_n]$ の範囲で積分すると

$$U_n - U_{n-1} = \int_{t_{n-1}}^{t_n} f(\tau, u(\tau))\mathrm{d}\tau \tag{5.7}$$

となる．$\Delta t = t_n - t_{n-1}$ が十分小さければ，右辺の被積分関数の値は端点の値やそれらの平均で置き換えてもよさそうである．そこでまず，$t \in [t_{n-1}, t_n]$ における f を $f(t_{n-1}, U_{n-1})$ で置き換えてみよう．$f(t_{n-1}, U_{n-1})$ は積分変数 τ に依存しないので，式 (5.5) と同じ近似関係式が得られる．この関係式を基にして, 漸化式 (5.6) を考える．これを**前進 Euler 法**と呼び，常微分方程式の数値解法の中で最も単純なものである．前進 Euler 法では，未知数である u_n が既知である u_{n-1} のみを使って陽的に表されているため，**陽解法 (explicit scheme)** と呼ばれる[1]．

[1]scheme の代わりに method という単語を用いている文献もある．

一方，$t \in [t_{n-1}, t_n]$ における f を $f(t_n, u_n)$ で近似すると，

$$u_n = u_{n-1} + f(t_n, u_n)\Delta t \tag{5.8}$$

を得る．式 (5.8) は**後退 Euler 法**と呼ばれる．前進 Euler 法と大きく異なるのは，式 (5.8) が u_n に対して陰的に表されていることであり，このような方法を**陰解法 (implicit scheme)** という．つまり，$u_n = \ldots$ と u_n について書き下すためには，式 (5.8) を u_n について解かなくてはならない．f が u_n に対して線形であればこれは容易だが，非線形である場合は，非線形方程式を解くために Newton 法などの反復解法が必要となることが多い．

前進 Euler 法は後退 Euler 法も導出過程で Δt^2 の項を無視しているので，数値解法の次数は 1 次精度である．一見すると後退 Euler 法を用いる利点は無いように思われる．しかし，実際には陰解法の方が陽解法に比べて安定であり，問題の性質によっては陰解法が好まれる場合がある．安定性については 5.4 節で詳述する．なお，Euler 法は u_n を求めるときに u_{n-1} より過去の情報（例えば u_{n-2}）を用いていない．このような方法を**一段法 (one-step scheme)** と呼ぶ．

5.3.2 Crank-Nicolson 法

Euler 法は 1 次精度であったが，これを 2 次精度に向上させる方法が **Crank-Nicolson 法**であり，これは**台形法**や **2 次の Adams-Moulton 法**とも呼ばれる．Adams-Moulton 法については 5.3.3 節で詳しく紹介する．式 (5.7) の中の f を，積分範囲の二つの端点

$$(t_{n-1}, f_{n-1}), \quad (t_n, f_n)$$

を通る 1 次関数で近似しよう[2]．このとき f は

$$f(\tau, u(\tau)) \approx \frac{\tau - t_{n-1}}{t_n - t_{n-1}} f_n + \frac{\tau - t_n}{t_{n-1} - t_n} f_{n-1} \tag{5.9}$$

と表すことができる．式 (5.9) は τ の 1 次関数なので，式 (5.7) の積分は容易に実行できて，

$$\int_{t_{n-1}}^{t_n} f(\tau, u(\tau))\mathrm{d}\tau \approx \frac{\Delta t}{2}(f_{n-1} + f_n) \tag{5.10}$$

を得る．つまり，U_n は

$$U_n \approx U_{n-1} + \frac{\Delta t}{2}(f_{n-1} + f_n)$$

を満たし, これを踏まえて方程式

$$u_n = u_{n-1} + \frac{\Delta t}{2}(f_{n-1} + f_n) \tag{5.11}$$

を考える．これが Crank-Nicolson 法である．

式 (5.9) は f の Taylor 展開において $O(\Delta t^2)$ を無視したものに他ならないので，式 (5.10) は結果的に $O(\Delta t^3)$ を無視している．つまり，Crank-Nicolson 法は 2 次精度の陰解法である．また，式 (5.11) の右辺に (u_n, u_{n-1}) しか含まれていないので，Crank-Nicolson 法は一段法である．

5.3.3 多段法

上記に紹介した一段法に対して，多段にして数値解析の精度の次数を向上させる**多段法（multi-step scheme）**がある．式 (5.7) の中の f を，τ に関する N 次多項式 $p_N(\tau)$ で近似する（N の意味は式 (5.4) と異なることに注意せよ）:

$$\int_{t_{n-1}}^{t_n} f(\tau, u(\tau))\mathrm{d}\tau \approx \int_{t_{n-1}}^{t_n} p_N(\tau)\mathrm{d}\tau. \tag{5.12}$$

$p_N(\tau)$ が τ の多項式なので，式 (5.12) の積分は解析的に実行できることに注意しよう．

p_N の構成方法は，u_n を用いるかどうかで二つに大別でき，それぞれ陽解法および陰解法となる．まず，最も一般的な陽解法について述べる．$p_N(\tau)$ を Lagrange 補間により構成することを考えよう．これには $N+1$ 個の点が必要である．陽解法なので，

$$(t_{n-1}, f_{n-1}), (t_{n-2}, f_{n-2}), \ldots, (t_{n-N-1}, f_{n-N-1})$$

を使うのが自然だろう．結果として，例えば $N = 1$ のときは

$$u_n = u_{n-1} + \frac{\Delta t}{2}(3f_{n-1} - f_{n-2}), \tag{5.13}$$

[2] 厳密には 2 点 $(t_{n-1}, f(t_{n-1}, U_{n-1}))$, $(t_n, f(t_n, U_n))$ を通る 1 次関数を考えるべきだが, ここでは記述の簡略化のため, 相当値 u_n から定まる 2 点 (t_{n-1}, f_{n-1}), (t_n, f_n) を通る 1 次関数を考えている．u_n が U_n の近似となっているため, この簡略化は問題とならない．以降の議論でもこの簡略化を採用する．

$N = 2$ のときは

$$u_n = u_{n-1} + \frac{\Delta t}{12}(23f_{n-1} - 16f_{n-2} + 5f_{n-3}) \tag{5.14}$$

がそれぞれ導出される．式 (5.13) および式 (5.14) をそれぞれ2次および3次の **Adams-Bashforth 法**と呼ぶ．このような高精度化の考え方は，数値積分法と密接に関連している．

次に陰解法について述べる．先の陽解法の例と同様に $N+1$ 個の点を用いて f を Lagrange 補間するが，ここでは

$$(t_n, f_n), (t_{n-1}, f_{n-1}), \ldots, (t_{n-N}, f_{n-N})$$

を用いる．f_n すなわち $f(t_n, u_n)$ を用いて多項式を構成するため，結果が陰的になることが想像できるだろう．また，$N = 1$ のときは Crank-Nicolson 法と一致するため，これは一段法であることも分かる．$N = 2$ のときは，

$$u_n = u_{n-1} + \frac{\Delta t}{12}(5f_n + 8f_{n-1} - f_{n-2}) \tag{5.15}$$

を得る．式 (5.15) を 3 次の **Adams-Moulton 法**と呼び，これは陰的な多段法である．

さて，Adams-Bashforth 法は陽解法なので，f の形に依らず簡単に構成できる．一方，Adams-Moulton 法は陰解法なので安定性は高いが，場合によっては非線形方程式を解く必要がある．また，過去に参照できる点を制限した場合，Adams-Moulton 法の方が精度が高い．これは，過去に参照している点が f_{n-2} である 2 次精度の Adams-Bashforth 法 (5.13) と 3 次精度の Adams-Moulton 法 (5.15) を比較すれば分かる．これら二つの方法の長所を組み合わせた方法として，**予測子・修正子法 (Predictor-Corrector method)** がある．予測子・修正子法では，まず Adams-Bashforth 法で u_n の予測値 $\tilde{u}_n$ を求めておく．次に，Adams-Moulton 法 (5.15) の右辺で，f_n に含まれる u_n の代わりに $\tilde{u}_n$ を用いて，修正値として u_n を求める．予測子・修正子法は完全に陽的な方法であることに注意しよう．また，修正値を求める段階の Adams-Moulton 法を繰り返すこともできる．この操作は反復法により非線形方程式 (5.15) を解くことに相当するため，数値解法は陰的になる．ただし，この反復法が収束するためには，予測値 $\tilde{u}_n$ が u_n に十分近く，また Δt が十分小さい必要がある（[4] の 5.2 章参照）．

本節で述べた多段法は，後に述べる Runge-Kutta 法に比べて 1 ステップにおける f の計算回数が少ないため，全計算時間における f の計算の割合が大きい問題では有用である．

5.3.4 高次精度一段法

これまで見てきたように，多段法を用いることで数値計算の精度の次数を向上させることができる．しかし，多段法においては，問題で与えられた初期値以外に，計算を開始するにあたって必要な初期値があり，これらを別途求めておく必要がある．これには，より低次の多段法を逐次的に適用することで対応できるが，計算全体の精度が低次部分の計算結果に依存してしまうため，例えばステップ幅を非一様にする工夫などが必要となる．このような問題を回避するためには，以下の高次精度一段法が便利である．

式 (5.6) を少し変形した次の形

$$\begin{cases} u_n = u_{n-1} + \frac{\Delta t}{2}(f_{n-1} + f(t_n, u_n^*)), \\ u_n^* = u_{n-1} + f_{n-1}\Delta t \end{cases} \tag{5.16}$$

を考えよう．これは，Crank-Nicolson 法 (5.11) に似ているが，右辺で u_n を使う代わりに，$n-1$ ステップ目の情報だけで構成される u_n^* を使っている点が異なる．式 (5.16) は **Heun 法**あるいは2次の **Runge-Kutta 法**と呼ばれ，高次精度一段法の代表的なものである．Heun 法の考え方は既出の予測子･修正子法と似ており，未知の $f(t_n, u_n)$ を既知の $f(t_n, u_n^*)$ で近似して高次精度化を試みている．最も一般的に使われる高次精度一段法は以下の 4 次の Runge-Kutta 法で，これを単に Runge-Kutta 法と呼ぶこともある．4 次の Runge-Kutta 法は

$$u_n = u_{n-1} + \frac{\Delta t}{6}(F_1 + 2F_2 + 2F_3 + F_4)$$

と表せる. ただし,

$$\begin{cases} F_1 = f(t_{n-1}, u_{n-1}), \\ F_2 = f(t_{n-1} + \Delta t/2, u_{n-1} + F_1 \Delta t/2), \\ F_3 = f(t_{n-1} + \Delta t/2, u_{n-1} + F_2 \Delta t/2), \\ F_4 = f(t_n, u_{n-1} + F_3 \Delta t) \end{cases}$$

である. 上述の解法は全て陽解法であることに注意する. 一般の Runge-Kutta 法の構築については [3] を参照するとよい.

5.4 安定性

数値解法の安定性は解法の性質を議論する上で重要な要素のひとつである. 本節では, 数値解法の安定性について簡単な例を題材に紹介する. より詳細な理論を知りたい場合は [3, 4] を参照せよ.

常微分方程式の解の安定性について復習する. 自励系の微分方程式

$$\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}t} = \boldsymbol{f}(\boldsymbol{u}(t)) \tag{5.17}$$

について, $\boldsymbol{f}(\boldsymbol{u}_*) = 0$ を満たす $\boldsymbol{u}_* \in \mathbb{R}^n$ を平衡点という. $\boldsymbol{u}(t) = \boldsymbol{u}_*$ は微分方程式 (5.17) を満たすことに注意する. 次に, この微分方程式の初期値問題

$$\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}t} = \boldsymbol{f}(\boldsymbol{u}(t)), \quad \boldsymbol{u}(0) = \boldsymbol{u}_0 \tag{5.18}$$

を考える. $\boldsymbol{u}_0 (\neq \boldsymbol{u}_*)$ を $\boldsymbol{u}_*$ の十分近くに取れば必ず $\lim_{t\to\infty} \boldsymbol{u}(t) = \boldsymbol{u}_*$ が成り立つとき, 平衡点 $\boldsymbol{u}_*$ は安定 (正確には漸近安定) という. 平衡点 $\boldsymbol{u}_*$ が安定であることは, 関数 $\boldsymbol{f}$ の $\boldsymbol{u}_*$ での Jacobi 行列 $D\boldsymbol{f}(\boldsymbol{u}_*)$ の固有値が全て負であることの必要十分条件となっている. すなわち平衡点の安定性は, 対応する線形化方程式

$$\frac{\mathrm{d}\boldsymbol{h}}{\mathrm{d}t} = D\boldsymbol{f}(\boldsymbol{u}_*)\boldsymbol{h}, \quad \boldsymbol{h}(0) = \boldsymbol{h}_0$$

の挙動と対応する. このことを踏まえ, 以下では簡単のため, 1 変数の常微分方程式の初期値問題

$$u' = -\alpha u + \beta, \quad u(0) = u_0 \quad (\alpha > 0, \beta \in \mathbb{R}) \tag{5.19}$$

に対する数値解法の安定性を議論することにする. なお, 式 (5.19) の解は

$$u(t) = \left(u_0 - \frac{\beta}{\alpha}\right)\exp(-\alpha t) + \frac{\beta}{\alpha}$$

であり, $\lim_{t\to\infty} u(t) = \beta/\alpha$ となる[3].

ところで, 式 (5.19) に対する一段法は一般に次の形で表される:

$$u_n = a_1 u_{n-1} + a_0. \tag{5.20}$$

例えば, 以下の例 5.4.1 で見るように, 前進 Euler 法 (5.6) や後退 Euler 法 (5.8) は式 (5.20) に対応する. $a_1 \neq 1$ のとき, $u_n = u_* := a_0/(1-a_1)$ は平衡点であり, 常微分方程式の場合に即して, $\lim_{n\to\infty} u_n = u_*$ となるとき, この一段法は安定であるということにする[4].

$a_1 \neq 1$ とする. $u_0 \in \mathbb{R}$ を与えると, 式 (5.20) の解は,

$$u_n = \left(u_0 - \frac{a_0}{1-a_1}\right)a_1^n + \frac{a_0}{1-a_1} \tag{5.21}$$

である. よって, この一段法が安定であるための必要十分条件は, $|a_1| < 1$ である. 係数 a_1 は一般に離散パラメータ Δt を含むため, 安定な数値計算を実行するためには, Δt の取り方には注意が必要である. このことを例題で確認してみよう.

例 5.4.1 Euler 法の安定性

まず, 式 (5.19) を前進 Euler 法で離散化してみよう. このとき,

$$\begin{aligned} u_n &= u_{n-1} + (-\alpha u_{n-1} + \beta)\Delta t \\ &= (1 - \alpha\Delta t)u_{n-1} + \beta\Delta t \end{aligned}$$

が得られるので, $a_1 = 1-\alpha\Delta t$, $a_0 = \beta\Delta t$ である. つまり, $|a_1| < 1$ となるためには, $0 \leq \Delta t < 2/\alpha$ が要求される. 一方, 後退 Euler 法では,

$$\begin{aligned} & u_n = u_{n-1} + (-\alpha u_n + \beta)\Delta t \\ \iff \quad & u_n = \frac{u_{n-1}}{1+\alpha\Delta t} + \frac{\beta\Delta t}{1+\alpha\Delta t} \end{aligned}$$

[3] $z = u - \beta/\alpha$ と変数変換して, $\beta = 0$ の場合に帰着してから安定性を議論するテキストが多い.

[4] $a_1 = 1$ の場合は必ず安定にならない.

と変形できるので，$a_1 = (1+\alpha\Delta t)^{-1}$, $a_0 = (\beta\Delta t)/(1+\alpha\Delta t)$ である．$\alpha > 0$ に対して，後退 Euler 法では常に $|a_1| < 1$ が満たされるため，後退 Euler 法は任意の $\Delta t > 0$ に対して安定である．

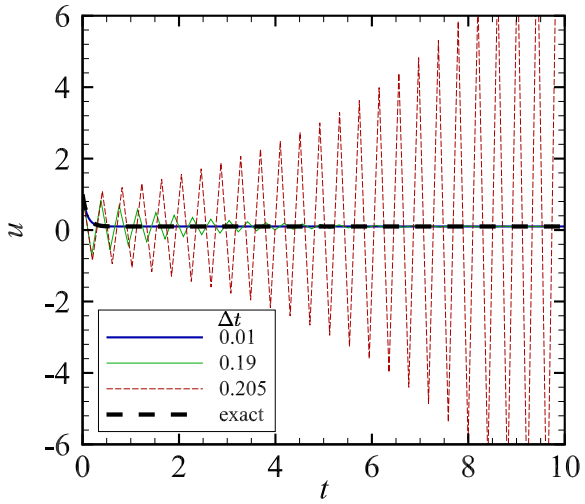


図 5.1: 前進 Euler 法による式 (5.19) の数値解．

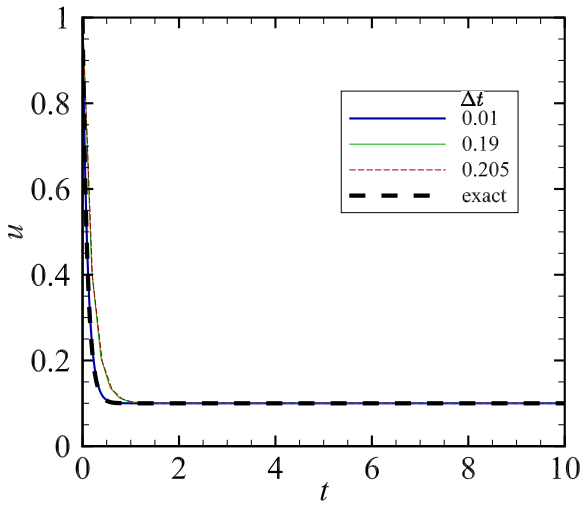


図 5.2: 後退 Euler 法による式 (5.19) の数値解．

図 5.1 と 5.2 にそれぞれ前進 Euler 法と後退 Euler 法による数値解を示す．パラメータは $u_0 = 1$, $\alpha = 10$, $\beta = 1$ として，$\Delta t = 0.01$, 0.19, 0.205 とした．図 5.1 より，前進 Euler 法で $\Delta t = 0.205 (> 2/\alpha = 0.2)$ の場合に，解が振動しながら発散していく様子が分かる．この振動は，$a_1 < -1$ であることと式 (5.21) の形からも想像できるだろう．一方，図 5.2 より，後退 Euler 法では数値解の発散は見られない．この一例からも分かるように，一般に陰解法の方が安定性が高い．

上記の例のように，陽解法を用いた場合，Δt を十分小さくとらなければ安定した数値解が得られないことがある．このような微分方程式は**硬い (stiff)** と表現される．式 (5.19) の場合は，α が Δt の大きさに制限を加えていることは明らかだが，一般の非線形連立常微分方程式系を扱う場合は安定性を議論することは容易ではない．

最後に，多段法の安定性について概説する．式 (5.19) の多段法は次の形で表される:

$$u_n = \sum_{j=1}^{N} a_j u_{n-j} + a_0. \tag{5.22}$$

ここで，$U_n = (u_n, u_{n-1}, \ldots, u_{n-N+1})^T$ とおけば，式 (5.22) は連立方程式 $U_n = AU_{n-1} + b_0$ の形に帰着される．ただし，

$$A = \begin{pmatrix} a_1 & a_2 & \cdots & a_N \\ 1 & & & \\ & \ddots & & \\ & & 1 & \end{pmatrix}, \quad b_0 = \begin{pmatrix} a_0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

である．一段法と同様に，ベクトル列 U_n が平衡点に収束するとき，この多段法は安定であるということにする．多段法が安定であるための必要十分条件は，行列 A の全ての固有値 λ_j $(j = 1, \ldots, N)$ に対して $|\lambda_j| < 1$ が成り立つことである．

ここで，行列 A の固有方程式は，

$$\det(\lambda I - A) = \lambda^N - \sum_{j=1}^{N} a_j \lambda^{N-j}$$

であるが，これは $(N+1)$ 項間漸化式 (5.22) の特性方程式でもある．すなわち，式 (5.22) において，$a_0 = 0$, $u_n = \lambda^n$ とおき，両辺を λ^{n-N} で除したものとなっている．また，$N = 1$ のときは固有値が $\lambda = a_1$ となることから，この考え方は一段法も含んでいることが分かる．

5.5 爆発解

有限の T に対して，$t \to T$ で $u = u(t)$ が発散するとき，解は**爆発**するという．数値計算上で

は，微小なステップ幅に対しても大きな u の変動が生じるため，爆発解の計算は一般には難しい．例えば，ステップ幅が解の振舞いに応じて自動的に変化するようなアルゴリズムが必要である．ここでは，適当な変数変換を施すことで爆発解の数値計算が容易になることを示そう [5]．

常微分方程式の初期値問題 (5.3) を考える．ある t の値（あるいは t の範囲）で u' が非常に大きくなる性質を持っているとしよう．このような場合，独立変数として，t の代わりに曲線の長さを用いることが考えられる．t は新しい従属変数になることに注意する．t-u 平面において解が描く軌道に沿った長さを s とする．つまり，

$$\frac{\mathrm{d}s}{\mathrm{d}t} = \sqrt{1+(u')^2} = \sqrt{1+(f(t,u))^2}$$

である．このとき，式 (5.3) は $u = u(s)$ および $t = t(s)$ を用いて

$$\frac{\mathrm{d}u}{\mathrm{d}s} = \frac{f(t,u)}{\sqrt{1+(f(t,u))^2}}, \quad u(0) = u_0, \tag{5.23a}$$

$$\frac{\mathrm{d}t}{\mathrm{d}s} = \frac{1}{\sqrt{1+(f(t,u))^2}}, \quad t(0) = 0 \tag{5.23b}$$

と表される．式 (5.23) は t, u を従属変数，s を独立変数とした連立常微分方程式に他ならない．式 (5.3) に比べて式 (5.23) が数値的に扱い易いことを，以下の例で確認してみよう．

例 5.5.1 Euler 法による爆発解の計算

非線形常微分方程式の初期値問題

$$u' = u^2, \quad u(0) = 1 \tag{5.24}$$

を考える．解析解は $u = (1-t)^{-1}$ なので，$t \to T = 1$ で $u \to \infty$ である．前進 Euler 法により数値解析した結果を図 5.3 に示す．変数変換 (5.23) を使わない場合，数値解は $t = T = 1$ を超えた範囲にまで進んでいる．本来このような挙動は許されない．一方，変数変換 (5.23) を用いた結果ではこのような不具合は生じておらず，解析解により近い数値解を得られていることが分かる．

5.6 カオス

最後に，解が興味深い振舞いをする例として，カオスを紹介する．数値計算の演習なので，物理的な背景や数学的な事柄はここでは紹介しない．興味がある人は，例えば [6] や，より数学的に発展的な内容については [7] を勉強するとよい．下記に示す Lorenz 方程式について，物理的な背景や数学的な性質が紹介されている．

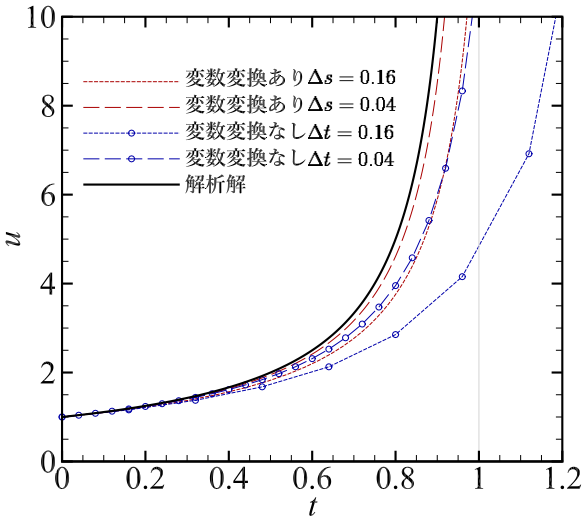


図 5.3: 前進 Euler 法による式 (5.24) の数値解．

3 つの未知数 $x(t)$, $y(t)$, $z(t)$ が，次の **Lorenz 方程式**

$$x' = \sigma(y-x), \quad x(0) = x_0, \tag{5.25a}$$

$$y' = rx - y - xz, \quad y(0) = y_0, \tag{5.25b}$$

$$z' = xy - bz, \quad z(0) = z_0 \tag{5.25c}$$

に従うとしよう．σ, r, b はパラメータであり，すべて正とする．式 (5.25) は非線形連立常微分方程式であり，本章で紹介してきた数値計算手法で解析できる．一見シンプルな方程式であるが，その解の挙動は非常に複雑でカオス的である．なお，文献 [6] では，カオスを「決定論的なシステムにおける非周期的な長時間挙動であり，初期条件への鋭敏な依存性を示す」と定義している．式 (5.25) には乱数が用いられていないため，決定論的なシステムであることは明らかである．以下の課題では，カオス的な性質のうち，非周期的な長時間挙動と初期条件への鋭敏な依存性について調べることにしよう．なお，初期条件に鋭敏に依存する方程式は，解が数値計算上の誤差にも強く影響を受ける．そのため，数値計算の精度にも十分に気を配る必要がある．

5.7 演習問題

課題 1 身の回りにある現象からひとつ選んで，例に倣って常微分方程式の初期値問題を定式化せよ．

課題 2 式 (5.14) や式 (5.15) にならって，4 次の Adams-Bashforth 法と Adams-Moulton 法を構成せよ．また，4 次の Adams-Bashforth 法と 4 次の Adams-Moulton 法を用いて，予測子・修正子法を構成せよ．ただし, 対応する Lagrange 多項式も含めて, 構成の過程も記述すること．

課題 3 常微分方程式の初期値問題

$$u' = u, \quad u(0) = 1 \tag{5.26}$$

を考える．解析解は $u = \exp(t)$ で与えられる．前進 Euler 法 (FE)，2 次および 3 次の Adams-Bashforth 法 (AB2 および AB3)，Heun 法 (H)，4 次の Runge-Kutta 法 (RK) の 5 種類の方法で，式 (5.26) を $t \in [0, 1]$ の範囲で数値的に解き，理論的に予測される次数を達成していることを確認せよ．ただし, 多段法で必要となる初期条件に関しては，解析解の値を用いても構わない．結果は分かりやすく表にまとめよ．

確認は以下の要領で行うこと．まず，ステップ幅を $\Delta t = 1/2^i$，ステップ数を $N = 2^i$，離散化した時間変数を $t_n = n\Delta t$ $(n = 0, ..., N)$ と定義する．また，ステップ幅が $\Delta t = 1/2^i$ の条件の下で得られた数値解を $u_n^{(i)}$ と表す．精度の確認は，i の値を $i = 1, 2, ..., i_{\max}$ の範囲で様々に変更した計算を実行し，それらの結果を比較すればよい．比較には，$t = 1$ における数値解 $u_N^{(i)}$ と解析解との差 $E^{(i)} = |u_N^{(i)} - u(1)|$ を用いる．p を計算手法の次数とし，$E_r^{(i)} = E^{(i)}/E^{(i-1)}$ $(i = 2, ..., i_{\max})$ と定義すると，$E_r^{(i)}$ は i の増加とともに $1/2^p$ に近づくはずである．

ヒント： 単精度でなく倍精度で計算すること．$i_{\max}$ の値は自由に決めてよいが，$i \approx 10$ 程度より i が大きくなると，倍精度演算でも結果を正しく評価できない場合がある．また, 多段法で必要となる初期条件の生成で低次の数値計算法 (例えば前進 Euler 法) を用いると精度が悪くなることがある．

課題 4 式 (5.19) に対して，Crank-Nicolson 法，Heun 法, 予測子・修正子法を適用した場合に安定性がどのように変わるかを, 5.4 節に倣って解析せよ．また，この解析結果を数値的に確認せよ．予測子および修正子に用いる数値計算法の次数は任意に選んで構わない．

課題 5 式 (5.7) の代わりに，

$$u_n = u_{n-2} + \int_{t_{n-2}}^{t_n} f(\tau, u(\tau))\mathrm{d}\tau$$

として，$t \in [t_{n-2}, t_n]$ の範囲で $f \approx f_{n-1}$ と近似すれば，

$$u_n = u_{n-2} + 2f(t_{n-1}, u_{n-1})\Delta t \tag{5.27}$$

を得る．式 (5.27) は陽的な多段法であり, 一見問題なさそうだが，$t \in [0, \infty)$ における常微分方程式の初期値問題

$$u' = -2u + 1, \quad u(0) = 1$$

を解くことができない．このことを 5.4 節の議論に則って示し，数値計算で確かめてみよ．ただし, 式 (5.27) の初期値 u_1 は, 解析解の値を用いても構わない．

課題 6 生物の個体数の増減に関する数理モデルを考える．$t \geq 0$ を時刻，$u(t)$ を時刻 t における個体数，$u(0) = u_0 > 0$ を初期個体数とする．個体数の時間変化 u' は，個体数が多いほど大きいと考えるのが自然である．そこで，$u' = ru$ $(r > 0)$ と仮定するのが最も単純であろう．しかし，ある種の生物では，u が小さいと交配相手が見つかる確率が減り，結果として r が小さくなるという仮定が成り立つ．逆に，u が大きいと交配相手が見つかる確率が高まるので，r は増加する．これを Allee 効果という．一例として，

$$u' = (u - 1)u, \quad u(0) = u_0 \tag{5.28}$$

という数理モデルが考えられる．ただし，式 (5.28) は適切に無次元化されているとする．問題 (5.28) を数値解析し，解の振舞いが初期値 u_0 によってどのように変化するか予想せよ．ただし,

解が有限時刻で爆発することが予想される場合は, 5.5 節の内容を踏まえた数値解析を行うこと．また, 問題 (5.28) の解析解を求め, 予想が正しいかどうか検討せよ．

課題 7 Lorenz 方程式 (5.25) において，$\sigma = 10$, $b = 8/3$, $r = 28$, $x_0 = 1 + \epsilon$, $y_0 = 0$, $z_0 = 0$ として，数値解析を行う．手法としては，前進 Euler 法と Runge-Kutta 法を用いよ．また，計算領域は $t \in [0, 100]$ とする．

1. まず，$\epsilon = 0$ とする．$x(t)$ を t の関数として図示せよ．また，$(x(t), y(t), z(t))$ の軌道を x-y-z 空間上に図示し，非周期的な長時間挙動を確認せよ．

2. $\Delta t = 0.01 \times 2^{-i}$ $(i = 0, ..., 13)$ として，前進 Euler 法と Runge-Kutta 法でそれぞれの i に対する数値解を求めよ．$t = 15$, 30, 60 の 3 つの時刻において，Δt を小さくするとともに $x(t)$ がどのように変化するか図示し，数値計算の誤差の影響が $x(t)$ に与える影響を議論せよ．

3. $\epsilon = 0$, 0.1, 0.01, 0.001 として，ϵ が $x(t)$ の振舞いに与える影響を議論せよ．ただし，解の振舞い対する数値計算の誤差の影響が ϵ の違いによる影響より小さい必要があるため，Runge-Kutta 法を用いて Δt も十分小さく設定すること．

参考文献

[1] 笠原晧司, ”微分方程式の基礎”, 朝倉書店 (1982)

[2] 高橋陽一郎, ”微分方程式入門”, 東京大学出版会 (1988)

[3] 齊藤宣一，”数値解析入門”，東京大学出版会（2012）

[4] G. H. Golub and J. M. Ortega, ”Scientific Computing and Differential Equations: An Introduction to Numerical Methods”, Academic Press (1992)

[5] 伊理正夫，藤野和建，”数値計算の常識”，共立出版（1985）

[6] ストロガッツ，”非線形ダイナミクスとカオス”，田中久陽，中尾裕也，千葉逸人訳，丸善出版（2015）

[7] アリグッド，サウアー，ヨーク，”カオス第 1 - 3 巻，力学系入門”，津田一郎監訳，シュプリンガー・ジャパン（2006）

[川越大輔]

第6章　組合せ最適化

6.1　目的

組合せ最適化問題（離散最適化問題）とは，順序や割当のように「組合せ的な構造」によって解が表現される最適化問題を指す．現実の多くの場面において自然に現れる問題であるが，問題の構造をうまく捉えなければ効率よく解くことは難しい．本実験の目的は**最短経路問題**を通して組合せ最適化問題の難しさを体感し，代表的な解法の一つである分枝限定法について学ぶことである．

6.2　準備

実数の集合を $\mathbb{R}$ とする．節点集合 $V = \{1, 2, \ldots, n\}$ と有向枝集合 $E \subseteq V \times V$ から成る有向グラフ $G = (V, E)$ が与えられたとする．任意の二節点 $s, t \in V$ について，節点の系列 $P = \langle v_1 = s, v_2, \ldots, v_k = t \rangle$ が

(条件 1) $(v_i, v_{i+1}) \in E,\ i \in \{1, 2, \ldots, k-1\}$

を満たすとき，P を s から t への**有向路** (あるいは単に**路**[1]) と呼ぶ．また $s = v_1$ を P の**始点**，$t = v_k$ を P の**終点**と呼ぶ．さらに P が

(条件 2) すべての節点 $v_1, v_2, \ldots, v_k$ が相異なる

とき，P を**単純有向路** (あるいは単に**単純路**) と呼ぶ．ただし $v_1 = v_k$ のみ成り立つとき，これを**閉路**と呼ぶ．

有向グラフ $G = (V, E)$ における各枝 $(u, v) \in E$ の長さ $d(u, v) \in \mathbb{R}$ が与えられたとき，s から t への路 $P = \langle v_1 = s, v_2, \ldots, v_k = t \rangle$ に含まれる枝の長さの総和を路の**長さ**と呼び，$d(P)$ と書く．すなわち $d(P)$ は，

$$d(P) \triangleq \sum_{i=1}^{k-1} d(v_i, v_{i+1})$$

と定義される．二つの節点 s, t をそれぞれ始点，終点とするような路のうち，長さが最小となるようなものを s から t への**最短経路**と呼ぶ．

[1] **道**と呼ぶ場合もある．

6.3　最短経路問題

有向グラフと枝の長さが与えられたとき，最短経路問題は文字通り最短経路を問う問題だが，始点と終点の指定の仕方によって以下の四通りの問題が考えられる．

- 単一頂点対最短経路問題: 指定された始点と終点の間の最短経路を問う問題．
- 単一始点最短経路問題: 始点のみが指定され，始点からそれ以外の節点すべてへの最短経路を問う問題．
- 単一終点最短経路問題: 終点のみが指定され，終点を除くすべての節点を始点として最短経路を問う問題，
- 全点対最短経路問題: すべての二節点間について最短経路を問う問題．

最短経路問題は乗り換え検索システムやカーナビの設計，PERT 図などのタスクスケジューリングに応用を持つ．**本実験では単一頂点対最短経路問題を取り扱う．**

最短経路問題を考えるとき，枝の長さを非負の値に限定することもあるが，**本実験では枝の長さが負の値を取ることも許容する．**すると長さ $d(C)$ が負になるような閉路 C が存在し得る（そのような閉路を**負閉路**と呼ぶ）．この場合最適解は存在しない．なぜならその C を繰り返し回ることで，路の長さをいくらでも小さくできてしまうからである．そこで**本実験では，単純路に限定して最短経路問題を考える**ことにする．すなわち始点と終点が与えられたとき，(条件 1), (条件 2) の両方を満たす節点の系列のうち，長さが最小となる

ものを問う．この問題は一般に NP 困難 [1] であることが知られている[2]．本実験では，この問題に対して分枝限定法に基づいた厳密解法を設計して実装し，簡単な性能評価を行う．

メモ． 単一始点最短経路問題に関して，枝の長さが非負に限定されている場合，ダイクストラ法を用いれば $O(m + n \log n)$ 時間で最適解を得ることができる（ただし $m = |E|, n = |V|$）．また枝の長さが負の値を取ることを許容し，長さが最小となる路を問う場合，ベルマン・フォード法を用いれば $O(nm)$ 時間で以下を行うことができる :「負閉路が存在するならばその旨を出力し，そうでなければ最適解を出力」．詳しくは [2, 5] などを参照せよ．

6.4 分枝限定法

分枝限定法は，組合せ最適化問題全般に用いることのできる厳密解法（厳密な意味での最適解を求める解法）の枠組みである．その動作原理は，組織的な場合分け（分枝）に基づく列挙法に，不要な場合の排除機能（限定）を付したものとなっている．

本稿では [4] に基づく説明を示す．興味のある者は [2, 3, 5] なども参照されたい．

分枝限定法は，問題をいくつかの小規模な問題に分割し，その全てを解くことで等価的に元の問題を解くという考え方に基づいている．小規模な問題への分割は，例えば，ある一つの決定変数の値を固定し（グラフ探索の場合，枝や節点を決め），それぞれの場合を個別に考察することによって実現できる．このように問題を分割する操作を**分枝操作**（branching operation）という．分枝操作を繰り返し行うことで，すべての場合を列挙することができるが，その過程は生成木と呼ばれる根付き木を用いて表現できる．生成木において，根は元の問題に対応し，その二つの子はそれぞれある変数の値を 0 または 1 に固定した 2 つの場合に対応する．その他の節点も同様である．よって，内部の節点は根からその節点への路に対応していくつかの変数の値を 0 か 1 に固定した問題に対応し，葉は全ての変数の値が定まった解に対応する．一部の変数の値が固定された問題を**部分問題**（partial problem）と呼び，この例の場合，部分問題も元の問題と等しい構造である．

分枝限定法では，実際に全ての葉を列挙するわけではなく，その一部分だけを生成する．生成木のうち実際に生成された節点のみから成るものを**探索木**（search tree）という．ある部分問題（節点）に対して，

(i) その最適値，

(ii) その部分問題が実行不可能であること，

(iii) その部分問題の最適解が元の問題の最適解とはならないこと，

のいずれかが分かれば，その部分問題をこれ以上調べる必要はなく，その下の節点（子孫と呼ぶ）の探索を省略できる（**終端する**（terminate）という）．これを**限定操作**（bounding operation）と呼ぶ．探索の過程で，まだ終端されていない部分問題を活性（active）であるといい，全ての部分問題が終端されたとき，つまり活性部分問題がなくなったとき，分枝限定法は厳密な最適解を与える（もしくは実行可能解が存在しないことを証明する）．

限定操作の代表例である**下界値テスト**について簡単に述べる．近似解法等を用いて実行可能解を一つでも得ることができれば，その目的関数値は最適値の上界（upper bound）値となる．また，探索が生成木の葉に到達したときにも実行可能解が得られる場合がある．これまでの探索で得られている最良の上界値を暫定値（incumbent value）という．一方，制約を緩めた緩和問題を解くなどして得られた最適値は，元の問題の最適値の下界（lower bound）値となる．このようにして得られた上下界値を用い，「ある部分問題の下界値が暫定値以上であれば，その子孫を調べる必要はない」ことが結論できる．その部分問題の子孫を全て調べても，暫定値よりもよい目的関数値を持つ解は見つからないからである．以上が下界値テストで

[2] ある問題が NP 困難であるとき，その問題には多項式時間アルゴリズムは存在しないと予想される．

ある.(最大化問題の場合は上界と下界の役割が入れ替わる.)

なお，計算時間の都合などにより活性部分問題が残っている状態で探索を終了し，その時点での暫定値を出力する方法もしばしば用いられる．このとき，出力した値が最適である保証はなく，分枝限定法を近似解法として用いることになるが，この場合でも，活性部分問題の下界値の最小値が元の問題の下界値となり，出力値の精度を測る指標としてこれを用いることができる．

6.5 最短路問題に対する分枝限定法

有向グラフ $G = (V, E)$，枝重み $d : E \to \mathbb{R}$，始点 $s \in V$ および終点 $t \in V$ $(s \neq t)$ が与えられたとする．以下では始点 s から終点 t への最短路を求めるための分枝限定法を設計するが，本節では s から t へ至る有向路をすべて列挙するためのアルゴリズムを示す．次節において，そのアルゴリズムを実装して最短路を求めることと，限定操作の導入によって計算を高速化することを実験課題とする．

単純路 $P = \langle v_1, v_2, \ldots, v_k \rangle$ の最後に節点 v を追加する操作を $P.\mathsf{push}(v)$，$P = \langle v_1, v_2, \ldots, v_k \rangle$ の最後尾の節点を削除する操作を $P.\mathsf{pop}()$ のように書くことにする．この二つの操作の結果，P はそれぞれ $P = \langle v_1, v_2, \ldots, v_k, v \rangle$，$P = \langle v_1, v_2, \ldots, v_{k-1} \rangle$ となる．このような操作は，単純路をスタックによって表現することで実現できる．また文脈に応じて単純路 P を非順序集合とみなし，その記法を用いる．たとえば，節点 $v \in V$ が P に属することを $v \in P$，属さないことを $v \notin P$ と表す．また空の単純路を $\emptyset$ で表す．

始点 s から終点 t へ至るすべての単純路を得るには，アルゴリズム 1 の擬似コードで示される再帰アルゴリズムにおいて OutputAllPaths$(\emptyset, s)$ を実行すればよい．

6.6 実験課題

あとに述べる注意事項をよく読んだ上で，次の課題 1 から課題 4 に取り組むこと．

アルゴリズム 1 OutputAllPaths(P, u)

入力: 単純路 P および節点 $u \in V \setminus P$;
出力: 単純路 P に属する節点を順に訪れた後に u を訪れ，やがて t へ至るすべての単純路.

```
1: P.push(u);
2: if u = t then
3:    P を出力
4: else
5:    for each (u, v) ∈ E, v ∉ P: do
6:       OutputAllPaths(P, v)
7:    end for
8: end if
9: P.pop()
```

課題 1
有向グラフ $G = (V, E)$，節点 $s, t \in V$ 及び各枝 $(u, v) \in E$ の長さ $d(u, v)$ が与えられたとき，点 s から点 t へ至る最短路を求める分枝アルゴリズムを設計し，その擬似コードを書け．アルゴリズム 1 を用いて設計するとよい．

課題 2
課題 1 で設計したアルゴリズムを実装し，グラフの節点数及び枝数に対して実行時間及び探索したノード数を調べ，表にまとめよ．

- 節点数の目安はおおむね 5 から 15 程度だが，数十分経っても計算が終わらないなど，実行時間があまりに長すぎる場合は適当な基準を設けて計算を打ち切ること．
- 枝をどこに張るか，および枝の重みは疑似乱数を使って決定するとよい．この場合，乱数によって計算結果が変わり得るので，各節点数に対して最低でも五回程度の計算を行うこと．表にまとめるべき実行時間やノード数は，その平均と標準偏差を示すこと．
- 枝の張り方について，ちょうど m 本の枝をランダムに張る方法として，すべての可能な有向枝の集合 $\{(1,2), (2,1), (1,3), \ldots, (n-1, n), (n, n-1)\}$ をシャッフルし (n は節点数)，最初の m 本を採用する，という手順が考えられる．このとき始点 s と終点 t の間

に路の存在を保証したいのであれば，必ず有向枝 (s,t) を含めるようにするとよい．

- 枝の重みは，適当な上限と下限を定めた上でランダムに決めるとよい．

課題 3
課題 1 で設計したアルゴリズムに関して限定操作を導入し，その妥当性を説明せよ．限定操作別の擬似コードを書き，最後に全体の限定操作アルゴリズムの擬似コードを書け．

課題 4
限定操作を実装し，実行時間及び探索したノード数を課題 2 の実装によるものと比較せよ．結果を表にまとめよ．

注意事項 (1) 擬似コード　擬似コードを書く時以下の点に注意すること：

- 代入 (substitution) には,「:=」を使う (「←」を使うこともある)．
- **goto** 文又は **break** 文は使わないようにする．
- 構造を分かりやすくするため，
  if ... **then** ... **end if**
  if ... **then** ... **else** ... **end if**
  for ... **do** ... **end for**
  while ... **do** ... **end while**
  の形を守り，字下げ (indentation) を深さに応じて使う．とくに入れ子構造が分かりやすくなるようにする．
- 関数の返す出力には **return**，印字・ファイルへの出力には **output** を使う．
- 連続するコマンドは「;」 で区切る（機械的にコマンドの終わりに付けるのではない)．

注意事項 (2) プログラムの実行時間の計測方法　ここでは，`gettimeofday()` を使用する方法を説明する．`gettimeofday()` を使用するにはプログラムの最初で以下のように記述し，`time.h` と `sys/time.h` をインクルードし，関数 `gettimeofday_sec()` を定義する必要がある．

```
#include <time.h>
#include <sys/time.h>

double gettimeofday_sec(){
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec
      + (double)tv.tv_usec*1e-6;
}
```

下の記述のように実行時間を計測したい操作の前後で `gettimeofday_sec()` を呼び出して値を保存すると，その差に 1000 をかけたものが実行時間（ミリ秒）になる．

```
/* 時間計測開始 */
double start =
gettimeofday_sec();

(計測したい操作の記述)

/* 時間計測終了 */
double end = gettimeofday_sec();

printf("time = %lf msec.\n",
(end-start)*1000);
```

この方法で計測される時間は実時間ではなく，プロセスによって消費された CPU 時間であるので，計算機で同時に実行されている他のプロセスによる影響は受けない．

参考文献

[1] M. R. Garey, D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, 1979.

[2] 浅野孝夫, 情報の構造 (上) (下), 日本評論社, 1994.

[3] 茨木俊秀, C によるアルゴリズムとデータ構造（改訂 2 版）, オーム社, 2019.

[4] 柳浦睦憲，野々部 宏司，分枝限定法— さらなる計算効率の希求 —,「システム／制御／情報」 第 50 巻 第 9 号，350-356，2006.

[5] T. コルメン，C. ライザーソン，R. リベスト，C. シュタイン，アルゴリズムイントロダクション第 3 版 総合版，近代科学社, 2013.

[Aleksandar Shurbevski, 原口 和也]

第7章　モンテカルロシミュレーション

7.1　はじめに

沢山のものが集まった物理的システムについて、統計物理学では、確率的なものとして記述することで、多くの興味深い結果を得ることに成功している。

モンテカルロ法は、そのような確率的システムに対するユニークな計算手法として発展してきた。現在では、ものだけなく事の集まりである情報システムも含めて、幅広い対象に対する強力なツールとして用いられている。

本章では、最初に、疫病の確率的な伝染モデル(**有向浸透現象モデル，パーコレーション**) を例に、確率変数の期待値をサンプル平均として近似的に評価する**モンテカルロシミュレーション**の具体例を示し、その数理的な仕組み（**重み付きモンテカルロ法**）について説明する。その後、相互作用する磁性体の研究やパターン認識などの情報処理に利用されている統計物理学の**イジングモデル**を例に、**マルコフ連鎖**を利用したサンプル作成を行う**マルコフ連鎖モンテカルロ法**について、その代表的なアルゴリズムと実装を説明する。本章で扱うモンテカルロシミュレーションは、for 文を含むアルゴリズムが必要になる。とくに Python を用いる学生は素朴にコードを書くと計算速度が著しく遅くなるため、for 文を避けるコードを意識して書くこと。

7.2　疫病の確率的な伝染モデル

疫病の伝染をモデル化することを考える。簡単のため、人の移動はないとして、図 7.1 のように人が 1 列に並んだ状況を考える。場所 i にいる人が疫病にかかった状況を、状態変数 s_i を使って、$s_i = 1$、また、疫病でない状態を $s_i = 0$ で表す。さらに時間進行も離散的であるとする。

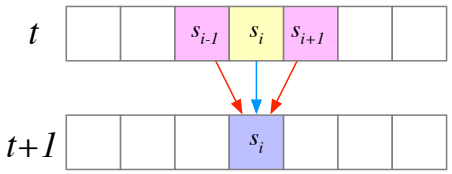


図 7.1: 疫病の確率的な 1 次元格子伝染モデル。

特に重要なこととして、疫病にかかった人が、確率的に、近隣の人を感染させたり治ったりすると仮定する。したがって、次の時刻 $(t+1)$ の場所 i の人の感染状況 $s_i(t+1)$ は、前の時刻 t での場所 $[i-1, i, i+1]$ での感染状況 $[s_{i-1}(t), s_i(t), s_{i+1}(t)]$ にのみ依存して確率的に決まる。簡単のために左右を区別しないことにすると、この条件付き確率は両隣の和 $n_i(t) = s_{i-1}(t) + s_{i+1}(t)$ と $s_i(t)$ に依存する形になる:$P(s_i(t+1)|s_i(t), n_i(t))$。このとき、$(s_i, n_i)$ の組み合わせは 2×3 通りあり、それぞれの場合で、次時刻に疫病にかかる確率を決定すると疫病の確率的伝染モデルが完成する。ただし、自分と近隣が疫病でない場合 $(s_i, n_i) = (0, 0)$ は、疫病になることはないので、その場合の確率は必ず 0 になる。

$$
\begin{aligned}
&P(s_i(t+1) = 1|(s_i(t), n_i(t)) = (0,0)) \\
&= P(1|(0,0)) = 0
\end{aligned}
\tag{7.1}
$$

このとき、このモデルには非常に特殊な状態が存在する。それは、疫病にかかっている人が全くいない状態で、この状態に一旦なるとずっとその状態を維持する。つまり、システムはこの特殊な状態から脱出することができない。そのため、このような状態を**吸収状態**という。

まとめると、この疫病の確率的伝染モデルは、$(s_i, n_i) = (0, 0)$ 以外の 5 つの場合に次時刻に疫病にかかる確率 $P(1|(s_i, n_i))$ をパラメータとした確率的時間発展をするシステムで、疫病が全くない吸収状態をもつ。なお、この例のように、有限の過去の状態のみ依存して、次状態が確率的に決定する**確率過程**は、**マルコフ過程**（**マルコフ連鎖**）と呼ばれている。

例えば、遷移確率を次のように 1 つのパラメータ p を用いて定義する[1]。

$$
\begin{aligned}
&P(1|(0,1)) = p^2,\\
&P(1|(0,2)) = P(1|(1,0)) = p^2(2-p^2),\\
&P(1|(1,1)) = p^2(p^3-2p^2-p+3),\\
&P(1|(1,2)) = p^2(2-p)(p^3-2p^2+2) \qquad (7.2)
\end{aligned}
$$

このとき、システムに含まれる人の数（システムサイズ）が十分大きければ $p_c = 0.6447\cdots$ を境に挙動が変わり、$p < p_c$ ならば十分時間が経つと吸収状態になり、$p > p_c$ ならばいつまでも疫病の人がいることが知られている。つまり、p_c を境にシステムの振る舞いが大きく変わる。このようにシステム全体の振る舞いが変わる現象を統計物理学では、**相転移現象**と呼ぶ。この例では、吸収状態になる相と、そうでない状態である活性状態相の間の相転移現象で、特に**吸収相転移**と呼ばれることもある。実は、このモデルの時間方向を下向き方向と解釈すると、確率的に存在する隙間を通って物質の中を液体が重力に引かれ下方向に浸透する現象のモデルとして解釈することができ、**有向浸透現象**モデルと言われている。特に、$p = p_c$ の相転移点近傍の振る舞いは、種々の量が**ベキ則**に従うなど興味深い振る舞いを示し、**臨界現象**と呼ばれる。臨界現象は、特定のモデルだけではなく、似たモデル全てに共通する性質（ベキ則）があり、普遍性を持つと言われる。そのため、このモデルについては、**非平衡臨界現象**の雛形モデルとして多くの研究がある [2]。

7.2.1 サンプル状態列

このモデルでは、時間順に状態を並べた状態列として、様々なものが確率的に出現することになる。以下では、状態列のサンプルを確率的に作る方法を考える。

この例では、次状態への遷移確率が局所的に定義されているので、時間の順にサンプル状態を確率的に作成できる。つまり、現在の状態 $\{s_i(t)\}$ がある時に、次の時刻の状態 $\{s_i(t+1)\}$ は、(7.2) のような遷移確率 $P(s_i(t+1)|(s_i(t), n_i(t)))$ にしたがって確率的に決定される。例えば、$s_i(t) = 0$, $n_i(t) = 2$ なら、確率 $P(1|(0,2))$ で $s_i(t+1) = 1$ に、確率 $P(0|(0,2)) = 1 - P(1|(0,2))$ で $s_i(t+1) = 0$ にすることで、次時刻のサンプル状態 $\{s_i(t+1)\}$ を確率的に作成することができる。この手続きは、具体的には、次の**擬似乱数**を使うことで実装できる。

乱数

ある区間にでたらめ（一様）に値をとる数を乱数と呼ぶ。乱数同士は定義より独立であるが、ルールに基づいて乱数を作成するとそのような事は原理的にはできない。しかし、実際上、ほぼ独立に見えるような**擬似乱数**の作り方がいくつか提案されている。例えば、C 言語関数 xor128 は、区間 $[0, 2^{32}-1]$ の疑似乱数を作成する[2]。

```
uint32_t xor128(void) {
 static uint32_t x = 123456789;
 static uint32_t y = 362436069;
 static uint32_t z = 521288629;
 static uint32_t w = 88675123;
 uint32_t t;
 t = x ^ (x << 11);
 x = y;
 y = z;
 z = w;
 return w = (w ^ (w >> 19)) ^ (t ^ (t >> 8));
}
```

この擬似乱数を区間の長さ (2^{32}) で割る事で、区間 $[0,1]$ の一様分布に従う疑似確率変数を作成することができる。現在、擬似乱数の研究は非常に進んでおり、標準的な関数として言語に内包されていたりライブラリー等の形で使用できるのでそれらの利用を勧める。

遷移確率に従ったサンプル状態の作成方法

コイン投げで見られるように、ある確率で表、そうでなければ裏になるような試行を、一様分布

[1] 有向ボンド浸透をモデル化した Domany-Kinzel オートマトンの 2 ステップ遷移確率 [1]

[2] 本コードは unsigned long が 4 バイトである C 言語処理系を前提にしている。また、x, y, z, w は種と呼ばれる。ここでは適当な初期値を設定してある。

に（擬似的に）従う変数（擬似乱数からで作成）を使って再現することができる。

例えば、確率 α で $s_i(t+1) = 1$ とし、確率 $1-\alpha$ で $s_i(t+1) = 0$ とするためには、

アルゴリズム 1

1. 区間 $[0,1]$ の擬似一様乱数 r を作る。
2. もし、$r \leq \alpha$ ならば、$s_i(t+1) = 1$ とし、そうでなければ、$s_i(t+1) = 0$ とする。

この方法を場所ごとに繰り返し、次時刻のサンプル状態 $\{s_i(t+1)\}$ を作れるので、時刻 $t = 0$ から順にサンプル状態列を得ることができる。例えば、図 7.2 は (7.2) の遷移確率をもつモデルにおいて、$p = p_c$ として、1 人だけ疫病にかかった初期状態から作成したサンプル状態列である。

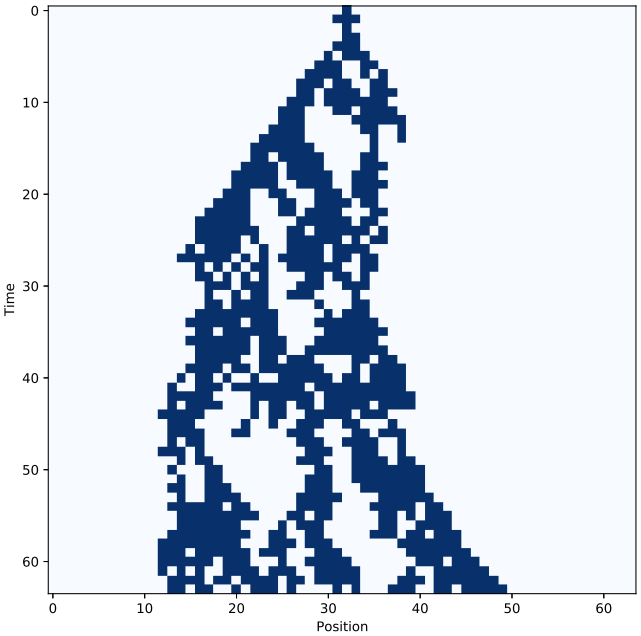


図 7.2: 疫病の確率的な伝染モデルの時間発展例。縦軸は時間、横軸は人の場所を表し、疫病にかかっている人の場所を濃青色で表している。場所の数は 64 で、時刻 $t = 0$ に中心の 1 人だけが疫病にかかっている人がいる設定にしている。遷移確率は (7.2) で $p = p_c$ とした。

病人数のサンプル平均

時刻 t で疫病にかかっている人数 $m(t) = \sum_i s_i(t)$ のサンプル値平均は、サンプル数が無限の極限では、確率変数である $m(t)$ の期待値に収束する。逆に言えば、有限のサンプルから計算したサンプル値平均は、確率変数 $m(t)$ の期待値の近似と考えることができる。このように、確率変数の期待値をサンプル値平均として近似的に評価する方法を、**モンテカルロ法**と呼ぶ。また、モンテカルロ法の実行を**モンテカルロシミュレーション**という。今の場合、アルゴリズム 1 を用いることで、疫病の確率的な伝染モデルに従ったサンプル状態列を作成することができているので、時刻 t のサンプル状態 $\{s_i(t)\}$ を用いて、確率変数 $m(t)$ の期待値の近似的評価として、$m(t)$ のサンプル値平均を計算することができる。

図 7.3 は、サンプル数 M を 100、10000 とした場合の (7.2) に従うモデルでの各時刻の $m(t)$ のサンプル値平均とその分散の平方根を同時にプロットしている。このように、十分なサンプル数を用いたサンプル値平均は、時刻 t で疫病にかかっている人数の期待値 $\langle m(t) \rangle = \sum_i \langle s_i(t) \rangle$ に収束していそうなことが分かる[3]。

問題 1

1. 遷移確率 (7.2) で $p = 0.7$ に従ったサンプル状態列を 10 個作成し、各サンプルに対する各時刻での $m(t)$ を同時にプロットせよ。ただし、システムサイズは 64、最終時刻は 256 までとし、初期状態は、中心の 1 人だけが疫病にかかっているとする。また、境界条件は周期的とする。つまり、$s_{65}(t) = s_1(t)$、$s_0(t) = s_{64}(t)$ と定義する。
2. $p = 0.64, 0.66, 0.68, 0.7$ の場合で、各時刻の $m(t)$ のサンプル数 $M = 10000$ のサンプル値平均を両対数グラフでプロットせよ。ただし、システムサイズ、最終時刻、初期状態、境界条件は前問と同じとする。

[3] $\langle \cdot \rangle$ は統計物理学での期待値を表す記号である。

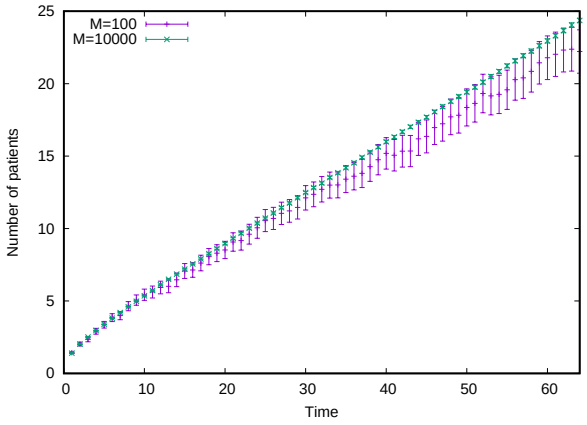


図 7.3: 疫病の確率的な伝染モデルの病人数のサンプル値平均。横軸は時間とし、場所の数は 64 で、時刻 $t = 0$ に中心の 1 人だけが疫病にかかっている人がいる設定にしている。遷移確率は (7.2) で $p = 0.7$ とした。サンプル数 M は、100, 10000 としている。

7.3 期待値のモンテカルロ計算

この節では、確率変数の期待値計算について、改めて、サンプル値平均による確率的近似計算としてのモンテカルロ法を定式化する。

システムが状態 X になる確率を $P(X)$ とする。その場合、状態 X に依存する量 $f(X)$ の期待値 $\langle f(X)\rangle$ は次式で定義される[4]。

$$\langle f(X)\rangle \equiv \sum_X f(X)P(X)$$
$$\text{又は} \equiv \int dX\ f(X)P(X). \qquad (7.3)$$

ここで、後者は状態変数 X が連続変数だった場合を表す。

状態の総数が少ない場合は全ての X について $f(X)P(X)$ を足し上げる事で計算することができるが、例えば、$X = (x_1, x_2, ..., x_D)$ のような多成分の場合、多重和となり全列挙による計算は困難になる。

[4]例えば、$f(X)$ は状態が X の時の対象からの出力値等。

$$\langle f(X)\rangle \equiv \sum_{x_1} \cdots \sum_{x_D} f(x_1, \ldots, x_D)P(x_1, \ldots, x_D),$$
$$\text{又は} \equiv \int \cdots \int \prod_{i=1}^{D} dx_i f(x_1, \ldots, x_D)P(x_1, \ldots, x_D). \qquad (7.4)$$

実際、状態変数 X の各成分 x_i が K 種類の値を取りうる時、状態総数は K^D となり、多重度 D に対して指数関数的に増加する。連続変数の場合も同様である。例えば、D 重積分の場合、それぞれの積分区間を K 個ずつに分割すると積分領域は K^D 個の積分要素に分割される。これは多重度 D に対して指数関数的に大きくなるため、その全列挙は困難である。従って、多重度が大きな場合、全列挙しない方法が必要である。

元々、各状態 X が出現する確率が $P(X)$ として与えられていたので、素朴に、確率 $P(X)$ に従って生成されたサンプル集団 $\{X_t\}$ の平均として、期待値を近似的に求めることができるように思われる。

$$\langle f(X)\rangle \approx \frac{1}{M} \sum_{t=1,\ldots,M} f(X_t). \qquad (7.5)$$

このような計算手法を**重み付きモンテカルロ法**と呼ぶ[5]。

(7.5) の右辺のサンプル値平均が真の期待値に収束することは容易に示す事ができる。サンプル値平均は確率変数の集合 $\{X_t\}$ によって定義されているのでこれ自身も確率変数と見なす事ができる。従って、その期待値を計算すると次の様になる。

$$\left\langle \frac{1}{M} \sum_{t=1,\ldots,M} f(X_t) \right\rangle = \frac{1}{M} \sum_{t=1,\ldots,M} \langle f(X_t)\rangle$$
$$= \langle f(X)\rangle. \qquad (7.6)$$

分散も同様に計算すると、

$$\frac{1}{M}\left[\langle f^2(X)\rangle - \langle f(X)\rangle^2\right] \qquad (7.7)$$

[5]確率的な方法であるため、カジノで有名な都市にちなんだ名前が付いている。

となることが分かる。従って、サンプル数 M が大きな極限で期待値 $\langle f(X)\rangle$ に収束する事が示せた。もう少し正確に言えば、**中心極限定理**により、重み付きモンテカルロ法で求めたサンプル平均値が、サンプル数 M が大きい極限で、次の平均と分散をもつガウス分布に収束する事がわかる。

$$\begin{aligned} &\text{平均 } \langle f(X)\rangle, \\ &\text{分散 } \frac{1}{M}\left[\langle f^2(X)\rangle - \langle f(X)\rangle^2\right]. \end{aligned} \tag{7.8}$$

この結果からモンテカルロ法の誤差はサンプル数 M が大きくなると $\frac{1}{\sqrt{M}}$ で減少していくことが分かる。

このように、モンテカルロ法は期待値の近似的な値を出すだけで、その誤差はサンプル数と共に減少していくが速度は早くはない。そのため、メリットがないように思われるが、上で述べたように全列挙が困難な場合、つまり、多重度が高い場合は誤差が多重度によらない事が大きなメリットになる。

これまでは、期待値計算の手法として、モンテカルロ法を紹介してきたが、**通常の多重積分や多重和**も、適時、確率分布 $P(X)$ に相当するものを設定すれば、モンテカルロ法で計算可能となる。例えば、多重和の状態総数や多重積分の積分範囲の大きさがわかっている場合は、$P(X) = 1/$(状態総数)、又は、$1/$(積分範囲の大きさ) に設定することができ、モンテカルロ法で多重和や多重積分が評価できる。

上で述べたように (7.8) のガウス分布の分散のルートがモンテカルロ法の精度を意味するが、与えられた確率分布による期待値を解析的に得ることは一般的にはできず、(7.8) 自身の評価は通常できない。そこで, (独立な) モンテカルロ計算の平均値の集合 $\{\bar{f}_i\}$ から、近似的に分散を推定することで精度の評価を行うことがしばしば行われる。その場合、期待値の推定値とその精度は次式で与えられる.

$$\begin{aligned} &\text{推定値 } \bar{\bar{f}} \equiv \frac{\sum_{i=1}^{N_M} \bar{f}_i}{N_M}, \\ &\text{精度 } \quad \sqrt{\frac{\sum_i \left(\bar{f}_i - \bar{\bar{f}}\right)^2}{N_M(N_M-1)}}. \end{aligned} \tag{7.9}$$

ここで、N_M は独立なモンテカルロ計算 $\{\bar{f}_i\}$ の回数である。精度は中心極限定理における標本標準偏差に相当する。

7.3.1 円周率の計算

以下の積分にモンテカルロ法を適用し、円周率をモンテカルロ法で求める事を考える。

$$\int_0^1 dx\ \frac{4}{1+x^2} = \pi. \tag{7.10}$$

いま、状態変数を区間 $[0,1]$ に属する 1 次元実数 X と考える。そして、確率密度を $P(X) = 1$ とすれば、円周率を $\pi = \langle 4/(1+X^2)\rangle$ のように期待値と見なすことができ、モンテカルロ法によって期待値（円周率）を評価する事ができる。

区間 $[0,1]$ の一様分布に従うサンプル列 $\{X_t\}$ はすでに述べたように擬似乱数から作成することができるので、それを用いて円周率をモンテカルロ法で評価することができる。

問題 2

1. 円周率を求める為に (7.10) の左辺のモンテカルロ計算を行う。サンプル数は $10, 10^3, 10^5, 10^7$ とした独立な計算を 10 回おこなうことで、(7.9) から各サンプル数での推定値とその精度を報告せよ。

2. 前問の結果を (7.8) との比をとって比較し、(7.9) の妥当性を検証せよ。

7.3.2 サンプルの作成方法

重み付きモンテカルロ法では、確率分布 $P(X)$ に従ったサンプルを作成する方法が必要である。その手法としては、下記で述べる直接的なものと、次節で紹介する**マルコフ過程**を用いるものの 2 種類がある。

低次元確率分布の場合

与えられた確率分布が指数分布やガウス分布などの低次元の確率分布の場合は、一様分布等を使う事で直接的にサンプルを作成することができる。このような手法には、**逆関数法**と**棄却法**が有名である。

逆関数法　区間 [0,1] の一様分布に従う確率変数 U とターゲット確率分布の累積分布関数 $F(x) = \Pr(X < x)$ の逆関数による次の関係式より、ターゲット確率分布のサンプルを作ることができる。

$$X = F^{-1}(U) \tag{7.11}$$

具体的には、次の手順で作成する。

アルゴリズム 2

1. サンプル v を区間 [0,1] の一様分布に従って作成。
2. ターゲット確率分布のサンプル $x = F^{-1}(v)$。

累積分布関数の逆関数が解析的に計算できる多くの 1 次元確率分布に対してこの手法は適用可能である。

棄却法　前提として、次のような式を満たす確率分布 $g(v)$ に従うサンプル v を生成できるとする。

$$^{\forall}x, p(x) \leq cg(x) \quad (c > 1) \tag{7.12}$$

すると、以下の手順で、作成可能な確率分布 $g(\cdot)$ からのサンプリングを用いて、ターゲット確率分布 $p(\cdot)$ のサンプルを作成することができる。

アルゴリズム 3

1. v を分布 $g(v)$ に従って作成。
2. 確率 $p(v)/(cg(v))$ で v をサンプル x として採用、そうでなければ棄却。
3. 採用されるまで繰り返す。

この手法で採用されるまでに作られるサンプル候補の平均数は c 回である。

高次元確率分布の場合

高次元では、各成分が独立である場合や、疫病の確率的な伝染モデルのように遷移確率により、サンプルを直接的に作れる場合もあるが、一般に、サンプルを効率的に作ることは難しい。このような場合によく用いられるのが、次節で述べる**確率過程**の 1 種である**マルコフ過程**（連鎖）を用いてサンプルを生成する**マルコフ連鎖モンテカルロ法** [3] である。

7.4　マルコフ連鎖モンテカルロ法

7.4.1　マルコフ過程とその状態の出現頻度の分布

確率過程は状態列を表す確率的モデルの一つで、特に、マルコフ過程は過去の有限ステップの状態にだけ依存して未来の状態が確率的に決定される確率過程として定義される。最もシンプルなマルコフ過程の例は、過去 1 ステップ分だけに依存する場合である。このようなマルコフ過程では、現在の状態 $X(t)$ に対し未来の状態が $X(t+1)$ である確率、遷移確率 $P(X(t) \to X(t+1))$ によって定義される。

定義より遷移確率 $P(X \to Y)$ は次の性質を満たす。

$$^{\forall}X, \quad \sum_{Y} P(X \to Y) = 1. \tag{7.13}$$

状態空間（全状態から構成される空間）の要素である各状態に番号をつけて X_i と書く事にして、遷移確率 P から次の遷移行列 T を定義すると便利である: $T_{ij} \equiv P(X_j \to X_i)$. このような行列 T は確率行列（stochastic matrix）とも呼ばれる。ここでは、遷移行列 T の成分は非負であるが、特に $T^k > 0$、つまり全ての成分が正となる整数 k が存在する事を仮定する（このような性質をもつ非負行列を既約であるという）。この仮定は有限回の遷移でどの状態からスタートしても任意の状態に到達する可能性があると言う事を意味する。T^k をあたらめて遷移行列とおけば、ペロン・フロベニウスの定理より、遷移行列の最大固有値は

1 であり対応する固有ベクトルは 1 つしかなく、その成分は全て正で、さらに他の固有値は絶対値が 1 未満である事が示される。今、ステップ t での各状態を取る確率を成分とするベクトルを $\mathbf{v}(t)$ とすると、ステップ 0 からスタートして、ステップ t のベクトル $\mathbf{v}(t)$ は次のようになる。

$$\mathbf{v}(t) = (T)^t \mathbf{v}(0). \tag{7.14}$$

従って、ステップ数を大きくすると固有値 1 に対応するベクトル $\mathbf{v}_T$ に収束することが分かる。つまり、マルコフ過程のサンプル列の分布は次の定常分布 W_T に収束する。

$$W_T(X_i) \equiv \frac{(\mathbf{v}_T)_i}{\sum_i (\mathbf{v}_T)_i} \tag{7.15}$$

ベクトル $\mathbf{v}_T$ の定義式を

$$\mathbf{v}_T = T\mathbf{v}_T. \tag{7.16}$$

と書き換えると、次式（**釣り合い条件**）になる。

$$^\forall X, \quad W(X) = \sum_Y P(Y \to X) W(Y). \tag{7.17}$$

従って、逆にこの式を満たす遷移確率に基づくマルコフ過程から、確率分布 W に従うサンプル列が生成されることが分かる。

(7.17) が釣り合い条件と呼ばれる理由は、右辺が状態 X に入ってくる割合、左辺が状態 X から出て行く割合を表すからである（$\sum_Y P(X \to Y) = 1$ より）。もし、状態の出現頻度がこの釣り合いを破っていても、負のフィードバックが働き、釣り合い条件 (7.17) が成立するような頻度に自動的に戻ってくる。例えば、状態 X が出現しすぎているということは、(7.17) で確率分布 W を頻度に置き換えた時に左辺が大きいということになっている。そのような時は出て行く割合の方が大きくなるので、最終的に釣り合いがとれる方向に状態 X の頻度が減少して行く。

7.4.2 マルコフ連鎖モンテカルロ法のアルゴリズム

釣り合い条件 (7.17) を満たす遷移確率は一つだけではない。実際、さまざまな手法が提案されている。代表的なものが**メトロポリス・ヘイスティングス法（メトロポリス法）**や**ギブスサンプリング（熱浴法）**である。以下では、それぞれを物理での名称であるメトロポリス法、熱浴法と呼ぶ。それぞれの手順は以下の通りである[6]。

メトロポリス法

初期状態 $X(0)$ を与え、以下の手順で状態列を更新する。

手順 1 現状態を $X(t)$ とし、次状態候補 X' を確率分布 $Q(X(t), X')$ に従って作成する。ただし、簡単のため、確率分布 $Q(X, Y)$ は対称 $(Q(X, Y) = Q(Y, X))$ と仮定する。

手順 2 次状態候補 X' を確率 $\min\left[1, \frac{W(X')}{W(X(t))}\right]$ で次の状態（$X(t+1) \equiv X'$）とし、そうでなければ、現在の状態 $X(t)$ を次の状態（$X(t+1) \equiv X(t)$）とする。

手順 3 手順 1 に戻る。

このとき、$P(X \to X') = Q(X, X') \min\left[1, \frac{W(X')}{W(X(t))}\right]$ であることに注意されたい。

熱浴法

初期状態 $X(0)$ を与え、以下の手順で状態列を更新する。

手順 1 現状態 $X(t) = (x_1, x_2, \cdots)$ を構成する状態変数から 1 つをランダムに選ぶ。例えば、x_i を選んだとすると、この i 番目の状態変数だけを更新する（それ以外はそのまま）。その際、更新後の値 x_i' を以下の確率にしたがって決定する。

$$P(x_i') = \frac{W(x_1, \cdots, x_i', \cdots)}{\sum_{x_i} W(x_1, \cdots, x_i, \cdots)} \tag{7.18}$$

手順 2 手順 1 に戻る。

[6] ここでは説明の為に設定を簡略化してある。

熱浴法は、もっと一般的に書くと、結局、選んだ変数をそれ以外が与えられた時の条件付き確率に従って更新していると言える。解析的に条件付き確率が計算できる場合に広く用いられている。

これらのアルゴリズムでは、次状態の作成に、確率分布の比だけしか必要ない。これは、確率分布が正規化をのぞいて定義されているような場合、例えば、統計物理学のモデルのようにボルツマン重みだけから確率分布（ボルツマン分布）が定義されている場合に、特に有用である。実際、ボルツマン重みの和である規格化因子は分配関数と呼ばれ、それ自身が物理量の生成関数として、非常に重要であるが、計算することは多重和の計算量の問題から困難であり、様々な計算手法の研究が現在もなされている。

問題 3

1. メトロポリス法の遷移確率が次の**詳細釣り合い条件**を満たす事を証明せよ。

$$^{\forall}X, ^{\forall}Y \text{ に対して}$$
$$P(X \to Y)W(X) = P(Y \to X)W(Y). \tag{7.19}$$

2. 熱浴法の遷移確率が詳細釣り合い条件を満たす事を証明せよ。

3. 遷移確率が詳細釣り合い条件 (7.19) を満たす場合は、釣り合い条件 (7.17) が成立する事を証明せよ。

7.4.3 マルコフ連鎖モンテカルロ法で注意すべき点

マルコフ過程を用いてサンプルを生成するとき、各サンプルは過去のサンプルに依存して確率的に生成されてきたので、当然、サンプル同士に統計的な相関がある。通常、有限個しかサンプルは生成できないので、相関が強い場合は、実質的な独立なサンプル数が少なくなる。(7.8) の分散の分母は統計的に独立なサンプル数なので、モンテカルロ計算の精度は相関が強くなると悪くなる。その為、相関の強さを確認しながら計算を進めることが必要である。もちろん、相関を弱める手法の研究も進んでいる[7]。

7.5 イジングモデル

統計物理は多数の要素からなる系の性質を対象とする物理学の理論の 1 つである。ボルツマンなど多くの研究者により、特に平衡系の性質に対する強力な理論が構築されている。その成果は、近年、物理現象だけではなく、ニューラルネットワークやネットワークコミュニティーなど、多数の要素からなる一般的なシステムの解析にも応用されるなど、様々な分野で活用されている。

統計物理での重要な概念の理解には理論モデルが有用である。その中でも**イジングモデル**は各要素が相互に影響する最も基本的なモデルであり、統計物理では極めて重要なモデルとなっている。また、近年注目されている機械学習分野でも**ボルツマンマシン**と呼ばれ、基礎的な計算モデルとして用いられている。以下では、イジングモデルのシミュレーションにマルコフ連鎖モンテカルロ法を適用する方法について述べる。

7.5.1 エネルギー

イジングモデルは多数の素子から構成されるシステムである。各素子の状態を表す変数 s_i は、± 1 の値のどちらかをとる。そして、システムのエネルギー E は以下の形の相互作用の総和になっている。

$$E(S) \equiv -\sum_{\langle i,j \rangle} J_{ij} s_i s_j \tag{7.20}$$

ここで、$\langle i, j \rangle$ は相互作用対 (i, j) を、J_{ij} は s_i と s_j の間の相互作用の強さを決める相互作用係数である。相互作用はダブルカウントしないことに注意。

統計物理の理論によると、周りの環境とのエネルギーのやりとりがあると、システムの状態

[7] 一般的にはレプリカ交換法などのアンサンブルを拡張するタイプの方法がいくつか提案されている [3, 5]。

$S = \{s_i\}$ は環境の温度 T によって支配される次の**ボルツマン分布（カノニカル分布）**に従う。

$$P(S) \equiv \exp(-E(S)/k_BT)/Z \tag{7.21}$$

ここで、k_B は**ボルツマン定数**と呼ばれる。また、Z は規格化因子、すなわち

$$Z = \sum_S \exp(-E(S)/k_BT) \tag{7.22}$$

であり、分配関数と呼ばれる。さらに、

$$F = -k_BT\log Z \tag{7.23}$$

という形で**自由エネルギー** F を定義する。以下、簡単のため、ボルツマン定数 $k_B = 1$ とする。また、$\exp(-E(S)/k_BT)$ を**ボルツマン重み**と呼ぶ。

したがって、状態変数による物理量 A の期待値は以下の式で定義される。

$$A(T) \equiv \sum_S A(S)P(S) \tag{7.24}$$

$$= \sum_S A(S)\exp(-E(S)/T)/Z \tag{7.25}$$

$$\equiv \langle A(S)\rangle \tag{7.26}$$

ここでは、記号 $\langle . \rangle$ は、統計物理で用いられる期待値を表す記法である。

7.5.2 トーラス上のイジングモデルの相転移現象（臨界現象）

一般に、要素数が無限大の極限（**熱力学的極限**）では、システムのコントロールパラメータ（イジングモデルでは温度）を変えると、システム全体の性質が突然切り替わる現象が現れる。システム全体の性質が本質的に同じである領域を**相**と呼び、それが突然に切り替わるこの現象を**相転移現象**と呼ぶ。イジングモデルの研究により、特に、相転移現象の一種である**臨界現象**に関する理解が非常に進んだ。特に 2 次元イジングモデルは臨界現象を示す最も簡単なモデルとして重要であり、以下で取り上げることにする。

7.5.3 対称性の自発的破れ

ドーナツ形状（トーラス）の表面に貼付けたグリッドを考える（図 7.4 参照）。グリッドの格子点の座標を (x, y) $(0 \leq x, y \leq (L-1))$ とすると、ドーナツ形状より格子点 (L, y) は $(0, y)$ と、(x, L) は $(x, 0)$ と同一点になる（**周期的境界条件**）。この各格子点にイジング素子（イジングスピン）が 1 つずつあり、上下左右とだけ相互作用をしているとする。特に相互作用定数 J が正で全て同じである場合は 2 次元**強磁性**イジングモデルと呼ばれる。

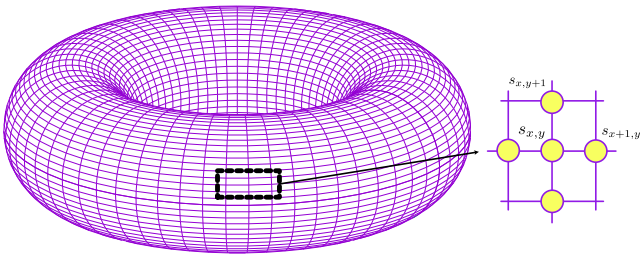


図 7.4: トーラス上のイジングモデル。各格子点上のイジングスピンが上下左右の 4 つの格子点上の各イジングスピンと相互作用している。

強磁性イジングモデルは全てのイジングスピンの符号を反転させてもエネルギーが変わらないという対称性をもつ。エネルギーが変わらないのでボルツマン分布による状態の出現確率は全反転でも同じである。つまり、ある状態とそれを全反転した状態は同じ確率で出現する。一方で、温度を下げていくと、エネルギーが低い状態の出現確率がボルツマン分布の式から上がることが分かる。したがって、温度が非常に低くなると、全てのイジングスピンが同符号になる状態が出現することになる。しかも、上に述べたように、対称性があるので、そのような状態は 2 つある。

しかし、環境とのエネルギーのやりとりは局所的である為、状態の変更は局所的にしかおきず、全反転するような状態に変化する為には途中にエネルギーが大きなところを通る必要がある。その為、低温では片方の状態からもう片方の状態に遷移することはほぼ起きない。このようにもともと

あった対称性が勝手に破れたような現象を自発的対称性の破れという。実際、2 次元イジングモデルは、熱力学的極限である温度以下で自発的対称性の破れが起きることが証明されている。

さらに、対称性が自発的に破れた相への相転移現象も興味深い。例えば、相転移が起きる温度（**臨界温度**）では、ちょうどイジングスピンが揃い始めるギリギリのところで、図 7.5 にあるように、様々なスケールのイジングスピンが揃ったクラスターが**自己相似的**（**フラクタル**）に現れることが知られている。

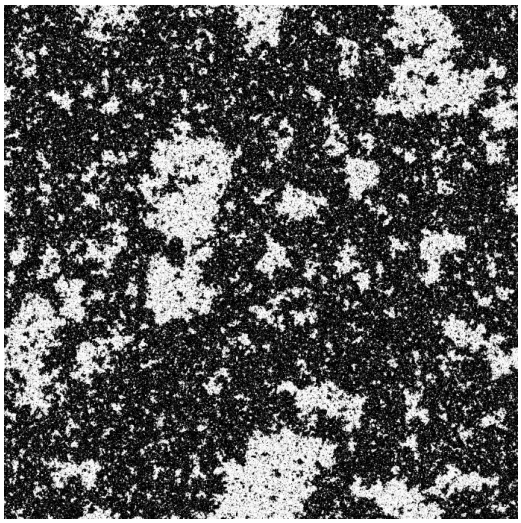


図 7.5: 臨界温度でのトーラス上のイジングモデルの状態例。イジングスピンが +1 になっている場所を黒く塗っている。システムサイズは $L = 2048$。

自発的な対称性の破れをみる為に磁化 m と呼ばれる以下の物理量を考える。

$$m = \frac{1}{N}\left|\sum_i s_i\right| \tag{7.27}$$

ここで、$|\cdot|$ は絶対値を表し、N はイジングスピン数を表す。有限系では対称性より絶対値をつけない場合はこの量の期待値は常に 0 になるが、絶対値をつけることで揃ったイジングスピンが多い場合は大きくなるので、対称性の破れを有限系でも見ることができる。

7.5.4 イジングモデルのメトロポリス法

以下の手順（1 **Monte Carlo Sweep**(MCS) と呼ぶ）で次の状態 $S(t+1)$ を確率的に作成する。

手順 1 ランダムに格子点を 1 つ選ぶ。

手順 2 選んだ格子点の状態変数 s をフリップしたものを次状態候補 s' とする。フリップとは、$s = 1$ なら -1 に、またはその逆の操作を意味する。

手順 3 区間 [0,1] の一様乱数 r を作成し、もし、

$$r \leq \min\left[1, \frac{P(\cdots, s', \cdots)}{P(\cdots, s, \cdots)}\right] \tag{7.28}$$

ならば s をフリップする。それ以外は何もしない。

手順 4 以上の手順を総格子点数の回数だけ行う。

ここでは、手順 2 でランダムに格子点を選んだが、これを格子を規則的に 1 巡するものに置き換えても良い。例えば、端から順番に更新しても良い。この場合も、格子を一巡（1MCS）するごとに、状態を見れば、詳細釣り合いを満たすからである。

問題 4

トーラス上の強磁性イジングモデル $J_{ij} = J = 1$（i, j は相互作用対）を考える。システムサイズは $L \times L$ とする。

1. イジングモデルの熱浴法の手順を書け。
2. マルコフ連鎖モンテカルロ法のサンプル列内の相関を見る為に、初期状態の影響が消える様子をグラフを用いて示せ。観察する量は各ステップでの磁化とする。ただし、温度 T は $z = \exp(2/T) - 1 = \sqrt{2}$ の解、システムサイズは $L = 64$ とし、初期状態 $S(t = 0)$ は全てのイジングスピンが 1 とする。特に、メトロポリス法、熱浴法それぞれでシミュレーションを行い、それらの結果を比較すること。

3. 磁化の期待値について、温度とシステムサイズへの依存性をモンテカルロ法により求めて報告せよ。ただし、z は区間 $(1.2, 1.5]$ 内に均等に 12 点以上、システムサイズは $L = 12, 16, 24, 32, 48, 64$ とし、各モンテカルロ計算でのサンプリング数はトータルで 10^6 MCS 以上（例: 10^5 MCS を 10 セット）とする。また、初期値の影響が残らないように、問題 (2) の結果を参考に、最初の（十分な）ステップを平均値の計算には含めないようにすること。

4. (3) の計算結果に基づき、システムサイズが大きくなった極限での磁化の割合の振る舞いを議論せよ（特に相転移が起きる温度とその近傍での振る舞いについて）。

7.6 おわりに

本章では数値計算手法としてのモンテカルロ法のさまざまな性質については証明などはせず、簡単に紹介だけをした。例えば、収束性などについて詳しく知りたい場合は、参考文献 [3] などを参考にされたい。

また、今回、取り上げなかったモンテカルロ法のアルゴリズムも多くある。特に、連続的な確率変数に対するマルコフ連鎖モンテカルロ法のアルゴリズムとしては、素粒子物理学での連続場の理論モデルのモンテカルロシミュレーションのために考案された**ハイブリッド（ハミルトニアン）モンテカルロ法**がよく用いられている。これについては、パターン認識などの機械学習分野での応用も多く、参考文献 [4] に詳しい。さらに、イジングモデルの 1 種で相互作用にランダムなフラストレーションがあるスピングラスモデルや不連続転移を示すモデルなど、マルコフ連鎖を用いてもサンプリングが困難な場合によく用いられる**拡張アンサンブル法**（**レプリカ交換法**等）についても取り上げなかった。これについては、最近の講義録 [5] を参考文献として上げておく。

冒頭に述べたように、モンテカルロ法は多数の要素からなる確率的なシステムの研究には欠かせないものとなっている。このような多数の要素からなるシステムは物理では**多体系**と呼ばれ、素粒子から物質、宇宙までのスケールを超える研究対象となっている。多体系では、各要素の振る舞い（相互作用）は単純であってもシステム全体としては、相転移現象などに見られる、相互作用から非自明な現象が創発し、興味深い研究の対象となっている。このような多体系の特性に関するアンダーソン博士の ***More is different*** の気持ちを、モンテカルロシミュレーションを通して、多くの人に知っていただければと思う [6]。

参考文献

[1] E. Domany and W. Kinzel: “Equivalence of Cellular Automata to Ising Models and Directed Percolation,” Phys.Rev.Lett., Vol. 53, No. 4, pp. 311 (1984).

[2] M. Henkel, H. Hinrichsen, and S. Lübeck: Non-Equilibrium Phase Transitions. Volume 1: Absorbing Phase Transitions, Springer (2008).

[3] 伊庭 幸人, 種村 正美, 大森 裕浩, 和合 肇, 佐藤 整尚, 高橋 明彦: 計算統計 II マルコフ連鎖モンテカルロ法とその周辺 (統計科学のフロンティア 12), 岩波書店 (2005 年).

[4] C.M. ビショップ: （日本語版）パターン認識と機械学習, 丸善出版 (2012).

[5] 福島 孝治: モンテカルロ法の基礎と応用 : 計算物理学からデータ駆動科学へ, 物性研究・電子版, Vol.7, No.2., pp. 072214 (2018 年).

[6] 原田 健自: 一般書籍「数理工学の世界」 第 3 章「沢山からできている世界」, 日本評論社 (2019 年).

[原田健自]

第8章　熱方程式の差分法

8.1　目的

偏微分方程式とは, 2 つ以上の独立変数と未知関数およびその未知関数の偏微分を含むような方程式である. 独立変数を x, t とし, 未知関数を $u(x,t)$ とすると, 偏微分方程式の一般系は

$$f\left(t, x, u, \frac{\partial u}{\partial t}, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial t^2}, \frac{\partial^2 u}{\partial x^2}, \frac{\partial^2 u}{\partial t \partial x}, \cdots\right) = 0$$

となる.

偏微分方程式は様々な現象の数学的記述に現れる. 具体例としては, 波動の時間発展を記述する波動方程式や, 静電ポテンシャルの関数形を与えるラプラス方程式, そして本演習で扱う熱方程式 (拡散方程式) などがある.

偏微分方程式を解くとは, 決められた (x,t) に対して $u(x,t)$ を求めることである. 解析的に解くのが困難な場合, 数値計算を用いて近似的に解くことになる.

本章では, 偏微分方程式の数値解法の一例として, 熱方程式 (拡散方程式) の差分法を学修する [1, 2].

8.2　拡散方程式

熱方程式については, 第 5 章で簡単に紹介した. 本節では, 拡散方程式について概説する.

拡散とは, 微粒子などが自発的に散らばり広がっていく現象である. 簡単のため, 1 次元空間内での拡散を考える. 微粒子の濃度を $C(x,t)$, 微粒子の流れを $J(x,t)$ とすると, 粒子数の保存を表す連続の式は

$$\frac{\partial C}{\partial t}(x,t) = -\frac{\partial J}{\partial x}(x,t) \tag{8.1}$$

と書ける. また, 流れ $J(x,t)$ は濃度勾配に比例すると仮定すると

$$J(x,t) = -D\frac{\partial C}{\partial x}(x,t) \tag{8.2}$$

と表される. これらから, 拡散方程式

$$\frac{\partial C}{\partial t}(x,t) = \frac{\partial}{\partial x}\left(D\frac{\partial C}{\partial x}\right)(x,t)$$

が得られる. 以下では, $D = 1$ とした拡散方程式

$$\frac{\partial u}{\partial t}(x,t) = \frac{\partial^2 u}{\partial x^2}(x,t) \tag{8.3}$$

を考える.

1 次元有界区間 $(0, L)$ 内で拡散方程式 (8.3) を解くためには, 初期条件

$$u(x,0) = u_0(x)$$

と, 境界 $x = 0, L$ における境界条件を考える必要がある. 代表的な境界条件には, 以下の 2 つがある.

- Dirichlet 境界条件: 境界上での u の値が指定されている;

$$u(0,t) = u_{\mathrm{L}}, \quad u(L,t) = u_{\mathrm{R}},$$

- Neumann 境界条件: 境界上での u の微分の値が指定されている;

$$\frac{\partial u}{\partial x}(0,t) = J_{\mathrm{L}}, \quad \frac{\partial u}{\partial x}(L,t) = J_{\mathrm{R}}.$$

8.3　数値解法

本節では, 時間と空間を離散化して拡散方程式 (8.3) を数値的に解くことを考える.

時間と空間の刻み幅をそれぞれ Δt, Δx とする $(L = N\Delta x)$. 拡散方程式 (8.3) の解 u に対して, 離散化された時空間上での平均濃度を

$$U_j^n \equiv \frac{1}{\Delta x}\int_{(j-1)\Delta x}^{j\Delta x} u\left(y, n\Delta t\right)\, dy \tag{8.4}$$

と書く. U_j^n は区間 $[(j-1)\Delta x, j\Delta x]$ に対して割り当てられていることに注意する.

方程式 (8.1) を $[n\Delta t,(n+1)\Delta t]$ で積分すると,

$$\begin{aligned} &u\left(x,(n+1)\Delta t\right)-u\left(x,n\Delta t\right)\\ =&-\int_{n\Delta t}^{(n+1)\Delta t}\frac{\partial J}{\partial x}J(x,s)\,ds \end{aligned} \tag{8.5}$$

となる. (8.5) の右辺の積分を時刻 $n\Delta t$ の値で近似すると

$$\begin{aligned} &u\left(x,(n+1)\Delta t\right)-u\left(x,n\Delta t\right)\\ \simeq&-\Delta t\frac{\partial J}{\partial x}\left(x,n\Delta t\right) \end{aligned}$$

となる. この場合,

$$\begin{aligned} &U_j^{n+1}-U_j^n\\ \simeq&-\frac{1}{\Delta x}\int_{(j-1)\Delta x}^{j\Delta x}\Delta t\frac{\partial J}{\partial y}\left(y,n\Delta t\right)\,dy\\ =&-\frac{\Delta t}{\Delta x}\left[J(j\Delta x,n\Delta t)-J((j-1)\Delta x,n\Delta t)\right] \end{aligned}$$

となる. ただし, (8.2) より

$$J(j\Delta x,n\Delta t)=-\frac{\partial u}{\partial x}(j\Delta x,n\Delta t)$$

である. この量はさらに

$$J(j\Delta x,n\Delta t)\simeq-\frac{U_{j+1}^n-U_j^n}{\Delta x}$$

と近似されるので, 近似関係式

$$U_j^{n+1}-U_j^n\simeq\frac{\Delta t}{\Delta x^2}\left(U_{j-1}^n-2U_j^n+U_{j+1}^n\right)$$

を得る. このことを踏まえ, 差分方程式

$$u_j^{n+1}-u_j^n=\frac{\Delta t}{\Delta x^2}\left(u_{j-1}^n-2u_j^n+u_{j+1}^n\right) \tag{8.6}$$

を考える. ただし, u_0^n および u_{N+1}^n は境界条件に応じて別途定める.

一方, (8.5) において積分をより精度の良い台形公式で近似すると,

$$\begin{aligned} &u\left(x,(n+1)\Delta t\right)-u\left(x,n\Delta t\right)\\ \simeq&-\frac{\Delta t}{2}\left[\frac{\partial J}{\partial x}\left(x,(n+1)\Delta t\right)+\frac{\partial J}{\partial x}\left(x,n\Delta t\right)\right] \end{aligned}$$

となる. これを用いると,

$$\begin{aligned} U_j^{n+1}-U_j^n\simeq&\frac{1}{2}\frac{\Delta t}{\Delta x^2}\left(U_{j-1}^{n+1}-2U_j^{n+1}+U_{j+1}^{n+1}\right)\\ &+\frac{1}{2}\frac{\Delta t}{\Delta x^2}\left(U_{j-1}^n-2U_j^n+U_{j+1}^n\right) \end{aligned}$$

が得られ, これに対応して差分方程式

$$\begin{aligned} u_j^{n+1}-u_j^n=&\frac{1}{2}\frac{\Delta t}{\Delta x^2}\left(u_{j-1}^{n+1}-2u_j^{n+1}+u_{j+1}^{n+1}\right)\\ &+\frac{1}{2}\frac{\Delta t}{\Delta x^2}\left(u_{j-1}^n-2u_j^n+u_{j+1}^n\right) \end{aligned} \tag{8.7}$$

を考える.

方程式 (8.6) は, $\left\{u_j^n\right\}_{j=1}^N$ から $\left\{u_j^{n+1}\right\}_{j=1}^N$ を直接求める漸化式になっている. このような数値解法を陽解法と呼ぶ. これに対して, 方程式 (8.7) は右辺にも $\left\{u_j^{n+1}\right\}_{j=1}^N$ が現れており, $\left\{u_j^{n+1}\right\}_{j=1}^N$ の連立方程式として解く必要がある. このような未知変数と既知変数の混在した差分方程式による数値解法を陰解法と呼ぶ.

8.3.1 Euler 陽解法

方程式 (8.6) による数値解法は, 特に Euler 陽解法と呼ばれる. 以下では,

$$c\equiv\frac{\Delta t}{\Delta x^2} \tag{8.8}$$

と書くことにする.

初期条件は $\left\{u_j^0\right\}_{j=1}^N$ と表される. また, 境界条件は次のように与える.

- Dirichlet 境界条件:

$$u_0^n=u_{\mathrm{L}},\quad u_{N+1}^n=u_{\mathrm{R}}.$$

- Neumann 境界条件:

$$u_0^n=u_1^n-J_{\mathrm{L}}\Delta x,\quad u_{N+1}^n=u_N^n+J_{\mathrm{R}}\Delta x$$

Euler 陽解法のアルゴリズムは, アルゴリズム 2 のようにまとめられる.

アルゴリズム 2 Euler 陽解法

Input: 初期条件 $\left\{u_j^0\right\}_{j=1}^N$, 境界条件
Output: 時刻 $n\Delta t$ $(n = 1, \cdots, T)$ における $\left\{u_j^n\right\}_{j=1}^N$

1: u_0^0, u_{N+1}^0 を境界条件に従って与える
2: **for** $n = 0$ to $T-1$ **do**
3: $\left\{u_j^n\right\}_{j=1}^N$ から $\left\{u_j^{n+1}\right\}_{j=1}^N$ を (8.6) に従って計算
4: u_0^{n+1}, u_{N+1}^{n+1} を境界条件に従って与える
5: $\left\{u_j^{n+1}\right\}_{j=1}^N$ を出力
6: **end for**

8.3.2 Crank-Nicolson 法

方程式 (8.7) による数値解法は Crank-Nicolson 法と呼ばれる. この方法は Euler 陽解法よりも精度が良くなっている.

方程式 (8.7) を書き直すと,

$$\begin{aligned} &-\frac{c}{2}u_{j-1}^{n+1} + (1+c)u_j^{n+1} - \frac{c}{2}u_{j+1}^{n+1} \\ =&(1-c)u_j^n + \frac{c}{2}u_{j-1}^n + \frac{c}{2}u_{j+1}^n \end{aligned} \tag{8.9}$$

となる. ただし, 境界条件は次のように与える.

- Dirichlet 境界条件: $u_0^n = u_{\mathrm{L}}$, $u_{N+1}^n = u_{\mathrm{R}}$ より,

$$\begin{aligned} &(1+c)u_1^{n+1} - \frac{c}{2}u_2^{n+1} \\ =&(1-c)u_1^n + cu_{\mathrm{L}} + \frac{c}{2}u_2^n, \end{aligned} \tag{8.10}$$

$$\begin{aligned} &-\frac{c}{2}u_{N-1}^{n+1} + (1+c)u_N^{n+1} \\ =&(1-c)u_N^n + cu_{\mathrm{R}} + \frac{c}{2}u_{N-1}^n. \end{aligned} \tag{8.11}$$

- Neumann 境界条件: $u_0^n = u_1^n - J_{\mathrm{L}}\Delta x$, $u_{N+1}^n = u_N^n + J_{\mathrm{R}}\Delta x$ より,

$$\begin{aligned} &\left(1+\frac{c}{2}\right)u_1^{n+1} - \frac{c}{2}u_2^{n+1} \\ =&\left(1-\frac{c}{2}\right)u_1^n - cJ_{\mathrm{L}}\Delta x + \frac{c}{2}u_2^n, \end{aligned} \tag{8.12}$$

$$\begin{aligned} &-\frac{c}{2}u_{N-1}^{n+1} + \left(1+\frac{c}{2}\right)u_N^{n+1} \\ =&\left(1-\frac{c}{2}\right)u_N^n + cJ_{\mathrm{R}}\Delta x + \frac{c}{2}u_{N-1}^n. \end{aligned} \tag{8.13}$$

非負整数 n を固定すると, 方程式 (8.9) と境界条件 (8.10)-(8.11) あるいは (8.12)-(8.13) を合わせた連立方程式は, $\boldsymbol{x} \equiv \left(u_1^{n+1}, \cdots, u_N^{n+1}\right)^{\mathsf{T}}$ と既知な量を成分とするベクトル $\boldsymbol{z}$, およびある行列 A を用いて

$$A\boldsymbol{x} = \boldsymbol{z} \tag{8.14}$$

のように表される. A は Dirichlet 境界条件では

$$A = \begin{pmatrix} 1+c & -\frac{c}{2} & 0 & \cdots & 0 \\ -\frac{c}{2} & 1+c & -\frac{c}{2} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\frac{c}{2} & 1+c & -\frac{c}{2} \\ 0 & \cdots & 0 & -\frac{c}{2} & 1+c \end{pmatrix},$$

Neumann 境界条件では

$$A = \begin{pmatrix} 1+\frac{c}{2} & -\frac{c}{2} & 0 & \cdots & 0 \\ -\frac{c}{2} & 1+c & -\frac{c}{2} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\frac{c}{2} & 1+c & -\frac{c}{2} \\ 0 & \cdots & 0 & -\frac{c}{2} & 1+\frac{c}{2} \end{pmatrix}$$

となる. また, $\boldsymbol{z}$ は Dirichlet 境界条件では

$$\begin{aligned} z_1 &= (1-c)u_1^n + cu_{\mathrm{L}} + \frac{c}{2}u_2^n, \\ z_j &= (1-c)u_j^n + \frac{c}{2}u_{j-1}^n + \frac{c}{2}u_{j+1}^n \\ &\quad (2 \le j \le N-1), \\ z_N &= (1-c)u_N^n + cu_{\mathrm{R}} + \frac{c}{2}u_{N-1}^n, \end{aligned} \tag{8.15}$$

Neumann 境界条件では

$$\begin{aligned} z_1 &= \left(1-\frac{c}{2}\right)u_1^n - cJ_{\mathrm{L}}\Delta x + \frac{c}{2}u_2^n, \\ z_j &= (1-c)u_j^n + \frac{c}{2}u_{j-1}^n + \frac{c}{2}u_{j+1}^n, \\ &\quad (2 \le j \le N-1), \\ z_N &= \left(1-\frac{c}{2}\right)u_N^n + cJ_{\mathrm{R}}\Delta x + \frac{c}{2}u_{N-1}^n \end{aligned} \tag{8.16}$$

である. 以下では, 行列の LU 分解を用いて, 方程式 (8.14) から $\boldsymbol{x}$ を求める.

LU 分解とは A を上三角行列 U と下三角行列 L の積 $A = LU$ に分解することである. 今, A を

$$A = \begin{pmatrix} a_1 & b_1 & 0 & \cdots & 0 \\ c_1 & a_2 & b_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & c_{N-2} & a_{N-1} & b_{N-1} \\ 0 & \cdots & 0 & c_{N-1} & a_N \end{pmatrix} \tag{8.17}$$

のように書く. 行列 L および U をそれぞれ

$$L = \begin{pmatrix} \alpha_1 & 0 & 0 & \cdots & 0 \\ c_1 & \alpha_2 & 0 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & c_{N-2} & \alpha_{N-1} & 0 \\ 0 & \cdots & 0 & c_{N-1} & \alpha_N \end{pmatrix},$$

$$U = \begin{pmatrix} 1 & \beta_1 & 0 & \cdots & 0 \\ 0 & 1 & \beta_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 0 & 1 & \beta_{N-1} \\ 0 & \cdots & 0 & 0 & 1 \end{pmatrix}$$

と置くと,

$$LU = \begin{pmatrix} \alpha_1 & \alpha_1\beta_1 & 0 & \cdots & 0 \\ c_1 & \alpha_2 + c_1\beta_1 & \alpha_2\beta_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & c_{N-2} & \alpha_{N-1} + c_{N-2}\beta_{N-2} & \alpha_{N-1}\beta_{N-1} \\ 0 & \cdots & 0 & c_{N-1} & \alpha_N + c_{N-1}\beta_{N-1} \end{pmatrix} \tag{8.18}$$

となる. (8.17) と (8.18) を比較すると

$$\begin{aligned} &\alpha_1 = a_1, \\ &\beta_1 = \frac{b_1}{\alpha_1}, \\ &\alpha_2 = a_2 - c_1\beta_1, \\ &\beta_2 = \frac{b_2}{\alpha_2}, \\ &\vdots \\ &\alpha_{N-1} = a_{N-1} - c_{N-2}\beta_{N-2}, \\ &\beta_{N-1} = \frac{b_{N-1}}{\alpha_{N-1}}, \\ &\alpha_N = a_N - c_{N-1}\beta_{N-1} \end{aligned} \tag{8.19}$$

のように昇順に L と U を求められる.

次に LU 分解を用いて連立方程式 (8.14) を解く方法を説明する. $LU\boldsymbol{x} = \boldsymbol{z}$ において $U\boldsymbol{x} \equiv \boldsymbol{y}$ とおくと,

$$L\boldsymbol{y} = \boldsymbol{z}$$

となる. この式から $\boldsymbol{y}$ を求め, $U\boldsymbol{x} = \boldsymbol{y}$ から $\boldsymbol{x}$ を求める. まず $L\boldsymbol{y} = \boldsymbol{z}$ は

$$\begin{pmatrix} \alpha_1 & 0 & 0 & \cdots & 0 \\ c_1 & \alpha_2 & 0 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & c_{N-2} & \alpha_{N-1} & 0 \\ 0 & \cdots & 0 & c_{N-1} & \alpha_N \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N-1} \\ y_N \end{pmatrix} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_{N-1} \\ z_N \end{pmatrix}$$

より, $\boldsymbol{y}$ は昇順に

$$\begin{aligned} &y_1 = \frac{z_1}{\alpha_1}, \\ &y_2 = \frac{z_2 - c_1 y_1}{\alpha_2}, \\ &\vdots \\ &y_{N-1} = \frac{z_{N-1} - c_{N-2} y_{N-2}}{\alpha_{N-1}}, \\ &y_N = \frac{z_N - c_{N-1} y_{N-1}}{\alpha_N} \end{aligned} \tag{8.20}$$

のように求まる. 次に $U\boldsymbol{x} = \boldsymbol{y}$ は

$$\begin{pmatrix} 1 & \beta_1 & 0 & \cdots & 0 \\ 0 & 1 & \beta_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 0 & 1 & \beta_{N-1} \\ 0 & \cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N-1} \\ y_N \end{pmatrix}$$

より, $\boldsymbol{x}$ は降順に

$$\begin{aligned} x_N &= y_N, \\ x_{N-1} &= y_{N-1} - \beta_{N-1} x_N, \\ &\vdots \\ x_2 &= y_2 - \beta_2 x_3, \\ x_1 &= y_1 - \beta_1 x_2 \end{aligned} \tag{8.21}$$

のように求まる. 実際の計算では $\{\alpha_j\}$, $\{\beta_j\}$ は配列に保存して置くと良い.

以上をまとめて, Crank-Nicolson 法はアルゴリズム 3 のように計算を行う.

アルゴリズム 3 Crank-Nicolson 法

Input: 初期条件 $\left\{u_j^0\right\}_{j=1}^N$, 境界条件
Output: 時刻 $n\Delta t$ $(n = 1, \cdots, T)$ における $\left\{u_j^n\right\}_{j=1}^N$

1: LU 分解 : 境界条件に合わせて $\{\alpha_j\}$, $\{\beta_j\}$ を (8.19) に従って計算
2: **for** $n = 0$ to $T-1$ **do**
3: $\left\{u_j^n\right\}_{j=1}^N$ から $\boldsymbol{z}$ を (8.15) または (8.16) に従って計算
4: (8.20) に従って $\boldsymbol{y}$ を計算
5: (8.21) に従って $\boldsymbol{x}$ を計算し, これを $\left\{u_j^{n+1}\right\}_{j=1}^N$ とする
6: $\left\{u_j^{n+1}\right\}_{j=1}^N$ を出力
7: **end for**

8.3.3 その他の陰解法

一般に (8.5) において $0 \le \theta \le 1$ を用いて

$$\begin{aligned} u_j^{n+1} - u_j^n =& \theta c \left(u_{j-1}^{n+1} - 2u_j^{n+1} + u_{j+1}^{n+1}\right) \\ &+ (1-\theta) c \left(u_{j-1}^n - 2u_j^n + u_{j+1}^n\right) \end{aligned} \tag{8.22}$$

という差分方程式を考えることができる. $\theta = 0$ が Euler 陽解法, $\theta = 1/2$ が Crank-Nicolson 法に対応する. また, $\theta = 1$ を完全陰的スキームと呼ぶ. (8.22) を書き直すと

$$\begin{aligned} & -\theta c u_{j-1}^{n+1} + (1+2\theta c) u_j^{n+1} - \theta c u_{j+1}^{n+1} \\ =& \{1 - 2(1-\theta)c\} u_j^n \\ &+ (1-\theta) c u_{j-1}^n + (1-\theta) c u_{j+1}^n \end{aligned} \tag{8.23}$$

となる. ただし, 境界条件は次のように与える.

- Dirichlet 境界条件: $u_0^n = u_{\mathrm{L}}$, $u_{N+1}^n = u_{\mathrm{R}}$ より,

$$\begin{aligned} & (1+2\theta c) u_1^{n+1} - \theta c u_2^{n+1} \\ =& \{1 - 2(1-\theta)c\} u_1^n + c u_{\mathrm{L}} \\ &+ (1-\theta) c u_2^n, \end{aligned} \tag{8.24}$$

$$\begin{aligned} & -\theta c u_{N-1}^{n+1} + (1+2\theta c) u_N^{n+1} \\ =& \{1 - 2(1-\theta)c\} u_N^n + c u_{\mathrm{R}} \\ &+ (1-\theta) c u_{N-1}^n. \end{aligned} \tag{8.25}$$

- Neumann 境界条件: $u_0^n = u_1^n - J_{\mathrm{L}} \Delta x$, $u_{N+1}^n = u_N^n + J_{\mathrm{R}} \Delta x$ より,

$$\begin{aligned} & (1+\theta c) u_1^{n+1} - \theta c u_2^{n+1} \\ =& \{1 - (1-\theta)c\} u_1^n - c J_{\mathrm{L}} \Delta x \\ &+ (1-\theta) c u_2^n, \end{aligned} \tag{8.26}$$

$$\begin{aligned} & -\theta c u_{N-1}^{n+1} + (1+\theta c) u_N^{n+1} \\ =& \{1 - (1-\theta)c\} u_N^n + c J_{\mathrm{R}} \Delta x \\ &+ (1-\theta) c u_{N-1}^n. \end{aligned} \tag{8.27}$$

これらの連立方程式を解く際にも LU 分解が用いられる. この場合, (8.14) の行列 A は, Dirichlet 境界条件では

$$A = \begin{pmatrix} 1+2\theta c & -\theta c & 0 & \cdots & 0 \\ -\theta c & 1+2\theta c & -\theta c & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\theta c & 1+2\theta c & -\theta c \\ 0 & \cdots & 0 & -\theta c & 1+2\theta c \end{pmatrix},$$

Neumann 境界条件では

$$A = \begin{pmatrix} 1+\theta c & -\theta c & 0 & \cdots & 0 \\ -\theta c & 1+2\theta c & -\theta c & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\theta c & 1+2\theta c & -\theta c \\ 0 & \cdots & 0 & -\theta c & 1+\theta c \end{pmatrix}$$

となる. また, $\boldsymbol{z}$ は Dirichlet 境界条件では

$$
\begin{aligned}
z_1 &= \{1-2(1-\theta)c\}\,u_1^n + cu_{\mathrm{L}} + (1-\theta)cu_2^n,\\
z_j &= \{1-2(1-\theta)c\}\,u_j^n + (1-\theta)cu_{j-1}^n\\
&\quad + (1-\theta)cu_{j+1}^n \qquad (2\le j\le N-1),\\
z_N &= \{1-2(1-\theta)c\}\,u_N^n + cu_{\mathrm{R}} + (1-\theta)cu_{N-1}^n,
\end{aligned}
$$

Neumann 境界条件では

$$
\begin{aligned}
z_1 &= \{1-(1-\theta)c\}\,u_1^n - cJ_{\mathrm{L}}\Delta x + (1-\theta)cu_2^n,\\
z_j &= \{1-2(1-\theta)c\}\,u_j^n + (1-\theta)cu_{j-1}^n\\
&\quad + (1-\theta)cu_{j+1}^n \qquad (2\le j\le N-1),\\
z_N &= \{1-(1-\theta)c\}\,u_N^n + cJ_{\mathrm{R}}\Delta x\\
&\quad + (1-\theta)cu_{N-1}^n
\end{aligned}
$$

である.

8.4 安定性解析

本節では, 拡散方程式に対する Euler 陽解法, Crank-Nicolson 法およびその他の陰解法の安定性解析を行う. 今, 先の解法で得られた方程式について, 波数 k の特解

$$u_j^n = \lambda_k^n e^{ikj} \tag{8.28}$$

を考える. 各方程式は線形であるから, 波数 k の成分ごとに議論を行える. （任意の解はそれらの重ね合わせとして得られる. ）係数 λ_k の絶対値 $|\lambda_k|$ が任意の波数 k に対して 1 より小さければ差分方程式は安定である (即ち, 任意の波数のモードは時間とともに減衰する) と言える. この λ_k は増幅因子と呼ばれる.

Euler 陽解法で数値計算を行う際, (8.8) で定義される c の値がある値を越えると, 差分方程式が不安定になることが知られている. 実際, 特解 (8.28) を (8.6) に代入すると,

$$
\begin{aligned}
&\lambda_k^{n+1}e^{ikj}\\
=&\lambda_k^n e^{ikj} + c\left(\lambda_k^n e^{ik(j-1)} - 2\lambda_k^n e^{ikj} + \lambda_k^n e^{ik(j+1)}\right)
\end{aligned}
$$

を得る. これより

$$\lambda_k = 1 - 4c\sin^2\frac{k}{2}$$

となる. 従って, $|\lambda_k|<1$ が任意の k に対して満たされるためには

$$c \le \frac{1}{2}$$

でなければならない. この条件は, Δx を小さくとるとき, Δt もそれに合わせて小さくしなければならないことを表す.

Crank-Nicolson 法については, (8.9) に特解 (8.28) を代入すると,

$$
\begin{aligned}
&-\frac{c}{2}\lambda_k^{n+1}e^{ik(j-1)} + (1+c)\lambda_k^{n+1}e^{ikj} - \frac{c}{2}\lambda_k^{n+1}e^{ik(j+1)}\\
&= (1-c)\lambda_k^n e^{ikj} + \frac{c}{2}\lambda_k^n e^{ik(j-1)} + \frac{c}{2}\lambda_k^n e^{ik(j+1)}
\end{aligned}
$$

となる. これから

$$\lambda_k = \frac{1-2c\sin^2\frac{k}{2}}{1+2c\sin^2\frac{k}{2}}$$

と求まる. $c>0$ より, 全ての k に対して $|\lambda_k|<1$ であることが分かり, Crank-Nicolson 法は全ての c について安定であると言える.

その他の陰解法については, (8.23) に特解 (8.28) を代入することで,

$$\lambda_k = \frac{1-4(1-\theta)c\sin^2\frac{k}{2}}{1+4\theta c\sin^2\frac{k}{2}}$$

が得られる. これより, 無条件安定（全ての $c>0$, k に対して $|\lambda_k|<1$ が成り立つ）であるための条件は, $1/2\le\theta\le 1$ であることが分かる.

8.5 演習問題

以下の 3 つの問題の解答をレポートとして提出せよ.

問題 1 (拡散方程式の数値解)

拡散方程式 (8.3) ($L=10$) を初期条件

$$u_0(x) = \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x-5)^2}$$

の下で数値的に解く. 刻み幅は $\Delta t = 0.01$, $\Delta x = 0.5$ ($N=20$) として, 以下の 4 つの場合を考えよ.

1. Euler 陽解法 (8.6) で境界条件は Dirichlet 境界条件 $u_{\mathrm{L}} = u_{\mathrm{R}} = 0$.
2. Euler 陽解法 (8.6) で境界条件は Neumann 境界条件 $J_{\mathrm{L}} = J_{\mathrm{R}} = 0$.
3. Crank-Nicolson 法 (8.7) で境界条件は Dirichlet 境界条件 $u_{\mathrm{L}} = u_{\mathrm{R}} = 0$.
4. Crank-Nicolson 法 (8.7) で境界条件は Neumann 境界条件 $J_{\mathrm{L}} = J_{\mathrm{R}} = 0$.

いずれの場合も, $t = 1, 2, 3, 4, 5$ ($n = 100, 200, 300, 400, 500$) における $u(x,t)$ の値を出力すること. また, 得られた $u(x,t)$ の値をそれぞれ一つの図（横軸 x, 縦軸 u）に図示せよ. 加えて, 数値計算法の違い, 境界条件の違によって数値解がどのように変化するか比較検討せよ.

注意 8.5.1 1. 本演習では (8.4) のように, u_j^n に区間 $[(j-1)\Delta x, j\Delta x]$ を割り当てるという離散化を行なっている. 従って, 初期条件は

$$u_j^0 = \frac{1}{2}\left[u_0((j-1)\Delta x) + u_0(j\Delta x)\right]$$

であると考えるのが自然である. (これは境界条件を必ずしも満たすものではないが, この点は気にしないことにする.) また, $\left\{u_j^n\right\}_{j=1}^{N}$ をプロットする場合は, 各 j に対応する座標として $x = (j-1/2)\,\Delta x$ を採用するのが自然である. ただし, これらと異なる方法をとったとしても減点はしない. 問題 2, 3 も同様である.

2. 本問題では方程式と初期条件の対称性より $x = 5$ について対称な結果が得られるはずである. もし解答のグラフが $x = 5$ について対称となっていない場合はプログラムが間違っていると考えてほしい.
3. グラフを生成する部分についてはソースコードに含める必要はない. 問題 2, 3 も同様である.
4. 余力があれば, 刻み幅を $\Delta t = 0.25$, $\Delta x = 0.5$ と変更して上記の 4 つの場合の数値計算を行い, その結果を比較検討せよ.

問題 2 (Fisher 方程式)

拡散方程式に非線形項 $f(u) = u(1-u)$ を加えた偏微分方程式 (Fisher 方程式)

$$\frac{\partial u}{\partial t} = u(1-u) + \frac{\partial^2 u}{\partial x^2}$$

を考える. この方程式は遺伝学における優性遺伝子の空間伝播を表現するために提案された. 初期条件

$$u_0(x) = \frac{1}{\left(1 + e^{bx-5}\right)^2}$$

および境界条件 ($L = 200$)

$$u(0,t) = 1, \quad u(L,t) = 0$$

の下で, $b = 0.25, 0.5, 1.0$ の場合に Fisher 方程式を, Euler 陽解法

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = f\left(u_j^n\right) + \frac{u_{j-1}^n - 2u_j^n + u_{j+1}^n}{\Delta x^2}$$

を用いて数値的に解け. ただし, 刻み幅は $\Delta x = 0.05$ ($N = 4000$), $\Delta t = 0.001$ とし, $t = 10, 20, 30, 40$ ($n = 10000, 20000, 30000, 40000$) での $u(x,t)$ の値を出力すること. また, 得られた $u(x,t)$ の値を b の値ごとに一枚の図（横軸 x, 縦軸 u）に図示せよ.

加えて, 得られた結果について考察せよ. （例えば, b の値を様々に変えた時に振る舞いがどのように変わるかを見るなど. ）

問題 3 (1 次元調和振動子の Schrödinger 方程式)

量子力学において, 1 次元調和振動子のダイナミクスは, 次の Schrödinger 方程式

$$i\hbar\frac{\partial \psi}{\partial t}(x,t) = \left(-\frac{\hbar^2}{2m}\frac{\partial^2}{\partial x^2} + \frac{k}{2}x^2\right)\psi(x,t) \tag{8.29}$$

に従う. $\psi(x,t) = \psi_{\mathrm{R}}(x,t) + i\psi_{\mathrm{I}}(x,t)$ ($\psi_{\mathrm{R}}, \psi_{\mathrm{I}} \in \mathbb{R}$) のように波動関数を実部と虚部に分けると, それぞれの従う方程式は

$$\hbar\frac{\partial \psi_{\mathrm{R}}}{\partial t}(x,t) = \left(-\frac{\hbar^2}{2m}\frac{\partial^2}{\partial x^2} + \frac{k}{2}x^2\right)\psi_{\mathrm{I}}(x,t),$$
$$-\hbar\frac{\partial \psi_{\mathrm{I}}}{\partial t}(x,t) = \left(-\frac{\hbar^2}{2m}\frac{\partial^2}{\partial x^2} + \frac{k}{2}x^2\right)\psi_{\mathrm{R}}(x,t)$$

となり，2 つの拡散型方程式の組で表される．この際，粒子の位置の確率密度は $P(x,t) \equiv |\psi(x,t)|^2 = \psi_{\mathrm{R}}(x,t)^2 + \psi_{\mathrm{I}}(x,t)^2$ で与えられる．

今, パラメータを $\hbar = 1$, $m = 1$, $k = 1$ とし, 初期条件

$$\psi(x,0) = \frac{\sqrt{2}}{\pi^{\frac{1}{4}}} e^{-2x^2}$$

の下で Schrödinger 方程式を数値的に解くことを考える．ただし, 空間領域は $[-L/2, L/2]$ とし, 境界条件は周期境界条件

$$\psi\left(\frac{L}{2}, t\right) = \psi\left(-\frac{L}{2}, t\right)$$

とする．

数値的に解く際には Visscher のスキーム [3] と呼ばれる, 実部と虚部の時間発展を半ステップだけずらして解く陽解法を用いる．即ち, 離散化された波動関数の実部, 虚部をそれぞれ $\left\{R_j^n\right\}$, $\left\{I_j^n\right\}$ と書くと, $1 \le j \le N$ $(L = N\Delta x)$ に対して時間発展は

$$\begin{aligned} R_j^{n+1} = R_j^n \\ + \Delta t \left(-\frac{1}{2} \frac{I_{j-1}^n - 2I_j^n + I_{j+1}^n}{\Delta x^2} + \frac{1}{2} x_j^2 I_j^n \right), \end{aligned}$$

$$\begin{aligned} I_j^{n+1} = & I_j^n - \Delta t \left(-\frac{1}{2} \frac{R_{j-1}^{n+1} - 2R_j^{n+1} + R_{j+1}^{n+1}}{\Delta x^2} \right. \\ & \left. + \frac{1}{2} x_j^2 R_j^{n+1} \right) \end{aligned}$$

と記述される．ただし,

$$x_j \equiv \left(j - \frac{1}{2}\right) \Delta x - \frac{L}{2}$$

であり, 境界条件は全ての n で

$$\begin{aligned} R_0^n = R_N^n, \quad R_{N+1}^n = R_1^n, \\ I_0^n = I_N^n, \quad I_{N+1}^n = I_1^n \end{aligned}$$

と表される．また, 初期条件は $1 \le j \le N$ で

$$\begin{aligned} R_j^0 &= \frac{1}{2} \left[\psi\left(x_j - \frac{\Delta x}{2}, 0\right) + \psi\left(x_j + \frac{\Delta x}{2}, 0\right) \right], \\ I_j^0 &= -\Delta t \left(-\frac{1}{2} \frac{R_{j-1}^0 - 2R_j^0 + R_{j+1}^0}{\Delta x^2} + \frac{1}{2} x_j^2 R_j^0 \right) \end{aligned}$$

と表される．$N = 400$, $\Delta x = 0.05$ $(L = 20)$, $\Delta t = 0.001$ の下でこの系を数値的に解き, $t = 1, 2, 3, 4, 5, 6, 7, 8$ $(n = 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000)$ における確率密度 $\left(R_j^n\right)^2 + I_j^n I_j^{n-1}$ $(1 \le j \le N)$ の値を出力せよ．また, 得られた確率密度 $\left(R_j^n\right)^2 + I_j^n I_j^{n-1}$ の値を一つの図（横軸は x_j）に図示せよ．

注意 8.5.2 1. Euler 陽解法

$$\begin{aligned} R_j^{n+1} = R_j^n + \Delta t \left(-\frac{1}{2} \frac{I_{j-1}^n - 2I_j^n + I_{j+1}^n}{\Delta x^2} \right. \\ \left. + \frac{1}{2} x_j^2 I_j^n \right), \end{aligned}$$

$$\begin{aligned} I_j^{n+1} = & I_j^n - \Delta t \left(-\frac{1}{2} \frac{R_{j-1}^n - 2R_j^n + R_{j+1}^n}{\Delta x^2} \right. \\ & \left. + \frac{1}{2} x_j^2 R_j^n \right) \end{aligned}$$

で確率密度 $\left(R_j^n\right)^2 + \left(I_j^n\right)^2$ の値を計算し, Visscher スキームで得られた結果と比較せよ．

2. 周期境界条件を仮定すると, Schrödinger 方程式 (8.29) の解 ψ は

$$\frac{d}{dt} \int_{-L}^{L} |\psi(x,t)|^2 \, dx = 0$$

を満たす．このことを踏まえて, 余力があれば, 和

$$\sum_{j=1}^{N} \left(R_j^n\right)^2 + I_j^n I_j^{n-1}$$

も n によって変化しないことを確認せよ．

参考文献

[1] 山本哲朗, 数値解析入門 (サイエンスライブラリ 現代数学への入門 14), サイエンス社 (1976)

[2] 高橋大輔, 数値計算 (理工系の基礎数学 8), 岩波書店 (1996)

[3] P. B. Visscher, Computers in Physics **5**, 596 (1991)

[川越大輔]

付 録A　最小二乗法

1.1　目的

最小二乗法とは，与えられたデータに合う数理モデルあるいはそのパラメータを決定するために，二乗誤差を目的関数とした最小化問題を解いて推定値を得る手法のことである．この手法は最小二乗誤差推定とも呼ばれ，原理は単純ながらも強力で，実験データを扱う多くの場面で標準的に用いられる手法の 1 つである．最小二乗法を単に最適化問題の 1 つとみなすのではなく，最適化問題を構成するデータの取得方法や用いるデータ数，得られた最適化解の評価の仕方を工夫することも重要であり，課題を通して簡単なデータ分析ができるようになってほしい．本実験では，最小二乗法の基礎から逐次的な処理方法，さらに交互最小二乗法までを使えるようになり，データ処理の初歩を学ぶことを目的とする．

本稿では，2 回前期配当科目の全学共通科目「確率論基礎」の知識があることが望ましいが，内容や課題の多くは高校数学の範囲内の確率・統計学の知識でカバーできるように記述した．そのため，回りくどい部分や天下り的な箇所もあるいが，それらを含む最小二乗法の統計的性質や発展的な内容は，3 回前期の「確率と統計」を受講するか，あるいは章末に挙げた参考文献 [1–5] などで自習してもらいたい．

- 課題ではデータをダウンロードする必要がある．担当者の指示に従い，適宜指定されたデータをダウンロードして用いること．
- 読み込まれたデータは,「入力，出力」の順か，あるいはその転置で並んでいる．入力や出力の次元が 2 以上の場合もあるので，読み込む際に注意すること．
- 課題を取り組むためのプログラミング言語は特に指定しない．
- 「課題 X」がレポートの対象で,「問題」は演習（必要に応じて行う課題）.
- 各課題において結果を示すのみでなく，結果から分かったことや改善できることなど，考察をすること．必要に応じて図を追加し議論すること．
- （担当者向け）時間がかかりそうなときは逐次最小二乗法までででいいです．Kalman フィルタ以降は早く終わってる人向けで．

記法

$\mathbb{R}$ は実数全体を表し，n 次の数ベクトル空間を $\mathbb{R}^n$ で表す．また，$m \times n$ の実行列全体を $\mathbb{R}^{m\times n}$ で表す．とくに I_m は m 次の単位行列を表す．行列 A の転置は，$A^\top$ で表す．正方行列 A に対して $A \geq 0$ で半正定値対称行列を，$A > 0$ で正定値対称行列を表す．$\| \bullet \|$ は Euclid ノルムを表す．ただしノルムに下添字をつけることで，別のノルムを表すことがある．$\mathbb{E}$ は適当な確率分布による期待値演算であり，確率分布あるいは確率密度関数を陽に表すときには $\mathbb{E}_P$ と書く．確率変数 $w \in \mathbb{R}^m$ に対し $\mathcal{N}(\mu, V)$ は平均 $\mu \in \mathbb{R}^m$, 共分散行列 $V \in \mathbb{R}^{m\times m}$ の正規分布を表す．

1.2　最小二乗法とその評価方法

次の関係式を満たす方程式が与えられているとする．

$$z = f(\theta, x) \tag{A.1}$$

ここで $z \in \mathbb{R}^m$, $\theta \in \mathbb{R}^n$, $x \in \mathbb{R}^p$ とし，$f : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^m$ とする．「適当に固定されたパラメータ

θ の下で，x を入れると z を返す」という意味で，これはパラメータを θ とし，x を入力，z を出力とするシステム f であるという．一般に観測には誤差を伴うため，入力 x や出力 z の値には誤差が乗る．最近の応用で多く見られるように，ここでは出力にのみ観測誤差のある場合を考えよう．入力に観測誤差がある場合も考えられるが，基本的には同じように議論できるため，本項では考えない．

誤差を $w \in \mathbb{R}^m$ とおき，加法的な誤差を考えると，観測値は次で与えられる．

$$y = z + w. \tag{A.2}$$

観測誤差は偶然誤差と系統誤差の 2 種類に分類されるが，本稿では偶然誤差のみを考える．特に断らない限り，本稿では偶然誤差は次の性質を持つものと仮定する．

1. 各試行ごとに観測誤差は同じ確率分布 P から確率的に発生する．
2. 入力 x やパラメータ θ とは独立で，かつ観測ごとに独立に定まる．
3. i 回目の観測に対する誤差を $w_i \in \mathbb{R}^m$ で表す．このとき，次の 2 つが成り立つものとする．

$$\mathbb{E}_P[w_i] = 0,\ \mathbb{E}_P[w_i w_j^\top] = \delta_{i,j} V. \tag{A.3}$$

ここで 0 は m 次のゼロベクトルを表し，V は m 次の正定値対称行列である．$\delta_{i,j}$ は Kronecker のデルタである．

同じ計測機器で計測することを考えるのならば，同じ条件で測定する限り，観測誤差は独立同一分布に従うものとみてもよいであろう．ただし，このような確率分布を正確に知ることは難しい．以降断らない限り，我々はこの観測誤差に関する確率分布は正確に分からないものとし，観測誤差は平均ゼロ，共分散行列が有界に存在することのみを知っているものとする．例として，図 A.1 に $f(\theta, x) = \theta x$，$\theta, x \in \mathbb{R}$ の場合の例を表す．

ここで関数 f が既知であるが，パラメータ θ が未知である場合を考えよう．このとき，入力と観測値のデータから θ を決定したい．例えば，図 A.1

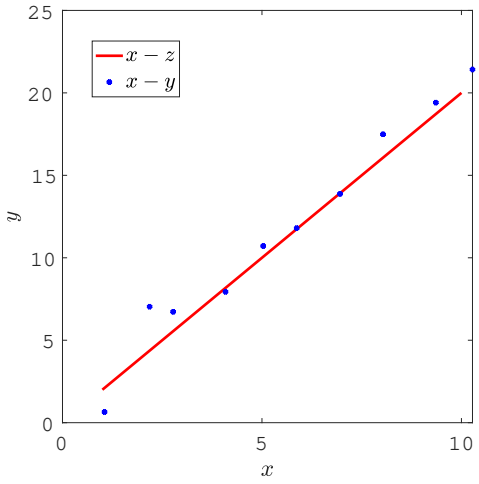


図 A.1: $f(\theta, x) = \theta x$ の場合．青色の点は観測値 y_i で，赤色の線が z_i の値．

の青点から赤線を求める問題を考えたい．θ さえ適切に決められれば，未知のデータに対する出力の予測ができるようになり，様々な場面で恩恵を受けることができるからである．N 組の入力および観測値のデータ $\{(x_i, y_i)\}_{i=1}^N$ が与えられているとしよう．このとき，

$$\min_{\theta \in \mathbb{R}^n} \sum_{i=1}^{N} \|y_i - f(\theta, x_i)\|^2 \tag{A.4}$$

を解くことで，真のパラメータ θ^* を得ることを考える．ここで $\|\bullet\|$ は Euclid ノルムである．このパラメータを求めることを，で f が θ に関して線形である場合を**線形最小二乗法（linear least squares）**あるいは単に**最小二乗法（least mean squares）**，非線形である場合を**非線形最小二乗法（non-linear least squares）**と呼ぶ（least の代わりに minimum が用いられることもある）．一般に最小二乗法は，適切な最適化の手法を使って解けばよい．しかし，以下で見るように，特別な場合は解を陽に書き下せたり，後からデータが追加されても逐次的に解を得られる場合があり，データ容量を抑えられるなどメリットがある．

注意 1.2.1 最小二乗法において誤差の確率「解釈」は，理解のしやすさのために導入されるもの

で，いつも必ず必要というわけではない．例えば，A 君は四条河原町に，B さんは吉田キャンパスに，C 君は御所にいるとき，それぞれが最短距離で落ち合う場所を求める問題も最小二乗法として解くことができるが，最適な落合場所からそれぞれの現在地が確率的に決まっているかどうかがアルゴリズムや解を決定するわけではない．

注意 1.2.2 実際のデータ分析の現場では，関数 f が既知であるというのも強い仮定であることに注意が必要である．関数 f はデータの統計的性質や物理的考察から決めるか，線形関数などの簡単なものから試していけばよい．深層ニューラルネットワークでは，関数を柔軟に表現できる基底の組み合わせを使うことで f を表現するし，最近では Gauss 過程回帰 [6] と呼ばれる手法も使われる．

1.2.1 回帰問題

まずは θ が f に対して線形である場合を扱おう．このとき，適当な行列値関数 $\varphi : \mathbb{R}^p \to \mathbb{R}^{m\times n}$ が存在して，

$$f(\theta, x) = \varphi(x)\theta$$

となる場合を考えていることになる．ここで，$\sum_{i=1}^{N} \varphi(x_i)^\top \varphi(x_i)$ が正則ならば，最適化問題 (A.4) の最適解は次で求められる．

$$\hat{\theta}_N = \left(\sum_{i=1}^{N} \varphi(x_i)^\top \varphi(x_i) \right)^{-1} \sum_{j=1}^{N} \varphi(x_j)^\top y_j. \tag{A.5}$$

この $\hat{\theta}_N$ を，**最小二乗誤差推定値（Minimum square error estimate）**あるいは**最小二乗推定値**という[1]．このデータをもらったときの処理の仕方（推定規則）のことを，**最小二乗誤差推定量（minimum square error estimator）**という．また，このとき観測誤差 w_i, $i = 1, \dots, N$ の共分散行列の推定値は，

$$\hat{V}_N = \frac{1}{N-n} \sum_{i=1}^{N} \left(y_i - \varphi(x_i)\hat{\theta}_N \right) \left(y_i - \varphi(x_i)\hat{\theta}_N \right)^\top \tag{A.6}$$

で与えられる[2]．以降，表記の簡単のために，

$$\Phi_N := \left(\sum_{i=1}^{N} \varphi_i^\top \varphi_i \right)^{-1}, \quad \varphi_i := \varphi(x_i)$$

とおく．

式 (A.5) より推定値 $\hat{\theta}_N$ が得られたからといって，喜んでばかりいてはいけない．推定値が得られることと，その推定精度が良いことは別問題だからである．そこで得られた推定値がどのくらいよいか，検討しよう．真のパラメータ θ と $\hat{\theta}_N$ の間には

$$\begin{aligned} \theta - \hat{\theta}_N =& \Phi_N \left(\Phi_N^{-1}\theta - \sum_{j=1}^{N} \varphi_j^\top y_j \right) \\ =& \Phi_N \sum_{j=1}^{N} \varphi_j^\top (\varphi_j\theta - y_j) \\ =& -\Phi_N \sum_{j=1}^{N} \varphi_j^\top w_j \end{aligned}$$

の関係がある．入力のデータは既知の数であるので，確率的な変動は w_j のみに依存することに注意されたい．したがって，w_j が平均ゼロであることは既知なので，推定が**不偏的（unbiased）**であるという性質

$$\mathbb{E}[\theta - \hat{\theta}_N] = 0$$

が成り立つ．また，もし観測誤差の共分散行列 V が既知であるならば，推定誤差の共分散行列 $\mathrm{Cov}(\hat{\theta}_N)$ は

$$\begin{aligned} \mathrm{Cov}(\hat{\theta}_N) :=& \mathbb{E}[(\theta - \hat{\theta}_N)(\theta - \hat{\theta}_N)^\top] \\ =& \Phi_N \sum_{i=1}^{N} \varphi_i^\top V \varphi_i \Phi_N \end{aligned} \tag{A.7}$$

[1] $\hat{\theta}_N$ を求める問題は，本質的には連立 1 次方程式の解を求める問題と同じになるため，逆行列の計算は Gauss の消去法などを利用して解けばよい．

[2] パラメータ数分をデータから引いて $N - n$ としているのは不偏推定値にするため．$N \gg n$ ならあまり気にしなくてもよい．

となる．観測誤差の推定値が分からない場合は，式 (A.7) 右辺の V を $\hat{V}_N$ に置き換えればよい．V の最大特異値 $\sigma_{\max}(V)$ を用いると，

$$\mathrm{Cov}(\hat{\theta}_N) \leq \sigma_{\max}(V)\Phi_N \tag{A.8}$$

が成り立つ（不等号の意味は半正定値行列の意味）．とくに $V = \sigma^2 I_n$ ならば，

$$\mathrm{Cov}(\hat{\theta}_N) = \sigma^2\Phi_N$$

となり，Φ_N は N に対して（正定値行列の意味で）単調減少なので，漸近的に推定誤差がゼロに近づくことが示唆される．

問題 5 式 (A.8) を示せ．

回帰推定誤差も求めておこう．元の最小化問題 (A.4) のコスト関数を正規化するため，データ数 N で割ると，

$$R_N := \frac{1}{N}\sum_{i=1}^{N}\left\|y_i - \varphi_i\hat{\theta}_N\right\|^2 \tag{A.9}$$

とおくと，

$$\begin{aligned} R_N =& \frac{1}{N}\boldsymbol{y}^\top\left(I_{mN} - \boldsymbol{\varphi}(\boldsymbol{\varphi}^\top\boldsymbol{\varphi})^{-1}\boldsymbol{\varphi}^\top\right)\boldsymbol{y} && \text{(A.10)}\\ =& \frac{1}{N}\left\|\left(I_{mN} - \boldsymbol{\varphi}(\boldsymbol{\varphi}^\top\boldsymbol{\varphi})^{-1}\boldsymbol{\varphi}^\top\right)\boldsymbol{y}\right\|^2 && \text{(A.11)}\\ =& \frac{1}{N}\left\|\left(I_{mN} - \boldsymbol{\varphi}(\boldsymbol{\varphi}^\top\boldsymbol{\varphi})^{-1}\boldsymbol{\varphi}^\top\right)\boldsymbol{w}\right\|^2 && \text{(A.12)} \end{aligned}$$

となる．ただし，

$$\begin{aligned} \boldsymbol{y} &:= \begin{bmatrix} y_1^\top & \dots & y_N^\top \end{bmatrix}^\top \in \mathbb{R}^{mN},\\ \boldsymbol{w} &:= \begin{bmatrix} w_1^\top & \dots & w_N^\top \end{bmatrix}^\top \in \mathbb{R}^{mN},\\ \boldsymbol{\varphi} &:= \begin{bmatrix} \varphi_1^\top & \dots & \varphi_N^\top \end{bmatrix}^\top \in \mathbb{R}^{mN\times n} \end{aligned}$$

とおいた．Φ_N の正則性は，$\boldsymbol{\varphi}$ が列フルランクであることを意味することに注意されたい．また，式 (A.10) から式 (A.11) への変形には $(I_{mN} - \boldsymbol{\varphi}(\boldsymbol{\varphi}^\top\boldsymbol{\varphi})^{-1}\boldsymbol{\varphi}^\top)$ が射影行列になることを用いた．式 (A.12) から，誤差の二乗平均を表す R_N は観測誤差と入力から定まる行列 $\boldsymbol{\varphi}$ で定まる．もし各ベクトルやデータの数が $mN = n$ を満たす場合，$\boldsymbol{\varphi}$ そのものも逆行列をもつため，$R_N = 0$ になる．R_N の最小値を達成していることになるが，これはよい推定と言えるだろうか？一方，データを増やすと $mN > n$ となり，$R_N > 0$ と一般にはなってしまうが，これは悪い推定値を得ていることになるのだろうか？

推定問題が「データが与えられたときの単なる最適化問題である」と言えないのは，推定問題の本来の目的は入出力関係を適切に定める関数 φ とパラメータ θ を決定することであり，最適化問題はそのための手段だからである．そのため，取得するデータ数や方法を含め，どのように最適化問題を定めれば適切に推定できるかを検討する必要がある．上に挙げた例は，観測誤差に対してパラメータを合わせてしまった**過学習（over learning）**あるいは**過適合（over fitting）**と呼ばれる現象が起こりうる例であり，一般的に歓迎するものではない．それでは，推定性能はどのように評価すればよいだろうか？θ が何かの分布に従って発生しており，雑音の分布も分かっているのならば，R_N の期待値，あるいは $\|\theta - \hat{\theta}_N\|^2$ の期待値を取って評価することができる．しかし，ここで考えている条件下では，確率分布が分からないため，（観測誤差を含む）観測値と答え合わせをするしかない．1 つのやり方が，**交差検証（cross validation）**である（和名よりもクロスバリデーションと呼ばれることが多い）．簡単なやり方は，データを学習用と検証用の 2 つに分け，学習用データでパラメータ推定を行い，検証用データで誤差を評価する方法である．検証用のデータを用いた誤差のことを**テスト誤差（testing error）**という．学習用のデータで得られた推定値が検証用のデータでも良い推定誤差をもつのならば，良い推定ができていると言えるだろう．学習用のデータはできるだけ多くとることは重要であるが，そのために検証用のデータがなくなるのは困るので，多くの場合はそれらの兼ね合いを考えて行う必要がある．学習データがどの程度あれば十分そうかを見積もるには，$\hat{\theta}_N$ がどのくらいのデータ数 N で収束しているかを確認すればよい．例えば，$y = 2x + w$, $w \sim \mathcal{N}(0, 4)$ の場合の推定値のデータ数に対する収束の様子を図 A.2 に

示す．また，区間 $[0, 10]$ 内から一様乱数を用いて x_i を発生させデータセットを 10000 組用意し，学習用データ N を増やしたときの観測誤差 R_N の様子と，次で定義される学習用データを除いた残りの $M = 10000 - N$ 個のデータを使った誤差を図 A.3 に示す．

$$R_N^M := \frac{1}{M} \sum_{i=N+1}^{10000} \|y_i - \varphi_i \hat{\theta}_N\|^2, \ M > 0$$

これによると大体 $N = 200$ 程度から $\hat{\theta}_N$ はほとんど変化しなくなるので，この例においては学習用のデータは 200 くらいあればよさそうである．また，この例では $R_1 = 0$ となるが，$\hat{\theta}_1 \simeq 1.2$ なので真値から大きく異なる．N を増やしていくと学習誤差はどこか一定の値に収束していく様子も見られる．また，テスト誤差もだいたい学習誤差と同じくらい値に収束していく様子も見られる．データが十分ある場合はテスト誤差と学習誤差は変わらないが，データが十分あるかどうかわからない場合は両方を併用するとよい．

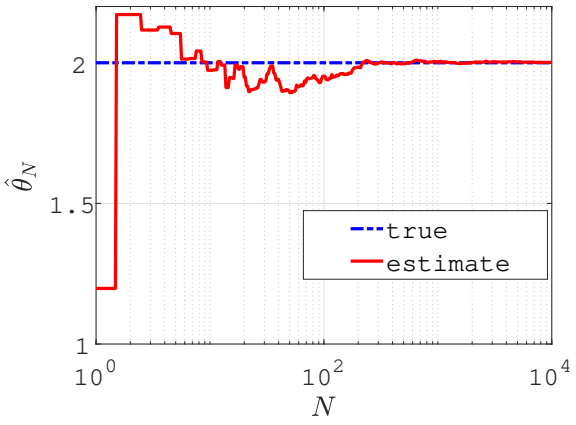


図 A.2: $\hat{\theta}_N$ の収束の様子

なお，上で用いた R_N や R_N^M の大きさはどの単位で測定しているかによって決まるので（例えば m か cm では 100 倍違うので），得られたパラメータがどれほどよくデータを説明するかの指標として用いるには向いてない．観測誤差の確率分布が不明であるならば，**決定係数（coefficient of determination, determination coefficient）**

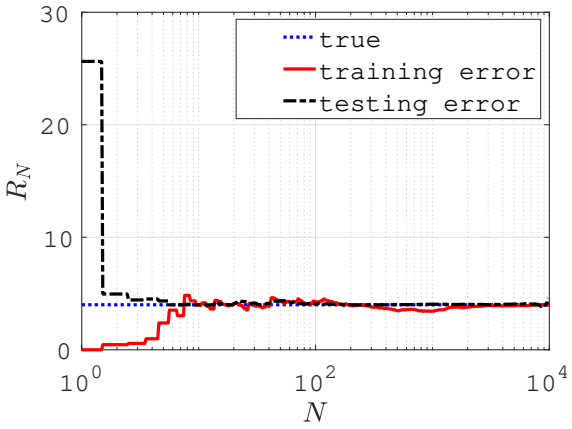


図 A.3: N に対する R_N と R_N^M

と呼ばれる量が用いられる．

$$R^2 := \frac{\sum_{i=1}^N \|\varphi_i \hat{\theta}_N - \bar{y}\|^2}{\sum_{i=1}^N \|y_i - \bar{y}\|^2} \tag{A.13}$$

ここで $\bar{y} := \frac{1}{N} \sum_{i=1}^N y_i$ とした．最小二乗推定値 $\hat{\theta}_N$ の決定係数は非負の値をとることはもちろん，とくに定数項も推定すると，$[0, 1]$ の範囲に値を取る（モデルが適切でデータが多いときは定数項を含めなくても $[0, 1]$ に値をとる）．決定係数の値が 1 に近いほど「回帰の当てはまりが良い推定」であるとみなされる．逆に 0 に近い場合は得られたパラメータ $\hat{\theta}_N$ が出力 y の推定に役に立たないことを意味するが，これはパラメータ推定がうまくいかないことを意味するわけではないことに注意されたい（課題 8 で確認されたい）．

なお，観測誤差が正規分布に従うのであれば，$\hat{\theta}_N$ の誤差の評価も様々な統計ツールで分析することができる [2]．以下の 2 つの問題を通して，観測誤差の大きさが異なる場合の最小二乗法を比較検討されたい．

課題 7 (重回帰問題) 各入力 $x_i \in \mathbb{R}^2$, $i = 1, 2, \ldots, 10000$ に対し，次の式で観測データ $y_i \in \mathbb{R}$ が生成されているとする．

$$y_i = x_i^\top \theta + w_i \tag{A.14}$$

ここで観測誤差は $\mathcal{N}(0, 1)$ に従って発生している（ただし，平均の情報のみ知っているとし，分散

も分布も知らないものとする）．このとき，次の問いに答えよ．

1. θ の最小二乗誤差推定値 $\hat{\theta}_N$ を，与えられた全てのデータ $\{(x_i, y_i)\}_{i=1}^{10000}$ を使って求めよ．また，そのときの推定誤差共分散行列 $\mathrm{Cov}(\hat{\theta}_{10000})$ を求めよ．

2. 図 A.2 のように用いるデータ $\{(x_i, y_i)\}_{i=1}^{N}$ を増やしたとき，$\hat{\theta}_N$ の各要素が収束していくかどうかをサンプル数 N を対数を用いて表す片対数グラフでプロットして確認せよ．ただし，$N = 2, 4, 8, \ldots, 2^{13} = 8192$ のときのみでよい．

3. 全てのデータを用いたときの決定係数 (A.13) を求めよ．また，$\{(x_i, y_i)\}_{i=1}^{10000}$ と $\{(x_i, \varphi(x_i)\hat{\theta}_{10000})\}_{i=1}^{10000}$ を同じグラフに重ねてプロットし，決定係数とグラフから推定結果の妥当性を議論せよ．

課題 8 (多項式回帰) 各入力 $x_i \in \mathbb{R}$, $i = 1, 2, \ldots, 10000$ に対し，次の式で観測データ $y_i \in \mathbb{R}$ が生成されているとする．

$$y_i = \varphi(x_i)\theta + w_i,\ \varphi(x) := \begin{bmatrix} 1 & x & x^2 & x^3 \end{bmatrix} \tag{A.15}$$

ここで観測誤差は $\mathcal{N}(0, 9)$ に従って発生している（ただし，平均の情報のみ知っているとし，分散も分布も知らないものとする）．このとき，次の問いに答えよ．

1. θ の最小二乗誤差推定値 $\hat{\theta}_N$ を，与えられた全てのデータ $\{(x_i, y_i)\}_{i=1}^{10000}$ を使って求めよ．また，そのときの推定誤差共分散行列 $\mathrm{Cov}(\hat{\theta}_{10000})$ を求めよ．

2. 図 A.2 のように用いるデータ $\{(x_i, y_i)\}_{i=1}^{N}$ を増やしたとき，$\hat{\theta}_N$ の各要素が収束していくかどうかをサンプル数 N を対数を用いて表す片対数グラフでプロットして確認せよ．ただし，$N = 4, 8, \ldots, 2^{13} = 8192$ のときのみでよい．

3. 全てのデータを用いたときの決定係数 (A.13) を求めよ．また，$\{(x_i, y_i)\}_{i=1}^{10000}$ と $\{(x_i, \varphi(x_i)\hat{\theta}_{10000})\}_{i=1}^{10000}$ を同じグラフに重ねてプロットし，決定係数とグラフから推定結果の妥当性を議論せよ．

また，推定値が収束しない例もあることを次の問題を通して確認されたい．このような問題に最小二乗法を適用することは意味がなく，別の推定手法が必要になる．

課題 9 (期待値の存在しない観測誤差の場合) 各入力 $x_i \in \mathbb{R}^2$, $i = 1, 2, \ldots, 10000$ に対し，次の式でデータ $y_i \in \mathbb{R}$ が生成されているとする．

$$y_i = x_i^\top \theta + w_i \tag{A.16}$$

ただし，観測誤差 w_i は標準 Cauchy 分布に従うとする．また，パラメータ $\theta \in \mathbb{R}^2$ は課題 7 と同じである．このとき，次の問いに答えよ．

1. 与えられた全データ $\{(x_i, y_i)\}_{i=1}^{10000}$ を用いて θ の最小二乗誤差推定値を式 (A.5) から求めよ．

2. 図 A.2 のように用いるデータ $\{(x_i, y_i)\}_{i=1}^{N}$ を増やしたとき，$\hat{\theta}_N$ の各要素が収束していかないことを片対数グラフでプロットして確認せよ．ただし，$N = 2, 4, 8, \ldots, 2^{13} = 8192$ のときのみでよい（必要に応じてプロット点数を増やしてもよい）．

3. (1) で得られた推定値を使って $\{(x_i, y_i)\}_{i=1}^{10000}$ と $\{(x_i, \varphi(x_i)\hat{\theta}_{10000})\}_{i=1}^{10000}$ を同じグラフに重ねてプロットし，得られたモデルが予測モデルとして意味があるかどうかを議論せよ．

期待値が存在しない観測誤差を扱ったが，仮に期待値が存在しても，分散が無限大になるような観測誤差の場合，式 (A.4) のコスト関数を N で割って，$N \to \infty$ の極限をとると発散するので，最小二乗法では適切な解を出せなくなる．

データが求めたい関数を表現するのに十分であるかどうかを確認することも重要である．

課題 10 課題 8 と同じ $\varphi(x)$，θ および観測誤差の分布 $\mathcal{N}(0, 9)$ を考える．また，$x_i \in [0, 1]$, $i =$

$1,\dots,1000$ のときに，データ $\{(x_i,y_i)\}_{i=1}^{1000}$ が与えられているものとする．このとき，

1. $\hat{\theta}_{N=1000}$ を求め，課題 8.1 の $N=1000$ および $N=10000$ の結果と比較せよ．
2. 課題 8 のデータ $\{(x_i,y_i)\}_{i=1}^{10000}$ と本課題のデータ $\{(x_i,y_i)\}_{i=1}^{1000}$ を重ねてプロットし，用いたデータの範囲を確認せよ．
3. 得られた推定値 $\hat{\theta}_{N=1000}$ から $\{(x_i,\varphi(x_i)\hat{\theta}_{N=1000})\}_{i=1}^{1000}$ と課題 8 のデータ $\{(x_i,y_i)\}_{i=1}^{10000}$ をプロットし，推定結果について考察せよ．
4. どのようにデータを取るべきか考察せよ．

注意 1.2.3 データの取り方が悪い場合やデータの数が少ない場合は，$\sum_{i=1}^{N}\varphi_i^\top\varphi_i$ が正則にならない，あるいはそれに近い状況（条件数が悪い）になることがあり，逆行列が求められないことがある．正則にならない正方行列は線形従属な行または列をもち，とくに統計学では**多重共線性 (multi-colinear)** があるという．これは $\hat{\theta}_N$ が求められないことを意味するのではなく，一意に求められないことを意味するので，そのような場合，下記のように**正則化 (regulization)** と呼ばれる項 $\lambda g(\theta)$ を追加して解くことがよく行われる．

$$\min_{\theta\in\mathbb{R}^n}\sum_{i=1}^{N}\|y_i-\varphi_i\theta\|^2+\lambda g(\theta)$$

ここで λ は正の実数であり，$g(\theta)$ は θ の何らかの（通常非負の）関数である．この正則化は，単に解を得るための手段としてのみでなく，過学習を防ぐ役割や求めたいパラメータの特徴を反映させるために付加することもある．例えば，$g(\theta)=\|\theta\|^2$ とすれば，リッジ回帰と呼ばれる種類の問題になり，0 から大きく外れる値に大きなペナルティを付加していると解釈できる．0 周りでなく，例えば a 周りにしたいならば $g(\theta)=\|\theta-a\|^2$ とすればよい．また，$g(\theta)=\|\theta\|_{\ell^1}$ と ℓ^1 ノルムを用いれば，Lasso と呼ばれるスパースな解を得るための問題になる [3, 7]．リッジ回帰は逐次最小二乗法の項で用いる．

注意 1.2.4 観測誤差が Cauchy 分布の場合を課題 9 に出し，最小二乗法ではパラメータの推定値が収束しないことを数値的に確認してもらった．実際のデータでは，観測誤差が概ね Gauss 分布に従うものの，ときに大きく外れた値が生じることがある．最小二乗法の弱点は，外れ値（outlier）と呼ばれる大きな誤差に弱い（大きな誤差に合わせるようにパラメータが選ばれてしまう）点なので，このような外れ値がデータに混入する場合，最小二乗法を適用する前に，事前にデータから外れ値を除いた方がよい．修正トンプソン法など，データから外れ値を自動的に判断して除去する方法はいくつか提案されているので，外れ値の影響が疑われる場合はデータの前処理を行うとよい．

1.2.2 重み付き最小二乗法

次に，最小二乗法における観測誤差に関する仮定を再考する．具体的には，次の 3 つを考える

- 観測誤差の同一分布性
- 観測誤差の独立性
- 観測誤差の共分散行列の情報の有無

これまでは観測誤差は独立同一分布であることや，共分散行列の値が未知であることを仮定していたが，経年劣化したセンサと新品のセンサで同じ対象を計測した場合，その観測誤差は独立ではあっても同じ統計量をもつとはいえない場合も出てくるだろう．データ数 N_1 までは同じ共分散行列 V_1 だったものが，センサを変えたので新たに取れるデータでは共分散行列が $V_2=0.1\times V_1$ になっているかもしれない．仮にどちらかの分散が小さくなることが分かっているのならば（あるいは推定されているのならば），得られるデータの質は同等とはみなせないだろう．このように，データ $\{(x_i,y_i)\}_{i=1}^{N}$ を得たとき，それぞれのデータの信用度を考えて最小二乗法を解きたい場合もある．このとき，一般には重み行列 $Q_i\geq 0$, $i=1,2,\dots,N$ を用いた次の**重み付き最小二乗法**

（**weighted least squares**）

$$\min_{\theta\in\mathbb{R}^n}\sum_{i=1}^{N}(y_i-\varphi_i\theta)^\top Q_i(y_i-\varphi_i\theta) \tag{A.17}$$

を考えればよい．もちろん，全ての重みがゼロ行列の場合は意味がないので，そのような例は本稿では考えない．この場合，$\sum_{i=1}^{N}\varphi_i^\top Q_i\varphi_i$ が正則ならば，最適な推定値は

$$\begin{aligned}\hat{\theta}_N^{\mathrm{wls}} =& \left(\sum_{i=1}^{N}\varphi_i^\top Q_i\varphi_i\right)^{-1}\sum_{j=1}^{N}\varphi_j^\top Q_j y_j \\ =&\theta+\left(\sum_{i=1}^{N}\varphi_i^\top Q_i\varphi_i\right)^{-1}\sum_{j=1}^{N}\varphi_j^\top Q_j w_j\end{aligned} \tag{A.18}$$

となる．重み付き最小二乗法とは通常，Q_i は対角行列とすることが多いが，本項では対角とは限らない重み行列の場合も重み付き最小二乗法と呼ぶ．観測誤差 w_j の共分散行列を V_j とすると，推定誤差共分散行列は

$$\begin{aligned}\mathrm{Cov}(\hat{\theta}_N^{\mathrm{wls}}) =&\mathbb{E}[(\theta-\hat{\theta}_N^{\mathrm{wls}})(\theta-\hat{\theta}_N^{\mathrm{wls}})^\top] \\ =&\Phi_{\{Q_i\}_{i=1}^N}\sum_{i=1}^{N}\varphi_j^\top Q_j V_j Q_j\varphi_j\Phi_{\{Q_i\}_{i=1}^N}\end{aligned} \tag{A.19}$$

となる．ここで $\Phi_{\{Q_i\}_{i=1}^N} := \left(\sum_{i=1}^{N}\varphi_i^\top Q_i\varphi_i\right)^{-1}$ とした．

観測誤差の共分散行列 V が既知である場合，適切な重み行列を用いて最小二乗法による推定誤差共分散行列 (A.7) よりもよい推定値を与えることができる（Gauss–Markov の定理，[2, 4]）．V は正定値対称行列であるので，$V=W^2$ を満たす正定値行列 W が一意に存在する（V の平方根行列）[3]．これを用いると，式 (A.2) は

$$W^{-1}y = W^{-1}\varphi(x)\theta+\underbrace{W^{-1}w}_{=:w'}, \tag{A.20}$$

と書き直すことができる．ここで新たな観測誤差 w_i' は単位行列を分散としてもつ観測誤差である．この解は，

$$\hat{\theta}_N^{\mathrm{wls}} = \left(\sum_{i=1}^{N}\varphi_i^\top W^{-2}\varphi_i\right)^{-1}\sum_{j=1}^{N}\varphi_j^\top W^{-2}y_j \tag{A.21}$$

となるので，$Q_i = aW^{-2} = aV^{-1}$ と選んだものに相当する（$a\in\mathbb{R}$ は正定数）．この観測誤差の統計量を用いて推定値 $\hat{\theta}_N$ を得る上記の手法は，**最良線形不偏推定量（best linear unbiased estimator, BLUE）**と呼ばれる推定量と一致する．「最良」の名前の通り，これは最も推定誤差および訓練誤差の分散を最小にする推定規則である．正しい観測誤差の情報を持っていれば，素朴な定式化から得られる推定規則よりもいい結果が得られることを意味する．

課題 11 $x_i\in\mathbb{R}, i=1,\ldots,1000$ に対し，観測値の次元が 2 次元となる場合を考える．

$$y_i = \varphi(x_i)\theta+w_i.$$

ここで w_i は $\mathcal{N}(0,V)$ の独立同一分布に従うとし，

$$\varphi(x)=\begin{bmatrix}1 & x\\ 1 & x^2\end{bmatrix},\ V=\begin{bmatrix}100 & 0\\ 0 & 1\end{bmatrix}$$

とする．このとき，式 (A.5) と式 (A.18) をそれぞれ用いて推定値を求め，それぞれの推定誤差共分散 $\mathrm{Cov}(\hat{\theta}_N)$, $\mathrm{Cov}(\hat{\theta}_N^{\mathrm{wls}})$ を求めよ．ただし，$Q_i = V^{-1}$ とする．また，$\hat{\theta}_N$ の各要素の収束の仕方をプロットし，収束の様子の違いについて検討せよ．

異なるそれぞれの推定結果の良さを比較したい場合，決定係数のみでなくクロスバリデーションも考えてみるとよい．

課題 12 先ほどの課題と同様，$x_i\in\mathbb{R}$, $i=1,\ldots,1000$ に対し，観測値の次元が 2 次元となる場合を考える．

$$y_i = \varphi(x_i)\theta+w_i.$$

ここで w_i は，$i=1,\ldots,500$ は $\mathcal{N}(0,V_1)$ の独立同一分布に，$i=501,\ldots,1000$ は $\mathcal{N}(0,V_2)$ の独

[3] 一般的に，正則行列 U で $V=UU^\top$ となるものは無数に存在する．本稿では平方根行列を用いるが，例えば Cholesky 分解を使った下三角行列 U を使っても議論は同じくできる．

立同一分布に従うとし，

$$\varphi(x) = \begin{bmatrix} 1 & x \\ 1 & x^2 \end{bmatrix},\ V_1 = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}, V_2 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

とする．このとき，

1. 式 (A.5) と式 (A.18) のそれぞれで推定値を求め，それぞれの収束の仕方を片対数グラフでプロットし，収束の違いについて検討せよ．
2. それぞれの推定量の精度を比較するため，適当なサンプル数まで学習データとして用いたクロスバリデーション，推定誤差共分散行列，および決定係数を求め，推定精度の良し悪しを議論せよ．

より一般的には，観測誤差の独立性が仮定できない場合も考えられる．観測誤差が独立でない場合，次のように相関をもつ場合がある．

$$V_{i,j} = \mathbb{E}[w_i w_j^\top]$$

$\boldsymbol{V} := \mathbb{E}[\boldsymbol{w}\boldsymbol{w}^\top]$ とおくと，重み付き最小二乗法の解法を拡張して，次で得られる推定値が BLUE であることが分かる．

$$\hat{\theta}_N = (\boldsymbol{\varphi}^\top \boldsymbol{V}^{-1} \boldsymbol{\varphi})^{-1} \boldsymbol{\varphi}^\top \boldsymbol{V}^{-1} \boldsymbol{y}^\top \tag{A.22}$$

ここで $\boldsymbol{V}$ はデータ数 N によってはかなり大きなサイズの行列になりえるので，仮にこの行列が分かっていたとしても，この逆行列を求めるのは時間がかかってしまう．そのため，計算時間がかかりすぎる場合や条件数が悪い場合には，何かしら工夫して逆行列を計算する必要がある．

1.3 推定値の合成と逐次最小二乗法

式 (A.5) のように得られている全てのデータをまとめて処理することは，**バッチ処理**と呼ばれる．新しくデータが追加されなければバッチ処理だけ知っておけば十分だが，データ処理の現場では入力と観測値のデータが追加されることはよくあるので，そのたびに最適化問題を解き直すことは煩わしい．とくに計測器からの情報が送られ続ける場合，データを保存するメモリの総量も膨大になるので，別の手段を用いる必要がある．幸いなことに，最小二乗推定値は出力に関して線形な解になる．このことを利用して，ここでは新たにデータが得られたときの処理の仕方について学ぶ．

1.3.1 異なるデータセットからの推定値の合成

データ $\{(x_i, y_i)\}_{i=1}^N$ を得たとき，$f(\theta, x) = \varphi(x)\theta$ の場合の式 (A.4) の問題を解くと，式 (A.5) で解が得られることがわかったが，実際のデータ処理では，別のデータセット $\{(x'_j, y'_j)\}_{j=1}^M$ を用いた推定値 $\hat{\theta}'_M$ が独立に存在する場合もある．それらは，適切な仮定の下で，最小二乗法を用いて以下のように推定値を得ることができる．

$$\hat{\theta}'_M = \underbrace{\left(\sum_{i=1}^M \varphi(x'_i)^\top \varphi(x'_i)\right)^{-1}}_{=:\Phi'_M} \sum_{j=1}^M \varphi(x'_j)^\top y'_j. \tag{A.23}$$

2 組のデータセットからそれぞれの推定値が得られているが，より多くのデータセットから推定する方が推定精度は改善することが期待される．このとき，データを合わせて改めて解き直してもよいが，2 つの推定値を用いて，推定値を更新することができる．便宜的に $\{(x'_j, y'_j)\}_{j=1}^M$ のラベルをつけ直して $\{(x_i, y_i)\}_{i=N+1}^{N+M} = \{(x'_j, y'_j)\}_{j=1}^M$ とす

ると，

$$\begin{aligned}
&\hat{\theta}_{N+M} \\
&= \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \sum_{i=1}^{N+M} \varphi_i^\top y_i \\
&= \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \sum_{i=1}^{N} \varphi_i^\top y_i \\
&\quad + \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \sum_{i=N+1}^{N+M} \varphi_i^\top y_i \\
&= \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \Phi_N^{-1}\Phi_N \sum_{i=1}^{N} \varphi_i^\top y_i \\
&\quad + \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \Phi_M'^{-1}\Phi_M' \sum_{i=N+1}^{N+M} \varphi_i^\top y_i \\
&= \left(\Phi_N^{-1} + \Phi_M'^{-1}\right)^{-1} \left(\Phi_N^{-1}\hat{\theta}_N + \Phi_M'^{-1}\hat{\theta}'_M\right)
\end{aligned}$$

となる．したがって，新たなデータを得ることが想定されるならば，推定値 $\hat{\theta}$ のみでなく Φ_N も保存しておけば，元のデータを保存しておく必要はない[4]．

課題 13 $x_i \in \mathbb{R}$, $i = 1, \ldots, 10000$ に対し，次の式で観測データ $y_i \in \mathbb{R}$ が生成されているとする．

$$y_i = \varphi(x_i)\theta + w_i \tag{A.24}$$

ここで

$$\varphi(x) := \left[1 \quad \exp\left(-\tfrac{(x_i-1)^2}{2}\right) \quad \exp\left(-(x_i+1)^2\right)\right]$$

であり，w_i は平均 0，分散有限の独立同一分布から生成されているものとする．10000 組のデータの組 $\{(x_i, y_i)\}_{i=1}^{10000}$ のうち，最初の 6000 組と残りの 4000 組のデータそれぞれで $\theta \in \mathbb{R}^3$ の推定値を求め，それらから推定値を合成せよ．また，全データを使った場合の推定値と比較し，一致することを確かめよ．

観測誤差の大きさが異なる場合，重み付きの最小二乗法を使った方が推定結果がよくなることが期待される．データ数がそれぞれ N_1, N_2 のデータセットが 2 組与えられているとき，重み行列 Q_1 と Q_2 を使ったときのそれぞれのデータセットにおける推定値を $\hat{\theta}_{N_1}$，$\hat{\theta}_{N_2}$ とおくと，

$$\begin{aligned}
\hat{\theta}_{N_1+N_2} = &\left(\Phi_{Q_1,N_1}^{-1} + \Phi_{Q_2,N_2}^{-1}\right)^{-1} \\
&\times \left(\Phi_{Q_1,N_1}^{-1}\hat{\theta}_{N_1} + \Phi_{Q_2,N_2}^{-1}\hat{\theta}_{N_2}\right)
\end{aligned} \tag{A.25}$$

となる．ここで

$$\Phi_{Q,N} := \left(\sum_{i=1}^{N} \varphi_i^\top Q \varphi_i\right)^{-1}$$

とした．次の問題もそれほど難しくないので，時間があるときに考えてもらいたい．

問題 6 式 (A.25) が成り立つことを示せ．

課題 14 $x_i \in \mathbb{R}$, $i = 1, \ldots, 10000$ に対し，次の式でデータ $y_i \in \mathbb{R}$ が生成されているとする．

$$y_i = \varphi(x_i)\theta + w_i \tag{A.26}$$

ここで w_i は最初の 6000 組と残りの 4000 組のデータで，平均ゼロの異なる独立な分布に従うものとする（$i = 1, \ldots, 6000$ では独立同一分布，$i = 6001, \ldots, 10000$ では異なる独立同一分布）．

$$\varphi(x) := \left[1 \quad \exp\left(-\tfrac{(x_i-1)^2}{2}\right) \quad \exp\left(-(x_i+1)^2\right)\right]$$

である．10000 組のデータの組 $\{(x_i, y_i)\}_{i=1}^{10000}$ のうち，最初の 6000 組と残りの 4000 組のデータそれぞれで $\theta \in \mathbb{R}^3$ の推定値と偶然誤差 w_i の分散の推定値を求め，推定された分散を用いて推定値を合成せよ．また，全データを使い，式 (A.5) を用いて得た $\hat{\theta}_{10000}$ と，どちらが優れているか比較考察せよ．クロスバリデーションを使う，推定誤差共分散を使う，決定係数を使う，など比較の仕方を考えること．

1.3.2 逐次最小二乗法

これまでの考え方を応用して，最小二乗法を実時間処理で解くことを考えよう．具体的には，

[4] あくまで最小二乗法で行う場合の結果であり，他の評価基準を用いる場合は全データを用いる必要が生じることがほとんどである．

N 個のデータから推定された $\hat{\theta}_N$ と新たなデータ (x_{N+1}, y_{N+1}) を用いて再帰的に $\hat{\theta}_{N+1}$ を求めることを考える．最小二乗誤差推定 (A.5) より，次のように式変形できる．

$$\begin{aligned}
&\hat{\theta}_{N+1}\\
=&\Phi_{N+1}\sum_{j=1}^{N+1}\varphi_j^\top y_j\\
=&\left(\Phi_N^{-1}+\varphi_{N+1}^\top\varphi_{N+1}\right)^{-1}\\
&\times\left(\varphi_{N+1}^\top y_{N+1}+\sum_{j=1}^{N}\varphi_j^\top y_j\right)\\
=&\Big\{\Phi_N-\Phi_N\varphi_{N+1}^\top\\
&\times\left(I_m+\varphi_{N+1}\Phi_N\varphi_{N+1}^\top\right)^{-1}\varphi_{N+1}\Phi_N\Big\}\\
&\times\left(\varphi_{N+1}^\top y_{N+1}+\sum_{j=1}^{N}\varphi_j^\top y_j\right)\\
=&\hat{\theta}_N+K_{N+1}\left(y_{N+1}-\varphi_{N+1}\hat{\theta}_N\right).
\end{aligned}$$

ただし

$$K_{N+1}:=\Phi_N\varphi_{N+1}^\top\left(I_m+\varphi_{N+1}\Phi_N\varphi_{N+1}^\top\right)^{-1}.$$

ここで式変形に逆行列補題[5]を用いて，

$$\begin{aligned}
&\Phi_{N+1}\\
=&\Phi_N-\Phi_N\varphi_{N+1}^\top\\
&\times\left(I_m+\varphi_{N+1}\Phi_N\varphi_{N+1}^\top\right)^{-1}\varphi_{N+1}\Phi_N
\end{aligned}$$

を得た．したがって，新たなデータ (x_{N+1}, y_{N+1}) が得られるたびに，新たな推定値 $\hat{\theta}_{N+1}$ はそれまでのデータによる推定値 $\hat{\theta}_N$ と Φ_N を用いることで更新できるので，改めてバッチ処理をする必要がない．オンラインで行う場合，重要なのは Φ_0 の選び方である．Φ_N の定義から，Φ_0 は定義できない．そこで Φ_N そのものを用いずに，小さな正の実数 ε を用いて $\tilde{\Phi}_0=\frac{1}{\varepsilon}I_n$ とし，次のように更新すればよい．

$$\tilde{\theta}_{N+1}=\tilde{\theta}_N+\tilde{K}_{N+1}\left(y_{N+1}-\varphi_{N+1}\tilde{\theta}_N\right), \tag{A.27}$$

$$\tilde{K}_{N+1}=\tilde{\Phi}_N\varphi_{N+1}^\top\left(I_m+\varphi_{N+1}\tilde{\Phi}_N\varphi_{N+1}^\top\right)^{-1} \tag{A.28}$$

$$\tilde{\Phi}_{N+1}=\tilde{\Phi}_N-\tilde{K}_{N+1}\varphi_{N+1}\tilde{\Phi}_N \tag{A.29}$$

ただし，$\tilde{\theta}_0=0$ とした．

ここでこの $\tilde{\Phi}_N$ と Φ_N の関係は，Φ_N が正則になる N に対して，

$$\tilde{\Phi}_N^{-1}=\Phi_N^{-1}+\varepsilon I_n=\sum_{i=1}^{N}\varphi_i^\top\varphi_i+\varepsilon I_n$$

であることに注意されたい．右辺最後の項は，注意 1.2.3 で行った正則化項を追加することに相当する．Φ_N^{-1} はデータを得るたびに正定値対称行列の意味で大きくなるため，$\tilde{\Phi}_N$ と Φ_N を Taylor 展開すると，

$$\tilde{\Phi}_N=\Phi_N-\varepsilon\Phi_N^2+O(\varepsilon^2)$$

であり，適切なデータ数 N が増えるにつれて Φ_N は正定値対称行列の意味で小さくなるので，ε の 1 次の項がゼロに近づくことが期待される．実際に ε の 1 次以上の項は N が増えると小さくなる項である．以下では，添字に時間の順序関係がある場合には k を用いる．

動的システムの制御問題では，システムのモデルを適切に作成することが重要である．その際，第一原理モデルから方程式は得られるが，そのパラメータの値は実際に測ってみないと分からないことがある．次の課題は，**システム同定（system identification）**と呼ばれる，動的システムのモデリングのうち，パラメータ推定に特化したものである．

課題 15 (システム同定) 次のバネ・マス・ダンパ系を表す直線上の運動方程式を考える．

$$M\frac{d^2}{dt^2}y(t)+D\frac{d}{dt}y(t)+Ky(t)=F(t)$$

ここで M, K, D は正定数で，$F(t)$ は外力である．$y(t)$ は観測でき，$F(t)$ はこちらで設計できるもの

[5] $A\in\mathbb{R}^{n\times n}$ を正則な行列とするとき，$(A+BC)^{-1}=A^{-1}-A^{-1}B(I+CA^{-1}B)^{-1}CA^{-1}$．ただし，$B$ と C は $BC\in\mathbb{R}^{n\times n}$ となり，$B, C^\top$ が同じサイズとなる行列．

とする．このとき，得られるデータから (M, K, D) を決定したい．微小な δt では

$$\frac{d}{dt}y(t) \simeq \frac{y((k+1)\delta t) - y(k\delta t)}{\delta t},$$
$$\frac{d^2}{dt^2}y(t) \simeq \frac{y((k+2)\delta t) - 2y((k+1)\delta t) + y(k\delta t)}{\delta t^2}$$

と近似できることを利用し，$y_k := y(k\delta t)$，$F_k := F(k\delta t)$ とすると，

$$\begin{aligned} y_k = &\left(2 - \frac{D}{M}\delta t\right) y_{k-1} \\ &- \left(1 - \frac{D}{M}\delta t + \frac{K}{M}\delta t^2\right) y_{k-2} \\ &+ \frac{\delta t^2}{M} F_{k-2} + w_k \end{aligned}$$

を得る．ここで w_k は近似の際に生じる誤差である．したがって，問題は

$$y_k = x_k^\top \theta + w_k,\ x_k := \begin{bmatrix} y_{k-1} \\ y_{k-2} \\ F_{k-2} \end{bmatrix}$$

の θ を求める問題にすることができる．パラメータの真値を $(M, D, K) = (2, 1, 3)$ とし，$\delta t = 0.01$，w_k は $[-1, 1]$ の一様分布に従う各時刻で独立な確率変数であるとして，以下の問いに答えよ．

1. $y_{-2} = 0$, $y_{-1} = 0$ とし，$F_k = 1$, $k = -2, -1, 0, 1, 2, \ldots$ として一定の外力を加えたとき，パラメータ θ を逐次最小二乗法で求めよ．ただし，パラメータの初期値は 0 とし，$k = 1000$ までによい．また正則化項は工夫せよ．

2. $y_{-2} = 0$, $y_{-1} = 0$ とし，$F_k = \sin(\pi k/5)$, $k = -2, -1, 0, 1, 2, \ldots$ として一定の外力を加えたとき，パラメータ θ を逐次最小二乗法で求めよ．ただし，パラメータの初期値は 0 とし，$k = 1000$ までによい．また正則化項は工夫せよ．

3. $y_0 = 0$, $y_1 = 0$ とする．このとき，どのような外力を加えればパラメータの推定がうまくいくか考え実装し，逐次最小二乗法で求めよ．ただし，パラメータの初期値は 0 とし，$k = 100$ までで上の結果と同等の推定値を出すように工夫せよ．

重み付き逐次最小二乗法の更新アルゴリズムも，逐次最小二乗法と同様の計算で求められる．とくに重み行列が単位行列の定数倍（$Q_i = \rho_i I$）のとき，次のように計算される．

$$\begin{aligned} \hat{\theta}_{N+1} =& \hat{\theta}_N + K_{N+1}(y_{N+1} - \varphi_{N+1}\hat{\theta}_N), \\ K_{N+1} =& \rho_{N+1}\Phi_N\varphi_{N+1}^\top \\ &\times \left(I_m + \rho_{N+1}\varphi_{N+1}\Phi_N\varphi_{N+1}^\top\right)^{-1}, \\ \Phi_{N+1} =& \Phi_N - K_{N+1}\varphi_{N+1}\Phi_N. \end{aligned}$$

とくに時系列データの場合には，古いデータの影響を小さく，新しいデータの影響を大きく使いたい場面が多い．そこで N 組のデータがあるときに $\rho_i = \gamma^{N-i}$，$\gamma \in (0, 1)$ とおくと，その推定値は

$$\hat{\theta}_{\gamma,N+1} = \left(\sum_{i=1}^{N} \gamma^{N-i}\varphi_i^\top\varphi_i\right)^{-1} \sum_{j=1}^{N} \gamma^{N-j}\varphi_j^\top y_j$$

となる．同様に $N+1$ 組のデータでは

$$\begin{aligned} \hat{\theta}_{\gamma,N+1} = &\left(\varphi_{N+1}^\top\varphi_{N+1} + \gamma\sum_{i=1}^{N}\gamma^{N-i}\varphi_i^\top\varphi_i\right)^{-1} \\ &\times \left(\varphi_{N+1}^\top y_{N+1} + \gamma\sum_{j=1}^{N}\gamma^{N-j}\varphi_j^\top y_j\right) \end{aligned}$$

となるので，これを整理すると次の更新アルゴリズムを得る．

$$\hat{\theta}_{\gamma,N+1} = \hat{\theta}_{\gamma,N} + K_{\gamma,N+1}(y_{N+1} - \varphi_{N+1}\hat{\theta}_{\gamma,N}) \tag{A.30}$$

$$K_{\gamma,N+1} = \Phi_{\gamma,N}\varphi_{N+1}^\top \left(\gamma I_m + \varphi_{N+1}\Phi_{\gamma,N}\varphi_{N+1}^\top\right)^{-1} \tag{A.31}$$

$$\Phi_{\gamma,N+1} = \frac{1}{\gamma}\Phi_{\gamma,N} - \frac{1}{\gamma}K_{\gamma,N+1}\varphi_{N+1}\Phi_{\gamma,N} \tag{A.32}$$

この γ を**忘却係数（forgetting factor）**と呼び，古いデータに指数的な重みをつけ，新しいデータの影響をパラメータ推定に強く反映させる効果を与える．

課題 16 (非定常時系列データの推定) 次の正弦波でゆっくりと変動する信号を考える．

$$y_k = \sin(0.0001k) + w_k,\ k = 1, \dots, 10000.$$

ただし，w_k は -1 と $+1$ を確率 $\frac{1}{2}$ で出す Bernoulli 過程であるとし，$\theta_k := \sin(0.0001k)$ は未知とする．このとき，$\gamma = 0.99$ の忘却係数付きの逐次最小二乗法で θ_k を推定し，各時刻の $\hat{\theta}_{\gamma,k}$ をプロットせよ．また，忘却係数を $\gamma = 0.5$ および $\gamma = 0.999$ と変更し，逐次最小二乗法の推定結果がどのように変化するかを調べよ．

1.4 Kalman フィルタと Kalman スムーザ

重み付け最小二乗法の項で述べたとおり，偶然誤差の確率分布を知っている場合（あるいは推定に必要な統計的性質を知っている場合），最小二乗誤差推定よりも**最小平均二乗誤差推定（minimum mean square error estimation）**と呼ばれる手法を使った方が推定精度が向上する[6]．議論を簡単にするため，N 個のデータ $\mathcal{D}_N := \{(x_i, y_i)\}_{i=1}^N$ の関数として，入力 x を入れたときの出力の推定値を $\hat{y}(\mathcal{D}_N, x)$ と表す．また，入力 x のときの出力を $y(x)$ で表す．このとき，入力 x に対する最小平均二乗誤差推定は，次の「関数 $\hat{y}$」に関する最適化問題の解になる（期待値の存在は仮定する）．

$$\min_{\hat{y}} \mathbb{E}_P \left[|y(x) - \hat{y}(\mathcal{D}_N, x)|^2 \right] \tag{A.33}$$

期待値を用いることで未知のデータに対しても二乗誤差の統計量を計算することができ，それを最小にするように推定規則を作成することができる．データとして用いている $\{y_i\}_{i=1}^N$ も確率変数であることに注意されたい．また，$\hat{y}$ は x にも依存する関数であるが，以下では $x_i = i$ と時間を表すパラメータだと思うことにする[7]．この問題 (A.33) の解は，条件付き期待値になることが知られている．条件付き期待値の厳密な定義や性質は「確率と統計」の授業や統計学の文献で勉強していただきたい．とくに $y(x) = \varphi(x)\theta + w$ と表される場合，最適化問題 (A.33) の解は θ の推定値を求める BLUE に等しい．そこで，重み行列を使った逐次最小二乗法の考え方を応用してみよう．とくにここでは線形なダイナミカルシステムの**潜在変数（hidden variable）**の推定問題を考える．

1.4.1 Kalman フィルタ

次の線形差分方程式を考える．

$$\theta_k = a_k \theta_{k-1} + v_k, \tag{A.34}$$

$$y_k = c_k \theta_k + w_k,\ k = 1, 2, \dots \tag{A.35}$$

ここで $a_k, c_k \in \mathbb{R}$ であり，$v_k, w_k \in \mathbb{R}$ は互いに独立な平均ゼロ，分散がそれぞれ σ_v^2, σ_w^2 の Gauss 分布に従って発生し，さらに異なる時刻で独立である（白色性をもつ）とする[8]．また，$c_k \neq 0$, $k = 1, 2, \dots$ とする．θ_0 は平均 $\bar{\theta}$, 分散 P を持つ分布に従って発生するとする（Gauss 性は不要）．ここで v_k や w_k および初期値 θ_0 は観測できないが，それらの “分布”の情報は既知であり，a_k, c_k も既知であるとしよう．観測値 y_k もセンサから各時刻で得られるものとするが，θ_k は潜在変数で直接知ることはできないとする．この節で扱う内容は，各時刻 k までの観測値 $\{y_\ell\}_{\ell=1}^k$ を用いて θ_k を推定することである．これまで扱ってきた問題は推定すべきパラメータが確定的だったが，ここでは θ_k も確率変数になっていることに注意されたい．

時刻 k までのデータによる推定値を $\hat{\theta}_k \in \mathbb{R}$ で表すとする．式 (A.27) の逐次最小二乗法では，1 ステップ前の推定値に，新たなデータ $(k, y_k) \equiv y_k$ による修正項が加えられる．式 (A.34) と (A.35)

[6] least mean square error (LMSE) と minimum mean square error (MMSE) は，異なる意味で用いられるので注意．LMSE は二乗誤差の「標本平均」に対する推定を意味し，MMSE は二乗誤差の「期待値」に対する推定を意味することが多いようである．

[7] そうでない場合は確率場の推定になり，Gauss 過程回帰などで用いられる．その場合，x の依存性を消すために x に関する積分や $\sup_x$ などを期待値の内側か外側につける．

[8] Gauss 性は本来は不要だが議論を簡単にするため．

より，

$$y_k = c_k(a_k\theta_{k-1} + v_k) + w_k$$

となるため，前の時刻の推定値 $\hat{\theta}_{k-1}$ を a_k 倍したものに新たなデータから修正する項を考え，次の規則で推定値を更新することを考える．

$$\begin{aligned}\hat{\theta}_k =& a_k\hat{\theta}_{k-1} + F_k(y_k - c_k a_k\hat{\theta}_{k-1}) \\ =& a_k\hat{\theta}_{k-1} + a_k c_k F_k(\theta_{k-1} - \hat{\theta}_{k-1}) \\ &+ F_k(w_k + c_k v_k)\end{aligned}$$

ここで $F_k \in \mathbb{R}$ は自由に設計できるパラメータである．このとき，推定誤差 $\theta_k - \hat{\theta}_k$ を最小二乗平均の意味で最小化する推定器を設計したい．推定誤差分散を $V_k := \mathbb{E}[(\theta_k - \hat{\theta}_k)^2]$ とし，各時刻 $k \geq 1$ で V_k を最小にするパラメータ F_k を求めると，

$$\begin{aligned}V_k =& a_k^2(1 - c_k F_k)^2 V_{k-1} \\ &+ (1 - c_k F_k)^2\sigma_v^2 + \sigma_w^2 F_k^2 \\ =& (a_k^2 c_k^2 V_{k-1} + c_k^2\sigma_v^2 + \sigma_w^2) \\ &\times \left(F_k - \frac{a_k^2 c_k V_{k-1} + c_k\sigma_v^2}{a_k^2 c_k^2 V_{k-1} + c_k^2\sigma_v^2 + \sigma_w^2}\right)^2 \\ &+ a_k^2 V_{k-1} + \sigma_v^2 - \frac{(a_k^2 c_k V_{k-1} + c_k\sigma_v^2)^2}{a_k^2 c_k^2 V_{k-1} + c_k^2\sigma_v^2 + \sigma_w^2}\end{aligned}$$

より，

$$F_k = \frac{a_k^2 c_k V_{k-1} + c_k\sigma_v^2}{a_k^2 c_k^2 V_{k-1} + c_k^2\sigma_v^2 + \sigma_w^2} \tag{A.36}$$

を得る．見やすくなるように変数 X_k を入れて整理すると，$\hat{\theta}_0 = \bar{\theta}, V_0 = P$ として，次の推定則を導くことができる．

$$\hat{\theta}_k = a_k\hat{\theta}_{k-1} + F_k(y_k - c_k a_k\hat{\theta}_{k-1}), \tag{A.37}$$

$$X_k = a_k^2 V_{k-1} + \sigma_v^2, \tag{A.38}$$

$$V_k = X_k - \frac{c_k^2 X_k^2}{c_k^2 X_k + \sigma_w^2} = \frac{\sigma_w^2 X_k}{c_k^2 X_k + \sigma_w^2}, \tag{A.39}$$

$$F_k = \frac{c_k X_k}{c_k^2 X_k + \sigma_w^2}. \tag{A.40}$$

この推定則は **Kalman フィルタ（Kalman filter）**と呼ばれ，システム制御や時系列解析，信号処理等で広く用いられる．θ_k, y_k が多次元であっても，行列やベクトルの平方完成を使って同様に求められる．Kalman フィルタは推定誤差分散を最小にする推定器であるが，これは定常誤差の有界性を保証するものではないことにも注意されたい．一般に，v_k が一定値に収束しなければ，推定値は真値に収束しないことにも注意されたい．

課題 17 (Kalman フィルタ) 次の離散時間ダイナミクスおよび観測方程式が与えられているとする．

$$\theta_k = 0.9\theta_{k-1} + v_k, \tag{A.41}$$

$$y_k = 2\theta_k + w_k \tag{A.42}$$

ここで $\theta_k, y_k \in \mathbb{R}$ であり，v_k, w_k は互いに独立な $\mathcal{N}(0,1)$ に従う確率変数である．また，θ_0 は平均 3, 分散 2 をもつ確率分布（何でもよい）に従うとする．このとき，$\{y_\ell\}_{\ell=1}^k$ までを使った θ_k, $k = 1, \ldots 100$ の推定値 $\hat{\theta}_k$ を求め，θ_k と共に時間変化をプロットせよ．

Kalman フィルタの Bayes 的な導出や応用例は，文献 [8, 9] とその参考文献を参照されたい．また，確率制御問題へどのように応用されるかは文献 [10] を参照されたい．

1.4.2 Kalman スムーザ

Kalman フィルタは，新たにデータ y_k を得るたびに，新たな時刻の潜在変数 θ_k の推定を行うアルゴリズムである．$\alpha_i := a_i(1 - c_i F_i)$ として $\hat{\theta}_k$, $k = 1, \ldots, N$ を改めて書き直すと，

$$\begin{aligned}\hat{\theta}_k =& \alpha_k\hat{\theta}_{k-1} + F_k y_k \\ =& \alpha_k\alpha_{k-1}\hat{\theta}_{k-2} + \alpha_k F_{k-1} y_{k-1} + F_k y_k \\ =& \prod_{i=1}^k \alpha_i\bar{\theta} + \sum_{\ell=1}^{k-1}\left(\prod_{j=\ell+1}^k \alpha_j\right) F_\ell y_\ell + F_k y_k\end{aligned}$$

となるので，次のように書き直すことができる．

$$\begin{bmatrix}\hat{\theta}_1\\ \hat{\theta}_2\\ \vdots\\ \hat{\theta}_N\end{bmatrix} = \begin{bmatrix}\alpha_1\\ \alpha_1\alpha_2\\ \vdots\\ \prod_{i=1}^N \alpha_i\end{bmatrix}\bar{\theta} + M_N \begin{bmatrix}y_1\\ y_2\\ \vdots\\ y_N\end{bmatrix}$$

$$M_N := \begin{bmatrix} F_1 & 0 & \cdots & 0\\ \alpha_2 F_1 & F_2 & \ddots & \vdots\\ \vdots & & \ddots & 0\\ \left(\prod_{i=2}^N \alpha_i\right) F_1 & \left(\prod_{i=3}^N \alpha_i\right) F_2 & \cdots & F_N\end{bmatrix}$$

M_N が下三角行列になることが，各時刻 k で k までのデータ $\{y_\ell\}_{\ell=1}^k$ を使っていることと等価になる．一方，差分方程式 (A.34) の解 θ_k は，過去の情報 $\theta_{k-\ell}$, $\ell = 1, \cdots$ に影響を受けているため，

$$\begin{aligned} y_k =& c_k\theta_k + w_k = c_k(a_k\theta_{k-1} + v_k) + w_k\\ =& c_k \prod_{j=i+1}^{k} a_j\theta_i + (\text{noise terms})\end{aligned}$$

より，y_k には過去の θ に関する情報も含まれている．そのため，新たなデータ y_k から推定値 $\hat{\theta}_\ell$, $\ell < k$ を更新することもできるはずである．これを可能にするのが **Kalman スムーザ (Kalman smoother)** であり，いくつかアルゴリズムは知られているが，ここではよく用いられる **Rauch–Tung–Striebel アルゴリズム**と呼ばれるオフラインの手法を紹介する（オンラインのアルゴリズムもあるが，あまり使われない）．

時刻 1 から N までのデータ $\{y_k\}_{k=1}^N$ から Kalman フィルタによる推定値 $\{\hat{\theta}_k\}_{k=1}^N$, 推定誤差分散 $\{V_k\}_{k=1}^N$, 一段先予測誤差分散 $\{X_k\}_{k=1}^N$ が得られているものとする．時刻 k の潜在変数 θ_k を，全てのデータを用いた推定を $\hat{\theta}_k^s$ で表す．このとき，終端条件 $\hat{\theta}_N^s = \hat{\theta}_N$, $V_N^s = V_N$ とした後退方程式

$$\hat{\theta}_k^s = \hat{\theta}_k + g_k(\hat{\theta}_{k+1}^s - a_k\hat{\theta}_k) \tag{A.43}$$

$$g_k = a_k\frac{V_k}{X_{k+1}} \tag{A.44}$$

$$V_k^s = V_k + g_k^2(V_{k+1}^s - X_{k+1}) \tag{A.45}$$

の解 $\hat{\theta}_k^s$, $k = 0, \ldots, N$ は，時刻 N までの全ての情報を用いた最小平均二乗誤差推定値である．導出に関しては文献 [9, 10] を参照されたい．また，V_k^s は推定誤差分散 $\mathbb{E}[(r_k - \hat{r}_k^s)^2]$ を表す．

課題 18 (Kalman スムーザ) 課題 17 の設定の下で，初期値 θ_0 の推定値とその推定誤差分散を求めよ．また，初期値の分散と比べて式 (A.43)–(A.45) からなる RTS アルゴリズムによる推定誤差分散がどれほど改善できたか，推定誤差分散と初期分散の比（分母が初期分散）で表せ．

1.5 交互最小二乗法

非線形最小二乗法は，問題によって様々な解き方があるので，一般論は存在しない．ここでは，機械学習などで多く用いられる **K–平均法（K-means clustering）**（あるいは K–平均クラスタリング）を題材に，非線形最小二乗法の一種である**交互最小二乗法（Alternating least squares）**を取り上げる．

1.5.1 交互最小二乗法

関数 $f(\theta, x)$ が $\theta = (\alpha, \beta)^\top$ に関して双線形であるとは，例えば

$$f(\theta, x) = \alpha\beta x + \beta$$

のように，推定したいパラメータ同士が掛け算の形になってしまっているというものである．$\alpha\beta$ を改めて別の変数に置き換えて推定してもよいが，それぞれのパラメータに意味のある場合もある．このような関数 $f(\theta, x)$ は，パラメータ $\theta \in \mathbb{R}^n$ を 2 つに分割すれば，分割したパラメータ $\alpha \in \mathbb{R}^{n_1}$, $\beta \in \mathbb{R}^{n_2}$, $n_1 + n_2 = n$ それぞれに対して線形になる．そこで，$f(\theta, x)$ を改めて $g(\alpha, \beta, x)$ とおいて，次のようにパラメータを求める．

Step 1 初期値 $\hat{\alpha}_0$, $\hat{\beta}_0$ を設定する．

Step 2 $j = 1, \ldots$ に対し，

Step 2-1 $\beta = \hat{\beta}_{j-1}$ に固定し，α を最小化する

$$\hat{\alpha}_j = \arg\min_{\alpha\in\mathbb{R}^{n_1}} \sum_{i=1}^N \|y_i - g(\alpha, \hat{\beta}_{j-1}, x_i)\|^2$$

Step 2-2 $j = 1, \ldots$ に対し，$\alpha = \hat{\alpha}_j$ を固定し，β を最小化する．

$$\hat{\beta}_j = \arg\min_{\beta \in \mathbb{R}^{n_2}} \sum_{i=1}^{N} \|y_i - g(\hat{\alpha}_j, \beta, x_i)\|^2$$

Step 3 収束判定条件（例えば事前に設定した $\varepsilon > 0$ に対して $(\|\alpha_{j-1} - \alpha_j\|^2 + \|\beta_{j-1} - \beta_j\|^2) < \varepsilon$）を満たすか判定し，満たさなければ j を 1 つ繰り上げて Step 2 に戻る．

同じデータを使ったまま，最小二乗問題を繰り返し解く問題になる．各最小化問題は，式 (A.5) の形で解けばよい．交互に最小化問題を解くため，交互最小二乗法と呼ばれる．アルゴリズムの性質から，$j = 1, \ldots$ に対して

$$\begin{aligned} &\sum_{i=1}^{N} \|y_i - g(\hat{\alpha}_j, \hat{\beta}_j, x_i)\|^2 \\ \geq &\sum_{i=1}^{N} \|y_i - g(\hat{\alpha}_{j+1}, \hat{\beta}_{j+1}, x_i)\|^2 \end{aligned}$$

なる関係も成り立つ．したがって交互最小二乗法は「悪くはならない」手法である．しかし，一般には局所的最適解への収束は必ずしも保証されるとは限らないため，初期値の工夫などが必要である．また，局所的最適解への収束が保証されないことから，得られた解の妥当性は別途検証すべきである．任意のゼロでない実数 $a \in \mathbb{R}$ に対し，(α, β) と $(a\alpha, a^{-1}\beta)$ は同じモデルを表すことに注意せよ．

課題 19 $x_i \in [-1, 1]$, $i = 1, \ldots, 10000$, に対して，次の方程式で定められる入出力データが与えられているとする．

$$y_i = \alpha^\top \varphi(x_i)\beta + w_i.$$

ここで $\alpha \in \mathbb{R}^2$, $\beta \in \mathbb{R}^3$，w_i は区間 $[-1, 1]$ の一様分布であるとし，

$$\varphi(x) = \begin{bmatrix} 1 & x & x^2 \\ x & x^2 & x^3 \end{bmatrix}$$

の場合を考える．

1. 10000 組の $\{(x_i, y_i)\}_{i=1}^{10000}$ から，交互最小二乗法を用いて α, β を推定せよ．このとき，10 組の異なる初期値を用意し，最もよかったパラメータとそのときに使った初期値を示せ．
2. 最もよかったときのパラメータを使ったとき，データ $\{(x_i, y_i)\}_{i=1}^{10000}$ と $\{(x_i, \alpha^\top \varphi(x_i)\beta)\}_{i=1}^{10000}$ を重ねてプロットし，推定の精度を議論せよ．

上の課題のように，異なる複数の初期値を使って最適化問題を解く方法をマルチスタート法あるいは多点スタート法という．

1.5.2 K-平均法

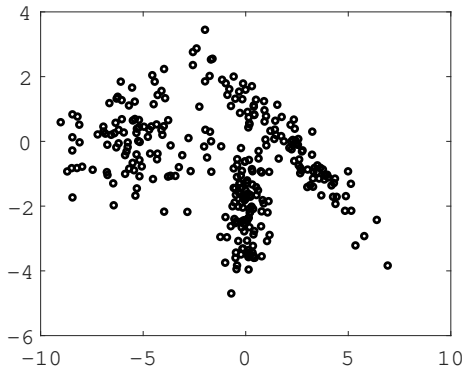


図 A.4: 観測されたデータ

K–平均法は，与えられたデータを K 個のクラスタに分類する手法であり，似たようなデータを集める代表的な方法である [4, 11]．例えば，図 A.4 は元々別々の分布から発生したデータが混在しており，元のデータは図 A.5 のように色分けされた 3 種類のデータからなる．$\{x_i\}_{i=1}^{N} \subset \mathbb{R}^p$ がデータとして与えられているものとする．これを K 個のクラスタに分類したい．ここで K はこちらで与える正整数とする．K をどのように決めるかの議論はここでは行わず，与えられているものとする．$\mathcal{V} := \{1, \ldots, N\}$ とし，$\mathcal{V}$ の空でない部分集合 $\mathcal{V}_1, \ldots, \mathcal{V}_K$ を，$\cup_{\ell=1}^{K} \mathcal{V}_\ell = \mathcal{V}$ かつ $\mathcal{V}_\ell \cap \mathcal{V}_k = \emptyset$, $\ell \neq k$ を満たすようにとる．この $\mathcal{V}_\ell$ がクラスタを

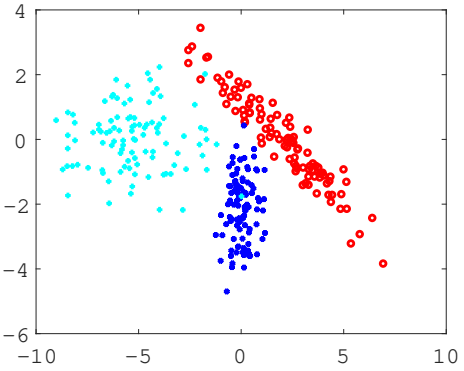


図 A.5: それぞれのデータ点を生成分布別に表した図

表し，そのクラスタを特徴づける量を次で定める．

$$\mu(\mathcal{V}_\ell) := \arg\min_{\mu\in\mathbb{R}^p} \sum_{i_\ell\in\mathcal{V}_\ell} \|x_{i_\ell} - \mu\|^2,\ \ell = 1,\dots,K \tag{A.46}$$

これは，次のようにクラスタの中心を表す．

$$\mu(\mathcal{V}_\ell) = \frac{1}{|\mathcal{V}_\ell|} \sum_{i_\ell\in\mathcal{V}_\ell} x_{i_\ell}. \tag{A.47}$$

ただし，$|\mathcal{V}_\ell|$ は $\mathcal{V}_\ell$ の要素数を表す．問題となるのはどのようにクラスタを選ぶかであるが，これは次の最小化問題を考えればよい．

$$\min_{\{\mathcal{V}_1,\dots,\mathcal{V}_K\}} \sum_{\ell=1}^{K} \sum_{i_\ell\in\mathcal{V}_\ell} \|x_{i_\ell} - \mu(\mathcal{V}_\ell)\|^2. \tag{A.48}$$

ここで

$$r_{i,\ell} := \begin{cases} 1, & i \in \mathcal{V}_\ell \\ 0, & i \notin \mathcal{V}_\ell \end{cases}$$

を導入すれば，式 (A.48) は

$$\sum_{\ell=1}^{K}\sum_{i=1}^{n} r_{i,\ell}\|x_i - \mu(\mathcal{V}_\ell)\|^2$$

となる．x_i がどのクラスタに入るかを更新するには，$\mathcal{V}_\ell$ をどう変更するかの問題になるが，これは $r_{i,\ell}$ と $\mu(\mathcal{V}_\ell)$ の両方を変更しなければならないので難しい．そこで現在のクラスタ中心 $\mu(\mathcal{V}_\ell)$ を固定して μ_ℓ と表し，$r_{i,\ell}$ を更新する．$r_{i,\ell} = 1$ は，x_i が $\mathcal{V}_\ell$ に所属することを意味したが，改めて x_i がどこのクラスタに所属するかは，クラスタの代表点の中で最も近いものに変更する．

$$r_{i,\ell} = \begin{cases} 1, & \ell = \arg\min\limits_{k=1,\dots,K} \|x_i - \mu_k\|^2 \\ 0, & \text{otherwise} \end{cases} \tag{A.49}$$

各クラスタに所属するデータが変更されたので，$\mu(\mathcal{V}_\ell)$ も計算し直す必要があり，交互に最適化問題を解くことになる．アルゴリズムをまとめると，次の通りである．

Step 1 初期値 $\mu_\ell \in \mathbb{R}^p,\ \ell = 1,\dots,K$ を設定する．

Step 2 $j = 1,\dots$ に対し，

Step 2-1 式 (A.49) より $r_{i,\ell}^{(j)},\ i = 1,\dots,N,\ \ell = 1,\dots,K$ を定める．

Step 2-2 次の式より $\mu_\ell^{(j)},\ \ell = 1,\dots,K$ を定める．

$$\mu_\ell^{(j)} = \frac{1}{\sum_{i=1}^N r_{i,\ell}} \sum_{i=1}^{N} r_{i,\ell} x_i.$$

Step 3 収束判定条件（例えば事前に設定した $\varepsilon > 0$ に対して $\max_\ell \|\mu_\ell^{(j-1)} - \mu_\ell^{(j)}\|^2 < \varepsilon$）を満たすか判定し，満たさなければ j を 1 つ繰り上げて Step 2 に戻る．

$U := [\mu_1,\dots,\mu_K]$，$R = \{r_{i,\ell}\}$，$X = [x_1,\dots,x_N]$ と置くと，K 平均法は次のコスト関数の最小化問題となる．

$$\|X - UR\|^2 \tag{A.50}$$

R が 0 か 1 の要素しか持たないことから，連続最適化と離散最適化の両方の性質を持った難しめの最小化問題になっており，大域的最適解を求められる保証はない．上述したアルゴリズムでは，データの近さを Euclid ノルムで表したが，これも本来はデータの形状などによって工夫すべきである．また，初期値の選び方も非常に重要になる．

課題 20 与えられたデータ $x_i \in \mathbb{R}^2$, $i = 1, \ldots, 1000$ を 3 つのクラスターに分類せよ．異なる初期値を 10 組以上選び，最も良さそうな分類のときに結果を図 A.5 のように色分けして図示し，うまくいかなかったならばその理由を考察せよ．

K 平均法の初期値の選び方は色々工夫されているので，インターネットでも調べてみるとよい．

参考文献

[1] 松原 望, 統計学, 東京図書, 2013.

[2] 久保川 達也, 現代数理統計学の基礎, 共立出版, 2017.

[3] 小西 貞則, 多変量解析入門, 岩波書店, 2010.

[4] 森, 黒田, 足立, 最小二乗法・交互最小二乗法, 共立出版, 2017.

[5] 北川 源四郎, 時系列解析入門, 岩波書店, 2005.

[6] 持橋, 大羽，ガウス過程と機械学習, 講談社, 2019.

[7] I. Rosh, G. Y. Grabarnik, Sparse Modeling: Theory, Algorithms, and Applications, Chapman & Hall/Crc, 2006.
（Rish, Grabarnik，スパースモデリング: 理論，アルゴリズム，応用, ジャムハウス, 2019）

[8] 足立, 丸田, カルマンフィルタの基礎, 東京電機大学出版, 2012.

[9] Särkkä, Bayesian Filtering and Smoothing, Cambridge University Press, 2013.

[10] F. L. Lewis, L. Xie, D. Popa, Optimal and Robust Estimation with an Introduction to Stochastic Control Theory, CRC Press, 2008.

[11] C. M. Bishop, Pattern Recognition and Machine Learning, Springer-Verlag, 2006.
（ビショップ，パターン認識と機械学習 上下, 丸善出版, 2012）

[大木健太郎]

付録　データの取り扱い

付録 1　データの読み込み

各課題に取り組むにあたって必要なデータファイルは panda に用意されている．それらデータファイルの C 言語上での読み込み方を示しておく．例として図 A.6 に示したようなデータファイル sample.txt を読み込むことを考える．このデータファイルはテキストファイルであり，3 列で数字が並んでいる．このとき，以下のようなプログラムで sample.txt の各列をそれぞれ double 型の配列 a[N]，b[N]，c[N] に読み込むことができる．

```
1.111 1.222 1.333
2.111 2.222 2.333
3.111 3.222 3.333
4.111 4.222 4.333
5.111 5.222 5.333
6.111 6.222 6.333
7.111 7.222 7.333
8.111 8.222 8.333
9.111 9.222 9.333
  ⋮     ⋮     ⋮
```

図 A.6: データファイル sample.txt

```
#include<stdio.h>
#define N 1000//データファイルの行数

int main(){

  double a[N],b[N],c[N];
  FILE *fp;

  fp=fopen("sample.txt","r");
```

```
    int i=0;
    while(fscanf(fp, "%lf %lf %lf",
          &a[i],&b[i],&c[i])!=EOF){
            i++;
    }
    fclose(fp);

}
```

読み込むデータが整数値でそれを int 型として読み込みたいときは，対応する配列を int 型に，対応する%lf を%d にすればよい．

付録 2　データのプロット

データのプロットについて説明する．方法は無数に考えられるが，ここでは学術的な文献などでよく用いられる Gnuplot の使い方の基礎を述べておく．インストールの方法は検索すればすぐに出てくるので割愛する．（例えば https://mtkbirdman.com/c-gnuplot-install などを参照されたい.）また，Gnuplot の機能もほとんどここでは説明しきれない．必要な時に検索し，少しずつ身に着けていくのが良い．

散布図

Gnuplot ではどのような図を描くかをコマンドで指示する．例えば図 A.6 の sample.txt の 1 列目と 2 列目を散布図として描きたいときには

```
plot "sample.txt" u 1:2
```

と入力する．なお，これはカレントディレクトリ（作業しているフォルダ）に sample.txt が置かれている場合であり，別のところに置いてある場合にはパスを指定する必要がある．同様に sample.txt の 2 列目と 3 列目の散布図を描くには

```
plot "sample.txt" u 2:3
```

と入力する．重ねて書きたい場合には

```
plot "sample.txt" u 1:2,
     "sample.txt" u 2:3
```

と入力する．

なお，何列目のデータを使うかを指定しているところ（‘‘u 1:2’’ や ‘‘u 2:3’’）を省略すると，自動的に 1 列目と 2 列目が使われる．

線グラフ

線グラフを描きたい場合には，

```
plot "sample.txt" u 1:2 w l
```

または

```
plot "sample.txt" u 1:2 w lp
```

のように書く．こうすると，散布図の点が順に線で結ばれる．線は sample.txt の一行目に対応する点から二行目に対応する点，二行目に対応する点から三行目に対応する点，三行目から四行目，··· というふうな順番で結ばれていく．そのため，関数のグラフを図示したい場合など（通常,「折れ線グラフ」という言葉でイメージするもの）を描きたい場合には横軸に対応するデータは大きさの順でソートされていなければならない．

また，

```
plot sin(x)
```

のように関数形を与えてやると直接そのグラフを描いてくれる．

その他

- 凡例の名称を変更することもできる．

  ```
  plot "sample.txt" u 1:2 title "hoge"
  ```

  と書くことによって名称が hoge になる．

- 横軸にラベル（横軸が何を表せているかの説明）を付けるには

  ```
  set xlabel "hoge"
  ```

  のように指定する．指定した後にもう一度描画してやるとラベルが付される．縦軸にラベルを付けるには xlabel を ylabel に置き換える．

- 描画する範囲を指定することもできる．

  plot [a : b][c : d] ”sample.txt”

  とすると横軸が a から b，縦軸が c から d の範囲の図が描かれる． なお，横軸の範囲のみを指定することもできる．

- 横軸を対数表示にするには

  set log x

  と入力した後に再度描画する．縦軸を対数表示したければ x を y に置き換える．x と y のどちらも書かなければ両方の軸が対数表示になる．

- その他，点や線の色・種類・大きさ・太さ，文字のサイズ等も変更できる．調べてみよ．

[岩崎淳]

付 録B　関数の補間と数値積分

2.1　目的

B 章では積分を何らかの近似により数値的に計算する手法である**数値積分**について学習する．積分は工学や物理学上の様々なモデルに現れるが，その不定積分が陽に得られるのは被積分関数が特別な形をしているときのみであり，多くの場合積分値を評価するためには何らかの近似を導入して数値的に計算する必要がある．広く用いられている数値積分法は積分領域の次元によって大きく異なるが，本章では積分領域が比較的低次元の場合（高々3,4 次元程度）に有効な，関数補間に基づく数値積分公式について述べる．なお本章では簡単のため一次元上の区間 (a,b) における積分

$$\int_a^b f(x)dx$$

の数値積分を取り扱う（2 次元以上の積分については 2.4.4 参照）．

一般に未知関数 $f(x)$ について，いくつかの点 x_i において関数値 $f(x_i)$ が与えられたときに，これらの点 x_i の間の部分における $f(x)$ の値を推定することを**補間**という．関数補間に基づく数値積分法は，被積分関数を積分計算が容易な既知関数で補間することで積分値を近似的に求める手法である．2.2 節で扱う Newton-Cotes 公式は，最も基本的な数値積分公式の一つであり，被積分関数を Lagrange 補間（多項式補間）することで積分値を計算する方法である．この積分公式では積分区間を等間隔に分割し，その分点上における被積分関数の値を使って積分値を求める．2.3 節で扱う Gauss 型積分公式は直交多項式による補間に基づく数値積分公式であり，積分区間の分割を工夫することで Newton-Cotes 積分公式よりも精度を高めた数値積分法であると捉えられる．2.4 節ではこれらの数値積分公式の代表的な応用例として，偏微分方程式の数値解法である有限要素法の解説を行う．

2.2　Lagrange 補間と Newton-Cotes 公式

本節では最も基本的な数値積分公式の一つである Newton-Cotes 公式について解説する．2.2.1 節でこの積分公式の基礎となる Lagrange 補間を解説し，2.2.2 節で Newton-Cotes 公式について解説を行う．

2.2.1　Lagrange 補間

相異なる n 個の点 $x_1 < \cdots < x_n$ において関数 f の値 $f(x_i), (i = 1, \ldots, n)$ が与えられたとき，

$$P(x_i) = f(x_i) \quad (i = 1, \cdots, n) \tag{B.1}$$

を満たす $n-1$ 次多項式 $P(x)$ を作り，$x \in [x_1, x_n]$ における $f(x)$ の値を $P(x)$ と推定することを**多項式補間**または **Lagrange 補間**と呼ぶ．このような $n-1$ 次の補間多項式 $P(x)$ は一意に存在する．実際，$P(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}$ とおくと，与えられた条件は

$$a_0 + a_1 x_i + \cdots + a_{n-1}x_i^{n-1} = f(x_i) \quad (i = 1, \cdots, n)$$

となり，この式をまとめて書くと

$$\begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ & \cdots & & \cdots & \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \cdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} f(x_1) \\ f(x_2) \\ \cdots \\ f(x_n) \end{pmatrix} \tag{B.2}$$

となる．係数行列の行列式は **Vandermonde の行列式**と呼ばれ

$$\begin{vmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ & \cdots & & \cdots & \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{vmatrix} = \prod_{i>j}(x_i - x_j) \neq 0$$

となるため，線形方程式 (B.2) の解は一意に存在する．

式 (B.1) を満たす $n-1$ 次の補間多項式 $P(x)$ は

$$P(x) = \sum_{i=1}^{n} f(x_i) l_i(x) \tag{B.3}$$

$$l_i(x) = \frac{\prod_{j\neq i}(x - x_j)}{\prod_{j\neq i}(x_i - x_j)} \tag{B.4}$$

で与えられる．実際，$l_i(x_j) = \delta_{ij}$ より $P(x_i) = f(x_i)$ となる．また $P(x)$ は $n-1$ 次多項式 $l_i(x)$ の線型結合であるため，高々 $n-1$ 次の多項式である．これらの事実と上で示した補間多項式の一意性より，(B.3) は $f(x)$ の $n-1$ 次補間多項式である．

2.2.2 Newton-Cotes 公式

定積分

$$I(f) = \int_{\widetilde{a}}^{\widetilde{b}} f(x)dx$$

を近似的に計算するために，2.2.1 節で述べた Lagrange 補間で関数 f を近似する方法を考える．$\widetilde{x}_i$ を n 個の等間隔分点

$$\widetilde{x}_i = \widetilde{x}_1 + (i-1)h \quad (1 \leq i \leq n,\, h > 0)$$

とし，この分点における関数 $f(x)$ の Lagrange 補間 (B.3) を用いると，

$$\begin{aligned} I(f) &= \int_{\widetilde{a}}^{\widetilde{b}} f(x)dx \\ &\approx \int_{\widetilde{a}}^{\widetilde{b}} P(x)dx \\ &= \int_{\widetilde{a}}^{\widetilde{b}} \sum_{i=1}^{n} f(\widetilde{x}_i) l_i(x)dx \\ &= \sum_{i=0}^{n} c_i f(\widetilde{x}_i) \end{aligned}$$

となる．ここに

$$c_i = \int_{\widetilde{a}}^{\widetilde{b}} l_i(x)dx \tag{B.5}$$

である．この数値積分公式を n 点 **Newton-Cotes 公式**という．

式 (B.4) より関数 l_i は分点 x_i から決まるので，等間隔分点 x_i の個数およびそのとり方を決めることで様々な積分公式を作ることができるが，実際によく使われるのは以下の 3 種類の公式である．

単一区間における中点公式

区間 $(\widetilde{a}, \widetilde{b})$ において，

$$\widetilde{x}_1 = \frac{\widetilde{a} + \widetilde{b}}{2}$$

の 1 点 Newton-Cotes 公式を考える．$l_1(x) \equiv 1$ に注意して，式 (B.5) より

$$c_1 = \int_{\widetilde{a}}^{\widetilde{b}} dx = \widetilde{b} - \widetilde{a}$$

となる．したがって以下の数値積分公式を得る．

$$\int_{\widetilde{a}}^{\widetilde{b}} f(x)dx \approx (\widetilde{b} - \widetilde{a}) f(\widetilde{x}_1). \tag{B.6}$$

これを**中点公式**または中点則という（図 B.1）．

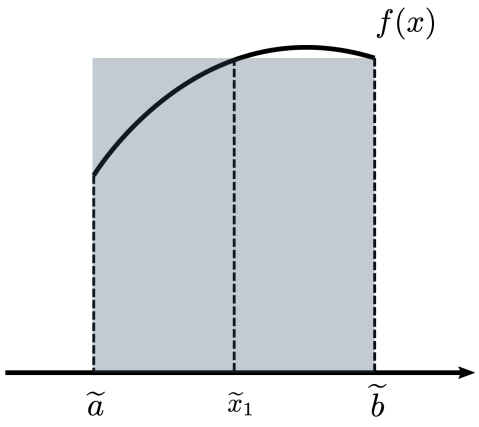


図 B.1: 積分区間 $(\widetilde{a}, \widetilde{b})$ における中点公式

単一区間における台形公式

区間 $(\widetilde{a}, \widetilde{b})$ において

$$\widetilde{x}_1 = \widetilde{a},\ \widetilde{x}_2 = \widetilde{b}$$

の 2 点 Newton-Cotes 公式を考える．分点の間隔は $\widetilde{h} = \widetilde{x}_2 - \widetilde{x}_1 = b - a$ であり，また式 (B.5) より，$c_1 = c_2 = h/2$．ゆえに以下の数値積分公式を得る．

$$\int_{\widetilde{a}}^{\widetilde{b}} f(x)dx \approx \frac{\widetilde{h}}{2}\{f(\widetilde{x}_1) + f(\widetilde{x}_2)\}. \qquad \text{(B.7)}$$

これを**台形公式**または台形則という（図 B.2）．

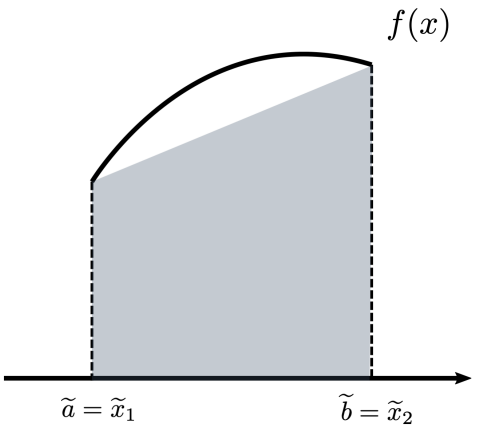


図 B.2: 積分区間 $(\widetilde{a}, \widetilde{b})$ における台形公式

単一区間における Simpson の公式

区間 $(\widetilde{a}, \widetilde{b})$ において

$$\widetilde{x}_i = \widetilde{a} + (i-1)\widetilde{h}, \quad (i = 1, 2, 3)$$
$$\widetilde{h} = \frac{\widetilde{b} - \widetilde{a}}{2}$$

の 3 点 Newton-Cotes 公式を考えると，式 (B.5) より

$$c_1 = \frac{\widetilde{h}}{3},\ c_2 = \frac{4}{3}\widetilde{h},\ c_3 = \frac{\widetilde{h}}{3}.$$

ゆえに以下の数値積分公式を得る．

$$\int_{\widetilde{a}}^{\widetilde{b}} f(x)dx \approx \frac{\widetilde{h}}{3}\{f(\widetilde{x}_1) + 4f(\widetilde{x}_2) + f(\widetilde{x}_3)\}. \qquad \text{(B.8)}$$

これは**Simpson の公式**または Simpson 則と呼ばれる（図 B.3）．

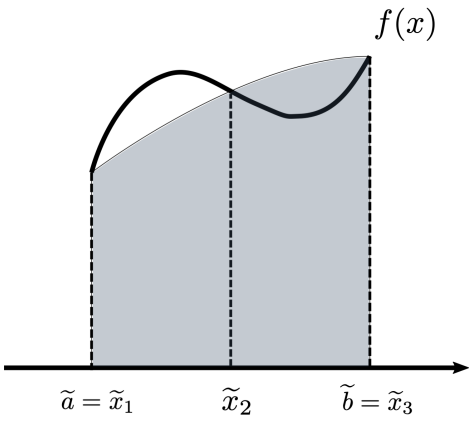


図 B.3: 積分区間 $(\widetilde{a}, \widetilde{b})$ における Simpson 公式

2.2.3 複合公式

前節において一般の n に対する n 点 Newton-Cotes 公式を求めたが，実際によく使われるのは中点公式，台形公式，Simpson の公式，すなわち $n = 1, 2, 3$ と小さい値の n に対する Newton-Cotes 公式である．「より大きな n に対する Newton-Cotes 公式を考えることで，より高精度な数値積分公式が得られないか」と考えるかもしれないが，これは大体の場合誤りである [1]（この資料の 2.2.4 節や 2.2.5 節の課題 1 も参照)．一方上で示した 3 つの公式は，被積分関数を定数，一次関数，二次関数で近似して得られる数値積分公式であるため，複雑な形状の被積分関数 $f(x)$ に対して，積分区間全体に上の公式をそのまま適用してもまともな精度は得られない．そこで実際の計算では積分区間 $[a, b]$ を等分して小区間を作り，各区間に対して上の数値積分公式を適用する．このようにして得られる積分公式を複合公式と言い，上の 3 つの積分公式の複合公式もそれぞれ複合中点公式，複合台形公式，複合 Simpson 公式と呼ぶ（但し，実際の計算ではほぼ必ず複合公式を用いるため，複合中点公式のことを単に中点公式などと呼ぶことが多い．）．

以下に 3 つの複合公式の陽な形を示す．

（複合）中点公式

$h=(b-a)/n$, $x_i=a+ih$, $(0\le i\le n)$ とおき，積分区間 (a,b) を n 個の小区間 (x_i,x_{i+1}) $(0\le i\le n-1)$ に分割し，各々の小区間に対して中点公式を適用すると，

$$\int_a^b f(x)dx \approx h\sum_{i=0}^{n-1} f\left(\frac{x_i+x_{i+1}}{2}\right) \qquad \text{(B.9)}$$

を得る．

（複合）台形公式

前例と同じ刻み幅 h と分点 x_i をとり，n 個の小区間 (x_i,x_{i+1}) に台形公式 (B.7) を適用すると，（複合）台形公式

$$\int_a^b f(x)dx \approx \frac{h}{2}\left(f(x_0)+2\sum_{i=1}^{n-1} f(x_i)+f(x_n)\right)$$

を得る．

（複合）Simpson 公式

前例と同じ刻み幅 h と分点 x_i をとり，各小区間にシンプソン公式 (B.8) を適用すると，（複合）シンプソン公式

$$\int_a^b f(x)dx \approx \frac{h}{6}\left(f(x_0)+2\sum_{i=1}^{n-1} f(x_i)+f(x_n) + 4\sum_{i=0}^{n-1} f\left(\frac{x_i+x_{i+1}}{2}\right)\right)$$

を得る．

2.2.4 Newton-Cotes 公式の誤差

区間 $(\widetilde{a},\widetilde{b})$ における単一区間の中点公式 (B.6) の左辺において $f(x)$ を $x=\widetilde{x}_1:=(a+b)/2$ 周りで Taylor 展開すると，

$$\begin{aligned}\int_{\widetilde{a}}^{\widetilde{b}} f(x)dx \approx& \int_{\widetilde{a}}^{\widetilde{b}} (f(\widetilde{x}_1)+f'(\widetilde{x}_1)(x-\widetilde{x}_1) \\ &+\frac{1}{2}f''(\widetilde{x}_1)(x-\widetilde{x}_1)^2)dx \\ =&(\widetilde{b}-\widetilde{a})f(\widetilde{x}_1)+\frac{(\widetilde{b}-\widetilde{a})^3}{24}f''(\widetilde{x}_1)\end{aligned}$$

となる．したがって積分公式 (B.6) の誤差は

$$\int_a^b f(x)dx-(b-a)f(x_1)\approx\frac{(b-a)^3}{24}f''(x_1)$$

となる．これを用いると n 点複合中点公式 (B.9) の誤差は

$$\begin{aligned}&\int_a^b f(x)dx-h\sum_{i=0}^{n-1} f\left(\frac{x_i+x_{i+1}}{2}\right)\\ =&\sum_{i=0}^{n-1}\left\{\int_{x_i}^{x_{i+1}} f(x)dx-hf\left(\frac{x_i+x_{i+1}}{2}\right)\right\}\\ =&\sum_{i=0}^{n-1}\frac{(x_{i+1}-x_i)^3}{24}f''\left(\frac{x_i+x_{i+1}}{2}\right)\\ =&\frac{n}{24}f''(\eta)h^3=\frac{(b-a)}{24}f''(\eta)h^2\end{aligned}$$

となる．ただし η は次式を満たすように選んだ．

$$\sum_{i=0}^{n-1} f''\left(\frac{x_i+x_{i+1}}{2}\right)=nf''(\eta).$$

これより，固定した積分区間 (a,b) における積分に対する（複合）中点公式の誤差は，分割幅 h に対して h^2 程度のオーダーであることがわかる．

（複合）台形公式,（複合）Simpson 公式についても同様の評価を得ることができ，それぞれ

$$-\frac{b-a}{12}f''(\eta)h^2,\quad -\frac{b-a}{2880}f^{(4)}(\eta)h^4$$

となり，それぞれ h^2, h^4 程度のオーダーである [1].

2.2.5 2.2 節の課題

1. $n=2,4,8,16$ に対して，以下の関数 $f(x)$ の $n-1$ 次多項式による Lagrange 補間のグラフを図示せよ．但し関数が定義される区間を (a,b) としたとき，分点 $x_i,(i=1,\cdots,n)$ は

$$x_i=a+\frac{b-a}{n-1}(i-1)$$

とする．また得られたグラフからわかることを述べよ．(Hint: Lagrange 補間で得られる関数 $P(x)$ は多項式なので，gnuplot などで

多項式をプロットさせるコードを書けばよいが，もしそれが難しければ，x_i よりもさらに細かい分点 y_i を取り，$P(y_i)$ 計算するコードを書いてもよい．)

(a) $f(x) = \log x \quad x \in (1, 2)$

(b) $f(x) = \frac{1}{x^3} \quad x \in (0.1, 2)$

(c) $f(x) = \frac{1}{1+25x^2} \quad x \in (-1, 1)$

2. 区間 (a, b) における定積分

$$I(f) := \int_a^b f(x)dx$$

を分割数 $n = 2^i\ (i = 0, 1, \cdots, 10)$ の（複合）中点公式，台形公式，Simpson 公式により求めるコードを作成し，以下の関数 $f(x)$ に対して適用した結果をグラフか表にまとめよ．(Hint: 結果の値によってグラフの方が見やすい場合と表の方が見やすい場合があるので適宜使い分けるとよい．またいくつかのグラフを重ねて表示するなど工夫して結果をまとめること)

(a) $f(x) = \frac{1}{x} \quad x \in (1, 2)$

(b) $f(x) = \mathrm{e}^{5x} \quad x \in (-1, 1)$

(c) $f(x) = 1 + \sin x \quad x \in (0, \pi)$

(d) $f(x) = 1 + \sin x \quad x \in (0, 2\pi)$

3. 課題 2. の定積分 $I(f)$ を分割数 n の数値積分で近似的に求めた値を $I_n(f)$ とし，その誤差を

$$E_n(f) = I_n(f) - I(f)$$

とする．課題 2 のときと同様に，$n = 2^i\ (i = 0, 1, \cdots, 10)$ に対する（複合）中点公式，台形公式，Simpson 公式を使って，(a)〜(d) に対する積分の誤差をグラフか表にまとめよ．

4. 課題 2. と課題 3. で作成したグラフや表からわかることを述べよ．

2.3 Gauss 型積分公式

前節で扱った Newton-Cotes 公式は等間隔分点上での被積分関数の値を使った積分公式であったが，本節で扱う Gauss 型積分公式は分点を適切に取ることで積分公式の精度をある意味で最良にした方法であると解釈できる．このアイデアに基づくと結果的に直交多項式の零点を分点とすればよいことがわかる．本節では 2.3.1 節で直交多項式について簡単に解説を行い，2.3.2 節で Gauss 型積分公式について解説する．

2.3.1 直交多項式

$w(x) > 0$ を区間 (a, b) で定義された連続な関数とし，実係数多項式 $p(x), q(x)$ に対して

$$(p, q)_w = \int_a^b w(x)p(x)q(x)dx \tag{B.10}$$

とおく．このとき，$(\cdot, \cdot)_w$ は内積を与える．すなわち次の性質を満たすことがわかる．

正値性 $(p, p)_w \geq 0$ で，$(p, p)_w = 0 \Rightarrow p \equiv 0$

対称性 $(p, q)_w = (q, p)_w$

線形性 多項式 $p(x), q(x), r(x)$ と実数 α, β に対して $(\alpha p + \beta q, r)_w = \alpha(p, r)_w + \beta(q, r)_w$

とくに $(p, q)_w = 0$ のとき，多項式 p, q は重み関数 w に関して**直交する**という．さらに $\phi_n(x),\ (n = 0, 1, 2, \cdots)$ が n 次多項式 $(\phi_0(x) \neq 0)$ で $(\phi_m, \phi_n)_w = 0,\ (m \neq n)$ を満たすとき，$\{\phi_n\}$ を直交多項式系という．

直交多項式系は以下の性質を満たす．

直交多項式系の存在 任意の内積に対して直交多項式系は存在する．

直交性 $\phi_n(x)$ は $n-1$ 次以下の任意の多項式と直交する．

一意性 二つの直交多項式系 $\{\phi_n(x)\}, \{\widetilde{\phi}_n(x)\}$ に対して，ある定数 $c_n \neq 0$ が存在して，$\phi_n(x) = c_n\widetilde{\phi}_n(x)$．すなわち直交多項式系は定数倍を除いて一意に定まる．

[直交多項式系の存在の証明] $\{1, x, x^2, \cdots\}$ に対して Gramm-Schmidt の直交化法を行えばよい．

[直交性の証明] $n-1$ 次以下の多項式 $p(x)$ は，$\phi_m(x)$, $(0 \le m \le n-1)$ の線型結合で表せる．すなわち $p(x) = \sum_{m=0}^{n-1} c_m \phi_m(x)$．したがって

$$\begin{aligned}(\phi_n, p)_w &= (\phi_n, \sum_{m=0}^{n-1} c_m \phi_m)_w \\ &= \sum_{m=0}^{n-1} c_m (\phi_n, \phi_m) \\ &= 0\end{aligned}$$

[一意性の証明] $\phi_n(x)$ と $\widetilde{\phi}_n(x)$ の x^n の係数をそれぞれ $c_n, \widetilde{c}_n (\neq 0)$ とすると，$q(x) = \widetilde{c}_n \phi_n(x) - c_n \widetilde{\phi}_n(x)$ は $n-1$ 次以下の多項式であるため，上で示した直交性より

$$\begin{aligned}(q, q)_w &= (q, \widetilde{c}_n \phi_n - c_n \widetilde{\phi}_n)_w \\ &= \widetilde{c}_n (q, \phi_n)_w - c_n (q, \widetilde{\phi}_n)_w \\ &= 0\end{aligned}$$

したがって $q \equiv 0$．$c_n/\widetilde{c}_n$ をあらためて c_n とおくことで $\phi_n(x) = c_n \widetilde{\phi}_n(x)$ を得る．

以上より内積（すなわち重み関数 w と区間 (a, b)）を一つ定めると直交多項式系が一つ定まることがわかる．よく用いられる直交多項式として，内積 (B.10) を

$$(p, q) = \int_{-1}^{1} p(x) q(x) dx$$

（すなわち，重み関数 $w \equiv 1$，区間 $(a, b) = (-1, 1)$）とすると得られる直交多項式系 $P_n(x)$ は **Legendre 多項式**と呼ばれる．他にも重み関数 $w = (1 - x^2)^{-1/2}$，区間 $(a, b) = (-1, 1)$ の **Chebyshev 多項式**，重み関数 $w = \mathrm{e}^{-x}$，区間 $(a, b) = (0, \infty)$ の **Laguerre 多項式**，重み関数 $w = \mathrm{e}^{-x^2}$，区間 $(a, b) = (-\infty, \infty)$ の **Hermite 多項式**などがよく知られている．

2.3.2 Gauss 型積分公式

Newton-Cotes 積分公式は，n 個の等間隔分点 x_i が与えられたときに，n 個の重み w_i を適切に設定することで，少なくとも $n-1$ 次までの多項式に対して誤差なく積分を行える積分公式と捉えることができる（→2.2.4 節）．そうすると，「重み w_i だけでなく分点 x_i も適切に決めれば，より高次の多項式に対しても誤差の無い積分公式が作れないか」と考えられる．これを実際に実現するのが **Gauss 型積分公式**である．ここではこのアイデアに基づき，Gauss 型積分公式を構成的に得る方法を示す．

n 個の重み w_i と分点 x_i, $(i = 1, \cdots, n)$ に対して積分公式

$$\int_{-1}^{1} f(x) dx = \sum_{i=1}^{n} w_i f(x_i) \tag{B.11}$$

が m 次までの多項式に対して正確であるとする（簡単のため積分範囲は $(-1, 1)$ とする）．このとき分点 x_i を零点とする n 次多項式を $G(x) = \prod_{i=1}^{n} (x - x_i)$，$r$ 次の Legendre 多項式を P_r とすると，$0 \le r \le m - n$ に対して

$$\int_{-1}^{1} G(x) P_r(x) dx = \sum_{i=1}^{n} w_i G(x_i) P_r(x_i) = 0 \tag{B.12}$$

が成り立つ（$\because G(x)P_r(x)$ は高々m 次多項式であるから）．つまり $G(x)$ と $P_r(x)$ $(0 \le r \le m-n)$ は直交する．これと Legendre 多項式の直交性より $G(x)$ は $m - n + 1$ 次以上の多項式でなければならない（$\because$ もし $G(x)$ が $m-n$ 次以下の多項式ならば，$G(x)$ は $P_r(x)$ $(0 \le r \le m-n)$ の線型結合で表されるため，いずれかの $P_r(x)$ と直交しないから）．すなわち

$$\begin{aligned}&n \ge m - n + 1 \\ &\Rightarrow m \le 2n - 1\end{aligned}$$

積分公式 (B.11) は m 次までの多項式に対して正確であるとしたので，m を最大化するために $m = 2n-1$ とする．このとき n 次多項式 $G(x)$ に対して式 (B.12) は，$0 \le r \le n-1$ で成り立たなければならない．すなわち $G(x)$ は n 次の Legendre 多項式（の定数倍）である必要がある．さらにこの事実と式 (B.12) より x_i は n 次 Legendre 多項式の零点となることがわかる．

数値積分公式の分点 x_i は n 次 Legendre 多項式の零点とすればよいことがわかったので，次に $m = 2n-1$ を実現する重み w_i を求める．$f(x)$ を $2n-1$ 次以下の多項式とするとき，$f(x)$ を $P_n(x)$ で割った商を $Q(x)$，余りを $R(x)$ とする：

$$f(x) = P_n(x)Q(x) + R(x).$$

ただし $Q(x), R(x)$ は $n-1$ 次以下の多項式である．$P_n(x)$ は $n-1$ 次以下の任意の多項式と直交するので，

$$\begin{aligned}\int_{-1}^{1} f(x)dx &= \int_{-1}^{1} (P_n(x)Q(x) + R(x))dx \\ &= \int_{-1}^{1} R(x)dx\end{aligned}$$

となる．一方，分点 x_i は Legendre 多項式の零点であったから

$$\begin{aligned}\sum_{i=1}^{n} w_i f(x_i) &= \sum_{i=1}^{n} w_i (P_n(x_i)Q(x_i) + R(x_i)) \\ &= \sum_{i=1}^{n} w_i R(x_i)\end{aligned}$$

となる．以上より，数値積分公式が $n-1$ 次以下の多項式 $R(x)$ に対して正確な値を与えるように重み w_i を定めれば，$2n-1$ 次以下の多項式 $f(x)$ に対しても正確な値を与えることがわかる．$n-1$ 次多項式 $R(x)$ の Lagrange 補間は $R(x)$ に等しいから，$R(x) = \sum_{i=1}^{n} R(x_i) l_i(x)$ である．これを積分すると

$$\int_{-1}^{1} R(x)dx = \sum_{i=1}^{n} R(x_i) \int_{-1}^{1} l_i(x)dx$$

となるので，重みは

$$w_i = \int_{-1}^{1} l_i(x)dx \tag{B.13}$$

で与えられる．n 次の Legendre 多項式の零点 x_i と重み (B.13) による積分公式 (B.11) を n 次の **Gauss 型積分公式**または **Gauss-Legendre 積分公式**と呼ぶ．

2.3.3 複合公式

Gauss 型積分公式も Newton-Cotes 公式と同様の理由（→ 2.2.3 節）で積分区間を小区間に分割し，それぞれの小区間に低次の積分公式を用いる複合公式を使うのが一般的である．一般に区間 (a, b) における積分を分点 $x_i = a + i(b-a)/n\ (i = 0, \cdots, N)$ で N 等分すると

$$\int_a^b f(x)dx = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x)dx$$

区間 (x_i, x_{i+1}) の積分は変数変換

$$y = 2\frac{x - x_i}{x_{i+1} - x_i} - 1$$

で $(-1, 1)$ での積分に変換できるので，2.3.2 節で求めた n 次 Gauss 型積分公式の積分点と重みをそれぞれ $y_m, w_m\ (m = 1, \cdots n)$ とすると，

$$\begin{aligned}&\sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x)dx \\ &= \sum_{i=0}^{N-1} \int_{-1}^{1} f\left\{\frac{(y+1)(x_{i+1} - x_i)}{2} + x_i\right\} \frac{x_{i+1} - x_i}{2} dy \\ &\approx \sum_{i=0}^{N-1} \sum_{m=1}^{n} w_m f\left\{\frac{(y_m+1)(x_{i+1} - x_i)}{2} + x_i\right\} \frac{x_{i+1} - x_i}{2}\end{aligned}$$

となり，Gauss 型積分公式に対する複合公式を得る．

2.3.4 Gauss 型積分公式の誤差

Gauss 型積分公式の誤差について以下の事実が知られている．区間 $(-1, 1)$ における積分の Gauss 型積分公式 (B.11) の誤差はある $\xi \in (a, b)$ に対して

$$\int_{-1}^{1} f(x)dx - \sum_{m=1}^{n} w_m f(y_m) \approx \frac{f^{(2n)}(\xi)}{(2n)!} \int_{-1}^{1} \phi_n(x)^2 dx$$

$$\phi_n(x) = (x - y_1)^2 \cdots (x - y_m)^2$$

となる．これを用いると（複合）Gauss 型積分公式の誤差は

$$\begin{aligned}
&\int_a^b f(x)dx - \sum_{i=0}^{N-1}\sum_{m=1}^{n} w_m f(x(y_m))\frac{x_{i+1}-x_i}{2} \\
=&\sum_{i=0}^{N-1}\int_{x_i}^{x_{i+1}} f(x)dx - \sum_{i=0}^{N-1}\sum_{m=1}^{n} w_m f(x(y_m))\frac{x_{i+1}-x_i}{2} \\
=&\sum_{i=0}^{N-1}\left\{\int_{-1}^{1} f(x(y))dy - \sum_{m=1}^{n} w_m f(x(y_m))\right\}\frac{x_{i+1}-x_i}{2} \\
\approx&\sum_{i=0}^{N-1}\frac{f^{(2n)}(\xi_i)}{(2n)!}\int_{-1}^{1}\phi_n(x)^2dx\left(\frac{x_{i+1}-x_i}{2}\right)^{2n+1} \\
=&N\frac{f^{(2n)}(\eta)}{(2n)!}\int_{-1}^{1}\phi_n(x)^2dx\left(\frac{x_{i+1}-x_i}{2}\right)^{2n+1}
\end{aligned}$$

となる．したがって複合公式の刻み幅を $h=(b-a)/N=x_{i+1}-x_i$ とすると，この式は

$$\frac{(b-a)f^{(2n)}(\eta)}{(2n)!2^{2n+1}}\int_{-1}^{1}\phi_n(x)^2dx\,h^{2n}$$

となり，誤差は h^{2n} 程度となることがわかる（2.2.4 節の Newton-Cotes 公式の誤差と比較せよ）．

2.3.5 2.3 節の課題

1. 2.2 節の課題 2 と同じ積分を分割数 $N=2^i\,(i=0,1,\cdots,10)$ の（複合）M 次 Gauss 型積分公式で計算し結果をグラフか表にまとめよ．ただし積分区間を N 分割した各小区間では表 B.1, B.2 に従い，$M=2$ または $M=3$ の Gauss 型積分公式を用いよ．

表 B.1: $M=2$ の Gauss 型積分公式

m	1	2
y_m	$-\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$
w_m	1	1

2. 以下の積分を考える．

$$\int_0^1 \frac{\mathrm{e}^{-x}}{\sqrt{x}}dx \qquad \text{(B.14)}$$

表 B.2: $M=3$ の Gauss 型積分公式

m	1	2	3
y_m	$-\sqrt{\frac{3}{5}}$	0	$\sqrt{\frac{3}{5}}$
w_m	$\frac{5}{9}$	$\frac{8}{9}$	$\frac{5}{9}$

(a) 定積分 (B.14) に対して，積分区間 $(0,1)$ を $N=2^i\,(i=0,1,\cdots,10)$ 等分した小区間に $m=2,3$ の Gauss 型積分公式を適用し，積分値とその誤差 E_n を求めよ．ただし

$$\int_0^1 \frac{\mathrm{e}^{-x}}{\sqrt{x}}dx \approx 1.49364826562$$

である．

また Gauss 型積分公式の代わりに台形則や Simpson 則を適用するとどうなるか？

(b) 定積分 (B.14) を $\sqrt{x}=t$ で変数変換した積分を求めよ．得られた不定積分に対して，その積分区間を $N=2^i\,(i=0,1,\cdots,10)$ 等分し，それぞれの小区間に $m=2,3$ の Gauss 型積分公式を適用し，積分値とその誤差 E_n を求めよ．

(c) 定積分 (B.14) を

$$\int_0^1 \frac{\mathrm{e}^{-x}}{\sqrt{x}}dx = \int_0^1 \frac{1}{\sqrt{x}}dx + \int_0^1 \frac{\mathrm{e}^{-x}-1}{\sqrt{x}}dx$$

と変形し，第一項の積分を手計算で，第二項の積分を (a),(b) と同じ数値積分でそれぞれ求めた積分値とその誤差 E_n を求めよ．

(d) (a),(b),(c) の 3 つの方法の精度にはどのような違いがあるか，理由と共に述べよ．

2.4 有限要素法

2.4.1 概要

本節では数値積分の応用例として，有限要素法について述べる．**有限要素法 (Finite Element**

Method, FEM) は変分問題に基づく数値解法であり，差分法と並んで代表的な偏微分方程式の数値解法である．元々はエンジニアによって開発された数値解法であり，機械工学，土木工学，電磁気学などに現れる様々な現象を解析するために広く用いられている．通常，有限要素法は偏微分方程式の解析に用いられるが，偏微分方程式は（当然「偏」微分を含むために）2 次元以上の領域で定式化されるため，実装が複雑になりがちである．そこで本節では，1 次元の Strum-Liouville 型の常微分方程式を例にとり，これに対する有限要素法について解説する．またこの資料では Strum-Liouville 型微分方程式に有限要素法を適用した結果の式のみを示し，有限要素法の理論については深く立ち入らない．有限要素法の詳しい解説は [1, 5] などを参照されたい．

2.4.2 Strum-Liouville 型微分方程式と弱形式

Strum-Liouville 型の 2 点境界値問題,

$$-\frac{d}{dx}\left(p(x)\frac{du}{dx}(x)\right)+q(x)u(x)=f(x),\qquad a<x<b \tag{B.15}$$

$$u(a)=u(b)=0 \tag{B.16}$$

を考える．ただし，$p(x), q(x), f(x)$ は与えられた滑らかな関数とする．

$v(a)=v(b)=0$ を満たす関数 v を式 (B.15) の両辺にかけ，区間 (a,b) で積分すると，左辺第一項は

$$\begin{aligned}&\int_a^b v(x)\left\{-\frac{d}{dx}\left(p(x)\frac{du}{dx}\right)\right\}dx\\ &=-\left[v(x)\left(p(x)\frac{du}{dx}\right)\right]_a^b+\int_a^b \frac{dv}{dx}(x)p(x)\frac{du}{dx}(x)dx\\ &=\int_a^b p(x)\frac{dv}{dx}(x)\frac{du}{dx}(x)dx\end{aligned}$$

となり，以下の式を得る:

$$\begin{aligned}&\alpha(u,v)\\ &:=\int_a^b\left(p(x)\frac{du}{dx}(x)\frac{dv}{dx}(x)+q(x)u(x)v(x)\right)dx\\ &=\int_a^b f(x)v(x)dx.\end{aligned} \tag{B.17}$$

式 (B.17) は**弱形式**や**変分形式**と呼ばれる．有限要素法では問題 (B.15),(B.16) を解く代わりに，任意の関数 v に対して弱形式 (B.17) を満たす関数 u を求めることで解を求める．

2.4.3 弱形式の離散化

任意の関数 v に対して，弱形式 (B.17) を満たす関数 u を計算機を使って求めたいが，このままでは関数 u, v の取りうる自由度が無限にあるため，計算機で扱うことは困難である．そこで**離散化**を行うことで関数 u, v が属する空間を有限次元空間に制限し，弱形式 (B.17) を計算機で扱える式に帰着する．

例えば関数 $u(x)$ が n 次元ベクトル空間に属する場合，u は**基底関数** $t_i(x), (i=1,\cdots n)$ と係数 $c_i\in\mathbb{R}$ を用いて，

$$u(x)=\sum_{i=1}^n c_i t_i(x) \tag{B.18}$$

と書ける．基底 t_i の選び方は様々なものが考えられるが，ここでは折れ線関数による近似を考えよう．区間 (a,b) の分割 $a=x_0<x_1<\cdots<x_{n-1}<x_{n+1}=b$ に対して，$h_i=x_i-x_{i-1}\,(i=1,\cdots n)$ とし，$t_i(x)$ を

$$t_i(x)=\begin{cases}\frac{x-x_{i-1}}{h_i} & (x_{i-1}\le x<x_i)\\ \frac{x_{i+1}-x}{h_{i+1}} & (x_i\le x<x_{i+1})\\ 0 & \text{その他}\end{cases} \tag{B.19}$$

とすると，式 (B.18) で表される関数 $u(x)$ は点 $u(x_i)=c_i$ を満たす折れ線関数になる（→2.4.4 節演習課題）.

関数 $u(x)$ に式 (B.18) で表される折れ線関数を代入し，さらに $v=t_i$ を代入すると，式 (B.17)

の左辺は

$$\alpha\left(\sum_{j=1}^{n} u_j t_j, t_i\right)$$
$$= \int_a^b \left\{ p(x)\frac{d}{dx}\left(\sum_{j=1}^{n} u_j t_j(x)\right)\frac{dt_i}{dx}(x) + q(x)\left(\sum_{j=1}^{n} u_j t_j(x)\right) t_i(x)\right\} dx$$
$$= \sum_{j=1}^{n} u_j \int_a^b \left(p(x)\frac{dt_j}{dx}(x)\frac{dt_i}{dx}(x) + q(x)t_i(x)t_j(x)\right) dx \quad (i = 1, \cdots n),$$

右辺は

$$\int_a^b f(x)t_i(x)dx \quad (i = 1, \cdots n)$$

となるので，これら n 本の式をまとめて行列表示で書き下すと

$$Au = b \tag{B.20}$$

となる．ここに n 次正方行列 A の ij 成分，および n 次ベクトル u, b の i 成分は，それぞれ

$$(A)_{ij} = \int_a^b \left(p(x)\frac{dt_i(x)}{dx}\frac{dt_j(x)}{dx} + q(x)t_i(x)t_j(x)\right) dx \tag{B.21}$$

$$(u)_i = u_i$$
$$(b)_i = \int_a^b f(x)t_i(x)dx \tag{B.22}$$

である．

2.4.4 2.4 節の課題

1. 式 (B.19) で定義された $t_i(x)$ によって，点 x_i を頂点とする任意の折れ線関数が式 (B.18) で表せることを確かめよ．すなわち，式 (B.18) で表される関数 $u(x)$ は $u(x_i) = c_i\,(i = 0, \cdots, n)$ を満たし，区間 $(x_{i-1}, x_i)\,(i = 0, \cdots, n)$ において一次関数になることを示せ．

2. 式 (B.20) の解を u_i とし，$\overline{u}(x) := \sum_{j=1}^{n} u_j t_j(x)$ とする．このとき任意の折れ線関数 $\widetilde{v}(x) = \sum_{i=1}^{n} \widetilde{v}_i t_i(x)$ に対して，

$$a(\overline{u}, \widetilde{v}) = \int_a^b f(x)\widetilde{v}(x)dx$$

が成り立つことを示せ．

3. 以下の 1 次元 Strum-Liouville 型境界値問題

$$-\frac{d}{dx}\left(\mathrm{e}^{-x^2}\frac{du}{dx}(x)\right) - 6\mathrm{e}^{-x^2}u(x) = 0, \quad 0 < x < 1$$
$$u(0) = 0, \quad u(1) = -4$$

を有限要素法で解くコードを作成する．

   (a) $u(x) = 8x^3 - 12x$ がこの境界値問題の解であることを確かめよ．

   (b) この境界値問題を式 (B.15), (B.16) の形の境界値問題へ変形せよ．(Hint: $v(x) = u(x) + 4x$ として，v の満たす境界値問題を導出する．)

   (c) $\frac{dt_i}{dx}(x)$ を求めよ．

   (d) 式 (B.21) の行列 A は対称かつ三重対角行列になることを示せ．

   (e) 式 (B.21) と (B.22) を計算するコードを作成せよ（Hint: 与えられた x に対して被積分関数を計算する関数を作り，これに数値積分を適用すれば良い．）．

   (f) 有限要素法のコードを完成させよ（Hint: (d) で作った線形方程式を Gauss の消去法などで解けば良い）．得られた $u(x)$ を図示し，解析解と比較せよ．

付録 A. 重積分

本章では全て 1 次元の数値積分を取り上げたが，より高次元の領域において数値積分を行う場合は，次元の大きさによって扱いが大きく異なる．次元が 4 以下程度の比較的低い場合においては，重積分を複数の一次元積分に帰着し，本章で紹介

した数値積分公式を用いる方法が一般的である．例えば $\mathbb{R}^2$ における長方形領域 $(x_1, x_2) \times (y_1, y_2)$ 上での積分は

$$\int\int_{(x_1,x_2)\times(y_1,y_2)} f(x,y)dxdy = \int_{y_1}^{y_2}\int_{x_1}^{x_2} f(x,y)dxdy$$

として，1 次元の数値積分公式をそれぞれの積分に適用すればよい．3 次元における直方体領域に対しても同じようにできることが容易にわかる．また長方形や直方体以外の領域上での数値積分も工夫することで 1 次元積分に帰着できることがある．特によく使われる 2 次元の三角形領域や 3 次元の四面体領域上での数値積分は，積分点や重みの数表が作られ，半ば数値積分公式として扱われている [5]．

また次元が 5 次以上の高次元領域における積分は，本章で扱った関数補間に基づく積分公式が使われることはあまりなく，**モンテカルロ法**などの統計的な方法で計算されることが多い．

付録 B. 被積分関数が特異性を持つ場合

被積分関数が積分区間内に特異性を持つ場合，Newton-Cotes 公式や Gauss 型積分公式をそのまま適用すると精度が悪化することが多い．この問題の対処法はいくつか考えられる．(→2.3.5 節問題 2)

変数変換 積分変数に対して適切な変数変換を施すことで特異性を消すことができる場合がある（→ 演習課題）．また上手い変数変換を施すことで特異性のある関数の積分の精度を高めた積分公式として，**二重指数関数型数値積分公式** [4] が知られている．

解析的なアプローチ 被積分関数の特異性がどの程度かわかっている場合は，被積分関数を “特異性があるが積分が解析的に行える関数” と “特異性の無い関数” の和に分割できることがある．

特異性に対応した Gauss 型数値積分公式 被積分関数の特異性がどの程度かわかっている場合には，その特異性に対応した Gauss 型積分公式を構成できることがある．例えば原点で $\log x$ の特異性 [6] や $1/x$ の特異性（Cauchy の主値積分）[7] を持っている場合の Gauss 型積分公式が知られている．

参考文献

[1] 山本哲朗, “数値解析入門 [増訂版]”, サイエンス社，2003 年 6 月．

[2] 杉原正顯，室田一雄，“数値計算法の数理”，岩波書店，1994 年 11 月．

[3] 一松信，“数値解析”，朝倉書店，1982 年 10 月．

[4] 森正武，“数値解析”，共立出版，1973 年 9 月．

[5] 菊地文雄，齊藤宣一 ，“数値解析の原理: 現象の解析をめざして”，岩波書店，2016 年 8 月

[6] S. Kapur and V. Rokhlin, “High-order corrected trapezoidal quadrature rules for singualr functions”, *Siam Journal on Numerical Analysis*, Vol. 34, No. 4, pp. 1331–1356, 1997

[7] D. Elliot and D. F. Paget, “Gauss type quadrature rules for Cauchy principal value integrals”, *Mathematics of computation*, Col. 33, Num. 145, pp. 301–309, 1979

[新納和樹]

付 録C　ニューラルネットワークによる機械学習

3.1　目的

本節では、ニューラルネットワーク（Neural Network; NN) を用いた機械学習の基礎を学習する。この枠組みでは、手書き文字認識など多くのタスクが、NN に含まれるパラメータに関する特殊な構造を有した最適化問題として定式化される。そこで、3.2 節においてこれらの最適化問題を効率よく解く手法である確率勾配法や誤差逆伝播法などの考え方とその実装方法を概説する。さらに、3.3 節以降では、Tensorflow と呼ばれるフレームワークを用いてアルゴリズムを実装する。ただし、独力でこれらのコードが書けるようになることは目指さず、準備されたサンプルコードを利用して、動作を理解することを目標とする。

3.2　数学的準備

3.2.1　ニューラルネットワーク

まずは図 C.1 で表現される関数を考えよう。それぞれのノード（円）は活性化関数と呼ばれる非線形関数の入力と出力の関係

$$h_1^{(1)} = f^{(1)}(u_1^{(1)}) \tag{C.1}$$

などを表現している。こうしたノードの縦方向の集まりのことを層と呼ぶ。層と層をつなぐエッジ（矢印）は重み $w_{ij}^{(l)}$ をもつ線型結合

$$u_j^{(l)} = \sum_i w_{ij}^{(l)} h_i^{(l-1)}$$

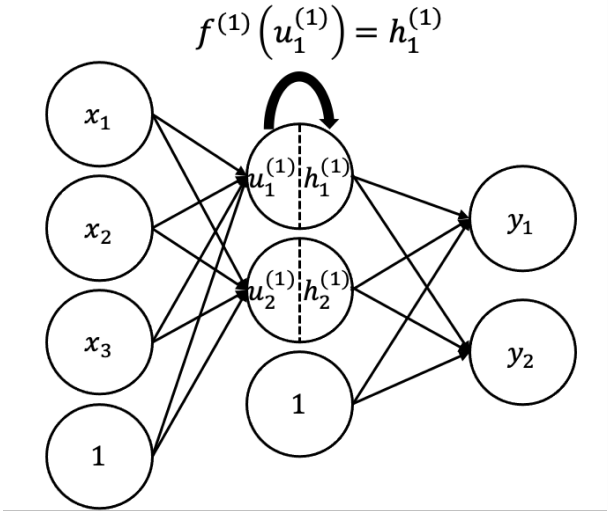


図 C.1: 3 層のニューラルネットワーク

をあらわす。NN はこうした構造をもつ関数であり、深層学習とは層の数が非常に多い NN を用いた機械学習の手法や、その周辺の研究領域のことを指す。

より一般的に L 層からなる NN を考え、第 l 層のノードの個数とそのノードを d_l, $\boldsymbol{u}^{(l)} = (u_1^{(l)}, \ldots, u_{d_l}^{(l)})^T$ と表記する。まず、入力層と呼ばれる第 1 層（input layer）は活性化関数を持たず、入力ベクトル $\boldsymbol{x} = (x_j)$ をそのまま出力する。

$$\boldsymbol{u}^{(1)} = \boldsymbol{x} \tag{C.2}$$

つぎに中間層 (internal layer) や隠れ層 (hidden layer) と呼ばれる第 l 層（$1 < l < L$）における演算処理は、パラメータ $w_{ij}^{(l)}$ と $b_j^{(l)}$ をまとめて $\boldsymbol{W}^{(l)}, \boldsymbol{b}^{(l)}$ と表記すると、

$$\boldsymbol{u}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}, \quad \boldsymbol{h}^{(l)} = f^{(l)}\left(\boldsymbol{u}^{(l)}\right) \tag{C.3}$$

と書ける。活性化関数 $f^{(l)}$ はノードごとに変えても構わないが、一般的には層ごとに共通の関数を用いる。最後の第 L 層が出力層に相当し、出力 $\hat{\boldsymbol{y}}$ は次で与えられる。

$$\hat{\boldsymbol{y}} = f^{(L)}(\boldsymbol{u}^{(L)}) \tag{C.4}$$

このように、入力が与えられたとき、NN の各層を順番に計算していき、出力まで計算を行うアー

キテクチャを順伝播 (feed forward; FF)NN と呼ばれる。

注意 3.2.1 隠れ層の活性化関数をタスクに応じて定める一般的な基準は未だ存在しない。現場では経験則や試行錯誤に頼ることが多いが、役に立つ活性化関数の選び方がいくつも知られている。例えば、学習をスムーズに進行させるには活性化関数として、正規化線形関数 (rectified linear unit; ReLU)

$$f(u) = \max\{0, u\} = \begin{cases} u & (u \geq 0) \\ 0 & (u < 0) \end{cases}$$

を用いると良いことがわかっている。活性化関数を持つノードを ReLU とも呼ぶが、関数自体も略して ReLU と呼ぶ。

3.2.2 関数回帰と最適化

設計する NN に課すタスクに応じてその構造を決定し、重み係数などのパラメータ集合 $\boldsymbol{w}$ に関する最適化問題を設定しなければならない。代表例として、訓練データ $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i\in\mathcal{B}}$, $\mathcal{B} = \{1, 2, \ldots\}$ に対して、$\hat{\boldsymbol{y}}(\boldsymbol{x}_i) \approx \boldsymbol{y}_i$ が成り立つ関数 $\hat{\boldsymbol{y}}$ を求める関数回帰問題を考える。この問題に対して、$\boldsymbol{w}$ によりパラメトライズされる関数の集合 $\{\hat{\boldsymbol{y}}(\boldsymbol{x}; \boldsymbol{w})\}_{\boldsymbol{w}}$ から求める関数を探すことにしよう。このとき、ある $\boldsymbol{w}$ を用いたモデルの予測値 $\hat{\boldsymbol{y}}(\boldsymbol{x}_i; \boldsymbol{w})$ と実際のデータ $\boldsymbol{y}_i$ ができるだけ近くなるように、平均二乗誤差

$$L(\boldsymbol{w}) = \frac{1}{|\mathcal{B}|} \sum_{i\in\mathcal{B}} \mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i; \boldsymbol{w}), \boldsymbol{y}_i), \tag{C.5}$$

$$\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \|\hat{\boldsymbol{y}} - \boldsymbol{y}\|^2 \tag{C.6}$$

に対して、

$$\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} L(\boldsymbol{w}) \tag{C.7}$$

とすることは妥当であろう。これは特別な場合として、アフィン関数

$$\hat{\boldsymbol{y}}(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \boldsymbol{W} \in \mathbb{R}^{n_y \times n_x}, \boldsymbol{b} \in \mathbb{R}^{n_y}$$

によりフィッティングを行う線形回帰を含む。以降では、線形回帰は困難なデータにも適用することを目指し、前節で導入した NN を関数集合として採用する。このとき、$\boldsymbol{w} = (\boldsymbol{W}^{(2)}, \boldsymbol{b}^{(2)}, \ldots, \boldsymbol{W}^{(L)}, \boldsymbol{b}^{(L)})$ であり、層の数や活性化関数などは適切に選択する必要がある。

他の代表的なタスクとして、データを 2 つのクラスに分類する 2 値分類がある。これは、各 $\boldsymbol{x}_i$ に対して $y_i \in \{0, 1\}$ が与えられていると考えれば良い。求めたい関数は $\{0, 1\}$ に値をとる関数であるため、出力層がシグモイド関数

$$\sigma(x) = 1/(1 + e^{-x}) \tag{C.8}$$

である NN が標準的に用いられ、$\hat{y}(\boldsymbol{x}; \boldsymbol{w}) < 1/2$ であるとき所属クラスを 0、それ以外の場合は 1 と判定する。損失関数としては、

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})) \tag{C.9}$$

を採用すれば $y_i = 1$ であれば $\hat{y}(\boldsymbol{x}_i, \boldsymbol{w}) \approx 1$、$y_i = 0$ であれば $\hat{y}(\boldsymbol{x}_i, \boldsymbol{w}) \approx 0$ となることが期待できる。

3.2.3 勾配降下法による学習

NN の学習は損失関数 $L(\boldsymbol{w})$ の最小化として定式化されていた。NN における損失関数は一般に非常に複雑な非凸関数であるため、大域的極小値 (global minimum) 以外にも、膨大な数の局所的極小値 (local minima) を持つ。実は、深層学習では真の最小値を見つけずとも、損失関数の良い極小値さえ見つければ十分であることが予想されている。そこで以下では方程式

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = 0$$

を満たす停留点を求める。さまざまな場面で用いられるニュートン法は、ヘシアン逆行列の評価にかかる計算コストが大きいため、深層学習ではほとんど用いられない。以下では、(C.7) が

- 決定変数 $\boldsymbol{w}$ が NN という特殊な関数に含まれるパラメータである、
- 損失関数 (C.5) が各データ i に対する誤差 $\mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i; \boldsymbol{w}), \boldsymbol{y}_i)$ の和である

という特徴を利用した効率的な最適化手法を紹介する。以下では記述を簡単化し、中心となる考え方に注目するために $d_l = 1,\ \boldsymbol{b}^{(l)} = 0$ とするが、一般の場合も同様の議論が成立する。

誤差逆伝播法　パラメータ $\boldsymbol{w}$ に関する損失関数 $\mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x};\boldsymbol{w}))$ の $\boldsymbol{w}$ に関する勾配を計算する。ここで、NN の特別な構造を用いると、素朴に $\boldsymbol{w}$ の各要素に関する勾配を独立に計算するよりも大幅に計算量を削減することができる。この手法は誤差逆伝播法 (back propagation) と呼ばれ、1986 年にラメルハートらによって発表され、NN の第 2 次ブームを巻き起こした。

まず、微分の連鎖律と $\boldsymbol{h}^{(l-1)}$ が $\boldsymbol{W}^{(l)}$ に依存しないという性質を用いると、

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial u_i^{(l)}} \frac{\partial u_i^{(l)}}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} h_j^{(l-1)} \qquad \text{(C.10)}$$

が $l \leq L$ に対して成り立つ。ここで、$\delta_i^{(l)} = \partial \mathcal{L}/\partial u_i^{(l)}$ とした。一方で、$l < L$ に対して

$$\begin{aligned}\delta_i^{(l)} &= \sum_{k=1}^{d_{l+1}} \frac{\partial \mathcal{L}}{\partial u_k^{(l+1)}} \frac{\partial u_k^{(l+1)}}{\partial u_i^{(l)}} \\ &= \sum_{k=1}^{d_{l+1}} \delta_k^{(l+1)} w_{ki}^{(l+1)} f'(u_i^{(l)}) \qquad \text{(C.11)}\end{aligned}$$

および $l = L$ に対して

$$\delta_i^{(L)} = \frac{\partial \mathcal{L}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial u_i^{(L)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}_i} f'(u_i^{(L)}) \qquad \text{(C.12)}$$

が成り立つ。

したがって、与えられた $(\boldsymbol{x}_i, \boldsymbol{y}_i)$ および $\boldsymbol{w}$ に対して、具体的に $\mathcal{L}(\boldsymbol{y}(\boldsymbol{x}_i;\boldsymbol{w}), \boldsymbol{y}_i)$ の $\boldsymbol{w}$ に関する勾配を求めるには、

1. 順方向に (C.2),(C.3),(C.4) により $\boldsymbol{h}^{(l)}, l < L$ を求める、
2. (C.12) により、$\delta^{(L)}$ を求める、
3. (C.11) により $\delta^{(l)}, l < L$ を逆方向に求める、
4. (C.10) により $\partial \mathcal{L}, \boldsymbol{y}_i)/\partial w_{ij}^{(l)}$ を求める、

という手順をたどればよい。この手順は L によらないが、層が深くなるに連れて勾配消失問題や勾配爆発問題などが起こることが知られている。パラメータの事前学習や活性化関数の工夫などにより、こうした問題を避ける手法の開発が盛んにおこなわれている。

課題 21 スカラー変数 $x,\ w_2,\ w_3$ に対して

$$\begin{aligned}&u^{(1)} = x,\ h^{(l)} = f(u^{(l)}), \\ &u^{(l)} = w^{(l)} h^{(l-1)},\ \hat{y} = f^{(3)}(u^{(3)})\end{aligned}$$

を定義し，$f(u) = u^2,\ \mathcal{L}(\hat{y}) = \hat{y}^2$ とする．このとき，$u^{(l)},\ h^{(l)},\ \hat{y}, \mathcal{L}$ を x, w_2, w_3 の関数としてあらわし、$\frac{\partial \mathcal{L}}{\partial w_l},\ l = 2, 3$ を求めよ。またこの例において

$$\delta^{(3)} = (2\hat{y})(2u^{(3)}),\ \delta^{(2)} = \delta^{(3)} w^{(3)} (2u^{(2)})$$

を用いて誤差逆伝播法の構造を説明せよ。

この課題において、たとえば $(x, w^{(2)}, w^{(3)}) = (1, 2, 3)$ における勾配を求める場合、$u,\ h,\ \delta$ は簡単な代入操作を繰り返すことで計算することができ、勾配もそれらの積で表現される。一方で、$\mathcal{L}$ およびその偏微分の解析表現を求め、値を代入することも可能であるが効率が悪い。また、変数が増えると、誤差逆伝播法においては使い回せる値が増えるため、その差はさらに顕著となる。

課題 22 活性化関数にシグモイド関数は勾配消失問題を起こすため、現在使われない。一方で、ReLU は勾配消失を起こす可能性が低いことが知られている。このことについて説明せよ。

確率的勾配降下法　勾配降下の過程において、局所的極小値にはまる (図 3.2.3 左) 状況をできるだけ回避するために確率的要素を取り入れる。各更新ステップ t で、ミニバッチと呼ばれる一部の訓練サンプル $\mathcal{B}^{(t)}$ のみを用いる方法をミニバッチ学習と呼ぶ（$\mathcal{B}^{(t)} = \mathcal{B}$ の場合をバッチ学習と呼ぶ)。通常ミニバッチは学習前にランダムに作成しておく。そしてステップ t では、ミニバッチ上で平均した誤差関数

$$L^{(t)}(\boldsymbol{w}) = \frac{1}{|\mathcal{B}^{(t)}|} \sum_{i \in \mathcal{B}^{(t)}} \mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i;\boldsymbol{w}), \boldsymbol{y}_i) \qquad \text{(C.13)}$$

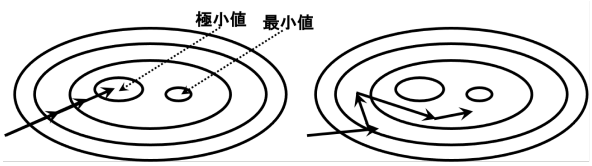


図 C.2: ミニバッチで極小値を避けるイメージ。左は極小値にはまってしまうが、ミニバッチによって極小値を避けて最小値に向かうことができる。

の勾配にもとづいて $\boldsymbol{w}$ を更新する。特に、各時刻のミニバッチに 1 つの訓練しか含まない $|\mathcal{B}^{(t)}| = 1$ という場合をオンライン学習や確率的勾配降下法と呼ぶ。ミニバッチ学習ではミニバッチ選択のランダム性により、時刻ごとに損失関数 $L^{(t)}(\boldsymbol{w})$ もランダムに変化し、望ましくない臨界点にはまり込む可能性がぐっと小さくなる (図 3.2.3)。なお、各 i に対する $\mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i;\boldsymbol{w}),\boldsymbol{y}_i)$ の勾配計算は容易に並列化できるため、並列計算環境がある場合には、ある程度のサイズのミニバッチを利用する方が良い。

ミニバッチ $\mathcal{B}^{(t)}$ 内の訓練データに対して計算された $\mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i;\boldsymbol{w}),\boldsymbol{y}_i)$ を用いて、勾配

$$\Delta \boldsymbol{W}^{(t,l)} = -\frac{1}{|\mathcal{B}^{(t)}|}\sum_{i\in\mathcal{B}^{(t)}}\frac{\mathcal{L}(\hat{\boldsymbol{y}}(\boldsymbol{x}_i;\boldsymbol{w}),\boldsymbol{y}_i)}{\partial \boldsymbol{W}^{(l)}}$$

を計算し

$$\boldsymbol{W}^{(t+1,l)} = \boldsymbol{W}^{(t,l)} + \eta^{(t)}\Delta \boldsymbol{W}^{(t,l)}$$

と重みパラメータを更新する。行列での微分は、∇ によるベクトルの成分ごとでの微分を行ったように行列の成分ごとの微分を表している．ベクトルステップサイズ $\eta^{(t)}$ の選び方や更新アルゴリズムによっても挙動は大きく変化するため、様々な手法が提案されている。

3.3 計算機実装

本節では、具体的に前節の最適化アルゴリズムを計算機上で実装する。

3.3.1 Google Colaboratory

Google Colaboratory (略称: Colab) はインターネット環境と Google のアカウントさえ持っていれば、環境構築なしでブラウザから Jupyther という Python のインターフェイスを使うことができるサービスである。深層学習の際には不可欠となる GPU も含めて無料で使うことができる。ブラウザに Google アカウントでログインした後に、以下のサンプルファイルを開く。

`https://bit.ly/39JsDg8`

サンプルファイルの中に、実装方法や本節以降の実装例も含まれている。

本節では MNIST (Mixed National Institute of Standards and Technology database) を使った画像認識を演習する。

注意 3.3.1 Colaboratory を立ち上げると、ノートブックを指定することが求められるので、「アップロード」のタブを選択する。Colaboratory が使用するノートブックはログインしている Google Drive 上にある。また、右上の「共有」のボタンを押すことにより、ノートブックの共有設定を変えられる。その上で、ノートブックが表示されているブラウザ上の URL を伝えることにより、教員や TA にノートブックを見せることができる。Colab でノートブックを実行する際は、実行環境として GPU や TPU が接続された環境を利用することができ、本演習では、GPU を接続した環境での環境での利用を想定している (デフォルトでは CPU の設定になっている)。

3.3.2 レポートの作成

Colab 内では、コード形式とテキスト形式で入力ができる。特に、テキスト形式ではマークダウン記法ができるので、レポートとして提出する際は第 1 章を参考にしてほしい。Google Drive 上のノートブックをパソコンにダウンロードするには、Colab のファイルメニューで「.ipynb をダウンロード」を選択し、提出することができる。

Colab による課題は、サンプルファイルを参照すること。

注意 3.3.2 本演習では、最適化の違いまでは解説していないが、実装で用いる TensorFlow にはいくつか Optimizer が登録されている。AdamOptimizer と FtrlOptimizer は、比較的安定して精度の高い値を与えることが知られているので、本演習では基本的には AdamOptimizer を用いることとする。

3.3.3 NN による画像認識

手書き文字認識や画像認識は典型的な多クラス分類である。K クラスの分類は、各 $\boldsymbol{x}_i$ に対して $y_i \in \{0, 1, \ldots, K\}$ が与えられていると考えれば良い。ここでは、$\hat{y}(\boldsymbol{x}; \boldsymbol{w})$ の出力は K 次元確率ベクトル (要素が非負かつ総和が 1) であるとして、その k 番目の要素が最大であるとき、所属クラスを k と判定する。出力層の活性化関数にソフトマックス関数（$\mathbb{R}^K \to \mathbb{R}^K$）

$$\mathrm{softmax}_i(x_1, \ldots, x_K) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}},\ i = 1, \ldots, K$$

を用いれば、このような NN が設計できる。

課題 23 損失関数を “MSE” にしてクロスエントロピーとの結果を比較せよ。

課題 24 次の関数

$$\begin{aligned} f(x_1, x_2) =& 10 - 10\exp(-0.2(x_1^2 + x_2^2)) \\ & - \exp((\cos(x_1) + \cos(x_2))/2) \end{aligned}$$

に対して回帰問題を考える。n 個のデータ (x_{i1}, x_{i2}), $i = 1, \ldots, n$ からこの関数上の値 $y_i = f(x_{i1}, x_{i2}) + \varepsilon$ を生成し、データセット $\{(x_{i1}, x_{i2}, y_i)\}_{i=1}^n$ をもとに，NN による関数 $f_{NN}(x_1, x_2)$ を学習し、$(x_1, x_2, f_{NN}(x_1, x_2))$ を 3 次元プロットとして図示せよ。

課題 25 TF や keras のフレームワークを使わずに、numpy を用いて、勾配法と SGD での実装せよ。 データは MNIST を使い、計算時間や損失の挙動について報告せよ。

課題 26 ノードの数を $10, 10^2, 10^3, 10^4, 10^5$ の 5 つで比較せよ。

課題 27 多層 NN の層数、ノード、バッチの設定において、エポック 100 としてエポックを増やしたときにの学習の様子を報告せよ。

課題 28 Optimizer クラスに 6 つの Optimizer（GradientDescent、Adadelta、Adagrad、Adam、Ftrl、RMSProp）を使用することで比較せよ。

課題 29 MNIST と CIFAR-10 それぞれで畳み込み層を入れたかどうかでどれほどの精度に違いがあるかを方向せよ。付録を参照のこと。

3.4 次元削減による特徴量抽出

前節までは入出力データ $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i \in \mathcal{B}}$ の関数回帰問題を念頭に、一般的に教師あり学習と呼ばれる問題に対する NN の議論を進めてきた。本節では s、（正解ラベルのついていない）データ集合 $\{\boldsymbol{x}_i\}_{i \in \mathcal{B}}$ の特徴量を次元削減により抽出する教師なし学習を考える。

3.4.1 オートエンコーダ

上記の目的のために、$\boldsymbol{y}_i = \boldsymbol{x}_i$ として前節と同様の関数回帰をおこなう。このように、$\hat{y}(\boldsymbol{x}_i) \approx \boldsymbol{x}_i$ を達成し、図 3.4.1 のように中間層のノード数が少ない「くびれた」構造をもつ NN をオートエンコーダ (Autoencoder; AE、自己符号化器) と呼ぶ。ここで、中間層の出力を潜在変数、入力から潜在変数への関数をエンコーダ、潜在変数から出力までの関数をデコーダと呼ぶ。与えられたデータの集合は、低次元の潜在変数をデコーダに入力することで再現できるという意味で、潜在変数をある種の特徴量と捉えることができる。

3.4.2 手書き文字の特徴量抽出

MNIST から 0 と 1 の画像をそれぞれ 100 個ずつ取り出し、エンコーダ、潜在変数、デコーダ、出

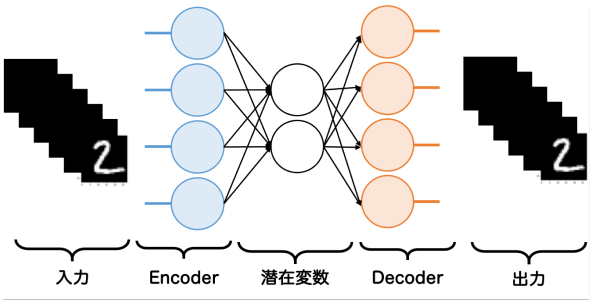


図 C.3: オートエンコーダのアーキテクチャ

力の順に結合し、学習を行なった結果が図 3.4.2 である。学習を行った結果エンコーダにより特徴が圧縮され、デコーダによりもとのデータが復元できるのであれば潜在変数には圧縮された特徴が得られていることがわかる。

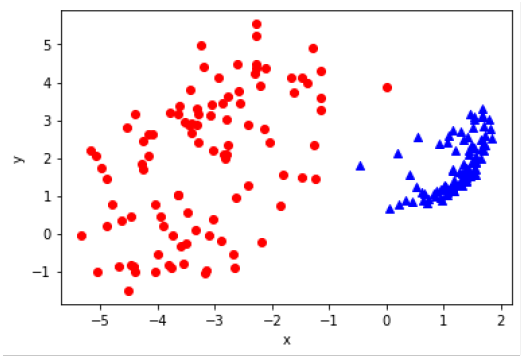


図 C.4: MNIST の 0 と 1 のそれぞれ 100 個のデータを AE で 2 次元に圧縮した散布図

課題 30 0, 1, 2 の 3 つの画像をそれぞれ 100 枚ずつ準備し、中間層で 2 次元まで圧縮し、散布図を表示せよ。(図 3.5 参照)

3.5 生成モデル学習

前節の手書き文字の特徴量抽出の例において、図 3.4.2 の赤（青）の特徴量をデコーダに入力すると 0（1）に「近い」画像が出力される。一方で、これらの特徴量の分布に関しては、$\{\boldsymbol{x}_i\}_i$ をエンコーダに入力することで知ることができるものの、学習時には特に考慮していない。そこで、本節ではこうした元データや特徴量の分布を陽に考慮した生成モデル学習と呼ばれる手法を概観する。

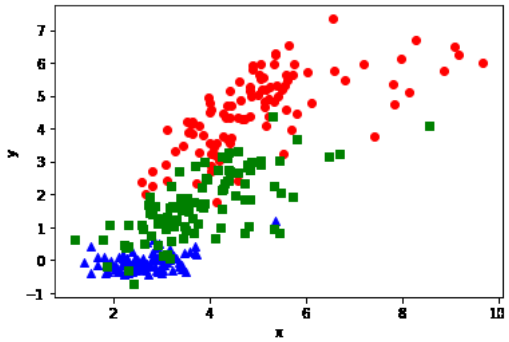


図 C.5: MNIST の 0 と 1 と 2 のそれぞれ 100 個のデータを AE で 2 次元に圧縮した散布図

3.5.1 変分オートエンコーダ

本節では、図 3.5.1 のような変分オートエンコーダ (Variational autoencorder; VAE) を紹介する。VAE での目的は潜在変数の分布が正規分布に従う（$\boldsymbol{z} \sim \mathcal{N}(0, I)$）ようなエンコーダ・デコーダを構成することである。準備として、確率密度関数 $q(x), p(x)$ に対してカルバック・ライブラー (Kullback-Leibler; KL) ダイバージェンスは次のように定義される。

$$D_{KL}(q||p) = \int q(x) \log(q(x)/p(x)) dx.$$

KL ダイバージェンスは 2 つの確率分布がどの程度似ているかを表す尺度である。エンコーダはデータ $\boldsymbol{x}$ に対して

$$\boldsymbol{z}(\boldsymbol{x}; \phi) = \mu(\boldsymbol{x}; \phi) + \sigma(\boldsymbol{x}; \phi)\varepsilon, \ \varepsilon \sim \mathcal{N}(0, I) \tag{C.14}$$

により、確率的に $\boldsymbol{z}$ を割り当てるとする。

デコーダの出力として生成されるデータの周辺尤度 $p_\theta(\boldsymbol{x}) = \int p(\boldsymbol{z}) p_\theta(\boldsymbol{x}|\boldsymbol{z}) d\boldsymbol{z}$ が与えられたデータの分布に一致するように、

$$\sum_i \log p_\theta(\boldsymbol{x}_i) \tag{C.15}$$

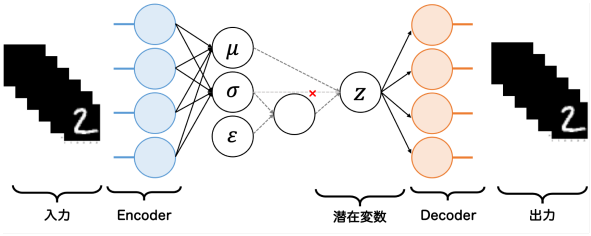


図 C.6: 変分オートエンコーダのアーキテクチャ

を最大化する θ が望ましい。ここで、推定困難な $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ の推定のために、その近似として $p_\phi(\boldsymbol{z}|\boldsymbol{x})$ を導入する。$\log p_\theta(\boldsymbol{x})$ に対して

$$\begin{aligned}\log p_\theta(\boldsymbol{x}) \geq & - D_{KL}(q_\phi(\boldsymbol{z}|\boldsymbol{x})||p_\theta(\boldsymbol{z})) \\ & + \int q_\phi(\boldsymbol{z}|\boldsymbol{x}) \log p_\theta(\boldsymbol{x}|\boldsymbol{z}) d\boldsymbol{z} \\ & (= L_{ELBO}(\boldsymbol{x};\theta,\phi)) \qquad \text{(C.16)}\end{aligned}$$

が任意の $\phi, \theta, \boldsymbol{x}$ に対して成り立つ。これを利用して、(C.15) の最大化の代わりに ELBO（Evidence Lower Bound）と呼ばれる L_{ELBO} の和

$$\sum_{i \in \mathcal{B}} L_{ELBO}(\boldsymbol{x}_i;\theta,\phi) \qquad \text{(C.17)}$$

の最大化を考える。第 1 項は解析的に求めることができる。例えば、潜在変数が 1 次元で $f_\theta(z)$ に $N(0,1)$、$q_\phi(z|x)$ に $N(\mu,\sigma^2)$ を仮定すると、$D_{KL}(q_\phi(\boldsymbol{z}|\boldsymbol{x})||p_\theta(\boldsymbol{z})) = (\sigma^2 + \mu^2 - \log\sigma^2 - 1)/2$ となる。一方で、再構成誤差に対応する第 2 項は積分計算が必要となるので、バッチ内のサンプルを用いてモンテカルロ法で近似される。以上の議論により、前節と同様の手法により θ, ϕ の最適化をおこなうことができる。学習後は、$\boldsymbol{z} \sim N(0,1)$ にしたがって、$\boldsymbol{z}$ のサンプルを生成し、それをデコーダに入力することでデータを生成できる。これが生成モデルと呼ばれる所以である。

課題 31 (C.16) を示せ。

課題 32 CIFAR-10 に対して VAE を用いて画像を生成せよ。

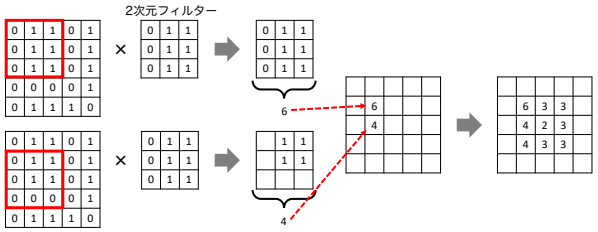


図 C.7: 5×5 の画像に対して 2 次元フィルターを適用した場合

(付録) 畳み込みニューラルネットワーク

画像ソフトにおけるぼかしやエッジ抽出などに応用されている画像処理手法にフィルターがある。畳み込みニューラルネットワーク (CNN) は図 3.5.1 のように画像入力データに 2 次元フィルターを適用して畳み込み演算を行うことで、元の画像の情報を保持しつつ、サイズの小さな画像に変換する手法であり、多くの場面で性能を向上させることができる。CNN の性能を引き上げるための様々な手法が考案されているが、最も効果があるとされているのは畳み込み層や全結合層の間に挿入するプーリング層（最大プーリングや平均プーリング）である。図 3.5.1 からもわかるように、フィルターを適用した後は画像のサイズが小さくなり、連続して適用していくと画像が小さくなりすぎてしまう。このような場合の対策として、あらかじめ元の画像の周りを 0 で埋めておくことで元のサイズを維持するゼロパディングなどがある。

参考文献

[1] 瀧雅人，機械学習スタートアップシリーズ これならわかる深層学習入門，講談社，2017.

[2] Jakub Langr，Vladimir Bok，大和田茂，実践 GAN 敵対的生成ネットワークによる深層学習，マイナビ出版，2020.

[3] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning (Adaptive Computation and

Machine Learning series) , The MIT Press, 2016.

[4] 中井悦司，TensorFlow で学ぶディープラーニング入門: 畳み込みニューラルネットワーク徹底解説，マイナビ出版，2019.

[中山、加嶋]